
DATA ANALYST

Série 2 — Volume III

Cahier d'exercices et d'applications

Exercices corrigés · Applications guidées · Projet
Compagnon du volume III : Analyse

À utiliser avec le livre : chaque chapitre renvoie ici
pour s'entraîner.

Table des matières

Mode d'emploi	1
Comment utiliser ce cahier	1
Numérotation et niveaux	1
Données et environnement	2
Lancer le code	3
Vérifier son installation	3
Mini-diagnostic de départ	4
Chapitre 1 : Analyse exploratoire des données — exercices et applications	7
Applications	7
Application 1.1 — Résumer le panier, honnêtement (section 1.1.1)	7
Application 1.2 — Choisir ses classes et lire des délais (sections 1.1.2 et 1.1.3)	8
Application 1.3 — Nuages, saison et corrélation « à saison égale » (section 1.2.1)	9
Application 1.4 — Comparer des groupes et croiser des variables (sections 1.2.2 et 1.2.3)	9
Application 1.5 — Profils hebdomadaires et indice de prix (sections 1.3.1 et 1.3.2)	10
Application 1.6 — Détecter des incidents pas à pas (section 1.3.4)	11
Application 1.7 — Le rapport d'exploration automatique (section 1.4)	11
Exercices	12
Exercice 1.1 ★ — Moyenne et médiane à la main (section 1.1.1)	12
Exercice 1.2 ★ — Moyenne géométrique (section 1.1.1)	12
Exercice 1.3 ★★ — Les classes selon Sturges (section 1.1.2)	12
Exercice 1.4 ★★ — Une hausse « à l'œil » (section 1.1.6)	12
Exercice 1.5 ★ — Pearson et Spearman (section 1.2.1)	12
Exercice 1.6 ★★ — La pluie, à mois égal (section 1.2.1)	13
Exercice 1.7 ★★ — Le V de Cramér à la main (section 1.2.3)	13
Exercice 1.8 ★★ — Fabriquer un paradoxe de Simpson (section 1.2.5)	13
Exercice 1.9 ★ — Un indice hebdomadaire à la main (section 1.3.1)	13
Exercice 1.10 ★★ — Une référence locale à la main (section 1.3.4)	13
Exercice 1.11 ★★ — Choisir un seuil par le coût (section 1.3.4)	13
Exercice 1.12 ★★ — Un calendrier d'événements (section 1.3.5)	13
Exercice 1.13 ★ — La liste de contrôle sur les produits (section 1.4)	13
Exercice 1.14 ★★ — Écrire le compte rendu (section 1.4)	14
Corrigés	14
Corrigé 1.1	14
Corrigé 1.2	14
Corrigé 1.3	14
Corrigé 1.4	15
Corrigé 1.5	15

Corrigé 1.6	15
Corrigé 1.7	16
Corrigé 1.8	16
Corrigé 1.9	16
Corrigé 1.10	17
Corrigé 1.11	17
Corrigé 1.12	17
Corrigé 1.13	18
Corrigé 1.14	18
Chapitre 2 : Tests d'hypothèses, tests A/B et corrélation — exercices et applications	19
Applications	19
Application 2.1 — Fabriquer une p-valeur en mélangeant (section 2.1)	19
Application 2.2 — Lire un test d'e-mail de bout en bout (section 2.2)	20
Application 2.3 — Défaut de répartition, appareils et sous-groupes (section 2.2)	21
Application 2.4 — Regarder en continu, soi-même (section 2.2)	21
Application 2.5 — Corrélation à saison égale (section 2.3)	21
Application 2.6 — Séries à tendance : corrélation fallacieuse (section 2.3)	22
Application 2.7 — Choisir le test pour cinq questions (section 2.4)	22
Application 2.8 — Taille et durée d'un test (section 2.5)	23
Exercices	23
Exercice 2.1 ★ — Lire une p-valeur (section 2.1)	23
Exercice 2.2 ★ — Le test z à la main (section 2.1)	23
Exercice 2.3 ★ — Intervalle de confiance de l'écart (section 2.1)	23
Exercice 2.4 ★ — Deux erreurs (section 2.1)	23
Exercice 2.5 ★★ — Défaut de répartition à la main (section 2.2)	24
Exercice 2.6 ★★ — La correction de Holm à la main (section 2.2)	24
Exercice 2.7 ★★ — Comparer deux moyennes (section 2.1)	24
Exercice 2.8 ★★ — Taille d'échantillon par la formule (section 2.5)	24
Exercice 2.9 ★★ — Pearson à la main (section 2.3)	24
Exercice 2.10 ★★ — Spearman et valeur aberrante (section 2.3)	24
Exercice 2.11 ★★★ — Le test de la promotion et la saison (section 2.3)	24
Exercice 2.12 ★★ — Choisir et lancer un test (section 2.4)	24
Exercice 2.13 ★★★ — Simuler une courbe de puissance (section 2.5)	25
Exercice 2.14 ★★★ — Rédiger le rapport d'un test (section 2.2)	25
Pistes des applications	25
Corrigés	26
Corrigé 2.1	26
Corrigé 2.2	26
Corrigé 2.3	26
Corrigé 2.4	26
Corrigé 2.5	26
Corrigé 2.6	27
Corrigé 2.7	27
Corrigé 2.8	27
Corrigé 2.9	27
Corrigé 2.10	28
Corrigé 2.11	28
Corrigé 2.12	28
Corrigé 2.13	29
Corrigé 2.14	29

Chapitre 3 : Régression pour les questions métier — exercices et applications	30
Applications	30
Application 3.1 — La droite à la main, puis à la machine (sections 3.1.1 et 3.1.2) . .	30
Application 3.2 — Promotion et publicité : du modèle naïf au modèle contrôlé (section 3.1.3)	31
Application 3.3 — Indicatrices, logarithmes et interactions (sections 3.1.4 à 3.1.6) .	32
Application 3.4 — Diagnostic : résidus, erreurs robustes, colinéarité, points in- fluents (section 3.1.7)	32
Application 3.5 — Expliquer ou prédire : 2025 mis de côté (section 3.1.8)	33
Application 3.6 — Présenter à la gérante (sections 3.2.1 à 3.2.6)	33
Application 3.7 — Les retours : régression logistique et seuil (section 3.3)	34
Exercices	35
Exercice 3.1 ★ — Une pente à la main (section 3.1.1)	35
Exercice 3.2 ★ — Un R^2 à la main (section 3.1.1)	35
Exercice 3.3 ★★ — Lire un tableau de résultats (section 3.1.2)	35
Exercice 3.4 ★ — Toutes choses égales par ailleurs (section 3.1.3)	35
Exercice 3.5 ★★ — De la valeur du coefficient à l'effet en pourcentage (section 3.1.5)	35
Exercice 3.6 ★★ — Les indicatrices (section 3.1.4)	35
Exercice 3.7 ★★ — La colinéarité à la main (section 3.1.7)	35
Exercice 3.8 ★★ — Détecter un défaut (section 3.1.7)	36
Exercice 3.9 ★★ — Mesurer une prévision (section 3.1.8)	36
Exercice 3.10 ★★★ — Une expérience pour trancher sur la publicité (section 3.1.9)	36
Exercice 3.11 ★ — Réécrire des phrases (section 3.2.5)	36
Exercice 3.12 ★★ — Relatif ou absolu (section 3.2.2)	36
Exercice 3.13 ★ — Un rapport de cotes à la main (section 3.3.1)	36
Exercice 3.14 ★★ — Un seuil par les coûts (section 3.3.5)	36
Corrigés	36
Corrigé 3.1	36
Corrigé 3.2	37
Corrigé 3.3	37
Corrigé 3.4	37
Corrigé 3.5	37
Corrigé 3.6	37
Corrigé 3.7	37
Corrigé 3.8	37
Corrigé 3.9	38
Corrigé 3.10	38
Corrigé 3.11	38
Corrigé 3.12	38
Corrigé 3.13	38
Corrigé 3.14	38
Chapitre 4 : Segmentation et analyse de cohortes — exercices et applications	39
Applications	39
Application 4.1 — Des segments par canal et fidélité (section 4.1)	39
Application 4.2 — Choisir k, un pas de plus (section 4.1)	40
Application 4.3 — Règles contre k-moyennes : qui prédit mieux ? (section 4.1) . . .	40
Application 4.4 — Cohortes mensuelles contre trimestrielles (section 4.2)	41
Application 4.5 — Revenu par client et concentration (section 4.2)	42
Application 4.6 — RFM : le piège des ex æquo (section 4.3)	42
Application 4.7 — Valeur vie client par segment (section 4.3)	43
Application 4.8 — L'entonnoir, source par appareil (section 4.4)	43

Exercices	44
Exercice 4.1 ★ — Un seuil de plus (section 4.1.2)	44
Exercice 4.2 ★★ — Une itération à la main, autres clients (section 4.1.3)	44
Exercice 4.3 ★★ — Standardiser ou non (section 4.1.4)	44
Exercice 4.4 ★★★ — Un critère extérieur pour les « chasseurs de promotions » (section 4.1.7)	44
Exercice 4.5 ★ — Une rétention à la main (section 4.2.2)	45
Exercice 4.6 ★★ — Les anciens clients et la période (section 4.2.5)	45
Exercice 4.7 ★★ — Quelles cases sont significatives ? (section 4.2.6)	45
Exercice 4.8 ★★★ — Le code de bienvenue retient-il ? (section 4.2.8)	45
Exercice 4.9 ★★ — Les gros clients à risque (section 4.3.1)	45
Exercice 4.10 ★★★ — Un seuil de relance (section 4.3.3)	45
Exercice 4.11 ★★ — Où perd-on le plus d'argent ? (section 4.4.2)	45
Exercice 4.12 ★ — Trois phrases sous le tableau (section 4.4.4)	45
Corrigés	46
Corrigé 4.1	46
Corrigé 4.2	46
Corrigé 4.3	46
Corrigé 4.4	47
Corrigé 4.5	47
Corrigé 4.6	48
Corrigé 4.7	48
Corrigé 4.8	48
Corrigé 4.9	49
Corrigé 4.10	50
Corrigé 4.11	50
Corrigé 4.12	51
Pistes des applications	51
Chapitre 5 : Séries temporelles et analyse de tendance — exercices et applications	53
Applications	53
Application 5.1 — Composantes et comparaison à l'an dernier (sections 5.1.1 à 5.1.3)	53
Application 5.2 — Indices saisonniers et décomposition (sections 5.1.4 et 5.1.5) . .	54
Application 5.3 — Calendrier et tendance (sections 5.1.6 et 5.1.7)	55
Application 5.4 — Trouver les incidents (section 5.1.8)	56
Application 5.5 — Moyennes mobiles et retard (section 5.2.1)	56
Application 5.6 — Prévoir 2025 avec 2023-2024 (sections 5.2.2 et 5.2.3)	57
Application 5.7 — Plusieurs origines et intervalle (sections 5.2.4 et 5.2.5)	58
Application 5.8 — Régression avec indicatrices (section 5.3.1)	59
Application 5.9 — Décembre prochain et planification (sections 5.3.5 et 5.3.6) . . .	59
Exercices	60
Exercice 5.1 ★ — Le mois court (section 5.1.2)	60
Exercice 5.2 ★ — Mois sur mois ou an sur an ? (section 5.1.3)	61
Exercice 5.3 ★★ — Indices saisonniers à la main (section 5.1.4)	61
Exercice 5.4 ★★ — Additif ou multiplicatif ? (section 5.1.5)	61
Exercice 5.5 ★★ — La croissance des commandes (section 5.1.6)	61
Exercice 5.6 ★★ — Choisir un seuil selon le coût (section 5.1.8)	61
Exercice 5.7 ★ — Lissage exponentiel à la main (section 5.2.1)	61
Exercice 5.8 ★★ — MAE, RMSE, MAPE (section 5.2.3)	61
Exercice 5.9 ★★ — Prévoir le canal Site (section 5.2.2)	61
Exercice 5.10 ★★★ — Comparer deux méthodes avec leur incertitude (sections 5.2.4 et 5.2.5)	62

Exercice 5.11 ★★ — L'effet de la pluie (section 5.3.1)	62
Exercice 5.12 ★★★ — Prévoir par canal et planifier (sections 5.3.3 à 5.3.6)	62
Corrigés	62
Corrigé 5.1	62
Corrigé 5.2	62
Corrigé 5.3	63
Corrigé 5.4	63
Corrigé 5.5	63
Corrigé 5.6	64
Corrigé 5.7	64
Corrigé 5.8	64
Corrigé 5.9	64
Corrigé 5.10	65
Corrigé 5.11	65
Corrigé 5.12	65
Chapitre 6 : Conception de KPI et cadres d'indicateurs — exercices et applications	67
Applications	67
Application 6.1 — Une fiche et un calcul unique (section 6.1)	67
Application 6.2 — Les pièges de définition (section 6.1)	68
Application 6.3 — Un arbre pour la marge (section 6.2)	68
Application 6.4 — Entonnoir et cadre AARRR (section 6.2)	69
Application 6.5 — Références et budget (section 6.3)	69
Application 6.6 — Cartes de contrôle et tableau de bord (section 6.3)	70
Exercices	70
Exercice 6.1 ★ — Quel chiffre garder ? (section 6.1.1)	70
Exercice 6.2 ★ — Une fiche pour le taux de rupture (section 6.1.2)	71
Exercice 6.3 ★ — La moyenne des moyennes (section 6.1.5)	71
Exercice 6.4 ★★ — Cibler un indicateur sans le casser (section 6.1.6)	71
Exercice 6.5 ★★ — Le « client actif » (section 6.1.5)	71
Exercice 6.6 ★★ — Décomposer le panier moyen (section 6.2.2)	71
Exercice 6.7 ★★ — Prix, volume et mix par catégorie (section 6.2.3)	71
Exercice 6.8 ★★★ — Un OKR sur les ruptures (sections 6.2.4 et 6.3.1)	71
Exercice 6.9 ★ — Lire un benchmark (section 6.3.2)	71
Exercice 6.10 ★★ — La précision du budget par canal (section 6.3.3)	72
Exercice 6.11 ★★ — Bruit d'un taux de rupture (section 6.3.4)	72
Exercice 6.12 ★★★ — Une carte de contrôle de la conversion (section 6.3.5)	72
Corrigés	72
Corrigé 6.1	72
Corrigé 6.2	72
Corrigé 6.3	72
Corrigé 6.4	73
Corrigé 6.5	73
Corrigé 6.6	73
Corrigé 6.7	74
Corrigé 6.8	74
Corrigé 6.9	74
Corrigé 6.10	75
Corrigé 6.11	75
Corrigé 6.12	75
Pistes des applications	76

Chapitre 7 : ☉ Analyse des écarts et des causes racines — exercices et applications	78
Applications	78
Application 7.1 — Lire un écart et fixer un seuil (section 7.1)	78
Application 7.2 — La décomposition à la main (section 7.2)	79
Application 7.3 — Décomposer la marge par canal (section 7.2)	79
Application 7.4 — Le prix et le coût derrière la marge unitaire (section 7.2)	80
Application 7.5 — Tester une hypothèse (section 7.3)	80
Application 7.6 — Rédiger le tableau d'hypothèses (section 7.3)	81
Exercices	81
Corrigés	82
Chapitre 8 : ☉ Pareto, analyse ABC et benchmarking — exercices et applications	85
Applications	85
Application 8.1 — Pareto d'une année à l'autre (section 8.1)	85
Application 8.2 — ABC par catégorie (section 8.1)	86
Application 8.3 — Les remboursements par classe (section 8.1)	86
Application 8.4 — Produits irréguliers (section 8.1)	87
Application 8.5 — Positionner la boutique (section 8.2)	87
Exercices	88
Corrigés	89
Chapitre 9 : ☉ Analyse financière — exercices et applications	92
Applications	92
Application 9.1 — Recouper les comptes avec les ventes (section 9.1 du livre)	93
Application 9.2 — Calculer les ratios à partir des comptes (section 9.2 du livre)	93
Application 9.3 — Séparer coûts fixes et coûts variables (section 9.3 du livre)	94
Application 9.4 — Rentabilité par canal et clés de répartition (section 9.3 du livre)	95
Application 9.5 — Hausse de prix ou hausse de volume ? (section 9.3 du livre)	96
Exercices	96
Exercice 9.1 ★ — Une cascade à la main (section 9.1 du livre)	96
Exercice 9.2 ★ — Marge ou marque ? (section 9.1 du livre)	96
Exercice 9.3 ★★ — Un bilan à équilibrer (section 9.1 du livre)	97
Exercice 9.4 ★ — Rotation et délais à la main (section 9.2 du livre)	97
Exercice 9.5 ★★ — La croissance consomme de la trésorerie (section 9.2 du livre)	97
Exercice 9.6 ★★ — Décomposer la rentabilité (section 9.2 du livre)	97
Exercice 9.7 ★ — Un point mort à la main (section 9.3 du livre)	97
Exercice 9.8 ★★★ — Le coût d'une livraison (section 9.3 du livre)	97
Exercice 9.9 ★★ — Arrêter ou garder un canal ? (section 9.3 du livre)	97
Corrigés	97
Corrigé 9.1	98
Corrigé 9.2	98
Corrigé 9.3	98
Corrigé 9.4	98
Corrigé 9.5	98
Corrigé 9.6	98
Corrigé 9.7	98
Corrigé 9.8	99
Corrigé 9.9	99
Pistes des applications	99
Chapitre 10 : ☉ Analytique marketing et web — exercices et applications	101
Applications	101

Application 10.1 — La conversion par source, avec son incertitude (section 10.1.2)	101
Application 10.2 — L'entonnoir par appareil (section 10.1.3)	102
Application 10.3 — La rentabilité par source payante (section 10.2.2 et 10.2.3)	102
Application 10.4 — Attribution et groupe témoin (sections 10.2.5 et 10.2.6)	103
Application 10.5 — Réconcilier un outil d'analyse web (section 10.3)	104
Exercices	105
Exercice 10.1 ★ — Un taux de conversion et son intervalle, à la main (section 10.1.2)	105
Exercice 10.2 ★ — Lire un entonnoir (section 10.1.3)	105
Exercice 10.3 ★★ — Deux sources, une vraie différence ? (section 10.1.2)	105
Exercice 10.4 ★★ — L'effet de mélange (section 10.1.5)	105
Exercice 10.5 ★ — ROAS et ROI (section 10.2.2)	105
Exercice 10.6 ★★ — Le seuil de rentabilité du ROAS (section 10.2.2)	105
Exercice 10.7 ★★ — CAC et durée de retour (section 10.2.3)	105
Exercice 10.8 ★★★ — Attribution à la main (section 10.2.5)	105
Exercice 10.9 ★★ — La couverture d'un outil (section 10.3.4)	106
Exercice 10.10 ★★★ — Des événements en double (section 10.3.5)	106
Corrigés	106
Corrigé 10.1	106
Corrigé 10.2	106
Corrigé 10.3	106
Corrigé 10.4	107
Corrigé 10.5	107
Corrigé 10.6	107
Corrigé 10.7	107
Corrigé 10.8	107
Corrigé 10.9	108
Corrigé 10.10	108

Chapitre 11 : ☉ Analytique des opérations et de la chaîne logistique — exercices et applications 109

Applications	109
Application 11.1 — Délais, centiles et taux à l'heure (sections 11.1.1 et 11.1.2)	109
Application 11.2 — Transporteurs, mode et décembre (sections 11.1.3 et 11.1.4)	110
Application 11.3 — Colis abîmés, retours et coût (section 11.1.5)	111
Application 11.4 — Ruptures et point de commande (sections 11.2.2 et 11.2.3)	111
Application 11.5 — Rejeu de la politique et quantité économique (sections 11.2.4 et 11.2.5)	112
Application 11.6 — La carte de performance des fournisseurs (section 11.3)	113
Exercices	114
Exercice 11.1 ★ — Le taux à l'heure selon la promesse (section 11.1.2)	114
Exercice 11.2 ★★ — Moyenne, médiane et 90e centile (section 11.1.2)	114
Exercice 11.3 ★ — Un intervalle de Wilson à la main (section 11.1.3)	114
Exercice 11.4 ★★ — Un paradoxe de Simpson de transporteurs (section 11.1.4)	114
Exercice 11.5 ★★ — Basculer les envois du transporteur C (section 11.1.5)	114
Exercice 11.6 ★ — Couverture et rotation à la main (section 11.2.1)	114
Exercice 11.7 ★★ — Un point de commande à la main (section 11.2.3)	114
Exercice 11.8 ★★ — La quantité économique et l'erreur sur le coût (section 11.2.5)	114
Exercice 11.9 ★★ — Deux fournisseurs, est-ce différent ? (section 11.3)	115
Exercice 11.10 ★★★ — Une note multicritère et sa sensibilité (section 11.3)	115
Corrigés	115
Corrigé 11.1	115
Corrigé 11.2	115

Corrigé 11.3	115
Corrigé 11.4	116
Corrigé 11.5	116
Corrigé 11.6	116
Corrigé 11.7	116
Corrigé 11.8	117
Corrigé 11.9	117
Corrigé 11.10	117
Chapitre 12 : ☉ Analytique RH et des ressources humaines — exercices et applications	119
Applications	119
Application 12.1 — Rotation, intervalle et motifs (sections 12.1.1 à 12.1.3)	119
Application 12.2 — Comparer des équipes, et l'absentéisme (sections 12.1.4 et 12.1.5)	120
Application 12.3 — Courbe de survie (section 12.1.6)	121
Application 12.4 — Écart brut, écart ajusté, compa-ratio (section 12.2)	121
Application 12.5 — Un modèle de départ et sa puissance (section 12.3)	122
Exercices	123
Exercice 12.1 ★ — Une rotation à la main (section 12.1.2)	123
Exercice 12.2 ★★ — Un intervalle de Poisson exact (section 12.1.2)	123
Exercice 12.3 ★★ — Kaplan-Meier à la main (section 12.1.6)	123
Exercice 12.4 ★ — Absentéisme et coût (section 12.1.5)	123
Exercice 12.5 ★★ — Brut contre ajusté sur un exemple (section 12.2.4)	123
Exercice 12.6 ★★ — Compa-ratio et petits groupes (section 12.2.5)	123
Exercice 12.7 ★★ — Combien d'années de données ? (section 12.3.1)	124
Exercice 12.8 ★★ — Puissance selon la taille (section 12.3.4)	124
Exercice 12.9 ★★ — Auditer un modèle (section 12.3.5)	124
Corrigés	124
Corrigé 12.1	124
Corrigé 12.2	124
Corrigé 12.3	124
Corrigé 12.4	125
Corrigé 12.5	125
Corrigé 12.6	125
Corrigé 12.7	126
Corrigé 12.8	126
Corrigé 12.9	126
Chapitre 13 : ☉ Analyse de sensibilité, simulations « et si » et scénarios — exercices et applications	128
Applications	128
Application 13.1 — Construire et vérifier le modèle (sections 13.1.1 et 13.1.2 du livre)	128
Application 13.2 — Tornade et seuil d'élasticité (sections 13.1.3 et 13.1.4 du livre)	129
Application 13.3 — Les lois de la simulation et un premier Monte-Carlo (sections 13.2.1 à 13.2.3 du livre)	131
Application 13.4 — Stabilité et dépendance (sections 13.2.4 et 13.2.5 du livre)	132
Application 13.5 — Scénarios et « et si » (sections 13.3.1 et 13.3.2 du livre)	132
Application 13.6 — Options, regret et seuils (sections 13.3.3 et 13.3.4 du livre)	133
Exercices	134
Exercice 13.1 ★ — Un modèle de poche, à la main (section 13.1.1 du livre)	134
Exercice 13.2 ★ — Un écart à expliquer (section 13.1.2 du livre)	134
Exercice 13.3 ★★ — La marge de contribution d'une commande (section 13.1.4 du livre)	134

Exercice 13.4 ★★ — Quelle plage pour que le trafic passe devant ? (section 13.1.6 du livre)	134
Exercice 13.5 ★ — Une loi normale pour un coût (section 13.2.2 du livre)	135
Exercice 13.6 ★★ — Combien de tirages pour 500 € près ? (section 13.2.4 du livre) .	135
Exercice 13.7 ★★★ — Répercuter les hausses de coût (sections 13.2.5 et 13.3.3 du livre)	135
Exercice 13.8 ★★ — Un autre scénario de transporteur (section 13.3.2 du livre) . .	135
Exercice 13.9 ★★ — Le regret selon l'élasticité (section 13.3.3 du livre)	135
Exercice 13.10 ★★★ — La note à la gérante (sections 13.3.4 et 13.3.5 du livre) . . .	135
Corrigés	135
Corrigé 13.1	135
Corrigé 13.2	136
Corrigé 13.3	136
Corrigé 13.4	136
Corrigé 13.5	137
Corrigé 13.6	137
Corrigé 13.7	137
Corrigé 13.8	138
Corrigé 13.9	138
Corrigé 13.10	138
Projet du volume et auto-évaluation — exercices et applications	140
Projet du volume	140
P.1 La question et le cahier des charges	140
P.2 Étape 1 : explorer	141
P.3 Étape 2 : l'effet sur les commandes	142
P.4 Étape 3 : le contrefactuel de marge	142
P.5 Étape 4 : l'incertitude et la sensibilité	143
P.6 Étape 5 : le seuil de bascule	144
P.7 Étape 6 : mesurer mieux la prochaine fois	144
P.8 Étape 7 : la recommandation à la gérante	144
P.9 Les limites de l'étude	145
P.10 La vérité programmée	145
P.11 Variante : l'objet d'e-mail	145
Auto-évaluation	146
Exploration (chapitre 1)	146
Tests, A/B et corrélation (chapitre 2)	146
Régression (chapitre 3)	147
Segmentation et cohortes (chapitre 4)	147
Séries temporelles (chapitre 5)	147
KPI (chapitre 6)	147
Chapitres complémentaires (7 à 13)	147
Corrigés des questions	148
Votre grille d'auto-évaluation	152

Mode d'emploi

« On ne mesure bien que ce que l'on a défini avant de regarder. »

Ce cahier est le **compagnon du livre** du volume III (*Analyse*). Le livre explique les méthodes ; le cahier les fait travailler. Il contient, chapitre par chapitre, des **applications guidées**, des **exercices** et leurs **corrigés**, puis le **projet du volume** (répondre de bout en bout à une vraie question métier) et l'auto-évaluation.

Comment utiliser ce cahier

1. **Lisez d'abord la section du livre.** Chaque section qui a un prolongement ici se termine par une ligne de ce genre : > ■ **Pour s'entraîner.** Cahier, chapitre 2 : application 2.1, exercices 2.1 à 2.4.
2. **Formulez la question avant de calculer.** Avant tout code, écrivez en une phrase : quelle décision cette analyse éclaire-t-elle, et de quel **type** est la question (décrire, comparer, expliquer, prévoir, décider) ? C'est l'habitude centrale du volume.
3. **Écrivez votre hypothèse, puis regardez les données.** Une hypothèse écrite à l'avance protège de la tentation de « trouver ce qui sort ».
4. **Essayez avant de regarder le corrigé.** Les énoncés sont regroupés dans la partie *Exercices*, les corrigés dans la partie *Corrigés* du même chapitre.
5. **Donnez toujours l'incertitude.** Un résultat se rend sous la forme *estimation, intervalle, ce que cela permet de conclure, ce que cela ne permet pas de conclure*.
6. **Recoupez.** Un chiffre obtenu par pandas se recoupe par SQL, par R ou par un calcul à la main ; un écart est un signal.
7. **N'ouvrez pas les fichiers de vérité pour décider.** Ce sont des corrigés : on les consulte après son analyse.

Numérotation et niveaux

Chaque chapitre du cahier correspond au chapitre du livre de même numéro.

Élément	Numérotation	Exemple
Application	Application N.k	Application 2.1 : première application du chapitre 2
Exercice	Exercice N.k	Exercice 2.3 : troisième exercice du chapitre 2

Élément	Numérotation	Exemple
Corrigé	Corrigé N.k	Corrigé 2.3 : correction de l'exercice 2.3

La difficulté des exercices est indiquée par des étoiles :

Niveau	Signification
★	application directe d'une idée ou d'un geste vu dans la section
★★	demande de combiner deux idées, ou un petit raisonnement
★★★	demande de la réflexion, un choix de méthode ou une petite expérience

Un exercice cite la section du livre qu'il exerce, par exemple « (section 2.2) ».

Données et environnement

Les données sont celles du livre, dans le dossier `donnees/`. Elles sont toutes **simulées**, avec des graines fixes (script `build/donnees_a3.py`) : vos résultats seront identiques à ceux du livre. Le catalogue complet figure dans la section « Carte du volume, données et environnement » du livre ; en voici l'essentiel.

Fichier	Contenu	Chapitres
<code>clients.csv</code> , <code>produits.csv</code> , <code>commandes.csv</code> , <code>lignes_commande.csv</code> , <code>retours.csv</code>	la base de la boutique (volumes I et II)	1, 3, 4, 7, 8
<code>jours_exploitation.csv</code> , <code>jours_incidents.csv</code>	une ligne par jour (et la même série avec des incidents injectés)	1, 3, 5
<code>ab_email.csv</code> , <code>ab_site.csv</code>	deux tests A/B	2
<code>sessions_web.csv</code> , <code>campagnes.csv</code>	trafic du site et dépenses publicitaires	4, 6, 10
<code>budget_reel_2025.csv</code> , <code>benchmark_secteur.csv</code>	budget contre réalisé ; indicateurs d'un secteur fictif	6, 7, 8
<code>compte_resultat_mensuel.csv</code> , <code>bilan_annuel.csv</code>	comptes de la boutique	9, 13
<code>livraisons.csv</code> , <code>reappro_fournisseur.csv</code> , <code>stock_quotidien.csv</code>	logistique	11
<code>employees.csv</code> , <code>employees_annees.csv</code> , <code>departs.csv</code>	ressources humaines (petits effectifs)	12
<code>verite_incidents.csv</code>	corrigé des incidents injectés	à la fin

Les chiffres sont fictifs. La boutique, ses clients et ses résultats sont inventés. Ne tirez de ces données aucune conclusion sur le monde réel : elles servent à apprendre une méthode.

Chaque chapitre du cahier est **autonome** : il commence par ses imports et recharge ses données. Les applications sont dimensionnées pour s'exécuter en **quelques secondes à quelques dizaines de secondes** sur un ordinateur ordinaire, sans réseau.

Lancer le code

Placez-vous dans le dossier du volume (celui qui contient `donnees/`), car les chemins sont relatifs (`donnees/commandes.csv`). Vous pouvez travailler dans un notebook Jupyter, dans un éditeur avec la commande `python`, dans RStudio pour R, ou dans un outil de bases de données pour SQL.

Les chemins. Si Python répond `FileNotFoundError`, c'est presque toujours que vous n'êtes pas dans le bon dossier : vérifiez avec `import os; print(os.getcwd())`.

Le hasard. Les exercices qui tirent au sort (simulations, rééchantillonnages) fixent une **graine** (`np.random.default_rng(0)`) : si vous la changez, vos nombres changeront un peu, et c'est normal ; ce qu'il faut retrouver, c'est l'ordre de grandeur et la conclusion.

Excel. Les exercices qui demandent un tableur se font dans Excel ou dans LibreOffice Calc (gratuit), qui lit les mêmes fichiers ; quand un menu ou une fonction diffère, la documentation de votre version fait foi.

Vérifier son installation

Quatre courts blocs vérifient que les bibliothèques sont installées et que les données se chargent, en Python, en SQL puis en R.

1. Les bibliothèques Python (le volume utilise celles des volumes précédents) :

```
import sys
from importlib import import_module
from importlib.metadata import version
print("python", sys.version.split()[0])
for p in ["numpy", "pandas", "scipy", "statsmodels", "sklearn", "matplotlib"]:
    import_module(p)
    print(f"{p:12s}", version("scikit-learn" if p == "sklearn" else p))
```

```
python 3.13.3
numpy      2.5.3
pandas     3.0.6
scipy      1.18.1
statsmodels 0.15.0
sklearn    1.9.1
matplotlib 3.11.2
```

2. Les données. Nous chargeons les commandes et les jours d'exploitation :

```
import pandas as pd

commandes = pd.read_csv("donnees/commandes.csv")
jours = pd.read_csv("donnees/jours_exploitation.csv", parse_dates=["date"])
print("commandes :", len(commandes), "| jours d'exploitation :", len(jours), "du",
      jours["date"].min().date(), "au", jours["date"].max().date())
```

```
commandes : 36395 | jours d'exploitation : 1096 du 2023-01-01 au 2025-12-31
```

3. Une requête SQL. Nous copions les commandes dans une base SQLite en mémoire, puis interrogeons-la :

```
import sqlite3
con = sqlite3.connect(":memory:")
commandes.to_sql("commandes", con, index=False)
```

```
SELECT canal, COUNT(*) AS commandes
FROM commandes
GROUP BY canal
ORDER BY commandes DESC;
```

canal	commandes
Boutique	16975
Site	15463
Réseaux	3957

4. Le même comptage en R, pour vérifier que R et le tidyverse sont là :

```
suppressPackageStartupMessages({library(readr); library(dplyr)})
commandes <- read_csv(file.path(Sys.getenv("DONNEES"), "commandes.csv"), show_col_types = FALSE)
print(count(commandes, canal, sort = TRUE))
```

```
# A tibble: 3 × 2
  canal      n
  <chr> <int>
1 Boutique 16975
2 Site     15463
3 Réseaux  3957
```

Vous devez lire une version pour chaque bibliothèque, **36 395 commandes** et **1 096 jours** d'exploitation, et **le même tableau** par canal en SQL et en R. Si tout concorde, vous êtes prêt.

Mini-diagnostic de départ

Huit questions pour vérifier que les réflexes de base de l'analyse sont en place. Répondez par écrit, puis comparez avec les corrigés plus bas ; chaque corrigé indique le chapitre à relire si la réponse vous a échappé.

1. Pour chacune de ces questions de la gérante, dites de quel **type** elle relève (décrire, comparer, expliquer, prévoir, décider) : (a) « Combien de commandes avons-nous eues en novembre ? » (b) « Le nouveau transporteur livre-t-il plus vite que l'ancien ? » (c) « Qu'est-ce qui pousse les clients à partir ? » (d) « Combien vendrons-nous en décembre ? » (e) « Faut-il refaire la promotion en juillet ? »
2. Un test A/B sur l'objet d'un e-mail donne un taux d'achat de 2,92 % pour A et 3,38 % pour B, avec 6 000 contacts par groupe. L'écart est-il « significatif » ? Que peut-on conclure ?
3. La dépense publicitaire et le chiffre d'affaires quotidiens sont corrélés (coefficient de 0,42). Peut-on en conclure que la publicité fait vendre ? Que regarder ?
4. Les ventes de sept jours consécutifs valent 20, 22, 19, 25, 40, 21 et 23. Calculez la **moyenne mobile sur trois jours** (là où elle est définie) et dites ce que le 40 devient.
5. Le « taux de conversion » du site est-il un bon indicateur ? Comment le définir sans ambiguïté, et quelle est sa valeur sur nos sessions ?

6. Le panier moyen est de 100,4 € mais la médiane est plus basse. Que cela indique-t-il, et lequel annoncer ?
7. Dans une régression de $\log(\text{nombre de commandes})$ sur la promotion, le coefficient de la promotion vaut 0,177. Que signifie-t-il en pourcentage ?
8. Un effet réel de +0,4 point sur un taux d'achat de 3 % existe, mais le test ne le détecte pas avec 6 000 contacts par groupe. Pourquoi, et que faire ?

Les **corrigés** s'appuient sur des calculs : le code est ci-dessous.

```
import numpy as np
import statsmodels.api as sm
from scipy import stats
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize, proportions_ztest

ab = pd.read_csv("donnees/ab_email.csv")
n = ab.groupby("groupe").size(); s = ab.groupby("groupe")["achat_7j"].sum()
stat, p = proportions_ztest([s["B"], s["A"]], [n["B"], n["A"]])
ecart = s["B"] / n["B"] - s["A"] / n["A"]
se = np.sqrt((s["A"] / n["A"]) * (1 - s["A"] / n["A"]) / n["A"] + (s["B"] / n["B"]) * (1 - s["B"] / n["B"]) / n["B"])
print("Q2 achats :", s.to_dict(), "| écart :", round(ecart * 100, 2), "points | p =", round(p, 3), "| IC 95 % :", round((ecart - 1.96 * se) * 100, 2), "à", round((ecart + 1.96 * se) * 100, 2), "points")
puissance = NormalIndPower().power(effect_size=proportion_effectsize(0.0338, 0.0292), nobs1=6000, alpha=0.05)
n_necessaire = NormalIndPower().solve_power(effect_size=proportion_effectsize(0.034, 0.030), power=0.8, alpha=0.05)
print("Q8 puissance observée :", round(puissance, 2), "| contacts par groupe pour +0,4 point (3,0 % -> 3,4 %) à 80 % de puissance :", round(n_necessaire))
```

```
Q2 achats : {'A': 175, 'B': 203} | écart : 0.47 points | p = 0.143 | IC 95 % : -0.16 à 1.09 points
Q8 puissance observée : 0.3 | contacts par groupe pour +0,4 point (3,0 % -> 3,4 %) à 80 % de puissance : 30362
```

```
pub = jours["depense_pub"].corr(jours["chiffre_affaires"])
jours["mois"] = jours["date"].dt.to_period("M")
a_mois_egal = (jours["depense_pub"] - jours.groupby("mois")["depense_pub"].transform("mean")).corr(jours["chiffre_affaires"] - jours.groupby("mois")["chiffre_affaires"].transform("mean"))
print("Q3 corrélation brute :", round(pub, 2), "| à mois égal (écarts à la moyenne du mois) :", round(a_mois_egal, 2))
ventes = pd.Series([20, 22, 19, 25, 40, 21, 23])
print("Q4 moyenne mobile sur 3 jours :", ventes.rolling(3).mean().dropna().round(2).tolist(), "| moyenne :", round(ventes.mean(), 2))
ses = pd.read_csv("donnees/sessions_web.csv")
print("Q5 conversion :", int(ses["commande"].sum()), "commandes /", len(ses), "sessions =", round(ses["commande"].mean() * 100, 2), "%")
lig = pd.read_csv("donnees/lignes_commande.csv")
paniers = lig.groupby("id_commande")["montant"].sum()
print("Q6 panier moyen :", round(paniers.mean(), 1), "| médiane :", round(paniers.median(), 1))
print("Q7 exp(0,177) - 1 =", round(np.exp(0.177) - 1, 3))
```

```
Q3 corrélation brute : 0.42 | à mois égal (écarts à la moyenne du mois) : 0.05
Q4 moyenne mobile sur 3 jours : [20.33, 22.0, 28.0, 28.67, 28.0] | moyenne : 24.29
Q5 conversion : 6078 commandes / 127022 sessions = 4.78 %
Q6 panier moyen : 100.4 | médiane : 79.8
Q7 exp(0,177) - 1 = 0.194
```

1. (a) **Décrire** ; (b) **comparer** ; (c) **expliquer** ; (d) **prévoir** ; (e) **décider**. Les chapitres associés : 1 et 6 pour décrire, 2 pour comparer, 3 et 4 pour expliquer, 5 pour prévoir, 6 et les chapitres complémentaires pour décider (introduction).

2. L'écart observé est de **+0,47 point** (203 achats sur 6 000 contre 175, soit 3,38 % contre 2,92 %), mais le test donne $p = 0,14$ et l'intervalle de confiance à 95 % de l'écart va de **-0,16 à +1,09 point** : il contient zéro. On **ne peut pas conclure** que B est meilleur, ni qu'il ne l'est pas : le test n'a pas assez de données. Une p-valeur supérieure à 0,05 signifie « données compatibles avec l'absence d'effet », pas « absence d'effet » (chapitre 2, sections 2.1 et 2.2).

3. Non, pas sur cette seule corrélation. La dépense publicitaire est **plus forte en novembre-décembre et au printemps**, les mêmes périodes où les ventes sont fortes : la **saison** (variable de confusion) explique une grande part du lien. À mois égal (en comparant chaque jour à la moyenne de son mois), la corrélation tombe à presque rien. Il faut une comparaison à saison égale, une régression ou, mieux, une expérience (chapitres 1, 2 et 3).

4. La moyenne mobile sur trois jours vaut 20,33 ; 22,0 ; 28,0 ; 28,67 ; 28,0 (cinq valeurs, centrées sur les jours 2 à 6). Le 40 est « étalé » sur trois valeurs, qui montent toutes à 28 environ (la moyenne de la série est de 24,29) : la moyenne mobile **lisse** le pic, au prix d'un retard et d'une perte de netteté (chapitre 5, section 5.2).

5. Oui, à **condition d'une définition sans ambiguïté** : nombre de commandes divisé par le nombre de sessions, sur la même période et le même périmètre (ici, le site). Sur nos sessions, 6 078 commandes pour 127 022 sessions donnent **4,78 %**. Un bon indicateur est défini, mesurable, comparable dans le temps et relié à une action (chapitre 6, section 6.1).

6. Le panier moyen (100,4 €) est tiré vers le haut par quelques gros paniers (médiane de 79,8 €) : la distribution est **asymétrique**. La **médiane**, de 79,8 €, décrit mieux le panier « typique » (la moitié des commandes en dessous) ; on annonce l'une et l'autre, avec leur définition, ou la médiane seule pour parler d'une commande ordinaire (chapitre 1, section 1.1 ; volume I, section 1.1).

7. Un coefficient b sur le logarithme du nombre de commandes se lit comme une variation **relative** : $e^{0,177} - 1 \approx +19,4 \%$. Les jours de promotion, il y a environ 19 % de commandes de plus, **toutes choses égales par ailleurs** (chapitre 3, section 3.2).

8. Parce qu'un petit effet demande de **gros échantillons** : avec 6 000 contacts par groupe, la puissance du test pour distinguer 2,92 % de 3,38 % est d'environ **30 %** (code ci-dessus), loin des 80 % visés. Pour détecter un écart de 0,4 point (de 3,0 % à 3,4 %), il faudrait environ **30 000 contacts par groupe**, soit cinq fois plus que la liste. On peut augmenter l'échantillon, allonger la durée, regrouper des tests, ou accepter qu'un effet de cette taille ne vaille pas le coût de le mesurer (chapitre 2, section 2.5).

Comment vous situer. Six bonnes réponses sur huit indiquent que vous pouvez aborder le volume directement. Si les questions 2, 3 et 8 vous ont posé problème, lisez d'abord le chapitre 2 (tests et A/B) avant les chapitres 3 à 6.

Chapitre 1 : Analyse exploratoire des données — exercices et applications

Ce chapitre du cahier prolonge le chapitre 1 du livre. Les **applications** sont de petites études guidées sur les données de la boutique, à refaire pas à pas ; les **exercices** (★ facile, ★★ moyen, ★★★ difficile) sont corrigés à la fin. Chaque élément renvoie à la section du livre qui le prépare. Tout le code s'exécute à partir du dossier du volume.

```
import os, sys, warnings
sys.path.insert(0, "build")
warnings.filterwarnings("ignore")
import numpy as np, pandas as pd
from scipy import stats
import outils_ch01 as O

donnees = O.charger()
cmd, lig, prod, ret, j, cli, liv, ji, vi = (donnees[k] for k in ["cmd", "lig", "prod", "ret", "j",
↳ "cli", "liv", "ji", "vi"])
print(len(cmd), "commandes |", len(j), "jours |", len(liv), "livraisons")
```

```
36395 commandes | 1096 jours | 19420 livraisons
```

Applications

Application 1.1 — Résumer le panier, honnêtement (section 1.1.1)

Objectif. Produire en quelques lignes le résumé du panier que l'on remettrait à la gérante : des chiffres qui ne mentent pas.

Étape 1 — Les résumés qui comptent. Moyenne, médiane, moyenne tronquée à 5 %, quartiles, centiles 95 et 99.

```
x = cmd["panier"]
res = {"moyenne": x.mean(), "médiane": x.median(), "moyenne tronquée 5 %": stats.trim_mean(x, 0.05),
      "Q1": x.quantile(0.25), "Q3": x.quantile(0.75), "centile 95": x.quantile(0.95), "centile 99":
↳ x.quantile(0.99)}
print({k: round(float(v), 1) for k, v in res.items()})
```

```
{'moyenne': 100.4, 'médiane': 79.8, 'moyenne tronquée 5 %': 92.4, 'Q1': 42.9, 'Q3': 135.4, 'centile
↳ 95': 254.5, 'centile 99': 383.7}
```

Étape 2 — Combien de commandes sont « au-dessus de la moyenne » ? Et quelle part du chiffre d'affaires font les 10 % de paniers les plus élevés ?

```
print("part des commandes au-dessus de la moyenne :", round((x > x.mean()).mean() * 100, 1), "%")
tri = x.sort_values(ascending=False)
print("part du chiffre d'affaires des 10 % de paniers les plus élevés :", round(tri.head(int(0.1 *
↳ len(tri))).sum() / x.sum() * 100, 1), "%")
```

```
part des commandes au-dessus de la moyenne : 39.2 %
part du chiffre d'affaires des 10 % de paniers les plus élevés : 27.9 %
```

Lecture. Quatre commandes sur dix seulement dépassent la moyenne, et le dixième supérieur des paniers pèse plus du quart du chiffre d'affaires : un portrait par la seule moyenne serait trompeur.

À vous. Refaites l'étape 1 pour **chaque canal** et dites si la phrase « le panier typique est de 80 € » s'applique aux trois.

Application 1.2 — Choisir ses classes et lire des délais (sections 1.1.2 et 1.1.3)

Objectif. Comparer des règles de choix du nombre de classes, puis résumer une variable discrète (les délais de livraison) par ses centiles.

Étape 1 — Combien de classes selon la règle ? Pour des tailles d'échantillon très différentes.

```
rng = np.random.default_rng(0)
for n in (100, 1000, len(cmd)):
    e = x.sample(n, random_state=1) if n < len(cmd) else x
    print(f"n = {n:6d} | Sturges : {len(np.histogram_bin_edges(e, bins='sturges')) - 1:3d} classes |
    ↳ Freedman-Diaconis : {len(np.histogram_bin_edges(e, bins='fd')) - 1:3d} classes")
```

```
n =    100 | Sturges :    8 classes | Freedman-Diaconis :   14 classes
n =   1000 | Sturges :   11 classes | Freedman-Diaconis :   28 classes
n = 36395 | Sturges :   17 classes | Freedman-Diaconis :  177 classes
```

Étape 2 — Les délais par transporteur. Médiane, centile 90, part de livraisons à plus de 8 jours.

```
t = liv.groupby("transporteur")["delai"].agg(médiane="median", centile90=lambda s: s.quantile(0.9),
↳ plus_de_8_jours=lambda s: (s > 8).mean() * 100)
print(t.round(1).to_string())
```

	médiane	centile90	plus_de_8_jours
transporteur			
Transporteur A	5.0	7.0	1.7
Transporteur B	6.0	8.0	3.5
Transporteur C	7.0	8.0	9.1

Lecture. Les deux règles diffèrent dès 100 observations (8 contre 14 classes) et l'écart s'élargit avec l'échantillon (17 contre 177 pour les 36 395 commandes), parce que Freedman-Diaconis tient compte de la dispersion et de la taille. Pour les délais, les centiles révèlent ce que la moyenne cache : le transporteur C a une médiane de deux jours de plus que le transporteur A et dépasse **environ cinq fois plus** souvent 8 jours que le transporteur A.

À vous. Quelle phrase écririez-vous pour promettre un délai aux clients, pour chaque transporteur ?

Application 1.3 — Nuages, saison et corrélation « à saison égale » (section 1.2.1)

Objectif. Mesurer ce qui reste d'une corrélation une fois la saison retirée.

Étape 1 — La corrélation brute.

```
print({c: round(float(j[c].corr(j["nb_commandes"])), 2) for c in ["depense_pub", "temperature_moy",
↳ "pluie_mm", "promo_active"]})
```

```
{'depense_pub': 0.53, 'temperature_moy': -0.21, 'pluie_mm': -0.03, 'promo_active': 0.07}
```

Étape 2 — La corrélation à mois égal. On retire de chaque variable la moyenne de son mois.

```
jm = j.assign(mois=j["date"].dt.month)
def a_mois_egal(a, b):
    ra = jm[a] - jm.groupby("mois")[a].transform("mean")
    rb = jm[b] - jm.groupby("mois")[b].transform("mean")
    return round(float(ra.corr(rb)), 2)
print({c: a_mois_egal(c, "nb_commandes") for c in ["depense_pub", "temperature_moy", "pluie_mm",
↳ "promo_active"]})
```

```
{'depense_pub': 0.1, 'temperature_moy': -0.04, 'pluie_mm': -0.02, 'promo_active': 0.18}
```

Lecture. La corrélation de la publicité passe de 0,53 à 0,10, celle de la température de -0,21 à -0,04 : la saison portait l'essentiel du lien. La pluie, qui n'avait pas de lien brut, n'en a pas davantage. La promotion, dont le lien brut est faible (0,07), gagne en force **à mois égal** : c'est le signe d'une variable de confusion qui **cachait** l'effet.

À vous. Refaites l'étape 2 en retirant la moyenne par **mois et jour de la semaine**. Quelle corrélation de la publicité obtenez-vous ?

Application 1.4 — Comparer des groupes et croiser des variables (sections 1.2.2 et 1.2.3)

Objectif. Décider, avec des intervalles de confiance, quels groupes diffèrent.

Étape 1 — Intervalles de la moyenne par groupe.

```
def moyenne_ic(df, groupe, valeur):
    g = df.groupby(groupe)[valeur].agg(["mean", "std", "count"])
    g["bas"] = g["mean"] - 1.96 * g["std"] / np.sqrt(g["count"])
    g["haut"] = g["mean"] + 1.96 * g["std"] / np.sqrt(g["count"])
    return g[["mean", "bas", "haut"]].round(2)
print(moyenne_ic(cmd, "canal", "panier").to_string())
print(moyenne_ic(liv, "transporteur", "delai").to_string())
```

	mean	bas	haut
canal			
Boutique	100.90	99.68	102.11
Réseaux	100.52	97.98	103.05
Site	99.77	98.50	101.03
	mean	bas	haut
transporteur			
Transporteur A	5.21	5.18	5.24
Transporteur B	5.82	5.79	5.85
Transporteur C	6.68	6.64	6.72

Étape 2 — Un tableau croisé et sa force. Le taux de retour par canal et le V de Cramér.

```
lc = lig.merge(cmd[["id_commande", "canal"]], on="id_commande")
lc["retourne"] = lc["id_ligne"].isin(ret["id_ligne"])
tab = pd.crosstab(lc["canal"], lc["retourne"])
```

```
chi2 = stats.chi2_contingency(tab)[0]
print((tab[True] / tab.sum(axis=1) * 100).round(1).to_dict(), "| V de Cramér :", round((chi2 /
↪ tab.values.sum()) ** 0.5, 3))
```

```
{'Boutique': 3.1, 'Réseaux': 6.7, 'Site': 9.0} | V de Cramér : 0.118
```

Lecture. Les intervalles du panier se recouvrent presque entièrement entre canaux ; ceux du délai ne se recouvrent pas du tout entre transporteurs. Le canal est lié au retour avec un V de 0,118 : modeste en valeur absolue, mais net sur plus de 80 000 lignes.

À vous. Croisez le **mode de livraison** et le **retard** (colonnes de liv) et calculez le V de Cramér. Le point relais retarde-t-il les livraisons ?

Application 1.5 — Profils hebdomadaires et indice de prix (sections 1.3.1 et 1.3.2)

Objectif. Mesurer un rythme, puis isoler un changement de niveau à composition constante.

Étape 1 — L'indice par jour de la semaine, par année. La forme est-elle stable ?

```
jj = j.assign(an=j["date"].dt.year, js=j["date"].dt.dayofweek)
idx = jj.groupby(["an", "js"])["nb_commandes"].mean().unstack(0)
print((idx / idx.mean()).round(2).T.to_string())
```

js	0	1	2	3	4	5	6
an							
2023	0.93	0.89	0.95	1.02	1.17	1.35	0.69
2024	0.97	0.90	0.91	0.97	1.17	1.42	0.67
2025	0.96	0.92	0.95	0.98	1.13	1.41	0.66

Étape 2 — L'indice de prix par catégorie. Le prix payé rapporté au prix catalogue, par année et catégorie.

```
l2 = lig.merge(cmd[["id_commande", "date_commande"]], on="id_commande").merge(prod[["id_produit",
↪ "prix_vente", "categorie"]], on="id_produit")
l2["indice"] = l2["prix_unitaire"] / l2["prix_vente"]
print(l2.groupby([l2["date_commande"].dt.year,
↪ "categorie"])["indice"].mean().unstack(0).round(3).to_string())
```

date_commande	2023	2024	2025
categorie			
Bien-être	1.0	1.0	1.03
Cuisine	1.0	1.0	1.03
Décoration	1.0	1.0	1.03
Jardin	1.0	1.0	1.03
Maison	1.0	1.0	1.03
Papeterie	1.0	1.0	1.03

Lecture. Le profil de la semaine est stable d'une année à l'autre (samedi fort, dimanche faible). La hausse de prix de 3 % se retrouve dans **toutes** les catégories : elle est générale, pas propre à un produit.

À vous. Calculez le panier moyen 2025 contre 2024 à **catégorie et canal constants** et comparez à la hausse brute de 3,5 %.

Application 1.6 — Détecter des incidents pas à pas (section 1.3.4)

Objectif. Refaire la détection du livre en détaillant chaque étape.

Étape 1 — Référence locale et score.

```
jz, doublons = 0.incidents_jours(ji)
print(jz[["date", "nb_commandes", "chiffre_affaires", "z_commandes",
↪ "z_panier"]].round(2).loc[jz["date"].isin(vi["date"])].to_string(index=False))
```

date	nb_commandes	chiffre_affaires	z_commandes	z_panier
2025-03-12	11	1293.02	-4.35	1.29
2025-03-13	8	742.37	-5.64	-0.14
2025-03-14	16	1512.58	-2.72	-0.35
2025-04-28	14	1298.36	-4.12	-0.65
2025-04-29	15	1359.77	-2.93	-1.30
2025-06-18	31	6752.36	-0.29	4.75
2025-09-09	29	30982.80	-0.15	15.26
2025-10-20	38	3684.52	0.58	-0.09

Étape 2 — Choisir un seuil. Précision et rappel pour plusieurs seuils.

```
vrais = set(vi["date"])
for seuil in (3, 4, 5):
    s = 0.signaler(jz, doublons, seuil)
    p, r = 0.precision_rappel(s, vrais)
    print(f"seuil {seuil} : {len(s):2d} jours signalés, précision {p * 100:.0f} %, rappel {r *
↪ 100:.0f} %")
```

```
seuil 3 : 25 jours signalés, précision 24 %, rappel 75 %
seuil 4 : 11 jours signalés, précision 55 %, rappel 75 %
seuil 5 : 4 jours signalés, précision 75 %, rappel 38 %
```

Lecture. Les scores de commandes sont négatifs pour les pannes et fermetures mais modestes (de -2,7 à -5,6), tandis que le score de panier du jour est très élevé pour la commande B2B et l'erreur de saisie. Plus le seuil monte, plus la précision monte et plus le rappel baisse.

À vous. Ajoutez un troisième signal : le score du **chiffre d'affaires** (*z_ca*). Améliore-t-il le rappel au seuil 4 ?

Application 1.7 — Le rapport d'exploration automatique (section 1.4)

Objectif. Appliquer le rapport à trois fichiers et décider des suites.

Étape 1 — Trois fichiers, trois jeux d'alertes.

```
for nom, t, cle in [("produits", prod, "id_produit"), ("livraisons", liv[["id_commande",
↪ "transporteur", "delai", "colis_abime", "delai_promis_j"]], "id_commande"), ("retours", ret,
↪ "id_retour")]:
    r = 0.rapport_eda(t, cle=cle)
    print(f"== {nom} ({r['lignes']} lignes)")
    print(*(r["alertes"] or ["aucune alerte"]), sep="\n")
```

```
== produits (120 lignes)
id_produit : une valeur différente par ligne (identifiant ?)
date_lancement : des dates stockées en texte (convertir)
prix_vente et cout_achat : corrélation forte (0.97)
== livraisons (19420 lignes)
id_commande : une valeur différente par ligne (identifiant ?)
delai_promis_j : une seule valeur (colonne inutile)
== retours (5002 lignes)
id_retour : une valeur différente par ligne (identifiant ?)
id_ligne : une valeur différente par ligne (identifiant ?)
```

```
date_retour : des dates stockées en texte (convertir)
montant_rembourse : très asymétrique (2.54) : regarder la médiane et l'échelle logarithmique
id_retour et id_ligne : corrélation forte (1.0)
```

Étape 2 — Les noms de produits. Le rapport ne dit pas tout : comptons les noms distincts.

```
print("produits :", len(prod), "| noms distincts :", prod["nom_produit"].nunique(), "| identifiants  
↪ distincts :", prod["id_produit"].nunique())
```

```
produits : 120 | noms distincts : 60 | identifiants distincts : 120
```

Lecture. Le rapport signale ce qui se mesure sur une colonne ; il n'a pas vu que **120 produits se partagent 60 noms** : une jointure sur le nom doublerait les lignes (volume II, chapitre 2). L'alerte manquante est un rappel : une exploration automatique ne remplace pas le regard.

À vous. Ajoutez à la fonction une alerte « le nom n'est pas unique » pour les colonnes de texte censées identifier une ligne, et relancez-la sur produits.

Exercices

Exercice 1.1 ★ — Moyenne et médiane à la main (section 1.1.1)

Sept paniers (en €) : 20, 25, 30, 35, 40, 45, 1 500. Calculez la moyenne et la médiane à la main, puis vérifiez avec Python. Laquelle annonceriez-vous ? Que deviennent-elles si l'on remplace 1 500 par 150 ?

Exercice 1.2 ★ — Moyenne géométrique (section 1.1.1)

La moyenne **géométrique** d'une variable positive est l'exponentielle de la moyenne de ses logarithmes. Calculez-la pour le panier et comparez avec la moyenne et la médiane. Laquelle des trois est la plus proche du « montant typique » ?

Exercice 1.3 ★★ — Les classes selon Sturges (section 1.1.2)

La règle de Sturges propose $1 + \log_2 n$ classes. Calculez à la main le nombre de classes pour $n = 100$, $n = 1\,000$ et $n = 36\,395$, et comparez à `np.histogram_bin_edges`. Pourquoi cette règle sous-estime-t-elle le nombre de classes utile quand n est très grand ?

Exercice 1.4 ★★ — Une hausse « à l'œil » (section 1.1.6)

Pour les paniers moyens 2024 (98,9 €) et 2025 (102,3 €), calculez le rapport des **hauteurs de barre** pour un axe qui commence à zéro, puis à 97 €, puis à 90 €. À partir de quelle origine l'écart visuel dépasse-t-il le double ? Que concluez-vous ?

Exercice 1.5 ★ — Pearson et Spearman (section 1.2.1)

Pour les couples (1, 1), (2, 4), (3, 9), (4, 16), (5, 25), calculez les coefficients de Pearson et de Spearman. Pourquoi diffèrent-ils ? Ajoutez le point (6, 300) : que deviennent-ils ?

Exercice 1.6 ★★ — La pluie, à mois égal (section 1.2.1)

Calculez la corrélation entre la pluie (`pluie_mm`) et le nombre de commandes du canal Boutique par jour (à reconstruire à partir de `cmd`), brute, puis à mois égal. La pluie a-t-elle un effet détectable sur la boutique ? (Indice : la vérité programmée parle de -8% les jours de pluie pour la boutique.)

Exercice 1.7 ★★ — Le V de Cramér à la main (section 1.2.3)

Dans un tableau 2×2 dont les lignes sont les canaux Boutique et Site, les colonnes « retour » et « pas de retour », les effectifs sont 120 et 3 880 pour la Boutique, 360 et 3 640 pour le Site. Calculez à la main le khi-deux, le V de Cramér, puis vérifiez avec `scipy`.

Exercice 1.8 ★★★ — Fabriquer un paradoxe de Simpson (section 1.2.5)

Construisez une table de deux traitements A et B appliqués à deux groupes de patients (légers et graves) où **B réussit mieux que A dans chaque groupe**, mais **A réussit mieux que B globalement**. Donnez des effectifs et des taux ; vérifiez par le calcul. Quelle variable de confusion avez-vous créée ?

Exercice 1.9 ★ — Un indice hebdomadaire à la main (section 1.3.1)

Voici les commandes de deux semaines consécutives : lundi 30, 28 ; mardi 29, 27 ; mercredi 31, 29 ; jeudi 32, 33 ; vendredi 38, 40 ; samedi 46, 44 ; dimanche 22, 24. Calculez l'indice de chaque jour de la semaine (moyenne du jour sur la moyenne générale) et dites ce que signifie l'indice du samedi.

Exercice 1.10 ★★ — Une référence locale à la main (section 1.3.4)

Les commandes de six samedis consécutifs sont 44, 47, 45, 12, 46, 48. Avec une référence locale de deux samedis avant et deux après (le samedi lui-même exclu), calculez la référence du quatrième samedi, l'écart relatif en logarithme, et dites s'il est « anormal » si l'écart-type robuste typique des écarts vaut 0,15.

Exercice 1.11 ★★ — Choisir un seuil par le coût (section 1.3.4)

Une fausse alerte coûte 1 unité (dix minutes d'enquête), un incident raté 20 unités. En utilisant les résultats des seuils 3, 3,5, 4, 5 et 6 (à recalculer), quel seuil minimise le coût total pour les données de 2025 ? Que changerait un incident raté à 5 unités ?

Exercice 1.12 ★★★ — Un calendrier d'événements (section 1.3.5)

Retirez des jours signalés ceux qui tombent entre le 1^{er} et le 7 janvier, que l'on déclare « événement connu » (soldes et jours de l'an). Recalculez la précision et le rappel au seuil 4 contre `verite_incidents`. Que gagne-t-on, et que risque-t-on avec cette règle ?

Exercice 1.13 ★ — La liste de contrôle sur les produits (section 1.4)

Appliquez `rapport_eda` à `produits.csv` avec `cle="id_produit"`, puis complétez à la main les contrôles de la liste (les six temps) en deux lignes par temps.

Exercice 1.14 ★★ — Écrire le compte rendu (section 1.4)

À partir de `livraisons.csv`, écrivez le compte rendu d'une page de l'exploration (trame de la section 1.4.4) : une phrase par temps, avec au moins quatre chiffres calculés par du code.

Corrigés

Corrigé 1.1

```
v = np.array([20, 25, 30, 35, 40, 45, 1500])
print("moyenne :", round(v.mean(), 1), "| médiane :", np.median(v))
w = v.copy(); w[-1] = 150
print("avec 150 au lieu de 1 500 : moyenne", round(w.mean(), 1), "| médiane", np.median(w))
```

```
moyenne : 242.1 | médiane : 35.0
avec 150 au lieu de 1 500 : moyenne 49.3 | médiane 35.0
```

La somme vaut 1 695, soit une moyenne de 242,1 € ; la médiane (quatrième valeur) est 35 €. La valeur extrême **change tout** pour la moyenne et **rien** pour la médiane. On annonce la médiane (35 €), en signalant la commande de 1 500 € à part. En remplaçant par 150, la moyenne tombe à 49,3 €.

Corrigé 1.2

```
g = np.exp(np.log(x).mean())
print("moyenne :", round(x.mean(), 1), "| médiane :", round(x.median(), 1), "| moyenne géométrique",
      ↪ " :", round(g, 1))
```

```
moyenne : 100.4 | médiane : 79.8 | moyenne géométrique : 71.9
```

La moyenne géométrique (71,9 €) est le **centre multiplicatif** : elle est moins sensible aux gros paniers que la moyenne arithmétique (100,4 €), qui s'interprète comme « total divisé par le nombre de commandes ». Elle tombe même **sous la médiane** (79,8 €), parce que le logarithme du panier est lui-même asymétrique, vers la gauche (−0,71). Pour **prévoir un chiffre d'affaires total**, c'est la moyenne arithmétique qui compte, car elle conserve la somme ; pour décrire un panier typique, la médiane est la plus simple à expliquer.

Corrigé 1.3

```
for n in (100, 1000, 36395):
    print(n, "classes de Sturges à la main :", round(1 + np.log2(n), 1), "| numpy :",
          ↪ len(np.histogram_bin_edges(np.arange(n), bins="sturges")) - 1)
```

```
100 classes de Sturges à la main : 7.6 | numpy : 8
1000 classes de Sturges à la main : 11.0 | numpy : 11
36395 classes de Sturges à la main : 16.2 | numpy : 17
```

$1 + \log_2 100 \approx 7,6$, $1 + \log_2 1\,000 \approx 11,0$ et $1 + \log_2 36\,395 \approx 16,2$: numpy arrondit à l'entier supérieur (8, 11, 17). La règle ne dépend que de n et croît très lentement : avec 36 395 observations, elle propose 17 classes alors que les données en supporteraient bien davantage ; elle suppose en outre une distribution **à peu près symétrique**.

Corrigé 1.4

```
for origine in (0, 97, 90):
    h24, h25 = 98.9 - origine, 102.3 - origine
    print(f"origine {origine:3d} € : hauteur 2024 = {h24:5.1f}, hauteur 2025 = {h25:5.1f}, rapport =
    ↳ {h25 / h24:.2f}")
```

```
origine  0 € : hauteur 2024 =  98.9, hauteur 2025 = 102.3, rapport = 1.03
origine 97 € : hauteur 2024 =   1.9, hauteur 2025 =   5.3, rapport = 2.79
origine 90 € : hauteur 2024 =   8.9, hauteur 2025 =  12.3, rapport = 1.38
```

À zéro, le rapport est de 1,03 : l'œil voit la hausse réelle. À 97 €, il vaut 2,8 ; à 90 €, 1,4. Le rapport dépasse 2 dès que l'origine dépasse environ 95 €. Une barre dont l'origine n'est pas zéro **ment sur les proportions** : on la réserve aux courbes, ou on le dit explicitement.

Corrigé 1.5

```
a = np.array([1, 2, 3, 4, 5]); b = a ** 2
print("sans extrême : Pearson", round(stats.pearsonr(a, b)[0], 3), "| Spearman",
    ↳ round(stats.spearmanr(a, b)[0], 3))
a2 = np.r_[a, 6]; b2 = np.r_[b, 300]
print("avec (6 ; 300) : Pearson", round(stats.pearsonr(a2, b2)[0], 3), "| Spearman",
    ↳ round(stats.spearmanr(a2, b2)[0], 3))
```

```
sans extrême : Pearson 0.981 | Spearman 1.0
avec (6 ; 300) : Pearson 0.707 | Spearman 1.0
```

La relation $y = x^2$ est **monotone mais pas droite** : Spearman vaut exactement 1, Pearson un peu moins (0,98). Avec le point extrême (6 ; 300), Pearson tombe à 0,71 : un seul point domine la droite ; Spearman, qui ne regarde que les rangs, reste à 1. Spearman est plus robuste aux extrêmes, mais il ne dit pas si la relation est droite.

Corrigé 1.6

```
cb = cmd[cmd["canal"] == "Boutique"].groupby("date_commande").size().rename("boutique")
jb = j.set_index("date").join(cb).fillna({"boutique": 0}).reset_index()
jb["mois"] = jb["date"].dt.month
brut = jb["pluie_mm"].corr(jb["boutique"])
ra = jb["pluie_mm"] - jb.groupby("mois")["pluie_mm"].transform("mean"); rb = jb["boutique"] -
    ↳ jb.groupby("mois")["boutique"].transform("mean")
print("corrélation brute :", round(brut, 3), "| à mois égal :", round(ra.corr(rb), 3))
jb["pluvieux"] = jb["pluie_mm"] > 1
print("commandes Boutique par jour : sans pluie", round(jb.loc[~jb["pluvieux"], "boutique"].mean(),
    ↳ 1), "| avec pluie", round(jb.loc[jb["pluvieux"], "boutique"].mean(), 1))
```

```
corrélation brute : -0.062 | à mois égal : -0.057
commandes Boutique par jour : sans pluie 15.9 | avec pluie 14.4
```

La corrélation est faible dans les deux cas (−0,06), parce que **le hasard quotidien (Poisson) et la saison noient** un effet de −8 %. La comparaison des moyennes (15,9 commandes sans pluie contre 14,4 avec pluie, soit −9 %) est, elle, compatible avec la vérité programmée, mais elle ne tient pas compte de la saison : un jour de pluie n'est pas réparti au hasard dans l'année. Un effet réel de cette taille ne se voit pas à l'œil sur un nuage ; il faut un modèle qui contrôle la saison et le jour de la semaine (chapitre 3) ou un test sur un grand nombre de jours (chapitre 2).

Corrigé 1.7

```
t = np.array([[120, 3880], [360, 3640]])
att = np.outer(t.sum(1), t.sum(0)) / t.sum()
chi2 = ((t - att) ** 2 / att).sum()
print("effectifs attendus :", att.round(0).tolist(), "| khi-deux :", round(chi2, 1), "| V :",
      ↪ round((chi2 / t.sum()) ** 0.5, 3))
print("scipy (sans correction) :", round(stats.chi2_contingency(t, correction=False)[0], 1))
```

```
effectifs attendus : [[240.0, 3760.0], [240.0, 3760.0]] | khi-deux : 127.7 | V : 0.126
scipy (sans correction) : 127.7
```

Les effectifs attendus sont 240 retours et 3 760 non-retours dans chaque ligne ; le khi-deux est la somme des $(\text{observé} - \text{attendu})^2 / \text{attendu}$; le V de Cramér vaut $\sqrt{\chi^2 / (n(k-1))}$ avec $k = 2$. Le résultat (V proche de 0,13) est du même ordre que celui des données réelles (0,118) : la liaison est modeste mais indiscutable sur 8 000 lignes.

Corrigé 1.8

```
# groupe : (succès A, total A, succès B, total B)
t = {"légers": (234, 270, 81, 87), "graves": (55, 80, 192, 263)}
for g, (sa, na, sb, nb) in t.items():
    print(f"{g:7s} A : {sa / na * 100:.0f} % ({sa}/{na}) | B : {sb / nb * 100:.0f} % ({sb}/{nb})")
SA, NA = sum(v[0] for v in t.values()), sum(v[1] for v in t.values())
SB, NB = sum(v[2] for v in t.values()), sum(v[3] for v in t.values())
print(f"global A : {SA / NA * 100:.0f} % ({SA}/{NA}) | B : {SB / NB * 100:.0f} % ({SB}/{NB})")
```

```
légers A : 87 % (234/270) | B : 93 % (81/87)
graves A : 69 % (55/80) | B : 73 % (192/263)
global A : 83 % (289/350) | B : 78 % (273/350)
```

Dans **chaque** groupe, B réussit mieux que A (93 % contre 87 % chez les cas légers, 73 % contre 69 % chez les cas graves) ; **globalement**, A réussit mieux (83 % contre 78 %). L'explication tient dans la **composition** : A est appliqué surtout à des cas légers (270 sur 350), B surtout à des cas graves (263 sur 350). La variable de confusion est la **gravité**, qui détermine à la fois le traitement reçu et la réussite. D'autres effectifs conviennent, pourvu que les proportions de groupes soient très différentes d'un traitement à l'autre.

Corrigé 1.9

```
sem = pd.DataFrame({"jour": ["lun", "mar", "mer", "jeu", "ven", "sam", "dim"], "s1": [30, 29, 31, 32,
      ↪ 38, 46, 22], "s2": [28, 27, 29, 33, 40, 44, 24]})
sem["moyenne_jour"] = sem[["s1", "s2"]].mean(axis=1)
sem["indice"] = (sem["moyenne_jour"] / sem["moyenne_jour"].mean()).round(2)
print(sem[["jour", "moyenne_jour", "indice"]].to_string(index=False))
```

```
jour  moyenne_jour  indice
lun      29.0      0.90
mar      28.0      0.87
mer      30.0      0.93
jeu      32.5      1.00
ven      39.0      1.21
sam      45.0      1.39
dim      23.0      0.71
```

La moyenne générale vaut environ 32,4 commandes par jour ; l'indice du samedi (45 sur 32,4) signifie que le samedi compte environ **39 % de commandes de plus** que le jour moyen, et celui

du dimanche (23 sur 32,4) environ 29 % de moins.

Corrigé 1.10

```
s = np.array([44, 47, 45, 12, 46, 48], dtype=float)
ref = np.median(np.r_[s[1:3], s[4:6]])
print("référence du 4e samedi :", ref, "| écart relatif (log) :", round(np.log(12 / ref), 2), "| score
↳ z :", round(np.log(12 / ref) / 0.15, 1))
```

```
référence du 4e samedi : 46.5 | écart relatif (log) : -1.35 | score z : -9.0
```

La référence est la médiane de 47, 45, 46 et 48, soit 46,5. L'écart relatif est $\ln(12/46,5) \approx -1,35$, soit un score de $-9,0$ pour un écart-type robuste de 0,15 : ce samedi est **très** anormal (et cela se justifie : on a divisé les ventes par près de quatre).

Corrigé 1.11

```
vrais = set(vi["date"])
for seuil in (3, 3.5, 4, 5, 6):
    s = 0.signaler(jz, doublons, seuil)
    fausses, ratees = len(s - vrais), len(vrais - s)
    print(f"seuil {seuil} : {fausses:2d} fausses alertes, {ratees} incidents ratés | coût (1 ; 20) =
↳ {fausses + 20 * ratees:3d} | (1 ; 5) = {fausses + 5 * ratees:3d} | (1 ; 1) = {fausses +
↳ ratees:3d}")
```

```
seuil 3 : 19 fausses alertes, 2 incidents ratés | coût (1 ; 20) = 59 | (1 ; 5) = 29 | (1 ; 1) = 21
seuil 3.5 : 7 fausses alertes, 2 incidents ratés | coût (1 ; 20) = 47 | (1 ; 5) = 17 | (1 ; 1) = 9
seuil 4 : 5 fausses alertes, 2 incidents ratés | coût (1 ; 20) = 45 | (1 ; 5) = 15 | (1 ; 1) = 7
seuil 5 : 1 fausses alertes, 5 incidents ratés | coût (1 ; 20) = 101 | (1 ; 5) = 26 | (1 ; 1) = 6
seuil 6 : 0 fausses alertes, 6 incidents ratés | coût (1 ; 20) = 120 | (1 ; 5) = 30 | (1 ; 1) = 6
```

Avec un incident raté qui coûte 20 fois une fausse alerte, le coût total est minimal au **seuil 4** (45 unités) ; avec 5 fois, c'est encore le seuil 4 (15), mais l'écart avec les autres se réduit. Si les deux erreurs coûtaient autant (1 contre 1), les seuils 5 et 6 (coût 6) battraient le seuil 4 (coût 7) : plus l'incident raté est bon marché, plus le seuil optimal monte. Un seuil n'est pas une propriété du jeu de données : c'est une **décision** qui encode un coût.

Corrigé 1.12

```
connus = {d for d in jz["date"] if d.month == 1 and d.day <= 7}
s4 = 0.signaler(jz, doublons, 4)
s4b = s4 - connus
for nom, s in [("sans calendrier", s4), ("avec calendrier", s4b)]:
    p, r = 0.precision_rappel(s, vrais)
    print(f"{nom} : {len(s)} jours signalés, précision {p * 100:.0f} %, rappel {r * 100:.0f} %")
```

```
sans calendrier : 11 jours signalés, précision 55 %, rappel 75 %
avec calendrier : 7 jours signalés, précision 86 %, rappel 75 %
```

Écarter les premiers jours de janvier retire les faux positifs qui tombent à cette période : la **précision** monte, le **rappel** ne change pas (aucun incident injecté ne tombe là). Le risque est de **masquer un vrai incident** survenant un 3 janvier : le calendrier doit être **précis** (par exemple le 1^{er} janvier seulement) et revu chaque année.

Corrigé 1.13

```
r = 0.rapport_eda(prod, cle="id_produit")
print(r["types"].to_string()); print(*(r["alertes"] or ["aucune alerte"]), sep="\n")
print("noms distincts :", prod["nom_produit"].nunique(), "sur", len(prod), "produits")
```

	type	manquants_pct	distincts
id_produit	int64	0.0	120
nom_produit	str	0.0	60
categorie	str	0.0	6
prix_vente	float64	0.0	64
cout_achat	float64	0.0	117
fournisseur	str	0.0	8
date_lancement	str	0.0	118

id_produit : une valeur différente par ligne (identifiant ?)
date_lancement : des dates stockées en texte (convertir)
prix_vente et cout_achat : corrélation forte (0.97)
noms distincts : 60 sur 120 produits

Les six temps : **tableau** (120 lignes, une par produit, id_produit unique) ; **types** (date_lancement est en texte : à convertir) ; **manques** (aucun) ; **variables** (six catégories de 20 produits chacune, huit fournisseurs) ; **relations** (le prix de vente et le coût d'achat sont corrélés à 0,97 : l'un déduit presque l'autre) ; **temps et anomalies** (la date de lancement permet d'étudier l'âge du catalogue ; les noms en double, 60 pour 120 produits, sont une anomalie que le rapport ne voit pas).

Corrigé 1.14

```
d = liv["delai"]
print("1. grain :", len(liv), "livraisons, une par commande ; période", liv["date_commande"].min(),
      ↪ "à", liv["date_commande"].max())
print("2. manquants :", int(liv.isna().sum().sum()), "| types : dates en texte")
print("3. délai : médiane", d.median(), "j, centile 90 :", d.quantile(0.9), "j, plus de 8 jours :",
      ↪ round((d > 8).mean() * 100, 1), "%")
print("4. transporteur C : délai moyen", round(liv.loc[liv["transporteur"] == "Transporteur C",
      ↪ "delai"].mean(), 2), "j contre", round(liv.loc[liv["transporteur"] == "Transporteur A",
      ↪ "delai"].mean(), 2), "j pour A")
print("5. colis abîmés :", round(liv["colis_abime"].mean() * 100, 1), "% | retards sur délai promis
      ↪ :", round(liv["retard"].mean() * 100, 1), "%")
```

1. grain : 19420 livraisons, une par commande ; période 2023-01-01 à 2025-12-31
2. manquants : 0 | types : dates en texte
3. délai : médiane 6.0 j, centile 90 : 8.0 j, plus de 8 jours : 3.8 %
4. transporteur C : délai moyen 6.68 j contre 5.21 j pour A
5. colis abîmés : 1.7 % | retards sur délai promis : 26.6 %

Un compte rendu d'une page reprend ces cinq lignes en phrases : *grain et période* ; *qualité* (rien ne manque, dates à convertir) ; *niveau de service* (médiane de 6 jours, 9 commandes sur 10 sous 8 jours) ; *relation notable* (le transporteur C est plus lent, à confirmer par un test et par une régression) ; *anomalies* (les retards se concentrent en décembre, à explorer au chapitre 11) ; *question ouverte* (le retard coûte-t-il des retours ?).

Chapitre 2 : Tests d'hypothèses, tests A/B et corrélation — exercices et applications

Ce chapitre du cahier accompagne le chapitre 2 du livre. Il contient **huit applications guidées** (de petites études que vous refaites sur les données de la boutique) et **quatorze exercices** de difficulté croissante (★ calcul à la main, ★★ calcul et interprétation, ★★★ étude complète), tous corrigés à la fin. Chaque exercice renvoie à la section du livre dont il prolonge le propos.

Une seule cellule charge les bibliothèques et les données. Elle est reprise au début de chaque application : si vous travaillez dans un notebook, exécutez-la une fois.

```
import sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
from scipy import stats
import outils_ch02 as O

d = O.charger()
email, site, jours, cmd, lig, ret, prod = d.email, d.site, d.jours, d.commandes, d.lignes, d.retours,
↳ d.produits
print(len(email), "contacts ;", len(site), "sessions ;", len(jours), "jours ;", len(cmd), "commandes")

12000 contacts ; 38622 sessions ; 1096 jours ; 36395 commandes
```

Applications

Application 2.1 — Fabriquer une p-valeur en mélangeant (section 2.1)

Objectif. Retrouver la p-valeur d'un test **sans formule**, en mélangeant les étiquettes, sur le clic de l'e-mail (et non sur l'achat).

Étape 1 — l'écart observé.

```
clic = email["clic"].values
est_b = (email["groupe"] == "B").values
ecart = clic[est_b].mean() - clic[~est_b].mean()
print("taux de clic A :", round(clic[~est_b].mean(), 4), "| B :", round(clic[est_b].mean(), 4), "|
↳ écart :", round(ecart, 4))
```

```
taux de clic A : 0.0388 | B : 0.0488 | écart : 0.01
```

Étape 2 — le mélange. On mélange les 12 000 résultats et l'on recalcule l'écart, 5 000 fois.

```
rng = np.random.default_rng(11)
ecarts = np.empty(5000)
for k in range(5000):
    m = rng.permutation(clic)
    ecarts[k] = m[:6000].mean() - m[6000:].mean()
print("part des mélanges dont l'écart absolu dépasse l'écart observé :", np.mean(np.abs(ecarts) >=
    ↪ abs(ecart)))
```

```
part des mélanges dont l'écart absolu dépasse l'écart observé : 0.0094
```

Étape 3 — comparer à la formule.

```
x = email.groupby("groupe")["clic"].sum()
print({k: round(float(v), 4) for k, v in 0.deux_proportions(x["A"], 6000, x["B"], 6000).items() if k
    ↪ in ("ecart", "z", "p")})
```

```
{'ecart': 0.01, 'z': 2.6754, 'p': 0.0075}
```

À vous. Les deux p-valeurs sont-elles voisines ? Pourquoi la p-valeur du clic est-elle bien plus petite que celle de l'achat (0,143) ? (*Piste en fin d'application : regardez l'écart en nombre de clics et l'effectif.*)

Piste. Il y a 60 clics de plus en B ; l'écart relatif est grand (+26 %) alors que, pour l'achat, 28 acheteurs de plus représentent +16 % : à effectif égal, un écart relatif plus grand, sur un événement plus fréquent, est plus difficile à attribuer au hasard.

Application 2.2 — Lire un test d'e-mail de bout en bout (section 2.2)

Objectif. Écrire une petite fonction qui rédige la ligne de résultat d'un test (taux, écart, intervalle, p-valeur, effet minimal détectable), et l'appliquer à trois populations.

```
from scipy.optimize import brentq

def lecture(df, col):
    t = df.groupby("groupe")[col].agg(["sum", "size"])
    r = 0.deux_proportions(t.loc["A", "sum"], t.loc["A", "size"], t.loc["B", "sum"], t.loc["B",
    ↪ "size"])
    emd = brentq(lambda p2: 0.taille_deux_proportions(r["pa"], p2) - t["size"].mean(), r["pa"] + 1e-6,
    ↪ 0.9) - r["pa"]
    return [f"{r['pa']:.2%}", f"{r['pb']:.2%}", f"{r['ecart']*100:+.2f}", f"{r['ic_bas']*100:+.2f} ;
    ↪ {r['ic_haut']*100:+.2f}", round(r["p"], 3), f"{emd*100:.2f}"]

lignes = [[nom] + lecture(sous, "achat_7j") for nom, sous in [("tous les contacts", email), ("clients
    ↪ seulement", email[email["est_client"] == 1]), ("autres contacts", email[email["est_client"] ==
    ↪ 0])]]
print(pd.DataFrame(lignes, columns=["population", "A", "B", "écart (pts)", "IC 95 %", "p", "EMD
    ↪ (pts)"]).to_string(index=False))
```

population	A	B	écart (pts)	IC 95 %	p	EMD (pts)
tous les contacts	2.92%	3.38%	+0.47	[-0.16 ; +1.09]	0.143	0.92
clients seulement	3.00%	3.63%	+0.63	[-0.27 ; +1.54]	0.172	1.36
autres contacts	2.83%	3.13%	+0.30	[-0.56 ; +1.16]	0.492	1.33

À vous. Dans quelle population l'écart est-il le plus grand ? Est-il plus **démonstré** ? Comparez l'EMD (l'effet minimal détectable à 80 % de puissance) à l'écart observé, et dites pourquoi on ne

peut pas conclure à une différence **entre** les populations à partir de ce tableau.

Application 2.3 — Défaut de répartition, appareils et sous-groupes (section 2.2)

Objectif. Appliquer la séquence de lecture d'un test A/B à la page de paiement.

```
n = site["groupe"].value_counts().sort_index()
print("répartition :", n.to_dict(), "| khi-deux 50/50, p =",
      ↪ f"{stats.chisquare(n.values).pvalue:.1e}")
lignes, pv = [], []
for dev in ["mobile", "ordinateur", "tablette"]:
    t = site[site["appareil"] == dev].groupby("groupe")["commande"].agg(["sum", "size"])
    r = 0.deux_proportions(t.loc["A", "sum"], t.loc["A", "size"], t.loc["B", "sum"], t.loc["B",
    ↪ "size"])
    lignes.append([dev, round(r["ecart"] * 100, 2), round(r["p"], 3)]); pv.append(r["p"])
res = pd.DataFrame(lignes, columns=["appareil", "écart (pts)", "p
    ↪ brute"]).assign(holm=np.round(0.holm(pv), 3))
print(res.to_string(index=False))
```

```
répartition : {'A': 20048, 'B': 18574} | khi-deux 50/50, p = 6.4e-14
  appareil  écart (pts)  p brute  holm
  mobile      0.57      0.025 0.075
ordinateur   -0.51      0.090 0.179
  tablette   -0.91      0.267 0.267
```

À vous. Refaites le découpage **par visiteur** (nouveau_visiteur : 0 ou 1) : quels écarts, quelles p-valeurs, et quelle correction appliquez-vous ? Concluez en une phrase pour la gérante.

Application 2.4 — Regarder en continu, soi-même (section 2.2)

Objectif. Mesurer le risque de faux positif selon le nombre de regards, et tester deux remèdes.

```
rng = np.random.default_rng(5)
P = np.array([0.p_aa(rng, n_jours=21, par_jour=900) for _ in range(3000)])
print("faux positifs, un seul regard (jour 21) :", round((P[:, -1] < 0.05).mean(), 3))
print("trois regards (jours 7, 14, 21) :", round((P[:, [6, 13, 20]] < 0.05).any(axis=1).mean(), 3))
print("chaque jour :", round((P < 0.05).any(axis=1).mean(), 3))
print("trois regards, seuil de Bonferroni (0,05 / 3) :", round((P[:, [6, 13, 20]] < 0.05 /
    ↪ 3).any(axis=1).mean(), 3))
```

```
faux positifs, un seul regard (jour 21) : 0.047
trois regards (jours 7, 14, 21) : 0.099
chaque jour : 0.252
trois regards, seuil de Bonferroni (0,05 / 3) : 0.036
```

À vous. Quel seuil faudrait-il pour que le contrôle **quotidien** (21 regards) garde un risque global proche de 5 % avec la correction de Bonferroni ? Quel est le prix en puissance ? (Calculez le seuil, puis l'effectif nécessaire à ce seuil avec `0.taille_deux_proportions`, qui accepte l'argument `alpha`.)

Application 2.5 — Corrélation à saison égale (section 2.3)

Objectif. Comparer la corrélation brute et la corrélation « à mois égal », et mesurer leur incertitude.

```
def a_mois_egal(v):
    return v - v.groupby(jours["mois"]).transform("mean")

x, y = jours["pluie_mm"], jours["nb_commandes"]
for nom, (u, w) in {"brute": (x, y), "à mois égal": (a_mois_egal(x), a_mois_egal(y))}.items():
    r, p = stats.pearsonr(u, w)
```

```
z, se = np.arctanh(r), 1 / np.sqrt(len(u) - 3)
print(f"pluie × commandes, {nom} : r = {r:.3f} (IC {np.tanh(z - 1.96 * se):.3f} à {np.tanh(z + 1.96 * se):.3f}), Spearman = {stats.spearmanr(u, w)[0]:.3f}")
```

```
pluie × commandes, brute : r = -0.034 (IC -0.093 à 0.025), Spearman = -0.046
pluie × commandes, à mois égal : r = -0.015 (IC -0.074 à 0.045), Spearman = -0.029
```

À vous. La pluie réduit-elle les commandes ? La vérité programmée dit : -8% pour la Boutique, $+5\%$ pour le Site. Que devient l'effet global quand les deux canaux se compensent ? Refaites le calcul **par canal** (nombre de commandes de la Boutique, puis du Site, par jour) à partir de cmd.

Application 2.6 — Séries à tendance : corrélation fallacieuse (section 2.3)

Objectif. Mesurer la fréquence des fausses corrélations sur des séries indépendantes de longueurs différentes.

```
def part_correlee(longueur, essais=2000):
    r = np.array([np.corrcoef(np.random.default_rng(g).normal(size=(2, longueur)).cumsum(axis=1)))[0,
↪ 1] for g in range(essais)])
    v = np.array([np.corrcoef(np.diff(np.random.default_rng(g).normal(size=(2,
↪ longueur)).cumsum(axis=1)))[0, 1] for g in range(essais)])
    return (np.abs(r) > 0.5).mean(), (np.abs(v) > 0.5).mean()

for longueur in (12, 36, 120):
    a, b = part_correlee(longueur)
    print(f"{longueur:4d} points : |r| > 0,5 sur les niveaux {a:.2f} | sur les variations {b:.3f}")
```

```
12 points : |r| > 0,5 sur les niveaux 0.40 | sur les variations 0.106
36 points : |r| > 0,5 sur les niveaux 0.41 | sur les variations 0.001
120 points : |r| > 0,5 sur les niveaux 0.38 | sur les variations 0.000
```

À vous. Comment la fréquence des fausses corrélations évolue-t-elle quand la série s'allonge ? Pourquoi une série plus longue **n'arrange pas** les choses pour des niveaux (alors qu'elle aide pour des observations indépendantes) ?

Application 2.7 — Choisir le test pour cinq questions (section 2.4)

Objectif. Répondre à cinq questions de la gérante avec le bon test, en justifiant chaque choix.

```
c25 = cmd[cmd["date_commande"] >= "2025-01-01"]
l25 = lig.merge(cmd[["id_commande", "canal", "date_commande"]], on="id_commande")
l25 = l25[l25["date_commande"] >= "2025-01-01"].assign(retour=lambda t:
↪ t["id_ligne"].isin(ret["id_ligne"]).astype(int))
q1 = stats.chi2_contingency(pd.crosstab(l25["canal"], l25["retour"]))[1]
↪ # retours selon le canal : khi-deux
q2 = stats.ttest_ind(c25.loc[c25["canal"] == "Site", "panier"], c25.loc[c25["canal"] == "Boutique",
↪ "panier"], equal_var=False).pvalue # panier Site/Boutique : Welch
q3 = stats.mannwhitneyu(jours.loc[jours["promo_active"] == 1, "nb_commandes"],
↪ jours.loc[jours["promo_active"] == 0, "nb_commandes"]).pvalue # promotion : Mann-Whitney
q4 = stats.ttest_rel(*[c25[c25["canal"] ==
↪ c].groupby("date_commande").size().reindex(sorted(c25["date_commande"].unique()), fill_value=0)
↪ for c in ("Site", "Boutique")]).pvalue # Site/Boutique par jour : apparié
q5 = stats.f_oneway(*[g["panier"] for _, g in c25.groupby("canal")]).pvalue
↪ # panier selon les trois canaux : ANOVA
print({"retours × canal": q1, "panier Site vs Boutique": round(q2, 3), "commandes promo": round(q3,
↪ 3), "Site vs Boutique par jour": q4, "panier × 3 canaux": round(q5, 3)})

{'retours × canal': np.float64(4.3134986108746563e-75), 'panier Site vs Boutique': np.float64(0.345),
↪ 'commandes promo': np.float64(0.029), 'Site vs Boutique par jour':
↪ np.float64(3.787053788491122e-09), 'panier × 3 canaux': np.float64(0.639)}
```

À vous. Pour chaque question, dites quelle information supplémentaire (taille de l'effet, intervalle) vous donneriez à la gérante en plus de la p-valeur.

Application 2.8 — Taille et durée d'un test (section 2.5)

Objectif. Planifier un test de la page de paiement : effectif, durée, effet minimal.

```
sessions_jour = len(site) / site["date"].nunique()
base = site.loc[site["groupe"] == "A", "commande"].mean()
lignes = []
for rel in (0.10, 0.15, 0.25):
    n = 0.taille_deux_proportions(base, base * (1 + rel))
    lignes.append([f"+{rel:.0%}", round(n), round(2 * n / sessions_jour, 1)])
print("conversion de référence :", round(base * 100, 2), "% | sessions par jour pendant le test :",
      ↪ round(sessions_jour))
print(pd.DataFrame(lignes, columns=["amélioration relative", "sessions par groupe", "jours
      ↪ nécessaires"]).to_string(index=False))
```

```
conversion de référence : 3.53 % | sessions par jour pendant le test : 1839
amélioration relative sessions par groupe jours nécessaires
+10%                  44938                48.9
+15%                  20427                22.2
+25%                  7679                 8.4
```

À vous. Le test réel a duré 21 jours. Quelle amélioration relative aurait-il pu détecter avec 80 % de puissance ? Et si l'on ne considère que les sessions **mobiles** (58 % du trafic) ?

Exercices

Exercice 2.1 ★ — Lire une p-valeur (section 2.1)

Un test donne $p = 0,03$ pour l'écart de taux de conversion entre deux pages. Dites pour chaque phrase si elle est correcte : (a) « il y a 3 % de chances que les pages soient équivalentes » ; (b) « si les pages étaient équivalentes, un écart au moins aussi grand serait observé dans 3 % des tirages » ; (c) « l'effet est important » ; (d) « le résultat prouve que la nouvelle page est meilleure » ; (e) « avec $p = 0,08$ on aurait conclu à l'absence d'effet ».

Exercice 2.2 ★ — Le test z à la main (section 2.1)

Groupe A : 180 acheteurs sur 6 000 ; groupe B : 204 sur 6 000. Calculez les deux taux, la proportion globale, l'erreur type sous H_0 , la statistique z et la p-valeur approximative (lue dans la loi normale : $P(|Z| > 1,25) \approx 0,21$).

Exercice 2.3 ★ — Intervalle de confiance de l'écart (section 2.1)

Pour les données de l'exercice 2.2, calculez l'intervalle de confiance à 95 % de l'écart $p_B - p_A$ (avec l'erreur type **sans** hypothèse nulle). Contient-il zéro ? Que conseillez-vous ?

Exercice 2.4 ★ — Deux erreurs (section 2.1)

Classez : (a) vous déployez une nouvelle page qui n'apporte rien ; (b) vous abandonnez une nouvelle page qui aurait apporté +0,5 point ; (c) vous annoncez un effet « significatif » trouvé en testant 20 sous-groupes. Pour chaque cas : erreur de type I ou II ? Que fait augmenter chaque décision : seuil, effectif, puissance ?

Exercice 2.5 ★★ — Défaut de répartition à la main (section 2.2)

Un test prévu à 50/50 donne 10 300 sessions en A et 9 700 en B. Calculez le khi-deux de la répartition (valeur attendue : 10 000 chacun), puis la p-valeur ($P(\chi_1^2 > 18) \approx 2 \times 10^{-5}$). Que faites-vous avant de lire les conversions ?

Exercice 2.6 ★★ — La correction de Holm à la main (section 2.2)

Quatre sous-groupes donnent les p-valeurs 0,010 ; 0,020 ; 0,030 et 0,200. Appliquez la correction de **Bonferroni** (multiplier par 4), puis celle de **Holm** (multiplier la plus petite par 4, la suivante par 3, etc., en gardant les p ajustées croissantes). Quels sous-groupes restent significatifs à 5 % dans chaque cas ?

Exercice 2.7 ★★ — Comparer deux moyennes (section 2.1)

Panier moyen du groupe A : 100 € (écart-type 80, 400 commandes) ; groupe B : 108 € (écart-type 85, 400 commandes). Calculez l'erreur type de la différence, la statistique t de Welch et la p-valeur approximative ($P(|t| > 1,37) \approx 0,17$). Que dire de l'écart de 8 € ?

Exercice 2.8 ★★ — Taille d'échantillon par la formule (section 2.5)

La conversion actuelle est de 4,0 %. Combien de sessions par groupe pour détecter une amélioration à 4,6 % (+15 % relatif) avec 80 % de puissance et un seuil de 5 % ? À 20 000 sessions par jour au total, combien de jours ?

Exercice 2.9 ★★ — Pearson à la main (section 2.3)

Six jours : dépense (100, 150, 200, 250, 300, 350 €) et commandes (22, 30, 29, 41, 38, 52). Calculez le coefficient de corrélation de Pearson à partir des écarts à la moyenne.

Exercice 2.10 ★★ — Spearman et valeur aberrante (section 2.3)

Avec les données de l'exercice 2.9, calculez le coefficient de Spearman par les rangs. Remplacez ensuite 52 par 150 : que deviennent Pearson et Spearman ?

Exercice 2.11 ★★★ — Le test de la promotion et la saison (section 2.3)

Calculez la corrélation entre `promo_active` et `nb_commandes` (brute, puis à mois égal), puis l'effet estimé de la promotion par régression de $\log(\text{nb_commandes})$ sur la promotion, le mois et le jour de la semaine. Comparez à l'effet programmé de +18 %. Que conclure sur la lecture d'une corrélation brute ?

Exercice 2.12 ★★ — Choisir et lancer un test (section 2.4)

Le taux de retour est-il différent entre le Site et les Réseaux en 2025 ? Précisez variable, groupes, test ; lancez-le ; rapportez l'écart, son intervalle et la p-valeur ; calculez le V de Cramér ou la taille d'effet.

Exercice 2.13 ★★★ — Simuler une courbe de puissance (section 2.5)

Pour un taux de référence de 3,0 % et une amélioration de 0,4 point, estimez par simulation la puissance pour 5 000, 10 000, 20 000 et 40 000 contacts par groupe ; comparez à la formule. À partir de quelle taille dépasse-t-on 80 % ?

Exercice 2.14 ★★★ — Rédiger le rapport d'un test (section 2.2)

Reprenez la page de paiement sur **mobile uniquement** : répartition, conversion A et B, écart, intervalle, p-valeur, effet minimal détectable. Rédigez le rapport en six lignes (objectif, conception, contrôles, résultats, interprétation, décision) et dites ce qu'il faudrait faire pour conclure.

Pistes des applications

- 2.1. Les deux p-valeurs sont voisines (0,0094 par mélange, 0,0075 par la formule) : le clic est « significatif » alors que l'achat ne l'est pas, parce que l'écart relatif est plus grand (+26 %) sur un événement plus fréquent (233 et 293 clics, contre 175 et 203 achats).
- 2.2. L'écart est le plus grand chez les clients (+0,63 point, contre +0,47 pour tous et +0,30 pour les autres contacts), mais aucun n'est démontré ($p = 0,17$ et $0,49$) et les intervalles se **recouvrent largement** : on ne peut pas conclure à une différence **entre** populations. Dans chaque sous-groupe l'effet minimal détectable (1,36 et 1,33 point) est supérieur à l'écart observé : les demi-échantillons sont trop petits.
- 2.3. Deux sous-groupes, donc deux tests : on applique Holm (ou Bonferroni) sur ces deux p-valeurs ; sans effet réel sur la nouveauté du visiteur, aucune des deux ne devrait passer sous 0,05 après correction. La phrase pour la gérante : « le test a un défaut de répartition ; rien n'est démontré globalement ; l'effet éventuel sur mobile est à reconfirmer ».
- 2.4. Avec 21 regards, le seuil de Bonferroni vaut $0,05/21 \approx 0,0024$; le prix en puissance se mesure en effectif : à puissance égale, il faut environ 1,9 fois plus de sessions (le facteur $(z_{0,0012} + z_{\beta})^2 / (z_{0,025} + z_{\beta})^2 \approx 15,0/7,85$).
- 2.5. La corrélation entre pluie et commandes totales est quasi nulle ($-0,03$ brute, $-0,02$ à mois égal) : les effets opposés par canal (-8 % Boutique, $+5$ % Site) se compensent presque dans le total ; il faut séparer les canaux pour les voir.
- 2.6. La part de paires de séries indépendantes avec $|r| > 0,5$ reste voisine de 40 % **quelle que soit la longueur** (0,40, 0,41 et 0,38) : allonger la série ne rend pas les marches aléatoires plus indépendantes ; sur les variations, la part tombe à presque rien dès 36 points.
- 2.7. Retours $\times$ canal : écart très net (p de l'ordre de 10^{-75}) : donner les taux par canal (3,3 %, 6,9 %, 8,8 %) ; panier Site/Boutique : écart de 1,45 € et son intervalle ; promotion : écart de moyennes (+2,6 commande par jour) et sa limite (saison) ; Site/Boutique par jour : écart moyen de 1,74 commande par jour ; panier selon les trois canaux : écarts deux à deux (Tukey).
- 2.8. Le test a duré 21 jours à 1 839 sessions par jour (38 622 sessions, 19 300 par groupe) : il pouvait détecter environ +0,55 point (+16 % relatif) ; sur le seul mobile (11 700 par groupe), l'effet minimal détectable monte à +0,72 point.

Corrigés

Corrigé 2.1

- (a) **Faux** : la p-valeur suppose l'équivalence, elle n'en donne pas la probabilité. (b) **Correct**, c'est la définition. (c) **Faux** : la p-valeur mesure la surprise, pas la taille. (d) **Faux** : une expérience bien conçue permet de parler de cause, mais « prouve » est trop fort ; l'écart reste soumis à l'incertitude. (e) **Faux** : $p > 0,05$ veut dire « on ne peut pas trancher », pas « pas d'effet ».

Corrigé 2.2

Taux : $p_A = 180/6\,000 = 0,030$, $p_B = 204/6\,000 = 0,034$; écart = 0,004. Proportion globale : $\hat{p} = 384/12\,000 = 0,032$. Erreur type sous H_0 : $\sqrt{0,032 \times 0,968 \times (1/6000 + 1/6000)} = \sqrt{1,0325 \times 10^{-5}} \approx 0,00321$. Donc $z = 0,004/0,00321 \approx 1,25$ et $p \approx 0,21$. Vérification :

```
r = 0.deux_proportions(180, 6000, 204, 6000)
print("z =", round(r["z"], 3), "| p =", round(r["p"], 3))
```

```
z = 1.245 | p = 0.213
```

Corrigé 2.3

Erreur type sans H_0 : $\sqrt{0,03 \times 0,97/6000 + 0,034 \times 0,966/6000} = \sqrt{4,85 \times 10^{-6} + 5,47 \times 10^{-6}} \approx 0,00321$. Intervalle : $0,004 \pm 1,96 \times 0,00321 = 0,004 \pm 0,0063$, soit de **-0,23 à +1,03 point**. Il contient zéro : l'écart n'est pas démontré. Mais l'intervalle contient aussi des valeurs qui compteraient (+1 point) : le test ne dit pas « pas d'effet », il dit « incertain » ; il faut un échantillon plus grand.

```
print(round(r["ic_bas"] * 100, 2), "à", round(r["ic_haut"] * 100, 2), "points")
```

```
-0.23 à 1.03 points
```

Corrigé 2.4

- (a) Faux positif : **type I** ; il augmente avec un seuil laxiste ou de nombreux tests. (b) Faux négatif : **type II** ; il augmente quand l'échantillon est petit (puissance faible). (c) Faux positif probable : **type I** gonflé par les **comparaisons multiples** (sur 20 tests sans effet, on en attend un à 5 %). Remède : corriger (Holm, Bonferroni) ou annoncer les sous-groupes à l'avance.

Corrigé 2.5

$\chi^2 = (10\,300 - 10\,000)^2/10\,000 + (9\,700 - 10\,000)^2/10\,000 = 9 + 9 = 18$; $p \approx 2 \times 10^{-5}$: le défaut est certain. Avant de lire les conversions : **chercher la cause** (par appareil, par jour, par source) et ne conclure qu'une fois la répartition expliquée ou corrigée.

```
print(stats.chisquare([10300, 9700]))
```

```
Power_divergenceResult(statistic=np.float64(18.0), pvalue=np.float64(2.2090496998585475e-05))
```

Corrigé 2.6

Bonferroni : 0,040 ; 0,080 ; 0,120 ; 0,800. Holm : $0,010 \times 4 = 0,040$; $0,020 \times 3 = 0,060$; $0,030 \times 2 = 0,060$ (on garde le maximum de ce qui précède) ; $0,200 \times 1 = 0,200$. À 5 % : **un seul** sous-groupe reste significatif dans les deux cas (le premier). Holm est moins conservateur que Bonferroni : ses p ajustées sont inférieures ou égales (0,06 contre 0,08 et 0,12).

```
print(0.holm([0.010, 0.020, 0.030, 0.200]).round(3), (np.array([0.010, 0.020, 0.030, 0.200]) *
↳ 4).clip(max=1))
```

```
[0.04 0.06 0.06 0.2 ] [0.04 0.08 0.12 0.8 ]
```

Corrigé 2.7

Erreur type : $\sqrt{80^2/400 + 85^2/400} = \sqrt{16 + 18,06} \approx 5,84$; $t = 8/5,84 \approx 1,37$; $p \approx 0,17$. L'écart de 8 € n'est pas démontré : l'intervalle à 95 % va d'environ -3,4 à +19,4 €. Il contient zéro et des écarts importants : il faudrait plus de données.

```
t = stats.ttest_ind_from_stats(108, 85, 400, 100, 80, 400, equal_var=False)
print(t, "| IC :", round(8 - 1.96 * 5.836, 1), "à", round(8 + 1.96 * 5.836, 1))
```

```
Ttest_indResult(statistic=np.float64(1.370729398009982), pvalue=np.float64(0.17084614867447198)) | IC
↳ : -3.4 à 19.4
```

Corrigé 2.8

$(1,96 + 0,84)^2 \approx 7,85$; $p_1(1 - p_1) + p_2(1 - p_2) = 0,04 \times 0,96 + 0,046 \times 0,954 = 0,0384 + 0,0439 = 0,0823$; $(p_2 - p_1)^2 = 0,006^2 = 3,6 \times 10^{-5}$; donc $n \approx 7,85 \times 0,0823 / 3,6 \times 10^{-5} \approx 17\,900$ par groupe, soit **35 900 sessions** au total, donc **environ 1,8 jour** de trafic à 20 000 sessions par jour (arrondir à **une semaine entière** pour couvrir le cycle).

```
n = 0.taille_deux_proportions(0.04, 0.046)
print(round(n), "par groupe |", round(2 * n / 20000, 2), "jours")
```

```
17940 par groupe | 1.79 jours
```

Corrigé 2.9

Moyennes : $\bar{x} = 225$, $\bar{y} = 35,33$. Écarts de x : -125, -75, -25, 25, 75, 125 ; écarts de y : -13,33, -5,33, -6,33, 5,67, 2,67, 16,67. Somme des produits = $1\,666,7 + 400 + 158,3 + 141,7 + 200 + 2\,083,3 = 4\,650$; $\sum (x - \bar{x})^2 = 43\,750$; $\sum (y - \bar{y})^2 = 563,3$; $r = 4\,650 / \sqrt{43\,750 \times 563,3} \approx 0,937$.

```
x = np.array([100, 150, 200, 250, 300, 350]); y = np.array([22, 30, 29, 41, 38, 52])
print(round(np.corrcoef(x, y)[0, 1], 3))
```

```
0.937
```

Corrigé 2.10

Rangs de x : 1, 2, 3, 4, 5, 6 ; rangs de y : 1, 3, 2, 5, 4, 6 ; différences au carré : 0, 1, 1, 1, 1, 0, soit 4 ; $\rho = 1 - 6 \times 4 / (6 \times 35) = 0,886$. En remplaçant 52 par 150, le **rang** du dernier point ne change pas (c'est toujours le plus grand) : Spearman reste à **0,886**, alors que Pearson tombe à **0,743** (le point extrême tire la droite). C'est l'intérêt de Spearman : la **robustesse**.

```
y2 = y.copy(); y2[-1] = 150
print(round(stats.spearmanr(x, y)[0], 3), round(np.corrcoef(x, y2)[0, 1], 3),
      ↪ round(stats.spearmanr(x, y2)[0], 3))
```

```
0.886 0.743 0.886
```

Corrigé 2.11

La corrélation brute entre promotion et commandes est faible ; à mois égal elle remonte, parce que les promotions tombent en saison creuse ; la régression qui contrôle mois et jour de semaine retrouve un effet voisin de +18 %.

```
import statsmodels.formula.api as smf
dm = jours["promo_active"] - jours.groupby("mois")["promo_active"].transform("mean")
dy = jours["nb_commandes"] - jours.groupby("mois")["nb_commandes"].transform("mean")
m = smf.ols("np.log(nb_commandes) ~ promo_active + C(mois) + C(jour_semaine)", data=jours).fit()
print("brute :", round(jours[["promo_active", "nb_commandes"]].corr().iloc[0, 1], 3), "| à mois égal
      ↪ :", round(np.corrcoef(dm, dy)[0, 1], 3), "| effet estimé :", f"{np.exp(m.params['promo_active'])
      ↪ - 1:+.1%}")
```

```
brute : 0.068 | à mois égal : 0.185 | effet estimé : +19.4%
```

La leçon : une corrélation brute proche de zéro n'est pas une preuve d'absence d'effet, quand une variable de confusion (la saison) agit en sens inverse.

Corrigé 2.12

Variable binaire (retour de la ligne), deux groupes indépendants : **test z de deux proportions** (ou khi-deux). On rapporte l'écart, l'intervalle et la p-valeur ; la taille d'effet est donnée par l'écart lui-même (en points) ou par le V de Cramér.

```
t = l25[l25["canal"].isin(["Site", "Réseaux"])]
r = 0.deux_proportions(t.loc["Réseaux", "sum"], t.loc["Réseaux", "size"], t.loc["Site", "sum"],
      ↪ t.loc["Site", "size"])
print(t.assign(taux=(t["sum"] / t["size"]).round(4)).to_string())
print("Site - Réseaux :", round(r["ecart"] * 100, 2), "points, IC [", round(r["ic_bas"] * 100, 2),
      ↪ " ; ", round(r["ic_haut"] * 100, 2), "], p =", round(r["p"], 4))
```

	sum	size	taux
canal			
Réseaux	228	3288	0.0693
Site	1229	13928	0.0882
Site - Réseaux : 1.89 points, IC [0.9 ; 2.88], p = 0.0005			

Le Site a un taux de retour supérieur à celui des Réseaux ; l'écart est significatif, d'une taille qui compte (près de 2 points sur 7 %).

Corrigé 2.13

Voir le tableau ci-dessous : la puissance croît avec l'effectif ; on dépasse 80 % autour de 30 000 contacts par groupe (la formule donne 30 400).

```
rng = np.random.default_rng(3)
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize
lignes = []
for n in (5000, 10000, 20000, 40000):
    sim = 0.puissance_simulee(rng, 0.030, 0.034, n)
    theo = NormalIndPower().power(proportion_effectsize(0.034, 0.030), nobs1=n, alpha=0.05)
    lignes.append([n, round(sim, 3), round(theo, 3)])
print(pd.DataFrame(lignes, columns=["par groupe", "simulée", "formule"]).to_string(index=False))
print("effectif pour 80 % :", round(0.taille_deux_proportions(0.030, 0.034)))
```

par groupe	simulée	formule
5000	0.205	0.206
10000	0.359	0.363
20000	0.620	0.623
40000	0.895	0.895

effectif pour 80 % : 30387

Corrigé 2.14

Le calcul :

```
m = site[site["appareil"] == "mobile"]
n = m["groupe"].value_counts().sort_index()
t = m.groupby("groupe")["commande"].agg(["sum", "size"])
r = 0.deux_proportions(t.loc["A", "sum"], t.loc["A", "size"], t.loc["B", "sum"], t.loc["B", "size"])
emd = brentq(lambda p2: 0.taille_deux_proportions(r["pa"], p2) - t["size"].mean(), r["pa"] + 1e-6,
             0.9) - r["pa"]
print("répartition :", n.to_dict(), "| p (50/50) =", round(stats.chisquare(n.values).pvalue, 3))
print(f"A {r['pa']:.2%} | B {r['pb']:.2%} | écart {r['ecart']*100:+.2f} pt, IC
      ↳ [{r['ic_bas']*100:+.2f} ; {r['ic_haut']*100:+.2f}], p = {r['p']:.3f} | EMD {emd*100:.2f} pt")
print("effectif par groupe pour détecter +0,6 point :", round(0.taille_deux_proportions(r["pa"],
      ↳ r["pa"] + 0.006)))
```

```
répartition : {'A': 11688, 'B': 11625} | p (50/50) = 0.68
A 3.64% | B 4.21% | écart +0.57 pt, IC [+0.07 ; +1.07], p = 0.025 | EMD 0.72 pt
effectif par groupe pour détecter +0,6 point : 16484
```

Rapport : (1) **Objectif** : la nouvelle page augmente-t-elle la conversion sur mobile ? (2) **Conception** : tirage au sort par session, métrique = commande, trois semaines ; ce sous-groupe a été choisi **après** avoir vu les données (point faible). (3) **Contrôles** : sur mobile, la répartition est équilibrée (11 688 contre 11 625) ; le défaut de répartition du test global vient de l'ordinateur. (4) **Résultats** : +0,57 point (IC +0,07 à +1,07), $p = 0,025$. (5) **Interprétation** : significatif à 5 % mais sous-groupe choisi a posteriori (l'ajustement de Holm donne 0,075) ; l'effet minimal détectable est supérieur à l'écart observé. (6) **Décision** : ne pas déployer sur la seule foi de ce test ; **relancer un test sur mobile uniquement**, sous-groupe annoncé à l'avance, avec l'effectif calculé pour +0,6 point (environ 16 600 sessions mobiles par groupe, selon le calcul ci-dessus).

Chapitre 3 : Régression pour les questions métier — exercices et applications

Ce chapitre du cahier accompagne le chapitre 3 du livre. Il contient **sept applications guidées** (de petites études que vous refaites sur les données de la boutique) et **quatorze exercices** (de ★ à ★★★) avec leurs corrigés. Les calculs à la main servent à comprendre ; le code sert à vérifier et à passer à l'échelle. Essayez toujours **avant** de lire le corrigé.

Une seule cellule charge les bibliothèques et les données. Elle est reprise telle quelle au début de chaque application : si vous travaillez dans un notebook, exécutez-la une fois.

```
import sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import statsmodels.formula.api as smf
import outils_ch03 as O

jr = O.charger_jours()                # un jour par ligne, dépense des 7 derniers jours en k€
↳ (pub_hebdo), pluie 0/1, mois, temps t
lg = O.charger_lignes()              # une ligne de commande par ligne, avec canal, catégorie et
↳ indicateur de retour
f = O.FORMULE
print(len(jr), "jours ;", len(lg), "lignes de commande")
```

```
1090 jours ; 83905 lignes de commande
```

Applications

Application 3.1 — La droite à la main, puis à la machine (sections 3.1.1 et 3.1.2)

Objectif. Refaire le calcul des moindres carrés sur six jours, le comparer à statsmodels, puis lire un tableau de résultats.

Étape 1 — Six jours, calcul direct. On prend les six jours du 2 au 7 juin 2025 : dépense publicitaire du jour et nombre de commandes.

```
six = jr[(jr["date"] >= "2025-06-02") & (jr["date"] <= "2025-06-07")]
x, y = six["depense_pub"].values, six["nb_commandes"].values.astype(float)
b = ((x - x.mean()) * (y - y.mean())).sum() / ((x - x.mean()) ** 2).sum()
a = y.mean() - b * x.mean()
```

```
e = y - (a + b * x)
print("pente :", round(b, 4), "| ordonnée :", round(a, 2), "| somme des résidus :", round(e.sum(), 6))
print("R2 :", round(1 - (e ** 2).sum() / ((y - y.mean()) ** 2).sum(), 3))
```

```
pente : 0.1318 | ordonnée : 21.63 | somme des résidus : 0.0
R2 : 0.198
```

Étape 2 — La même chose avec statsmodels. Vérifiez que l'on retrouve la pente, l'ordonnée et le R^2 .

```
m6 = smf.ols("nb_commandes ~ depense_pub", data=six).fit()
print(m6.params.round(4).to_dict(), "| R2 =", round(m6.rsquared, 3), "| intervalle de la pente :",
      ↪ m6.conf_int().loc["depense_pub"].round(4).tolist())
```

```
{'Intercept': 21.6251, 'depense_pub': 0.1318} | R2 = 0.198 | intervalle de la pente : [-0.2363, 0.4999]
```

Étape 3 — Sur tous les jours. Régressez le nombre de commandes sur la température moyenne et lisez chaque colonne du tableau.

```
mt = smf.ols("nb_commandes ~ temperature_moy", data=jr).fit()
print(mt.summary2().tables[1].round(3))
print("R2 =", round(mt.rsquared, 3))
```

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
Intercept	38.722	0.838	46.223	0.0	37.078	40.365
temperature_moy	-0.415	0.057	-7.328	0.0	-0.526	-0.304

R2 = 0.047

À vous. (1) Sur six jours, l'intervalle de confiance de la pente est-il étroit ? Qu'en concluez-vous ? (2) La pente de la température est négative : la chaleur fait-elle baisser les commandes ? Quelle variable cachée (le mois) pourrait l'expliquer ? (3) Le R^2 est-il élevé ? Un R^2 faible veut-il dire que le coefficient est mal estimé ?

Application 3.2 — Promotion et publicité : du modèle naïf au modèle contrôlé (section 3.1.3)

Objectif. Reproduire la construction « marche par marche » et voir le coefficient de la publicité s'effondrer quand on contrôle la saison.

```
etapes = {"A. promo + pub": "np.log(nb_commandes) ~ promo_active + pub_hebdo",
          "B. + jour de la semaine": "np.log(nb_commandes) ~ promo_active + pub_hebdo +
          ↪ C(jour_semaine)",
          "C. + mois": "np.log(nb_commandes) ~ promo_active + pub_hebdo + C(jour_semaine) + C(mois)",
          "D. modèle complet": f}
lignes = []
for nom, fm in etapes.items():
    m = smf.ols(fm, data=jr).fit(cov_type="HAC", cov_kws={"maxlags": 7})
    ic = m.conf_int().loc["pub_hebdo"]
    lignes.append((nom, 0.pct(m.params["promo_active"]), 0.pct(m.params["pub_hebdo"]), 0.pct(ic[0]),
    ↪ 0.pct(ic[1])))
print(pd.DataFrame(lignes, columns=["modèle", "promo %", "pub %", "pub bas", "pub
    ↪ haut"]).round(1).to_string(index=False))
```

	modèle	promo %	pub %	pub bas	pub haut
A. promo + pub		2.4	31.4	25.7	37.4
B. + jour de la semaine		2.0	31.5	25.9	37.4
C. + mois		18.8	-0.2	-6.5	6.6
D. modèle complet		19.2	0.1	-5.1	5.7

À vous. (1) À quelle étape l'effet de la publicité disparaît-il ? Quel est le point commun entre la

publicité et le mois ? (2) Remplacez `pub_hebdo` par la dépense **du jour** (`depense_pub`) : le résultat change-t-il ? (3) Ajoutez la température au modèle complet : que devient le coefficient de la pluie, et pourquoi la température seule est-elle un mauvais contrôle du mois ?

Application 3.3 — Indicatrices, logarithmes et interactions (sections 3.1.4 à 3.1.6)

Objectif. Changer de catégorie de référence, vérifier que le modèle ne change pas, estimer une élasticité, tester une interaction.

```
mod = smf.ols(f, data=jr).fit()
f2 = f.replace("C(jour_semaine)", "C(jour_semaine, Treatment(7))")          # le dimanche devient la
↳ référence
mod2 = smf.ols(f2, data=jr).fit()
print("mêmes valeurs prévues :", np.allclose(mod.fittedvalues, mod2.fittedvalues))
print("samedi contre dimanche :", round(0.pct(mod2.params["C(jour_semaine, Treatment(7))][T.6]"), 1),
↳ "%")
```

```
mêmes valeurs prévues : True
samedi contre dimanche : 109.9 %
```

```
from statsmodels.stats.anova import anova_lm
jr["weekend"] = jr["jour_semaine"].isin([6, 7]).astype(int)
f_int = f.replace("promo_active", "promo_active * weekend").replace("C(jour_semaine) + ", "")
m_int = smf.ols(f_int, data=jr).fit()
print("effet de la promotion en semaine :", round(0.pct(m_int.params["promo_active"]), 1), "%")
print("écart pour le week-end (coefficient d'interaction) :",
↳ round(m_int.params["promo_active:weekend"], 3), "| p-valeur :",
↳ round(m_int.pvalues["promo_active:weekend"], 3))
```

```
effet de la promotion en semaine : 17.4 %
écart pour le week-end (coefficient d'interaction) : 0.094 | p-valeur : 0.078
```

À vous. (1) Pourquoi les valeurs prévues sont-elles identiques malgré le changement de référence ? (2) L'interaction promotion × week-end est-elle significative ? Ajouteriez-vous cette interaction au modèle ? (3) Testez l'interaction promotion × mois d'hiver (janvier et février) : que remarquez-vous sur le nombre de jours concernés ?

Application 3.4 — Diagnostic : résidus, erreurs robustes, colinéarité, points influents (section 3.1.7)

Objectif. Appliquer les quatre contrôles du livre au modèle complet.

```
from statsmodels.stats.diagnostic import het_breuschpagan
from statsmodels.stats.stattools import durbin_watson
from statsmodels.stats.outliers_influence import variance_inflation_factor as vif
mco = smf.ols(f, data=jr).fit()
mha = smf.ols(f, data=jr).fit(cov_type="HAC", cov_kws={"maxlags": 7})
print("Breusch-Pagan p =", round(het_breuschpagan(mco.resid, mco.model.exog)[1], 4), "| Durbin-Watson
↳ =", round(durbin_watson(mco.resid), 2))
print(pd.DataFrame({"classique": mco.bse, "HAC": mha.bse}).loc[["promo_active", "pub_hebdo",
↳ "pluie_jour", "t"]].round(4))
```

```
Breusch-Pagan p = 0.0001 | Durbin-Watson = 1.99
classique    HAC
promo_active  0.0215 0.0248
pub_hebdo    0.0234 0.0274
pluie_jour   0.0123 0.0112
t            0.0066 0.0061
```

```
noms = mco.model.exog_names
print({n: round(float(vif(mco.model.exog, noms.index(n))), 1) for n in ["promo_active", "pub_hebdo",
↪ "pluie_jour", "t"]})
cook = mco.get_influence().cooks_distance[0]
print(jr.assign(cook=cook).nlargest(5, "cook")[["date", "nb_commandes", "promo_active",
↪ "cook"]].round(3).to_string(index=False))
```

```
{'promo_active': 1.9, 'pub_hebdo': 8.7, 'pluie_jour': 1.0, 't': 1.1}
  date nb_commandes promo_active cook
2024-01-01         14           0 0.034
2024-01-02         15           0 0.015
2025-02-09          9           0 0.012
2024-01-07         10           0 0.010
2024-04-10         16           0 0.009
```

À vous. (1) Les erreurs types robustes sont-elles plus grandes ou plus petites que les classiques ? Les coefficients changent-ils ? (2) Quelle variable a le VIF le plus élevé, et pourquoi ? (3) Retirez les cinq jours les plus influents et réajustez : la conclusion sur la promotion change-t-elle ?

Application 3.5 — Expliquer ou prédire : 2025 mis de côté (section 3.1.8)

Objectif. Juger une prévision sur des jours que le modèle n'a pas vus, et la comparer à des références.

```
app, test = jr[jr["annee"] <= 2024], jr[jr["annee"] == 2025]
m_app = smf.ols(f, data=app).fit()
prevu = np.exp(m_app.predict(test))
mae = (test["nb_commandes"] - prevu).abs().mean()
mape = ((test["nb_commandes"] - prevu).abs() / test["nb_commandes"]).mean() * 100
print("modèle complet : MAE =", round(mae, 2), "| MAPE =", round(mape, 1), "% | R2 d'ajustement =",
↪ round(m_app.rsquared, 3))
```

```
modèle complet : MAE = 4.44 | MAPE = 12.9 % | R2 d'ajustement = 0.757
```

```
ref = jr.set_index("date")["nb_commandes"].shift(364).reindex(test["date"]).values # même jour
↪ de semaine, un an plus tôt
print("même jour 364 jours avant : MAE =", round(np.nanmean(np.abs(test["nb_commandes"].values -
↪ ref)), 2))
for nom, fm in {"sans les mois": f.replace(" + C(mois)", ""), "sans le jour de la semaine":
↪ f.replace(" + C(jour_semaine)", "")}.items():
    m = smf.ols(fm, data=app).fit()
    print(nom, ": MAE =", round((test["nb_commandes"] - np.exp(m.predict(test))).abs().mean(), 2))
```

```
même jour 364 jours avant : MAE = 6.66
sans les mois : MAE = 6.24
sans le jour de la semaine : MAE = 7.35
```

À vous. (1) Quelle variable fait le plus perdre en prévision quand on la retire ? (2) Le modèle fait-il mieux que la référence « même jour, un an plus tôt » ? (3) Estimez l'erreur sur les seuls jours de promotion : le modèle y est-il meilleur ou moins bon ?

Application 3.6 — Présenter à la gérante (sections 3.2.1 à 3.2.6)

Objectif. Fabriquer automatiquement des phrases d'interprétation justes, avec effet, incertitude et conditions.

```
mod = smf.ols(f, data=jr).fit(cov_type="HAC", cov_kws={"maxlags": 7})
ic = mod.conf_int()
```

```
def phrase(nom, libelle, unite):
    e, lo, hi = (0.pct(v) for v in (mod.params[nom], ic.loc[nom, 0], ic.loc[nom, 1]))
    if lo < 0 < hi:
        return f"{libelle} : pas d'effet détecté ({e:+.1f} %, intervalle de {lo:+.1f} % à {hi:+.1f} %)."
    return f"{libelle} : {e:+.1f} % de commandes {unite}, intervalle de {lo:+.1f} % à {hi:+.1f} %."

print(phrase("promo_active", "Jour de promotion", "par rapport à un jour comparable sans promotion"))
print(phrase("pub_hebdo", "1 000 € de publicité hebdomadaire de plus", "toutes choses égales par ailleurs"))
print(phrase("pluie_jour", "Jour de pluie", "toutes choses égales par ailleurs"))
```

```
Jour de promotion : +19.2 % de commandes par rapport à un jour comparable sans promotion, intervalle
↳ de +13.5 % à +25.1 %.
1 000 € de publicité hebdomadaire de plus : pas d'effet détecté (+0.1 %, intervalle de -5.1 % à +5.7
↳ %).
Jour de pluie : pas d'effet détecté (-1.7 %, intervalle de -3.8 % à +0.5 %).
```

À vous. (1) Ajoutez la phrase du samedi contre lundi. (2) Convertissez l'effet de la promotion en commandes par jour pour un jour moyen de promotion (voir le livre, 3.2.1). (3) Rédigez les cinq lignes pour la gérante.

Application 3.7 — Les retours : régression logistique et seuil (section 3.3)

Objectif. Estimer des rapports de cotes, juger le modèle et choisir un seuil par un calcul de coûts.

```
fl = "retour ~ C(canal, Treatment('Boutique')) + C(categorie) + np.log(prix_unitaire) + promo +
↳ quantite"
mlog = smf.logit(fl, data=lg).fit(dis=0)
rc = np.exp(pd.concat([mlog.params, mlog.conf_int()], axis=1)); rc.columns = ["rapport de cotes",
↳ "bas", "haut"]
print(rc.drop("Intercept").round(2).head(6))
```

	rapport de cotes	bas	haut
C(canal, Treatment('Boutique'))[T.Réseaux]	2.25	2.04	2.49
C(canal, Treatment('Boutique'))[T.Site]	3.11	2.91	3.33
C(categorie)[T.Cuisine]	1.01	0.90	1.12
C(categorie)[T.Décoration]	0.99	0.89	1.10
C(categorie)[T.Jardin]	1.01	0.91	1.13
C(categorie)[T.Maison]	0.95	0.85	1.06

```
from sklearn.metrics import roc_auc_score
lg["p"] = mlog.predict(lg)
print("AUC :", round(roc_auc_score(lg["retour"], lg["p"]), 3), "| taux de retour :",
↳ round(lg["retour"].mean() * 100, 1), "%")
cout_retour, cout_action, succes = 18.0, 0.5, 0.4
b = lg.groupby("canal").agg(lignes=("retour", "size"), retours=("retour", "sum"))
b["gain net (€)"] = (b["retours"] * succes * cout_retour - b["lignes"] * cout_action).round(0)
print(b)
```

```
AUC : 0.636 | taux de retour : 6.0 %
lignes retours gain net (€)
canal
Boutique 39362 1211 -10962.0
Réseaux 9071 605 -180.0
Site 35472 3186 5203.0
```

À vous. (1) Quel est le seuil de probabilité rentable ? Quels canaux dépassent ce seuil ? (2) Si l'action n'évite le retour que dans 25 % des cas, que devient la décision ? (3) Ajoutez le mois de la commande au modèle : la saison joue-t-elle sur les retours ?

Exercices

Exercice 3.1 ★ — Une pente à la main (section 3.1.1)

Quatre jours : dépense publicitaire x (en centaines d'euros) 1, 2, 3, 4 et commandes y : 20, 26, 29, 37. Calculez la pente et l'ordonnée de la droite des moindres carrés, puis la prévision pour une dépense de 5.

Exercice 3.2 ★ — Un R^2 à la main (section 3.1.1)

Avec les données de l'exercice 3.1, calculez les résidus, leur somme et la somme de leurs carrés, puis le R^2 . Que veut dire cette valeur ?

Exercice 3.3 ★★ — Lire un tableau de résultats (section 3.1.2)

Un tableau indique, pour la pente de la publicité du jour : coefficient 0,061, erreur type 0,003. (1) Calculez le t et un intervalle de confiance approximatif à 95 %. (2) Un collègue dit « la p-valeur est nulle, donc la publicité est très efficace ». Que lui répondez-vous ?

Exercice 3.4 ★ — Toutes choses égales par ailleurs (section 3.1.3)

Un modèle de régression sur les commandes quotidiennes donne : promotion +0,175 (en logarithme), samedi +0,375 par rapport au lundi. Un jour de promotion tombe un samedi. Que prévoit le modèle pour ce jour par rapport à un lundi sans promotion, en pourcentage ?

Exercice 3.5 ★★ — De la valeur du coefficient à l'effet en pourcentage (section 3.1.5)

- (1) Convertissez en pourcentage les coefficients du logarithme suivants : 0,05, 0,175, $-0,017$, 0,754. (2) À partir de quelle taille l'approximation « le coefficient est le pourcentage » devient-elle trompeuse ? (3) Quelle élasticité faut-il pour qu'une hausse de 10 % de la dépense publicitaire fasse monter les commandes de 1 % ?

Exercice 3.6 ★★ — Les indicatrices (section 3.1.4)

On régresse les commandes sur le canal d'acquisition d'un client (Boutique, Site, Réseaux) avec la Boutique en référence. (1) Combien d'indicatrices crée-t-on ? (2) Que vaut le coefficient du Site si les moyennes des trois groupes sont 4,0, 5,5 et 3,5 (dans l'ordre Boutique, Site, Réseaux) ? (3) Que devient-il si le Site est la référence ?

Exercice 3.7 ★★ — La colinéarité à la main (section 3.1.7)

Deux variables explicatives ont une corrélation de 0,9. Avec seulement ces deux variables, le VIF de chacune vaut $1/(1 - r^2)$. (1) Calculez-le. (2) De combien l'erreur type d'un coefficient est-elle multipliée par rapport au cas indépendant ? (3) La publicité a un VIF de 8,7 dans le modèle du livre : que cela signifie-t-il ?

Exercice 3.8 ★★ — Détecter un défaut (section 3.1.7)

Sur une série de 8 résidus consécutifs +2, +1, +2, -1, -2, -1, +1, +2, calculez la statistique de Durbin-Watson $\sum_{t \geq 2} (e_t - e_{t-1})^2 / \sum e_t^2$. Qu'indique-t-elle ? Et qu'indiquerait un résultat proche de 2 ?

Exercice 3.9 ★★ — Mesurer une prévision (section 3.1.8)

Quatre jours : commandes observées 30, 40, 50, 60 ; prévues 33, 36, 55, 57. Calculez l'erreur absolue moyenne et l'erreur relative moyenne (MAPE). Un modèle qui prévoit toujours 45 fait-il mieux ?

Exercice 3.10 ★★★ — Une expérience pour trancher sur la publicité (section 3.1.9)

L'intervalle de confiance de l'effet de la publicité va de -5 % à +6 % par millier d'euros. On veut distinguer un effet de +1,5 % de zéro. (1) Si l'on pouvait répartir **au hasard** la dépense hebdomadaire entre deux niveaux, combien de semaines faudrait-il, en supposant que l'écart-type du logarithme des commandes hebdomadaires autour de leur moyenne est de 0,06 ? (2) Pourquoi une expérience aléatoire résout-elle le problème de la saison ?

Exercice 3.11 ★ — Réécrire des phrases (section 3.2.5)

Corrigez : (a) « La pluie ne compte pas, son coefficient est de -1,7 % ». (b) « Chaque euro de publicité rapporte 0,06 commande ». (c) « $p < 0,05$ donc la promotion est très importante ».

Exercice 3.12 ★★ — Relatif ou absolu (section 3.2.2)

La promotion augmente les commandes de 19 % et le chiffre d'affaires de 8 %. (1) Pour un jour à 20 commandes sans promotion, combien de commandes de plus ? Et pour un jour à 50 ? (2) Comment l'effet sur le chiffre d'affaires peut-il être plus faible que sur les commandes ? (3) Quel indicateur faudrait-il ajouter pour répondre à « est-ce que ça rapporte ? »

Exercice 3.13 ★ — Un rapport de cotes à la main (section 3.3.1)

Sur 1 000 lignes du Site, 90 sont retournées ; sur 1 000 lignes de la Boutique, 30. Calculez les deux probabilités, les deux cotes, le rapport de cotes et le rapport de probabilités.

Exercice 3.14 ★★ — Un seuil par les coûts (section 3.3.5)

Un retour coûte 25 €, une vérification coûte 1 € par ligne et évite le retour dans 50 % des cas. (1) Quel est le seuil de probabilité rentable ? (2) Une ligne a une probabilité de retour de 6 % : faut-il la vérifier ? (3) Que devient le seuil si la vérification coûte 2 € ?

Corrigés

Corrigé 3.1

$\bar{x} = 2,5$, $\bar{y} = 28$; $S_{xy} = (-1,5)(-8) + (-0,5)(-2) + (0,5)(1) + (1,5)(9) = 12 + 1 + 0,5 + 13,5 = 27$; $S_{xx} = 2,25 + 0,25 + 0,25 + 2,25 = 5$. Pente $b = 27/5 = 5,4$ commandes par centaine d'euros ; ordonnée $a = 28 - 5,4 \times 2,5 = 14,5$. Prévision pour $x = 5$: $14,5 + 5,4 \times 5 = 41,5$ commandes.

Corrigé 3.2

Valeurs prévues : 19,9 ; 25,3 ; 30,7 ; 36,1. Résidus : 0,1 ; 0,7 ; -1,7 ; 0,9 (somme nulle). Somme des carrés : $0,01 + 0,49 + 2,89 + 0,81 = 4,2$. Variabilité totale : $\sum (y - \bar{y})^2 = (-8)^2 + (-2)^2 + 1^2 + 9^2 = 150$, donc $R^2 = 1 - 4,2/150 = 0,972$. La droite explique 97 % de la variabilité de ces quatre jours (un R^2 très élevé sur quatre points ne prouve pas grand-chose).

Corrigé 3.3

- (1) $t = 0,061/0,003 \approx 20,3$; intervalle approximatif $0,061 \pm 1,96 \times 0,003$, soit de 0,055 à 0,067.
 (2) Une p-valeur nulle dit seulement que la pente est **distinguable de zéro** dans ce modèle ; elle ne dit ni que l'effet est grand (0,061 commande par euro), ni qu'il est **causal** : sans contrôle de la saison, la publicité capte l'effet des mois chargés (livre, 3.1.3).

Corrigé 3.4

Dans un modèle sur le logarithme, les effets se **multiplient** : $e^{0,175+0,375} = e^{0,55} \approx 1,73$, soit environ **+73 %** par rapport à un lundi sans promotion. Attention : additionner les pourcentages ($+19,1 \% + 45,5 \% = 64,6 \%$) est faux, car $1,191 \times 1,455 \approx 1,733$.

Corrigé 3.5

- (1) $e^{0,05} - 1 = 5,1 \%$; $e^{0,175} - 1 = 19,1 \%$; $e^{-0,017} - 1 = -1,7 \%$; $e^{0,754} - 1 = 112,5 \%$. (2) Pour un coefficient inférieur à 0,05 en valeur absolue, l'approximation est bonne (écart inférieur à 0,2 point) ; au-delà de 0,2, elle sous-estime nettement (0,754 donne 75 % au lieu de 112 %). (3) Une élasticité de 0,1 : $10 \% \times 0,1 = 1 \%$ (approximation valable pour de petites variations).

Corrigé 3.6

- (1) **Deux** indicatrices (Site et Réseaux) ; la Boutique est la référence. (2) Coefficient du Site : $5,5 - 4,0 = +1,5$ commande en moyenne ; Réseaux : $3,5 - 4,0 = -0,5$. (3) Avec le Site en référence : Boutique $4,0 - 5,5 = -1,5$ et Réseaux $3,5 - 5,5 = -2,0$; le modèle (les moyennes prévues) est inchangé.

Corrigé 3.7

- (1) $VIF = 1/(1 - 0,81) = 1/0,19 \approx 5,26$. (2) L'erreur type est multipliée par $\sqrt{VIF} \approx 2,3$. (3) Un VIF de 8,7 multiplie l'erreur type par $\sqrt{8,7} \approx 2,9$: le coefficient de la publicité est près de **trois fois plus incertain** qu'il le serait si la publicité ne dépendait pas du mois ; c'est l'une des raisons de la largeur de son intervalle.

Corrigé 3.8

Différences successives : -1, +1, -3, -1, +1, +2, +1 ; carrés : $1 + 1 + 9 + 1 + 1 + 4 + 1 = 18$. Somme des carrés des résidus : $4 + 1 + 4 + 1 + 4 + 1 + 1 + 4 = 20$. $DW = 18/20 = 0,9$. Une valeur nettement **inférieure à 2** indique une **autocorrélation positive** (les résidus d'un jour ressemblent à ceux de la veille) : les erreurs types classiques sont trop petites, d'où les erreurs robustes. Un résultat proche de 2 indique l'absence d'autocorrélation.

Corrigé 3.9

Erreurs absolues : 3, 4, 5, 3 ; $MAE = 15/4 = 3,75$. Erreurs relatives : $3/30 = 0,10$; $4/40 = 0,10$; $5/50 = 0,10$; $3/60 = 0,05$; $MAPE = 8,75\%$. Le modèle « toujours 45 » a des erreurs absolues 15, 5, 5, 15 : $MAE = 10$: bien pire.

Corrigé 3.10

- (1) On compare deux niveaux de dépense qui diffèrent de 1 000 € par semaine, avec n semaines par niveau ; l'erreur type de la différence de moyennes de logarithmes est $0,06\sqrt{2/n}$; on veut que l'effet de 0,015 (environ 1,5 % en logarithme) soit à 2,8 erreurs types (puissance de 80 % au seuil de 5 %) : $0,015 \geq 2,8 \times 0,06\sqrt{2/n}$, soit $n \geq 2 \times (2,8 \times 0,06/0,015)^2 = 2 \times 125 = 250$ semaines par groupe. C'est irréaliste : il faudrait **des années** ; on augmente la puissance en expérimentant par **zone géographique** ou **par jour** plutôt que par semaine, ou en visant un effet plus grand. (2) Le tirage au hasard rend la dépense **indépendante** de la saison (en moyenne, chaque niveau tombe autant en décembre qu'en janvier) : la comparaison n'a plus besoin de « contrôler » la saison.

Corrigé 3.11

- (a) « Pas d'effet net de la pluie détecté ($-1,7\%$, intervalle de $-3,8\%$ à $+0,5\%$) ; s'il existe, il est petit. » (b) « Sans contrôle de la saison, la publicité semblait faire gagner 0,06 commande par euro ; après contrôle, aucun effet n'est détecté. » (c) « La promotion augmente d'environ 19 % les commandes (intervalle de 13 % à 25 %) ; la p-valeur dit seulement que l'effet n'est pas dû au hasard d'échantillonnage. »

Corrigé 3.12

- (1) Pour 20 commandes sans promotion : $0,19 \times 20 = 3,8$ commandes de plus ; pour 50 : 9,5. L'effet **absolu** dépend de la base. (2) Les promotions s'accompagnent de remises de 5 à 20 % : chaque commande rapporte moins, de sorte que le chiffre d'affaires augmente moins vite que le nombre de commandes ($1,19 \times (1 - r) \approx 1,08$ avec une remise moyenne r d'environ 9 %). (3) La **marge** (chiffre d'affaires hors taxe moins coût des produits) : voir le chapitre 9.

Corrigé 3.13

Probabilités : Site $90/1\,000 = 9\%$, Boutique 3 %. Cotes : Site $90/910 \approx 0,0989$, Boutique $30/970 \approx 0,0309$. Rapport de cotes : $0,0989/0,0309 \approx 3,20$. Rapport de probabilités : $9/3 = 3,0$. Le rapport de cotes est un peu supérieur au risque relatif, parce que l'événement n'est pas infiniment rare.

Corrigé 3.14

- (1) $p^* = c/(qs) = 1/(0,5 \times 25) = 0,08$, soit 8 %. (2) Une ligne à 6 % est **sous le seuil** : la vérification coûterait 1 € pour un gain espéré de $0,06 \times 0,5 \times 25 = 0,75$ € : non rentable. (3) Avec un coût de 2 € : $p^* = 2/12,5 = 0,16$, soit 16 % : presque aucune ligne ne dépasse ce seuil.

Chapitre 4 : Segmentation et analyse de cohortes — exercices et applications

Ce chapitre du cahier prolonge le chapitre 4 du livre. Les **applications** sont de petites études guidées sur les clients de la boutique : chacune annonce un objectif, avance par étapes courtes (au plus vingt-cinq lignes par bloc) et se termine par une lecture et une invitation à aller plus loin. Les **exercices** vont de ★ (application directe) à ★★★ (à réfléchir) ; chacun renvoie à la section du livre qui l'éclaire. Les **corrigés** montrent un calcul à la main quand il est court, puis le code. Les données sont **simulées**.

Une seule cellule charge les bibliothèques et les tables ; les autres cellules en dépendent.

```
import sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import outils_ch04 as O      # fonctions du livre, regroupées (voir build/outils_ch04.py)
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, adjusted_rand_score
T = O.charger()
cli, cmd, lig, prod, sess = T["cli"], T["cmd"], T["lig"], T["prod"], T["sess"]
g = O.table_clients(cmd, lig)      # une ligne par client ayant commandé (4 806 clients) au
↳ 31/12/2025
print(len(cli), "clients |", len(g), "ont commandé |", len(cmd), "commandes |", len(sess), "sessions")
```

```
6000 clients | 4806 ont commandé | 36395 commandes | 127022 sessions
```

Applications

Application 4.1 — Des segments par canal et fidélité (section 4.1)

Objectif. Construire des segments par règles à partir du canal dominant et de la carte de fidélité, puis comparer leur chiffre d'affaires par client, sans conclure trop vite.

Étape 1 — le canal dominant. Un client est « Site » si au moins 70 % de ses commandes passent par le site, « Boutique » si au plus 30 %, « Mixte » sinon.

```
g["canal_dominant"] = np.select([g["site"] >= 0.7, g["site"] <= 0.3], ["Site", "Boutique", "Mixte"])
g["fidelite"] = g.index.map(cli.set_index("id_client")["fidelite"])
print(g["canal_dominant"].value_counts().to_dict())
```

```
{'Mixte': 2603, 'Boutique': 1499, 'Site': 704}
```

Étape 2 — comparer. Le chiffre d'affaires moyen par client et la part de clients, selon le canal dominant et la carte.

```
t = g.groupby(["canal_dominant", "fidelite"]).agg(clients=("n", "size"), commandes=("n", "mean"),
↪ ca=("ca", "mean"))
print(t.round(1))
```

		clients	commandes	ca
canal_dominant	fidelite			
Boutique	0	969	5.0	510.9
	1	530	4.9	487.7
Mixte	0	1676	10.3	1029.6
	1	927	10.5	1050.3
Site	0	462	2.9	280.8
	1	242	3.0	292.0

À vous. La carte de fidélité va-t-elle avec plus de commandes ? Peut-on dire que la carte les **fait** commander ? (Indice : la carte est proposée à des clients qui ne sont pas choisis au hasard.)

Application 4.2 — Choisir k, un pas de plus (section 4.1)

Objectif. Comparer plusieurs nombres de segments non seulement par la silhouette mais par la **taille du plus petit segment** et l'**accord** entre solutions voisines.

```
X = 0.variables_kmeans(g)
Z = StandardScaler().fit_transform(X)
sol = {k: KMeans(k, n_init=10, random_state=0).fit(Z) for k in range(3, 7)}
for k, m in sol.items():
    print(k, "| silhouette", round(silhouette_score(Z, m.labels_, sample_size=3000, random_state=0),
↪ 3), "| plus petit segment :", int(np.bincount(m.labels_).min()))
```

```
3 | silhouette 0.264 | plus petit segment : 1189
4 | silhouette 0.276 | plus petit segment : 392
5 | silhouette 0.213 | plus petit segment : 343
6 | silhouette 0.224 | plus petit segment : 318
```

```
print("accord k=4 / k=5 :", round(adjusted_rand_score(sol[4].labels_, sol[5].labels_), 2), "| k=4 /
↪ k=3 :", round(adjusted_rand_score(sol[4].labels_, sol[3].labels_), 2))
```

```
accord k=4 / k=5 : 0.5 | k=4 / k=3 : 0.82
```

À vous. Quel k retiendriez-vous si la gérante ne peut mener que trois actions distinctes ? Que gagne-t-on, que perd-on avec k = 3 ?

Application 4.3 — Règles contre k-moyennes : qui prédit mieux ? (section 4.1)

Objectif. Se placer au 30 juin 2025 et comparer le **pouvoir de prédiction** des segments par règles (réguliers, occasionnels, endormis) et des segments k-moyennes sur la commande du second semestre.

Étape 1 — les deux segmentations au 30 juin.

```
t0 = pd.Timestamp("2025-06-30")
c0 = cmd[cmd["date_commande"] <= t0]
g0 = 0.table_clients(c0, lig[lig["id_commande"].isin(c0["id_commande"])], t0)
g0["k4"] = 0.nommer_segments(g0, KMeans(4, n_init=10,
↪ random_state=0).fit(StandardScaler().fit_transform(0.variables_kmeans(g0))).labels_)
```

```
n12 = c0[c0["date_commande"] > t0 - pd.Timedelta(days=365)].groupby("id_client").size()
g0["n12"] = n12.reindex(g0.index).fillna(0)
g0["regle"] = np.select([g0["n12"] >= 3, g0["n12"] >= 1], ["Réguliers", "Occasionnels", "Endormis"])
```

Étape 2 — la commande du semestre suivant.

```
h2 = set(cmd.loc[cmd["date_commande"] > t0, "id_client"])
g0["achat_h2"] = g0.index.isin(h2).astype(int)
for col in ["regle", "k4"]:
    print(g0.groupby(col)["achat_h2"].agg(["size", "mean"]).round(3), "\n")
```

	size	mean
regle		
Endormis	763	0.362
Occasionnels	1897	0.521
Réguliers	1749	0.854

	size	mean
k4		
Chasseurs de promotions	431	0.492
Dormants de la Boutique	1064	0.435
Dormants du Site	896	0.450
Réguliers actifs	2018	0.833

À vous. Quelle segmentation sépare le mieux les clients qui recommandent de ceux qui ne recommandent pas ? Mesurez cet écart par la **différence entre le meilleur et le moins bon segment**.

Application 4.4 — Cohortes mensuelles contre trimestrielles (section 4.2)

Objectif. Voir le bruit d'une matrice mensuelle et le réduire en regroupant.

```
effm, tm, _ = 0.matrice_cohortes(cli, cmd, pas="M")
effq, tq, _ = 0.matrice_cohortes(cli, cmd, pas="Q")
print("taille des cohortes mensuelles :", int(effm.min()), "à", int(effm.max()), "| trimestrielles  
↪ :", int(effq.min()), "à", int(effq.max()))
```

taille des cohortes mensuelles : 42 à 66 | trimestrielles : 147 à 188

```
agelm = tm[1].dropna() * 100
agelq = tq[1].dropna() * 100
print("taux d'activité à l'âge 1 : mensuel, de", round(agelm.min()), "% à", round(agelm.max()), "% |  
↪ trimestriel, de", round(agelq.min()), "% à", round(agelq.max()), "%")
print("écart-type entre cohortes : mensuel", round(agelm.std(), 1), "| trimestriel",  
↪ round(agelq.std(), 1))
```

taux d'activité à l'âge 1 : mensuel, de 5 % à 33 % | trimestriel, de 30 % à 41 %
écart-type entre cohortes : mensuel 7.2 | trimestriel 3.6

À vous. Les cohortes mensuelles varient beaucoup plus que les trimestrielles : est-ce un vrai écart de comportement ? Calculez l'écart-type attendu **par pur hasard** d'une proportion : celle d'une cohorte mensuelle à l'âge 1 (environ 16 %, 57 clients) puis celle d'une cohorte trimestrielle (environ 36 %, 170 clients). Les variations observées dépassent-elles ce bruit ?

Application 4.5 — Revenu par client et concentration (section 4.2)

Objectif. Mesurer le revenu par client des cohortes annuelles, et regarder la **médiane** à côté de la moyenne.

```
new = cli[cli["date_inscription"] >= "2023-01-01"].set_index("id_client")
c = cmd[cmd["id_client"].isin(new.index)].copy()
c["age_j"] = (c["date_commande"] - c["id_client"].map(new["date_inscription"])).dt.days
c12 = c[c["age_j"] < 365].groupby("id_client")["ca"].sum()
base = new[new["date_inscription"] <= "2024-12-31"].copy() # observés au moins 12 mois
base["ca12"] = c12.reindex(base.index).fillna(0.0)
base["an"] = base["date_inscription"].dt.year
print(base.groupby("an")["ca12"].agg(["size", "mean", "median"]).round(1))
```

	size	mean	median
an			
2023	700	252.0	129.9
2024	666	240.2	125.9

```
srt = base["ca12"].sort_values(ascending=False)
print("part du CA des 10 % meilleurs :", round(srt.head(int(0.1 * len(srt))).sum() / srt.sum() * 100,
↪ 1), "% | clients à zéro :", round((base["ca12"] == 0).mean() * 100, 1), "%")
```

part du CA des 10 % meilleurs : 40.3 % | clients à zéro : 31.3 %

À vous. La moyenne annuelle de 12 mois est-elle un bon résumé du client « typique » ? Que diriez-vous à la gérante qui voudrait fixer un budget d'acquisition sur cette moyenne ?

Application 4.6 — RFM : le piège des ex æquo (section 4.3)

Objectif. Voir pourquoi la fréquence se classe par **rang**, et comparer deux définitions de « client à risque ».

```
print("clients n'ayant commandé qu'une fois :", round((g["n"] == 1).mean() * 100, 1), "%")
brut = pd.qcut(g["n"], 5, duplicates="drop")
print("quintiles sur les valeurs brutes :", brut.value_counts().sort_index().tolist())
rang = pd.qcut(g["n"].rank(method="first"), 5, labels=False)
print("quintiles sur les rangs :", pd.Series(rang).value_counts().sort_index().tolist())
```

clients n'ayant commandé qu'une fois : 17.3 %
quintiles sur les valeurs brutes : [1410, 881, 596, 1035, 884]
quintiles sur les rangs : [962, 961, 961, 961, 961]

```
s = 0.rfm(g)
a_risque = g[(s["R"] <= 2) & (s["F"] >= 4)]
print("clients à risque (R ≤ 2, F ≥ 4) :", len(a_risque), "| leur CA cumulé :",
↪ round(a_risque["ca"].sum()), "€ soit", round(a_risque["ca"].sum() / g["ca"].sum() * 100, 1), "%
↪ du total")
print(a_risque.sort_values("ca", ascending=False)[["n", "ca", "rec"]].head(5).round(0).astype(int))
```

clients à risque ($R \leq 2$, $F \geq 4$) : 257 | leur CA cumulé : 268151 € soit 7.3 % du total

	n	ca	rec
id_client			
4154	35	4162	181
12	32	3131	156
798	22	2690	164
3900	24	2519	147
2377	14	2494	418

À vous. Quels clients appelleriez-vous en premier parmi les clients à risque ? Quelles informations manquent pour décider ?

Application 4.7 — Valeur vie client par segment (section 4.3)

Objectif. Appliquer la formule de la valeur vie client aux segments de k-moyennes, avec un taux d'actualisation fictif de 8 %. Piège à éviter : les segments doivent être construits **avant** la période où l'on mesure la rétention. Segmentés avec les données de 2025, les « réguliers actifs » auraient une rétention 2024-2025 de près de 100 % par construction (ils sont actifs *parce que* leur dernière commande est récente) : c'est un raisonnement circulaire.

Étape 1 — segments au 31 décembre 2024.

```
t1 = pd.Timestamp("2024-12-31")
c1 = cmd[cmd["date_commande"] <= t1]
g1 = 0.table_clients(c1, lig[lig["id_commande"].isin(c1["id_commande"])], t1)
g1["segment"] = 0.nommer_segments(g1, KMeans(4, n_init=10,
↳ random_state=0).fit(StandardScaler().fit_transform(0.variables_kmeans(g1))).labels_)
x = lig.merge(cmd[["id_commande", "date_commande", "id_client"]],
↳ on="id_commande").merge(prod[["id_produit", "cout_achat"]], on="id_produit")
x["marge"] = x["montant"] / 1.2 - x["quantite"] * x["cout_achat"]
x["annee"] = x["date_commande"].dt.year
m25 = x[x["annee"] == 2025].groupby("id_client")["marge"].sum()
a24 = set(x.loc[x["annee"] == 2024, "id_client"]); a25 = set(x.loc[x["annee"] == 2025, "id_client"])
print(g1["segment"].value_counts().to_dict())
```

```
{'Réguliers actifs': 1813, 'Dormants de la Boutique': 981, 'Dormants du Site': 832, 'Chasseurs de
↳ promotions': 438}
```

Étape 2 — rétention, marge par client actif et valeur vie, par segment.

```
lignes = []
for seg, d in g1.groupby("segment"):
    act24 = a24 & set(d.index)
    rho = len(act24 & a25) / len(act24)
    m = m25.reindex(list(act24 & a25)).mean()
    lignes.append((seg, len(act24), round(rho, 3), round(m, 1), round(m / (1 - rho / 1.08))))
print(pd.DataFrame(lignes, columns=["segment", "actifs_2024", "retention", "marge_par_actif",
↳ "CLV_8pct"]).to_string(index=False))
```

segment	actifs_2024	retention	marge_par_actif	CLV_8pct
Chasseurs de promotions	345	0.713	80.0	235
Dormants de la Boutique	692	0.675	75.0	200
Dormants du Site	630	0.678	73.9	198
Réguliers actifs	1812	0.932	156.9	1142

À vous. Quel segment a la plus grande valeur vie, et à quel point la valeur d'un segment dont la rétention est proche de 100 % est-elle sensible à une petite erreur sur cette rétention ? (Indice : calculez la valeur vie avec une rétention de 0,92 et de 0,95.)

Application 4.8 — L'entonnoir, source par appareil (section 4.4)

Objectif. Croiser source et appareil, puis chiffrer, par source, la **valeur perdue** aux deux étapes où l'intention d'achat est démontrée.

```
ent = 0.entonnoir(sess, ["source", "appareil"])
print((ent["conversion"] * 100).round(1).unstack())
```

appareil	mobile	ordinateur	tablette
source			
direct	7.1	6.7	7.0
email	8.7	8.8	7.3
organique	4.0	4.1	3.9
payant	3.0	2.7	4.3

referent	3.7	4.0	3.2
reseaux	2.1	2.2	2.3

```
src = 0.entonnoir(sess, "source")
src["paniers_perdus"] = src["ajout_panier"] - src["debut_paielement"]
src["paiements_perdus"] = src["debut_paielement"] - src["commande"]
print(src[["sessions", "commande", "paniers_perdus",
↵ "paiements_perdus"]].sort_values("paiements_perdus", ascending=False))
```

	sessions	commande	paniers_perdus	paiements_perdus
source				
organique	43187	1736	2611	1455
direct	35566	2467	2205	1213
payant	17783	535	1121	579
reseaux	15243	329	971	508
email	8892	772	498	315
referent	6351	239	352	210

À vous. À quelle source un abandon de panier coûte-t-il le plus en nombre ? En proportion ? Les deux classements sont-ils les mêmes ?

Exercices

Exercice 4.1 ★ — Un seuil de plus (section 4.1.2)

Reprenez les segments par règles de la section 4.1.2, mais appelez « réguliers » les clients ayant **au moins deux** commandes sur douze mois (on ne retire pas ici les nouveaux inscrits, donc les effectifs diffèrent un peu de ceux du livre). Quelle part des clients et du chiffre d'affaires des douze derniers mois représentent-ils ? Comparez à la règle « au moins trois ».

Exercice 4.2 ★★ — Une itération à la main, autres clients (section 4.1.3)

Six clients : commandes 1, 3, 4, 10, 12, 15 ; panier 50, 70, 55, 90, 120, 105. Standardisez, prenez les clients **B** et **E** comme centres de départ, calculez les distances, les affectations et les nouveaux centres. L'algorithme a-t-il convergé après une itération ? Vérifiez avec KMeans.

Exercice 4.3 ★★ — Standardiser ou non (section 4.1.4)

Avec seulement deux variables, la récence (en jours) et le nombre de commandes, comparez les segments à $k = 3$ avec et sans standardisation. Quel est l'indice de Rand ajusté entre les deux solutions ? Quelle variable domine sans standardisation ?

Exercice 4.4 ★★★ — Un critère extérieur pour les « chasseurs de promotions » (section 4.1.7)

Au 30 juin 2025, reconstruisez les segments de k-moyennes. Les « chasseurs de promotions » sont-ils réellement plus sensibles aux promotions **après** cette date ? Mesurez, pour chaque segment, la part des commandes du second semestre passées avec un code promotionnel. Que concluez-vous, et quelle réserve faut-il garder ?

Exercice 4.5 ★ — Une rétention à la main (section 4.2.2)

Une cohorte de six clients : les trimestres avec commande sont (1) 0, 1 ; (2) 1, 2, 3 ; (3) aucun ; (4) 0, 3 ; (5) 2 ; (6) 0, 1, 2, 3. Calculez la rétention aux âges 0 à 3, puis la part de clients qui n'ont jamais commandé. Vérifiez avec pandas.

Exercice 4.6 ★★ — Les anciens clients et la période (section 4.2.5)

Calculez le taux d'activité **par trimestre civil** des 4 000 clients inscrits **avant 2023** (les clients que l'on exclut des cohortes) et comparez-le à celui des clients inscrits depuis 2023. Que dit la comparaison de l'effet de la saison ?

Exercice 4.7 ★★ — Quelles cases sont significatives ? (section 4.2.6)

Pour la cohorte trimestrielle de 2023T1 (167 clients), calculez l'intervalle de confiance à 95 % (Wilson) de la case d'âge 3 (50 %) et de la case d'âge 8 (26 %). Les deux intervalles se chevauchent-ils ? Que déduisez-vous de la lecture de cases isolées ?

Exercice 4.8 ★★★ — Le code de bienvenue retient-il ? (section 4.2.8)

Parmi les 2 000 clients inscrits depuis 2023 observables 180 jours, comparez la part qui repasse commande en 180 jours selon que la **première commande** utilisait un code promotionnel ou non. Quelle est la différence, avec son intervalle ? Peut-on dire que le code fait revenir ?

Exercice 4.9 ★★ — Les gros clients à risque (section 4.3.1)

Au 31 décembre 2025, listez les 10 clients de segment RFM « À risque (gros clients) » au plus fort chiffre d'affaires, avec leur récence. Combien d'euros de chiffre d'affaires annuel moyen (sur les trois ans) représentent-ils ensemble ? Que feriez-vous d'eux ?

Exercice 4.10 ★★★ — Un seuil de relance (section 4.3.3)

Une relance coûte 3 € et le client qui revient rapporte en moyenne 30 € de marge. (1) De combien de **points** la relance doit-elle augmenter la probabilité de revenir pour être rentable ? (2) À partir de la table « récence au 30 juin → probabilité de recommander » de la section 4.3.3, quelle est, pour chaque tranche de récence, la **marge de manœuvre maximale** ($1 - p$) ? (3) Peut-on, avec ces données seules, dire quelles tranches relancer ? Qu'ajoutez-vous pour décider ?

Exercice 4.11 ★★ — Où perd-on le plus d'argent ? (section 4.4.2)

Pour chaque source, la valeur d'un panier abandonné est le panier moyen des commandes de cette source multiplié par le nombre de paniers qui n'atteignent pas la commande. Classez les sources selon cette **valeur perdue**. Le classement est-il celui des taux d'abandon ?

Exercice 4.12 ★ — Trois phrases sous le tableau (section 4.4.4)

Calculez la part de clients actifs au trimestre suivant l'inscription (âge 1) pour chaque année d'inscription (2023, 2024, 2025, pour les cohortes observées à cet âge). Rédigez les **trois phrases** (le fait, la lecture, la limite) qui accompagneraient ce petit tableau.

Corrigés

Corrigé 4.1

On refait la table des douze derniers mois, avec deux seuils.

```
deb = 0.FIN - pd.Timedelta(days=365)
c12 = cmd[cmd["date_commande"] > deb].groupby("id_client").agg(n12=("id_commande", "size"),
    ↳ ca12=("ca", "sum"))
for seuil in (2, 3):
    r = c12[c12["n12"] >= seuil]
    print(f"au moins {seuil} commandes : ", len(r), "clients", round(len(r) / len(cli) * 100, 1), "%
    ↳ des 6 000 |", round(r["ca12"].sum() / c12["ca12"].sum() * 100, 1), "% du CA des 12 mois")
```

au moins 2 commandes : 2654 clients 44.2 % des 6 000 | 90.9 % du CA des 12 mois

au moins 3 commandes : 1848 clients 30.8 % des 6 000 | 78.6 % du CA des 12 mois

Lecture. Avec « au moins deux commandes », le groupe compte 2 654 clients (44,2 % des 6 000) et réalise 90,9 % du chiffre d'affaires des douze mois ; avec « au moins trois », 1 848 clients (30,8 %) et 78,6 %. Baisser le seuil ajoute 800 clients pour 12 points de chiffre d'affaires : le seuil est une **décision** (qui reçoit le traitement « fidèles » ?), pas une vérité. Les effectifs diffèrent de ceux du livre (1 726 réguliers) parce que les nouveaux inscrits ne sont pas retirés ici.

Corrigé 4.2

Moyennes 7,5 et 81,67 ; écarts-types 5,12 et 25,60. Après standardisation, les distances aux centres B et E donnent les affectations ; on vérifie par le code (les calculs à la main s'écrivent comme à la section 4.1.3).

```
P = pd.DataFrame({"commandes": [1, 3, 4, 10, 12, 15], "panier": [50, 70, 55, 90, 120, 105]},
    ↳ index=list("ABCDEF"))
Zp = ((P - P.mean()) / P.std(ddof=0)).values
cen = Zp[[1, 4]]
for it in range(3):
    lab = np.linalg.norm(Zp[:, None, :] - cen[None], axis=2).argmin(1)
    cen_new = np.array([Zp[lab == k].mean(0) for k in range(2)])
    print("itération", it + 1, "| affectations", lab.tolist(), "| centres", cen_new.round(2).tolist())
    if np.allclose(cen, cen_new):
        break
    cen = cen_new
print("KMeans :", KMeans(2, n_init=10, random_state=0).fit(Zp).labels_.tolist())
```

itération 1 | affectations [0, 0, 0, 1, 1, 1] | centres [[-0.94, -0.91], [0.94, 0.91]]

itération 2 | affectations [0, 0, 0, 1, 1, 1] | centres [[-0.94, -0.91], [0.94, 0.91]]

KMeans : [1, 1, 1, 0, 0, 0]

Lecture. Dès la première itération, les affectations sont {A, B, C} et {D, E, F} ; la seconde itération redonne les mêmes centres, (-0,94 ; -0,91) et (0,94 ; 0,91) : l'algorithme a **convergé en une itération**. KMeans retrouve les mêmes groupes (les numéros sont inversés, ce qui est sans importance).

Corrigé 4.3

```
d = pd.DataFrame({"rec": g["rec"], "n": g["n"]})
Zs = StandardScaler().fit_transform(d)
a = KMeans(3, n_init=10, random_state=0).fit(Zs).labels_
b = KMeans(3, n_init=10, random_state=0).fit(d.values).labels_
print("accord (indice de Rand ajusté) :", round(adjusted_rand_score(a, b), 2))
print("écarts-types sans standardisation :", d.std().round(1).to_dict())
```

```
print("moyenne de la récence et des commandes par segment, sans standardisation :")
print(d.assign(s=b).groupby("s").mean().round(1))
```

```
accord (indice de Rand ajusté) : 0.35
écarts-types sans standardisation : {'rec': 238.1, 'n': 7.9}
moyenne de la récence et des commandes par segment, sans standardisation :
      rec      n
s
0  780.8  1.6
1   57.5  9.7
2  342.7  3.6
```

Lecture. L'accord est de 0,35 seulement. Sans standardisation, la **récence domine** : son écart-type est de 238 jours contre 7,9 pour le nombre de commandes ; les trois groupes sont donc des tranches de récence (781, 58 et 343 jours en moyenne) et le nombre de commandes ne fait que les suivre (1,6 ; 9,7 ; 3,6).

Corrigé 4.4

```
t0 = pd.Timestamp("2025-06-30")
c0 = cmd[cmd["date_commande"] <= t0]
g0 = 0.table_clients(c0, lig[lig["id_commande"].isin(c0["id_commande"])], t0)
g0["seg"] = 0.nommer_segments(g0, KMeans(4, n_init=10,
↳ random_state=0).fit(StandardScaler().fit_transform(0.variables_kmeans(g0))).labels_)
h2 = cmd[cmd["date_commande"] > t0]
sh = h2.groupby("id_client").agg(n=("id_commande", "size"), promo=("promo", "sum"))
j = g0[["seg"]].join(sh).dropna()
print((j.groupby("seg")["promo"].sum() / j.groupby("seg")["n"].sum() *
↳ 100).round(1).sort_values(ascending=False))
```

```
seg
Chasseurs de promotions      15.9
Réguliers actifs             14.1
Dormants du Site             13.3
Dormants de la Boutique      12.5
dtype: float64
```

Lecture. Les « chasseurs de promotions » de juin passent **15,9 %** de leurs commandes du second semestre avec un code, contre 14,1 % pour les réguliers actifs, 13,3 % et 12,5 % pour les dormants : l'écart est de 2 à 3 points, très loin des 76 % qui avaient fait leur nom. L'étiquette tient mal : elle reposait sur deux ou trois commandes par client, et une part de 76 % sur trois commandes se produit facilement par hasard (un code **soldes** suffit aux soldes). Réserve : la part de commandes avec code dépend aussi du **calendrier** (les soldes) et de la carte de fidélité (code réservé), pas seulement du goût du client pour les rabais.

Corrigé 4.5

À l'âge 0, trois clients sur six commandent (1, 4, 6) : 50 % ; à l'âge 1, trois (1, 2, 6) : 50 % ; à l'âge 2, trois (2, 5, 6) : 50 % ; à l'âge 3, trois (2, 4, 6) : 50 %. Le client 3 n'a jamais commandé : 1/6, soit 17 %.

```
trim = {1: [0, 1], 2: [1, 2, 3], 3: [], 4: [0, 3], 5: [2], 6: [0, 1, 2, 3]}
ret = [sum(a in v for v in trim.values()) / len(trim) for a in range(4)]
print("rétention aux âges 0 à 3 :", [round(x * 100) for x in ret], "| jamais commandé :",
↳ round(sum(len(v) == 0 for v in trim.values()) / len(trim) * 100), "%")
```

rétenition aux âges 0 à 3 : [50, 50, 50, 50] | jamais commandé : 17 %

Lecture. Chaque âge compte trois clients actifs sur six, soit 50 % ; le client 3 n'a jamais commandé (1 sur 6, soit 17 %). Remarquez que la rétenition est constante à 50 % alors que les clients sont **différents** d'un âge à l'autre : la matrice donne une part, pas des personnes qui restent.

Corrigé 4.6

```
anc = cli[cli["date_inscription"] < "2023-01-01"]["id_client"]
nou = cli[cli["date_inscription"] >= "2023-01-01"]
cq = cmd.assign(per=cmd["date_commande"].dt.to_period("Q"))
res = {}
for nom, ids in [("anciens (avant 2023)", set(anc))]:
    a = cq[cq["id_client"].isin(ids)].groupby("per")["id_client"].nunique() / len(ids)
    res[nom] = a
print((pd.DataFrame(res) * 100).round(1).T.to_string())
print("nouveaux, âge ≥ 1 :", (0.activite_par_période(cli, cmd) * 100).round(1).values.tolist())
```

per	2023Q1	2023Q2	2023Q3	2023Q4	2024Q1	2024Q2	2024Q3	2024Q4	2025Q1	2025Q2
↪ 2025Q3 2025Q4										
anciens (avant 2023)	34.4	37.1	35.0	44.7	32.6	34.5	32.6	42.2	30.6	33.8
↪ 30.9 41.8										
nouveaux, âge ≥ 1 :	[39.5, 38.0, 42.7, 32.7, 34.0, 32.5, 41.8, 31.8, 34.4, 33.9, 40.7]									

Lecture. Les anciens clients montrent la **même saisonnalité** : 42 à 45 % d'actifs chaque quatrième trimestre contre 31 à 37 % aux autres trimestres, comme les nouveaux. L'effet de période n'est donc pas propre aux nouveaux clients : c'est la saison. On note aussi des niveaux plus élevés en 2023 qu'en 2025 (par exemple 37,1 % au deuxième trimestre de 2023 contre 33,8 % en 2025) : la dilution décrite à la section 4.2.8.

Corrigé 4.7

```
eff, taux, _ = 0.matrice_cohortes(cli, cmd, pas="Q")
n = int(eff.iloc[0])
for age in (3, 8):
    p = taux.iloc[0][age]
    lo, hi = 0.ic_proportion(p * n, n)
    print(f"âge {age} : {p * 100:.0f} %, intervalle à 95 % de {lo * 100:.0f} % à {hi * 100:.0f} %")
```

âge 3 : 50 %, intervalle à 95 % de 42 % à 57 %
 âge 8 : 26 %, intervalle à 95 % de 20 % à 34 %

Lecture. Les deux intervalles (42 à 57 % et 20 à 34 %) ne se chevauchent pas : la différence est probablement réelle. Mais l'âge 3 de cette cohorte tombe au quatrième trimestre de 2023 (la saison forte) et l'âge 8 au premier trimestre de 2025 (la saison faible) : c'est un effet de **période**, pas d'âge. Une case isolée ne se lit jamais seule, même quand elle est « significative ».

Corrigé 4.8

```
new = cli[cli["date_inscription"] >= "2023-01-01"]
n = cmd[cmd["id_client"].isin(new["id_client"])].sort_values("date_commande")
f = n.groupby("id_client").nth(0).set_index("id_client")
s = n.groupby("id_client").nth(1).set_index("id_client")
d = pd.DataFrame({"prem": f["date_commande"], "promot": f["promo"], "sec":
    ↪ s["date_commande"]}).query("prem <= '2025-07-04'")
d["retour"] = ((d["sec"] - d["prem"]).dt.days <= 180).fillna(False)
```

```
t = d.groupby("promol")["retour"].agg(["size", "sum", "mean"])
print(t.round(3))
p1, p0 = t.loc[True, "mean"], t.loc[False, "mean"]
se = np.sqrt(p1 * (1 - p1) / t.loc[True, "size"] + p0 * (1 - p0) / t.loc[False, "size"])
print("écart :", round((p1 - p0) * 100, 1), "points, intervalle à 95 % de", round((p1 - p0 - 1.96 *
↪ se) * 100, 1), "à", round((p1 - p0 + 1.96 * se) * 100, 1))
```

```
      size  sum  mean
promol
False    794  484  0.610
True     332  230  0.693
écart : 8.3 points, intervalle à 95 % de 2.3 à 14.3
```

Lecture. 69,3 % des nouveaux clients dont la première commande utilisait un code reviennent en 180 jours, contre 61,0 % des autres : un écart de 8,3 points, d'intervalle 2,3 à 14,3 (il exclut zéro). Mais on ne peut pas dire que le code fait revenir : les codes ne sont pas distribués au hasard (soldes, bienvenue, fidélité) et les clients qui en utilisent un ne ressemblent pas aux autres. Il faudrait un test A/B : donner le code au hasard à la moitié des nouveaux clients.

Corrigé 4.9

```
s = 0.rfm(g)
s["seg"] = [0.nom_segment_rfm(r, f) for r, f in zip(s["R"], s["F"])]
r = g.join(s)[lambda d: d["seg"] == "À risque (gros clients)"].sort_values("ca",
↪ ascending=False).head(10)
print(r[["n", "ca", "rec"]].round(0).astype(int))
print("CA annuel moyen de ces 10 clients (sur 3 ans) :", round(r["ca"].sum() / 3), "€")
```

```
      n   ca  rec
id_client
4154   35 4162  181
12     32 3131  156
798    22 2690  164
3900   24 2519  147
2377   14 2494  418
2882   24 2484  187
1735   20 2334  156
3113   15 2254  404
1900   20 2151  161
3393   17 2067  266
CA annuel moyen de ces 10 clients (sur 3 ans) : 8762 €
```

Lecture. Ces dix clients ont passé de 14 à 35 commandes, pour un chiffre d'affaires de 2 067 à 4 162 € chacun, et leur dernière commande a 147 à 418 jours : ensemble, 8 762 € par an. À ce niveau, un appel personnel se justifie même avec une faible chance de retour. Avant d'appeler, regardez le **rythme habituel** de chaque client (le client 2377 est silencieux depuis 418 jours, alors que le client 4154 l'est depuis 181 jours pour 35 commandes) et vérifiez le **consentement** à être contacté.

Corrigé 4.10

- (1) Il y a rentabilité quand $\Delta p \times 30 - 3 > 0$, soit $\Delta p > 3/30 = 10$ points, **quelle que soit** la tranche. (2) Une relance ne peut pas faire gagner plus que $1 - p$: 22 points pour les clients récents, 64 points pour les plus silencieux. (3) Non : ces données disent **combien** de clients reviennent, pas **combien reviennent à cause de la relance**. Les clients récents reviennent déjà à 78 % : le gain possible est faible, et le seuil de 10 points est difficile à atteindre ; les silencieux ont plus de marge, mais rien ne garantit qu'une relance les fasse revenir. Il faut un **test A/B** (chapitre 2, section 2.2) : relancer un échantillon au hasard dans chaque tranche, comparer au groupe non relancé, et chiffrer le gain en points avec son intervalle.

```
rr = 0.reachat(cmd, "2025-06-30")
rr["groupe"] = pd.cut(rr["rec"], [-1, 30, 90, 180, 365, 2000], labels=["0-30 j", "31-90 j", "91-180
↪ j", "181-365 j", "plus de 365 j"])
p = rr.groupby("groupe")["reachte"].mean()
print("seuil de rentabilité :", 3 / 30 * 100, "points")
print(pd.DataFrame({"p_sans_relance_%": (p * 100).round(1), "gain_maximal_points": ((1 - p) *
↪ 100).round(1)}))
```

```
seuil de rentabilité : 10.0 points
      p_sans_relance_%  gain_maximal_points
groupe
0-30 j                77.6                22.4
31-90 j               74.3                25.7
91-180 j              67.1                32.9
181-365 j             53.6                46.4
plus de 365 j         36.2                63.8
```

Corrigé 4.11

```
src = 0.entonnoir(sess, "source")
pm = cmd[cmd["canal"] == "Site"].groupby(sess.loc[sess["commande"] ==
↪ 1].set_index("id_commande")["source"].reindex(cmd.loc[cmd["canal"] == "Site",
↪ "id_commande"]).values)["ca"].mean()
src["pm"] = pm
src["perdu"] = (src["ajout_panier"] - src["commande"]) * src["pm"]
src["abandon"] = 1 - src["commande"] / src["ajout_panier"]
print(src[["ajout_panier", "pm", "abandon", "perdu"]].round(2).sort_values("perdu", ascending=False))
```

	ajout_panier	pm	abandon	perdu
source				
organique	5802	106.03	0.70	431126.44
direct	5885	100.74	0.58	344332.55
payant	2235	98.38	0.76	167239.45
reseaux	1808	105.46	0.82	155971.61
email	1585	96.21	0.51	78221.42
referent	801	98.38	0.70	55288.64

Lecture. La valeur perdue la plus forte est celle de la source **organique** (4 066 paniers non convertis, 431 126 € de paniers), devant le direct et la publicité ; le taux d'abandon le plus élevé est celui des **réseaux** (82 %), le plus faible celui de l'e-mail (51 %). Les deux classements ne sont pas les mêmes : le volume pèse autant que le taux. Précaution : ces montants sont des **paniers**, pas des ventes manquées (beaucoup de paniers ne seraient jamais devenus des commandes).

Corrigé 4.12

```
eff, taux, _ = 0.matrice_cohortes(cli, cmd, pas="Q")
t1 = taux[1].dropna() * 100
an = pd.Series([str(c)[:4] for c in t1.index], index=t1.index)
print(pd.DataFrame({"cohorte": t1.round(0), "an": an}).groupby("an")["cohorte"].agg(["size",
↪ "mean"]).round(1))
```

	size	mean
an		
2023	4	35.5
2024	4	35.2
2025	3	37.3

Lecture. Les trois phrases pourraient être : Pour les cohortes de 2023 (4 cohortes), 2024 (4) et 2025 (3 observées), 35,5 %, 35,2 % et 37,3 % des clients commandent au trimestre qui suit leur inscription. Le niveau est stable d'une année à l'autre ; les écarts sont du même ordre que le bruit attendu pour des cohortes de 150 à 190 clients. Ce tableau ne dit rien du comportement ultérieur ni de la saison : les cohortes de 2025 sont observées à des trimestres civils différents de celles de 2023.

Pistes des applications

Les « À vous » des applications n'ont pas de corrigé détaillé : voici une piste pour chacun, avec les ordres de grandeur à retrouver.

- **Application 4.1.** La carte de fidélité ne va pas avec plus de commandes : 5,0 contre 4,9 (clients de la Boutique), 10,3 contre 10,5 (mixtes), 2,9 contre 3,0 (Site). Et même si elle y allait, on ne pourrait pas conclure que la carte les **fait** commander : elle n'est pas distribuée au hasard. Remarquez aussi que la catégorie « Mixte » (10 commandes) est mécaniquement celle des gros acheteurs : un client qui n'a passé qu'une commande ne peut pas être mixte.
- **Application 4.2.** Avec trois actions, $k = 3$ convient : silhouette 0,264 (contre 0,276 pour $k = 4$), plus petit segment de 1 189 clients, accord de 0,82 avec $k = 4$. On perd l'isolement d'un petit groupe (392 clients) dont l'étiquette, on l'a vu, est fragile. $k = 5$ est moins bon (0,213) et s'accorde peu avec $k = 4$ (0,50).
- **Application 4.3.** Les règles séparent **mieux** que les k -moyennes : 85,4 % de recommandes chez les réguliers contre 36,2 % chez les endormis (49 points), contre 83,3 % et 43,5 % pour les k -moyennes (40 points). Les règles utilisent directement la fréquence et la récence, qui commandent le réachat ; les k -moyennes ajoutent le canal et les promotions, qui ne prédisent presque rien. Une méthode plus sophistiquée ne prédit pas mieux : elle sert à **découvrir** des profils.
- **Application 4.4.** Les cohortes mensuelles vont de 5 à 33 % à l'âge 1 (écart-type 7,2 points), les trimestrielles de 30 à 41 % (3,6). Le hasard seul donnerait 4,9 points pour une proportion de 16 % sur 57 clients et 3,7 points pour 36 % sur 170 : les cohortes trimestrielles varient comme le hasard le prévoit, les mensuelles davantage, parce que leur âge 1 tombe dans des mois civils très différents (décembre contre l'été).
- **Application 4.5.** La moyenne (252 € pour les inscrits de 2023, 240 € pour ceux de 2024) est deux fois la médiane (130 € et 126 €) : 31 % des clients inscrits ne rapportent **rien** en douze mois, et les 10 % meilleurs rapportent 40 % du total. Fixer un budget d'acquisition sur 250 € surestimerait ce que rapporte la moitié des clients.
- **Application 4.6.** Les quintiles sur valeurs brutes donnent des groupes de 1 410, 881, 596, 1 035 et 884 clients ; sur les rangs, des groupes égaux (961 ou 962). Parmi les 257 clients à risque (7,3 % du chiffre d'affaires), on appelle d'abord les plus gros (35 commandes, 4 162

€) mais il manque leur **rythme habituel**, la raison du silence (une livraison ratée ?) et le **consentement** à être contactés.

- **Application 4.7.** Les réguliers actifs ont la plus grande valeur vie (rétention de 93 %, marge de 157 € par client actif, valeur de 1 142 €) contre 198 à 235 € pour les trois autres segments. Elle est **très sensible** à la rétention : avec 0,92 la valeur tombe à 1 059 €, avec 0,95 elle monte à 1 303 €. Les rétentions des petits segments reposent sur 345 à 692 clients seulement.
- **Application 4.8.** Selon l'appareil, les conversions par source varient peu, sauf pour de petites cases (par exemple la publicité sur tablette, 4,3 %, repose sur très peu de sessions). En nombre, la source organique perd le plus de paniers et de paiements (2 611 et 1 455) ; en proportion, ce sont les réseaux. Les deux classements diffèrent.

Chapitre 5 : Séries temporelles et analyse de tendance — exercices et applications

Ce chapitre du cahier prolonge le chapitre 5 du livre. Les **applications** sont de petites études guidées sur le chiffre d'affaires de la boutique (la série quotidienne de 2023 à 2025, ses incidents, ses canaux) ; vous les refaites pas à pas, puis vous répondez aux questions « À vous ». Les **exercices** sont notés ★ (direct), ★★ (demande de la réflexion), ★★★ (étude plus longue). Les **corrigés** donnent le calcul à la main quand il existe, puis le code. Les données sont **simulées** et leur vérité est programmée (voir le livre).

Une première cellule charge les bibliothèques et les données ; les cellules suivantes reprennent les noms ainsi définis.

```
import os, sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import statsmodels.api as sm, statsmodels.formula.api as smf
import outils_ch05 as O

j, m = O charger() # j : chiffre d'affaires TTC quotidien (et
↳ commandes) ; m : chiffre d'affaires mensuel
y = j["chiffre_affaires"]
lig, cmd = O lire("lignes_commande.csv"), O lire("commandes.csv")
print(len(j), "jours,", len(m), "mois | CA 2025 :", round(y["2025"].sum(), 2), "€")
```

```
1096 jours, 36 mois | CA 2025 : 1324764 €
```

Applications

Application 5.1 — Composantes et comparaison à l'an dernier (sections 5.1.1 à 5.1.3)

Objectif. Comparer le chiffre d'affaires de 2025 à celui de 2024 de trois façons, et voir pourquoi deux d'entre elles trompent.

Étape 1 — Mois sur mois et variation annuelle. On calcule, pour chaque mois de 2025, la variation par rapport au mois précédent et par rapport au même mois de 2024.

```
mom = (m.pct_change() * 100)["2025"] # mois sur mois
yoy = (m["2025"].values / m["2024"].values - 1) * 100 # même mois, an dernier
```

```
print("mois sur mois : de", round(mom.min(), 1), "à", round(mom.max(), 1), "% | variation annuelle :  
↪ de", round(yoy.min(), 1), "à", round(yoy.max(), 1), "%")
```

```
mois sur mois : de -43.3 à 27.8 % | variation annuelle : de -1.2 à 18.5 %
```

Étape 2 — Le glissement annuel. On somme les douze derniers mois et on compare à la somme des douze mois précédents.

```
r12 = m.rolling(12).sum()  
gliss = ((r12 / r12.shift(12) - 1) * 100).dropna()  
print(gliss.iloc[[0, 6, 11]].round(1).to_string())
```

```
date  
2024-12-01    4.4  
2025-06-01    5.0  
2025-11-01    9.2
```

Lecture. Le mois sur mois va de -43,3 % à 27,8 % : il mesure surtout la saison. La variation annuelle va de -1,2 % à 18,5 % : la saison est écartée, mais le bruit mensuel reste. Le glissement annuel, lui, est lisible et montre l'accélération : 4,4 %, 5,0 % puis 9,2 % aux dates 12/2024, 06/2025 et 11/2025.

À vous. Refaites l'étape 2 pour le seul canal Site (chiffre d'affaires mensuel : `0.ca_par_canal()["Site"]`). Le glissement annuel du Site à fin 2025 est-il supérieur ou inférieur à celui de l'ensemble ? De combien ?

Application 5.2 — Indices saisonniers et décomposition (sections 5.1.4 et 5.1.5)

Objectif. Calculer à la main les indices saisonniers trimestriels, puis les comparer à ceux de `seasonal_decompose`.

Étape 1 — La moyenne mobile centrée sur quatre trimestres. Pour des trimestres, la moyenne centrée prend 0,5 fois les extrêmes, 1 fois les trois du milieu, le tout divisé par 4.

```
q = m.resample("QS").sum()  
ma = (0.5 * q.shift(2) + q.shift(1) + q + q.shift(-1) + 0.5 * q.shift(-2)) / 4  
rap = (q / ma).dropna()  
print(rap.round(3).to_string())
```

```
date  
2023-07-01    0.930  
2023-10-01    1.312  
2024-01-01    0.769  
2024-04-01    1.001  
2024-07-01    0.920  
2024-10-01    1.279  
2025-01-01    0.806  
2025-04-01    0.959  
Freq: QS-JAN
```

Étape 2 — Moyenner par trimestre et normaliser.

```
par_t = rap.groupby(rap.index.quarter).mean()  
iq = par_t / par_t.mean()  
print(iq.round(3).to_dict(), "| moyenne :", round(iq.mean(), 3))
```

```
{1: 0.79, 2: 0.983, 3: 0.928, 4: 1.299} | moyenne : 1.0
```

Étape 3 — Comparer à `seasonal_decompose` sur les mois. On regroupe ensuite les indices mensuels par trimestre.

```
from statsmodels.tsa.seasonal import seasonal_decompose
dm = seasonal_decompose(m, model="multiplicative", period=12)
idx_m = dm.seasonal.iloc[:12]; idx_m.index = range(1, 13)
print({t: round(idx_m[[3 * t - 2, 3 * t - 1, 3 * t]].mean(), 3) for t in range(1, 5)})
```

```
{1: np.float64(0.789), 2: np.float64(0.984), 3: np.float64(0.926), 4: np.float64(1.3)}
```

Lecture. Huit rapports (8) donnent les indices trimestriels 0,790, 0,983, 0,928 et 1,299 ; en regroupant les indices mensuels de `seasonal_decompose` par trimestre, on trouve 0,789, 0,984, 0,926 et 1,300 : les deux calculs racontent la même saison, à la différence près que les mois d'un même trimestre ont des indices différents (regrouper les mois perd du détail).

À vous. Ajoutez l'indice du mois de décembre à l'étape 3. Quel trimestre « perd » le plus de détail quand on regroupe les mois ?

Application 5.3 — Calendrier et tendance (sections 5.1.6 et 5.1.7)

Objectif. Mesurer l'effet du jour de semaine, puis la tendance du chiffre d'affaires désaisonnalisé avec son intervalle.

Étape 1 — Indice du jour de semaine.

```
dow = y.groupby(y.index.dayofweek).mean()
di = dow / dow.mean()
print(di.round(2).to_dict())
```

0 = lundi ... 6 = dimanche

```
{0: 0.97, 1: 0.88, 2: 0.93, 3: 0.99, 4: 1.17, 5: 1.39, 6: 0.66}
```

Étape 2 — Tendance avec erreurs robustes. On désaisonnalise par les indices mensuels, puis on ajuste une droite sur le logarithme.

```
idx = 0.indices_saisoniers(m)
des = m / idx.reindex(m.index.month).values
t = np.arange(len(m)) / 12
reg = sm.OLS(np.log(des.values), sm.add_constant(t)).fit(cov_type="HAC", cov_kws={"maxlags": 3})
print(round((np.exp(reg.params[1]) - 1) * 100, 1), "% par an ; intervalle :",
      ↪ np.round((np.exp(reg.conf_int()[1]) - 1) * 100, 1))
```

```
8.2 % par an ; intervalle : [ 6.4 10. ]
```

Étape 3 — La même régression, sans correction de l'autocorrélation. On compare la largeur des intervalles.

```
reg0 = sm.OLS(np.log(des.values), sm.add_constant(t)).fit()
print("largeur de l'intervalle : ordinaire", round(float(np.diff(reg0.conf_int()[1])[0]) * 100, 1),
      ↪ "| robuste", round(float(np.diff(reg.conf_int()[1])[0]) * 100, 1), "points de log %")
```

```
largeur de l'intervalle : ordinaire 3.1 | robuste 3.3 points de log %
```

Lecture. Le samedi pèse 1,39, le vendredi 1,17 et le dimanche 0,66 fois un jour moyen. La tendance est de 8,2 % par an (6,4 % à 10,0 %). L'intervalle de la régression ordinaire est large de 3,1 points, celui de la régression robuste de 3,3 : la correction de l'autocorrélation élargit l'intervalle, ce qui est honnête.

À vous. Refaites l'étape 2 sur le nombre de commandes mensuel plutôt que sur le chiffre d'affaires. La croissance est-elle plus proche de la vérité programmée (6 % par an) ? Pourquoi ?

Application 5.4 — Trouver les incidents (section 5.1.8)

Objectif. Détecter des jours anormaux, puis juger la détection avec la vérité programmée.

Étape 1 — Les doublons exacts.

```
ji = 0.jours(incidents=True)
print("dates en double :", ji.index[ji.index.duplicated()].strftime("%d/%m/%Y").tolist())
```

```
dates en double : ['20/10/2025']
```

Étape 2 — Le score z robuste, à plusieurs seuils. On compare les jours signalés aux incidents réels (la vérité n'est ouverte qu'ici).

```
z = 0.residus_robustes(ji)
v = 0.lire("verite_incidents.csv", parse_dates=["date"])
stat = set(v["date"]) - {pd.Timestamp("2025-10-20")} # le doublon est trouvé à l'étape 1
for s in (3, 3.5, 4):
    f = set(z[z.abs() > s].index)
    print(f"seuil {s} : {len(f)} signalés | vrais {len(f & stat)} | fausses alertes {len(f - stat)}")
```

```
seuil 3 : 18 signalés | vrais 7 | fausses alertes 11
seuil 3.5 : 8 signalés | vrais 4 | fausses alertes 4
seuil 4 : 3 signalés | vrais 2 | fausses alertes 1
```

Étape 3 — Corriger l'erreur de saisie. On remplace le jour le plus extrême par sa valeur attendue (la médiane des mêmes jours de semaine voisins) et l'on regarde l'effet sur le mois.

```
jour = z.abs().idxmax()
yi = ji[~ji.index.duplicated()][ "chiffre_affaires" ].asfreq("D")
voisins = yi[jour + pd.Timedelta(days=7 * k) for k in (-4, -3, -2, -1, 1, 2, 3, 4)]
yc = yi.copy(); yc[jour] = voisins.median()
print(jour.date(), "| valeur saisie :", round(yi[jour]), "€ | valeur corrigée :", round(yc[jour]), "€
↳ | CA du mois : avant", round(yi[jour.strftime("%Y-%m")].sum()), "après",
↳ round(yc[jour.strftime("%Y-%m")].sum()))
```

```
2025-09-09 | valeur saisie : 30983 € | valeur corrigée : 2773 € | CA du mois : avant 141337 après
↳ 113127
```

Lecture. Aux seuils 3, 3,5 et 4, on signale 18, 8 et 3 jours, dont 7, 4 et 2 vrais incidents (sur 7) : un seuil plus bas trouve plus d'incidents mais lève 11 fausses alertes au lieu de 1. Le jour le plus extrême est le 09/09/2025 (saisi 30 983 €, corrigé 2 773 €) : la correction ramène le mois de 141 337 € à 113 127 €.

À vous. Quelle valeur du seuil choisiriez-vous si une fausse alerte coûte 10 minutes de vérification et un incident manqué 2 heures ? Justifiez avec le tableau ci-dessus.

Application 5.5 — Moyennes mobiles et retard (section 5.2.1)

Objectif. Vérifier numériquement qu'une moyenne mobile arrière retarde de $(w - 1)/2$ jours, et mesurer l'effet du lissage.

Étape 1 — La moyenne arrière et la moyenne centrée.

```
d = y["2025-09-01":]
arriere = d.rolling(7).mean()
centree = d.rolling(7, center=True).mean()
dec = arriere.shift(-3).dropna()
print("moyenne arrière = moyenne centrée décalée de 3 jours :", np.allclose(dec,
↳ centree.loc[dec.index]))
```

```
moyenne arrière = moyenne centrée décalée de 3 jours : True
```

Étape 2 — La fenêtre et la variabilité. On compare l'écart-type du chiffre d'affaires lissé selon la fenêtre.

```
print({w: round(d.rolling(w).mean().std()) for w in (1, 7, 14, 28)})
```

```
{1: 1570, 7: 978, 14: 924, 28: 844}
```

Étape 3 — La moyenne exponentielle. On regarde l'âge moyen des données pour plusieurs α .

```
for a in (0.05, 0.15, 0.5):
    print("α =", a, "| âge moyen :", round((1 - a) / a, 1), "jours | écart-type :",
        ↳ round(d.ewm(alpha=a).mean().std()))
```

```
α = 0.05 | âge moyen : 19.0 jours | écart-type : 719
α = 0.15 | âge moyen : 5.7 jours | écart-type : 930
α = 0.5 | âge moyen : 1.0 jours | écart-type : 1144
```

Lecture. L'égalité est vérifiée (oui) : la moyenne arrière d'aujourd'hui est la moyenne centrée de **trois jours plus tôt**. Plus la fenêtre est longue, plus l'écart-type baisse : 1 570 € au jour, 978 € sur 7 jours, 924 € sur 14 jours, 844 € sur 28 jours ; la contrepartie est le retard.

À vous. Quelle fenêtre choisiriez-vous pour surveiller en temps réel le chiffre d'affaires d'une boutique ouverte tous les jours ? Pour détecter un début de promotion en moins de trois jours ?

Application 5.6 — Prévoir 2025 avec 2023-2024 (sections 5.2.2 et 5.2.3)

Objectif. Comparer six méthodes sur les douze mois de 2025 et mesurer l'effet d'une fuite d'information.

Étape 1 — Les six méthodes et leurs erreurs.

```
F, train, test = 0.previsions_mensuelles(m)
tab = 0.tableau_erreurs(F, train, test)
print(tab[["MAE", "MAPE %", "biais %"]].round(1).to_string())
```

méthode	MAE	MAPE %	biais %
naïve (dernier mois)	51330.5	52.8	42.5
naïve saisonnière	11487.8	10.2	-10.2
naïve saisonnière × croissance	8148.3	7.2	-6.2
moyenne des 12 derniers mois	20528.8	16.7	-10.2
tendance linéaire × indices	7792.3	7.0	-6.2
Holt-Winters	8019.8	7.1	-6.3

Étape 2 — La même méthode, avec fuite. On calcule les indices saisonniers sur les 36 mois au lieu de 24.

```
idx_tout = 0.indices_saisonniers(m)
b = np.polyfit(np.arange(24), (train /
    ↳ 0.indices_saisonniers(train).reindex(train.index.month).values).values, 1)
f_triche = np.polyval(b, np.arange(24, 36)) * idx_tout.reindex(test.index.month).values
print("MAPE honnête :", round(tab.loc["tendance linéaire × indices", "MAPE %"], 1), "| avec fuite :",
    ↳ round(0.mape(test, f_triche), 1))
```

```
MAPE honnête : 7.0 | avec fuite : 6.0
```

Étape 3 — Un intervalle de prévision. On prend le rapport réalisé/prévu des douze mois pour la méthode « tendance × indices » et l'on regarde sa dispersion.

```
r = (test.values / F["tendance linéaire × indices"])
print("rapport réalisé/prévu : de", round(r.min(), 2), "à", round(r.max(), 2), "| moyenne",
      ↪ round(r.mean(), 2))
```

```
rapport réalisé/prévu : de 0.95 à 1.14 | moyenne 1.07
```

Lecture. Les erreurs mensuelles (MAPE) sont de 10,2 % pour la naïve saisonnière, 7,2 % avec la croissance, 7,0 % pour tendance × indices et 7,1 % pour Holt-Winters ; le biais commun est de -6,2 %. La fuite ramène l'erreur de tendance × indices à 6,0 %. Le rapport réalisé/prévu va de 0,95 à 1,14 (moyenne 1,07) : l'intervalle qu'on en tirerait serait **décentré** (le réalisé est presque toujours au-dessus du prévu), signe d'un biais.

À vous. Ajoutez à la liste une méthode « moyenne des deux dernières années, mois par mois » et comparez son MAPE aux autres.

Application 5.7 — Plusieurs origines et intervalle (sections 5.2.4 et 5.2.5)

Objectif. Comparer deux méthodes quotidiennes à 48 origines, puis calibrer et vérifier un intervalle.

Étape 1 — Les 48 origines.

```
mae_o, tot_o = 0.origines(y)
print(pd.DataFrame({"MAE par jour": mae_o.mean(), "erreur absolue 28 j (%)":
  ↪ tot_o.abs().mean()}).round(1).to_string())
```

	MAE par jour	erreur absolue 28 j (%)
naïve saisonnière 7 j	891.5	10.4
moyenne des 4 mêmes jours	802.3	11.7
même jour l'an dernier	872.0	9.8
an dernier × niveau récent	897.1	7.2
Holt-Winters (7 j)	717.6	10.9

Étape 2 — Une différence appariée avec son incertitude. Pour comparer « an dernier × niveau » et Holt-Winters, on prend la différence des erreurs **absolues** à chaque origine, puis on estime l'incertitude par un bootstrap **par blocs** de 4 origines consécutives (les origines voisines se chevauchent).

```
dif = (tot_o["Holt-Winters (7 j)"].abs() - tot_o["an dernier × niveau récent"].abs()).values
rng = np.random.default_rng(0)
blocs = [dif[i:i + 4] for i in range(0, len(dif), 4)]
boot = [np.concatenate([blocs[k] for k in rng.integers(0, len(blocs), len(blocs))]).mean() for _ in
  ↪ range(3000)]
print("différence moyenne :", round(dif.mean(), 1), "points ; intervalle à 95 % :",
      ↪ np.round(np.percentile(boot, [2.5, 97.5]), 1))
```

```
différence moyenne : 3.7 points ; intervalle à 95 % : [0.9 6.7]
```

Étape 3 — La couverture d'un intervalle à 80 %.

```
r = 1 / (1 + tot_o["an dernier × niveau récent"] / 100)
bas, haut = r.iloc[:24].quantile([0.10, 0.90])
print("intervalle", round(bas, 2), "à", round(haut, 2), "| couverture sur la seconde moitié :",
      ↪ round(r.iloc[24:].between(bas, haut).mean() * 100), "%")
```

```
intervalle 0.89 à 1.1 | couverture sur la seconde moitié : 79 %
```

Lecture. « An dernier × niveau » a une MAE quotidienne de 897 € et Holt-Winters de 718 € ; sur les totaux de 28 jours, c'est l'inverse (7,2 % contre 10,9 %). La différence moyenne d'erreur

(Holt-Winters moins l'autre) est de 3,7 points, avec un intervalle de 0,9 à 6,7 : **zéro n'est pas dans l'intervalle**, la méthode saisonnière est donc réellement meilleure sur les totaux. L'intervalle à 80 % (0,89 à 1,10 fois la prévision) a une couverture de 79 % sur la seconde moitié.

À vous. Refaites l'étape 2 en comparant la naïve saisonnière de 7 jours à « la moyenne des 4 mêmes jours ». La différence est-elle significative ?

Application 5.8 — Régression avec indicatrices (section 5.3.1)

Objectif. Estimer l'effet de la promotion, de la publicité et de la tendance sur les commandes, puis comparer à la vérité programmée.

Étape 1 — Préparer les variables.

```
dj = j.reset_index().assign(mois=lambda x: x["date"].dt.month, jds=lambda x: x["date"].dt.dayofweek,
    ↪ t=lambda x: (x["date"] - x["date"].min()).dt.days / 365.25)
dj["pub7"] = dj["depense_pub"].rolling(7, min_periods=1).sum() / 1000
tr5, te5 = dj[dj["date"] < "2025-01-01"], dj[dj["date"] >= "2025-01-01"]
```

Étape 2 — Ajuster et lire les effets.

```
mod = smf.ols("np.log(nb_commandes) ~ C(mois) + C(jds) + promo_active + pub7 + t",
    ↪ tr5).fit(cov_type="HAC", cov_kws={"maxlags": 7})
ic = (np.exp(mod.conf_int()) - 1) * 100
for c in ("promo_active", "pub7", "t"):
    print(c, round((np.exp(mod.params[c]) - 1) * 100, 1), "% ; intervalle", np.round(ic.loc[c].values,
    ↪ 1))
```

```
promo_active 21.7 % ; intervalle [14.8 29. ]
pub7 -0.8 % ; intervalle [-6.4 5.2]
t 4.7 % ; intervalle [2.2 7.3]
```

Étape 3 — Prévoir 2025 et mesurer l'erreur.

```
pred = np.exp(mod.predict(te5)) * np.exp(mod.mse_resid / 2)
mens = pd.DataFrame({"réel": te5.set_index("date")["nb_commandes"], "prévu":
    ↪ pred.values}).resample("MS").sum()
print("MAPE mensuelle :", round(0.mape(mens["réel"], mens["prévu"]), 1), "% ; MAE par jour :",
    ↪ round(0.mae(te5["nb_commandes"], pred), 1), "commandes")
```

```
MAPE mensuelle : 4.4 % ; MAE par jour : 4.3 commandes
```

Lecture. La promotion augmente les commandes de 21,7 % (14,8 à 29,0 %), la vérité étant de +18 % : retrouvée. La publicité donne -0,8 % (-6,4 à 5,2 %) par millier d'euros sur 7 jours : indiscernable de zéro, alors que l'effet programmé est de +1,5 %. La tendance est de 4,7 % par an pour 6 % programmés. L'erreur de prévision de 2025 est de 4,4 % par mois et de 4,3 commandes par jour.

À vous. Ajoutez la température comme variable explicative. Son effet est-il significatif ? Retrouvez-on l'idée que la température agit sur le **mélange** des produits, pas sur le nombre de commandes ?

Application 5.9 — Décembre prochain et planification (sections 5.3.5 et 5.3.6)

Objectif. Construire trois scénarios pour décembre 2026, puis en déduire un besoin en personnel et en stock.

Étape 1 — Trois scénarios de croissance.

```
dec25 = m["2025-12-01"]
scen = {"prudent": dec25 * 1.044, "central": dec25 * 1.065, "hausse de prix": dec25 * 1.065 * 1.03}
print({k: round(v) for k, v in scen.items()})
```

```
{'prudent': 191934, 'central': 195795, 'hausse de prix': 201669}
```

Étape 2 — Du chiffre d'affaires aux commandes et au personnel. Avec un panier moyen de 100 € et 20 commandes préparées par personne et par jour (hypothèse illustrative), on dimensionne la journée de pointe.

```
dow = y.groupby(y.index.dayofweek).mean(); di = dow / dow.mean()
for k, v in scen.items():
    cmd_j = v / 100 / 31
    print(f"{k:15s} commandes/jour : {cmd_j:5.1f} | samedi : {cmd_j * di[5]:5.1f} | personnes le
    ↪ samedi : {int(np.ceil(cmd_j * di[5] / 20))}")
```

```
prudent      commandes/jour : 61.9 | samedi : 86.0 | personnes le samedi : 5
central      commandes/jour : 63.2 | samedi : 87.8 | personnes le samedi : 5
hausse de prix commandes/jour : 65.1 | samedi : 90.4 | personnes le samedi : 5
```

Étape 3 — Le stock de sécurité d'un produit. Pour le deuxième produit le plus vendu en novembre et décembre 2025, avec un délai de 14 jours et un niveau de service de 95 % ($z = 1,65$) :

```
x = lig.merge(cmd[["id_commande", "date_commande"]], on="id_commande"); x = x[x["date_commande"] >=
    ↪ "2025-11-01"]
p2 = x.groupby("id_produit")["quantite"].sum().sort_values(ascending=False).index[1]
dem = x[x["id_produit"] ==
    ↪ p2].groupby("date_commande")["quantite"].sum().reindex(pd.date_range("2025-11-01",
    ↪ "2025-12-31").strftime("%Y-%m-%d"), fill_value=0)
ss = 1.65 * dem.std() * np.sqrt(14)
print("produit", p2, "| demande moyenne :", round(dem.mean(), 2), "par jour | stock de sécurité :",
    ↪ round(ss), "| stock au moment de commander :", round(dem.mean() * 14 + ss))
```

```
produit 42 | demande moyenne : 3.57 par jour | stock de sécurité : 16 | stock au moment de commander :
    ↪ 66
```

Lecture. Les trois scénarios donnent 191 934 €, 195 795 € et 201 669 € pour décembre 2026. Un samedi de pointe compte environ 86, 88 ou 90 commandes, soit 5, 5 et 5 personnes. Pour le produit 42 (demande moyenne de 3,57 par jour), le stock de sécurité est de 16 unités et le stock au moment de commander de 66.

À vous. Quel scénario retiendriez-vous pour le recrutement d'intérimaires ? Et pour la commande de stock ? Une même prévision ne sert pas forcément à deux décisions de la même façon : pourquoi ?

Exercices

Exercice 5.1 ★ — Le mois court (section 5.1.2)

Un mois de 28 jours a rapporté 70 000 € ; le mois précédent, de 31 jours, a rapporté 77 000 €. De combien le chiffre d'affaires **par jour** a-t-il varié ? Que dire de la baisse du chiffre d'affaires mensuel ?

Exercice 5.2 ★ — Mois sur mois ou an sur an ? (section 5.1.3)

Les chiffres d'affaires mensuels de deux années sont, en k€ : année 1 : 60, 70, 100 ; année 2 : 66, 77, 108 (janvier, février, mars). Calculez la variation annuelle de chaque mois, puis la variation annuelle du trimestre, et la moyenne des trois variations mensuelles. Laquelle des deux dernières est la bonne, et pourquoi ?

Exercice 5.3 ★★ — Indices saisonniers à la main (section 5.1.4)

Voici douze trimestres de ventes (en k€), sur trois ans : année 1 : 80, 100, 90, 130 ; année 2 : 88, 108, 99, 143 ; année 3 : 97, 119, 109, 157. Calculez à la main (puis vérifiez par le code) la moyenne mobile centrée sur quatre trimestres pour les trimestres où elle existe, les rapports, les indices trimestriels normalisés, et la série désaisonnalisée de l'année 3.

Exercice 5.4 ★★ — Additif ou multiplicatif ? (section 5.1.5)

Pour le canal **Réseaux**, calculez le rapport et la différence entre décembre et février, pour 2023, 2024 et 2025. La saison du canal est-elle plutôt additive ou multiplicative ? Comparez à la conclusion du livre pour l'ensemble.

Exercice 5.5 ★★ — La croissance des commandes (section 5.1.6)

Estimez la croissance annuelle du **nombre de commandes** mensuel (série désaisonnalisée, logarithme, erreurs robustes) et donnez son intervalle à 95 %. Contient-il la vérité programmée de 6 % ? Pourquoi l'estimation est-elle plus proche de 6 % que celle du chiffre d'affaires ?

Exercice 5.6 ★★ — Choisir un seuil selon le coût (section 5.1.8)

Une fausse alerte coûte 1 unité (une vérification inutile) et un incident manqué en coûte 5. Parmi les seuils 3, 3,5 et 4 du score z robuste, lequel minimise le coût total sur les sept incidents statistiques de `jours_incidents.csv` ? Et si l'incident manqué ne coûte que 2 unités ?

Exercice 5.7 ★ — Lissage exponentiel à la main (section 5.2.1)

Avec $\alpha = 0,3$ et les valeurs 100, 120, 90, 110, calculez à la main les quatre valeurs lissées (avec $s_1 = y_1$), puis vérifiez avec `pandas`. Quel est l'âge moyen des données ?

Exercice 5.8 ★★ — MAE, RMSE, MAPE (section 5.2.3)

Le réalisé est (200, 150, 100, 50) et la prévision (180, 160, 120, 40). Calculez à la main la MAE, la RMSE et le MAPE. Pourquoi le MAPE est-il le plus sensible à la valeur 50 ? Que devient chaque mesure si l'on remplace 40 par 5 ?

Exercice 5.9 ★★ — Prévoir le canal Site (section 5.2.2)

Pour le canal **Site**, prévoyez 2025 avec 2023-2024 par la naïve saisonnière, la naïve saisonnière $\times$ croissance et « tendance $\times$ indices », et comparez les MAPE. Quelle méthode se trompe le plus, et pourquoi (pensez à la croissance du canal) ?

Exercice 5.10 ★★★ — Comparer deux méthodes avec leur incertitude (sections 5.2.4 et 5.2.5)

À 24 origines (une toutes les deux semaines en 2025), comparez la naïve saisonnière de 7 jours et « la moyenne des 4 mêmes jours » sur le total de 28 jours. Donnez la différence moyenne d'erreur absolue et son intervalle de bootstrap par blocs. Peut-on dire laquelle est meilleure ?

Exercice 5.11 ★★ — L'effet de la pluie (section 5.3.1)

Ajoutez la pluie (`pluie_mm > 1`, indicatrice) à la régression de 5.3.1, séparément pour le nombre de commandes **total**. Son effet est-il significatif ? La vérité programmée est -8% pour la Boutique et $+5\%$ pour le Site : que devriez-vous attendre sur le total, et pourquoi l'analyse a-t-elle du mal à le voir ?

Exercice 5.12 ★★★ — Prévoir par canal et planifier (sections 5.3.3 à 5.3.6)

Prévoyez décembre 2026 pour chaque canal par « tendance × indices » ajustée sur les 36 mois, additionnez, et comparez à la prévision directe du total et aux trois scénarios de l'application 5.9. Quel canal pèse le plus dans la croissance prévue ? Commentez en trois phrases ce que vous diriez à la gérante.

Corrigés**Corrigé 5.1**

Par jour : $70\,000/28 = 2\,500$ € contre $77\,000/31 \approx 2\,484$ € : le chiffre d'affaires **par jour** est quasiment stable ($+0,6\%$), alors que le chiffre d'affaires mensuel baisse de $70/77 - 1 \approx -9,1\%$. La « baisse » vient presque entièrement de la durée du mois.

```
print(round(70000 / 28), round(77000 / 31), round((70000 / 28) / (77000 / 31) * 100 - 100, 1),
      ↪ round((70 / 77 - 1) * 100, 1))
```

```
2500 2484 0.6 -9.1
```

Corrigé 5.2

Variations mensuelles : $66/60 - 1 = 10\%$, $77/70 - 1 = 10\%$, $108/100 - 1 = 8\%$. Trimestre : $(66 + 77 + 108)/(60 + 70 + 100) - 1 = 251/230 - 1 \approx 9,1\%$. La moyenne des trois variations vaut $9,33\%$. La **bonne** réponse est la variation du trimestre ($9,1\%$) : on somme les montants avant de calculer le pourcentage ; la moyenne des pourcentages pèse de la même façon un petit mois et un grand mois.

```
a1, a2 = np.array([60, 70, 100.]), np.array([66, 77, 108.])
print((a2 / a1 - 1).round(3), round(a2.sum() / a1.sum() - 1, 4), round((a2 / a1 - 1).mean(), 4))
```

```
[0.1  0.1  0.08] 0.0913 0.0933
```

Corrigé 5.3

Moyennes centrées ($0,5 \cdot y(t-2) + y(t-1) + y(t) + y(t+1) + 0,5 \cdot y(t+2)$, divisé par 4) pour les trimestres 3 à 10. Par exemple pour le trimestre 3 : $(0,5 \times 80 + 100 + 90 + 130 + 0,5 \times 88)/4 = (40 + 100 + 90 + 130 + 44)/4 = 101,0$; rapport $90/101 = 0,891$. On fait de même pour les autres et l'on moyenne par position.

```
s = pd.Series([80, 100, 90, 130, 88, 108, 99, 143, 97, 119, 109, 157.],
             ↪ index=pd.period_range("2001Q1", periods=12, freq="Q"))
ma = (0.5 * s.shift(2) + s.shift(1) + s + s.shift(-1) + 0.5 * s.shift(-2)) / 4
rap = (s / ma).dropna(); par = rap.groupby(rap.index.quarter).mean(); ind = par / par.mean()
print(ma.dropna().round(1).tolist()); print(rap.round(3).tolist()); print(ind.round(3).to_dict())
print("année 3 désaisonnalisée :", (s.iloc[8:].values / ind.reindex([1, 2, 3, 4]).values).round(1))
```

```
[101.0, 103.0, 105.1, 107.9, 110.6, 113.1, 115.8, 118.8]
[0.891, 1.262, 0.837, 1.001, 0.895, 1.264, 0.838, 1.002]
{1: 0.839, 2: 1.003, 3: 0.894, 4: 1.265}
année 3 désaisonnalisée : [115.7 118.7 121.9 124.2]
```

Le premier rapport est 0,891 (moyenne centrée 101,0). Les indices trimestriels valent 0,839, 1,003, 0,894 et 1,265 ; la série désaisonnalisée de l'année 3 est : 115,7 ; 118,7 ; 121,9 ; 124,2 k€. La série montre alors une progression régulière, sans le profil en dents de scie de la saison.

Corrigé 5.4

```
cc = 0.ca_par_canal()["Réseaux"]
for an in (2023, 2024, 2025):
    print(an, "rapport déc/fév :", round(cc[f"{an}-12-01"] / cc[f"{an}-02-01"], 2), "| différence :",
          ↪ round(cc[f"{an}-12-01"] - cc[f"{an}-02-01"]))
```

```
2023 rapport déc/fév : 2.48 | différence : 10284
2024 rapport déc/fév : 2.18 | différence : 9075
2025 rapport déc/fév : 3.73 | différence : 14932
```

Rapports : 2,48, 2,18, 3,73 ; différences : 10 284 €, 9 075 €, 14 932 €. Sur un petit canal, le bruit mensuel est grand et ni le rapport ni la différence ne sont aussi stables que pour l'ensemble : la conclusion est **moins nette**. On garde le modèle multiplicatif par prudence (la saison s'applique à un niveau qui croît) mais on le dit : avec un seul mois de chaque type par an, on ne peut pas trancher sur un canal de cette taille.

Corrigé 5.5

```
cmm = j["nb_commandes"].resample("MS").sum()
ic_ = 0.indices_saisoniers(cmm); dc = cmm / ic_.reindex(cmm.index.month).values
r_ = sm.OLS(np.log(dc.values), sm.add_constant(np.arange(36) / 12)).fit(cov_type="HAC",
↪ cov_kws={"maxlags": 3})
print(round((np.exp(r_.params[1]) - 1) * 100, 1), np.round((np.exp(r_.conf_int()[1]) - 1) * 100, 1))
```

```
6.9 [5.6 8.2]
```

La croissance des commandes est de 6,9 % par an (5,6 % à 8,2 %) : l'intervalle contient bien 6 %. L'estimation est plus proche de la vérité que celle du chiffre d'affaires (8,2 %, qui contient aussi la hausse de prix de 2025 et les remises) parce que le nombre de commandes **ne subit pas** les prix.

Corrigé 5.6

```
cout = {}
for s in (3, 3.5, 4):
    f = set(z[z.abs() > s].index); manque = len(stat - f); fausses = len(f - stat)
    cout[s] = (fausses * 1 + manque * 5, fausses * 1 + manque * 2, fausses, manque)
print(cout)
```

```
{3: (11, 11, 11, 0), 3.5: (19, 10, 4, 3), 4: (26, 11, 1, 5)}
```

Avec un coût de 5 par incident manqué, les coûts totaux sont 11 (seuil 3), 19 (3,5) et 26 (4) : le meilleur seuil est 3. Avec un coût de 2, ils sont 11, 10 et 11 : le meilleur seuil est 3.5. Plus l'incident manqué coûte cher par rapport à la fausse alerte, plus on baisse le seuil.

Corrigé 5.7

$s_1 = 100$; $s_2 = 0,3 \times 120 + 0,7 \times 100 = 106$; $s_3 = 0,3 \times 90 + 0,7 \times 106 = 101,2$; $s_4 = 0,3 \times 110 + 0,7 \times 101,2 = 103,84$. Âge moyen : $(1 - \alpha)/\alpha = 0,7/0,3 \approx 2,3$ périodes.

```
print(pd.Series([100, 120, 90, 110]).ewm(alpha=0.3, adjust=False).mean().round(2).tolist(), round(0.7 / 0.3, 1))
```

```
[100.0, 106.0, 101.2, 103.84] 2.3
```

Corrigé 5.8

Erreurs absolues : 20, 10, 20, 10 $\rightarrow$ MAE = $60/4 = 15$. Carrés : 400, 100, 400, 100 $\rightarrow$ somme 1 000 $\rightarrow$ RMSE = $\sqrt{250} \approx 15,8$. Erreurs relatives : $20/200 = 10\%$, $10/150 \approx 6,7\%$, $20/100 = 20\%$, $10/50 = 20\% \rightarrow$ MAPE $\approx 14,2\%$. La valeur 50 est la plus petite : une erreur de 10 y pèse 20 %, autant que 20 sur la valeur 100. En remplaçant 40 par 5, l'erreur de ce point passe à 45 (90 %).

```
def mesures(r, p):
    r, p = np.array(r, float), np.array(p, float)
    return round(0.mae(r, p), 1), round(0.rmse(r, p), 1), round(0.mape(r, p), 1)
print(mesures([200, 150, 100, 50], [180, 160, 120, 40]), mesures([200, 150, 100, 50], [180, 160, 120, 5]))
```

```
(15.0, 15.8, 14.2) (23.8, 27.0, 31.7)
```

Avec 40, on trouve MAE 15,0, RMSE 15,8, MAPE 14,2 % ; avec 5, MAE 23,8, RMSE 27,0 et MAPE 31,7 % : la RMSE réagit plus que la MAE à la grosse erreur.

Corrigé 5.9

```
cs = 0.ca_par_canal()["Site"]
tr_s, te_s = cs[:"2024-12-01"], cs[:"2025-01-01":]
sn = 0.prevision_naive_saisonniere(tr_s, 12)
g = tr_s.iloc[-12:].sum() / tr_s.iloc[-24:-12].sum()
ti = 0.prevision_tendance_indices(cs, "2024-12-01").values
print({"naïve saisonnière": round(0.mape(te_s, sn), 1), "× croissance": round(0.mape(te_s, sn * g), 1), "tendance × indices": round(0.mape(te_s, ti), 1)}, "| croissance passée :", round((g - 1) * 100, 1), "%")
```

```
{'naïve saisonnière': 19.0, '× croissance': 8.1, 'tendance × indices': 10.5} | croissance passée : 19.0 %
```

Erreurs : 19,0 % (naïve saisonnière), 8,1 % (avec croissance) et 10,5 % (tendance × indices). La

croissance passée du Site (+19,0 %) est inférieure à celle de 2025 (+22,9 %) : toutes les méthodes sous-estiment, et la naïve saisonnière sans croissance, qui suppose un canal immobile, se trompe le plus. Ici, la méthode la plus simple qui ajoute la croissance passée (8,1 %) fait même mieux que la droite ajustée (10,5 %), parce que cette droite sous-estime une croissance déjà forte. Plus un canal change de rythme, plus la prévision simple se trompe.

Corrigé 5.10

```
H = 28; origs = pd.date_range("2025-01-07", "2025-12-02", freq="14D")
dif = []
for o in origs:
    fut = y[o + pd.Timedelta(days=1): o + pd.Timedelta(days=H)]
    if len(fut) < H:
        continue
    f, _ = 0.previsions_28j(y, o, H)
    dif.append(abs(f["naïve saisonnière 7 j"].sum() / fut.sum() - 1) - abs(f["moyenne des 4 mêmes
    ↪ jours"].sum() / fut.sum() - 1))
dif = np.array(dif) * 100
rng = np.random.default_rng(1)
bl = [dif[i:i + 3] for i in range(0, len(dif), 3)]
boot = [np.concatenate([bl[k] for k in rng.integers(0, len(bl), len(bl))]).mean() for _ in
    ↪ range(3000)]
print(len(dif), round(dif.mean(), 2), np.round(np.percentile(boot, [2.5, 97.5]), 2))
```

```
24 -3.11 [-10.77  2.61]
```

Sur 24 origines, la différence moyenne d'erreur absolue (naïve de 7 jours moins moyenne des 4 mêmes jours) est de -3,11 points, avec un intervalle de -10,77 à 2,61 : **zéro est dans l'intervalle**. La naïve de 7 jours fait en moyenne un peu mieux, mais avec si peu d'origines et des fenêtres qui se chevauchent, on ne peut pas départager ces deux méthodes ; on garde la plus simple.

Corrigé 5.11

```
dj["pluie"] = (dj["pluie_mm"] > 1).astype(int)
tr5, te5 = dj[dj["date"] < "2025-01-01"], dj[dj["date"] >= "2025-01-01"]
mp = smf.ols("np.log(nb_commandes) ~ C(mois) + C(jds) + promo_active + pub7 + t + pluie",
    ↪ tr5).fit(cov_type="HAC", cov_kws={"maxlags": 7})
icp = (np.exp(mp.conf_int().loc["pluie"]) - 1) * 100
print(round((np.exp(mp.params["pluie"]) - 1) * 100, 1), "% ; intervalle", np.round(icp.values, 1), "|
    ↪ p =", round(mp.pvalues["pluie"], 3))
```

```
-1.6 % ; intervalle [-4.3  1.2] | p = 0.257
```

L'effet estimé de la pluie sur le total est de -1,6 % (intervalle -4,3 % à 1,2 %, $p = 0,26$) : non significatif. Attendu : la Boutique vend environ la moitié des commandes et perd 8 %, le Site, environ 40 %, gagne 5 % : l'effet net est de l'ordre de -2 % ($\approx 0,5 \times (-8) + 0,4 \times (+5)$), **trop petit** pour émerger du bruit quotidien (± 24 %). Pour voir l'effet, il faut séparer les canaux : l'effet a des signes opposés selon le canal, et leur somme s'annule presque.

Corrigé 5.12

```
cc = 0.ca_par_canal()
f26 = {c: 0.prevision_tendance_indices(cc[c], "2025-12-01")["2026-12-01"] for c in cc}
direct = 0.prevision_tendance_indices(cc.sum(axis=1), "2025-12-01")["2026-12-01"]
print({c: round(v) for c, v in f26.items()}, "| somme :", round(sum(f26.values()), 1), "| direct :",
    ↪ round(direct))
print("croissance prévue vs déc. 2025 par canal (%) :", {c: round((f26[c] / cc[c]["2025-12-01"] - 1) *
    ↪ 100, 1) for c in cc})
```

```
{'Boutique': 72118, 'Réseaux': 20939, 'Site': 96932} | somme : 189990 | direct : 190904  
croissance prévue vs déc. 2025 par canal (%) : {'Boutique': np.float64(-1.7), 'Réseaux':  
↪ np.float64(2.6), 'Site': np.float64(7.6)}
```

Les prévisions de décembre 2026 sont 72 118 € (Boutique), 20 939 € (Réseaux) et 96 932 € (Site), soit 189 990 € au total, contre 190 904 € par la prévision directe du total et 191 934 à 201 669 € pour les scénarios (application 5.9). Les croissances par canal par rapport à décembre 2025 sont de -1,7 % (Boutique), 2,6 % (Réseaux) et 7,6 % (Site) : le **Site** porte la croissance, la Boutique recule légèrement. À la gérante : « le chiffre d'affaires de décembre 2026 sera entre 190 000 € et 200 000 € environ, la fourchette dépendant surtout de la politique de prix ; le Site est le canal qui progresse, la Boutique stagne ou recule un peu ; prévoyez le stock et les équipes en conséquence plutôt que de répartir le total au prorata de l'an dernier. »

Chapitre 6 : Conception de KPI et cadres d'indicateurs — exercices et applications

Ce cahier prolonge le chapitre 6 : on y **écrit** des fiches d'indicateurs, on y **calcule** les pièges de définition, on **décompose** des chiffres en arbres, on **mesure** la précision d'un budget, et l'on trace des **cartes de contrôle**. Le cahier est autonome : chaque chapitre du cahier recharge ses données. Les données sont **simulées** et les références du secteur **fictives**.

```
import os, sys, warnings
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import outils_ch06 as 0
d = 0.charger(os.environ["DONNEES"])
x = d["x"]; x24, x25 = x[x["annee"] == 2024], x[x["annee"] == 2025]
print("lignes :", len(x), "| sessions :", len(d["sess"]), "| livraisons :", len(d["liv"]))
```

```
lignes : 83905 | sessions : 127022 | livraisons : 19420
```

Applications

Application 6.1 — Une fiche et un calcul unique (section 6.1)

Objectif. Écrire la fiche d'un indicateur, puis vérifier que trois outils donnent le même chiffre.

Étape 1 — la fiche. On la range dans un dictionnaire : c'est déjà un début de dictionnaire de KPI.

```
fiche = {"nom": "Panier moyen", "formule": "CA TTC / commandes distinctes", "perimetre": "tous canaux,
↳ commandes de la période",
        "periode": "mois de la commande", "source": "lignes_commande, commandes", "proprietaire": "la
↳ gérante", "frequence": "mensuelle",
        "sens_favorable": "à la hausse", "contre_indicateur": "chiffre d'affaires", "limites":
↳ "sensible aux promotions et au mix de canaux"}
print(pd.Series(fiche).to_string())
```

nom	Panier moyen
formule	CA TTC / commandes distinctes
perimetre	tous canaux, commandes de la période
periode	mois de la commande
source	lignes_commande, commandes
proprietaire	la gérante
frequence	mensuelle

sens_favorable à la hausse
 contre_indicateur chiffre d'affaires
 limites sensible aux promotions et au mix de canaux

Étape 2 — trois calculs. pandas, DuckDB (SQL) et un calcul « à la main » sur les sommes.

```
import duckdb
D = os.environ["DONNEES"]
pd_ = x25["montant"].sum() / x25["id_commande"].nunique()
sql = duckdb.sql(f"select sum(l.montant) / count(distinct c.id_commande) from
↳ read_csv_auto('{D}/lignes_commande.csv') l join read_csv_auto('{D}/commandes.csv') c using
↳ (id_commande) where c.date_commande >= '2025-01-01').fetchone()[0]
print("pandas :", round(pd_, 2), "| SQL :", round(sql, 2), "| fonction du chapitre :", round(0.kpis(d,
↳ 2025)["Panier moyen (€)"], 2))
```

pandas : 102.33 | SQL : 102.33 | fonction du chapitre : 102.33

À vous. Écrivez la fiche du **taux de conversion du site** : quel est le dénominateur ? Une session qui ajoute un article au panier sans commander compte-t-elle ? Calculez-le avec pandas et avec DuckDB.

Application 6.2 — Les pièges de définition (section 6.1)

Objectif. Constater qu'un même mot recouvre plusieurs chiffres.

Étape 1 — le taux de retour, trois définitions, par canal.

```
def trois_taux(g):
    par_cmd = g.groupby("id_commande")["retourne"].max()
    return pd.Series({"lignes (%)": g["retourne"].mean() * 100, "commandes (%)": par_cmd.mean() * 100,
↳ "euros (%)": g.loc[g["retourne"], "montant"].sum() / g["montant"].sum() * 100})
print(x25.groupby("canal").apply(trois_taux).round(2).to_string())
```

canal	lignes (%)	commandes (%)	euros (%)
Boutique	3.32	7.44	3.29
Réseaux	6.93	15.01	6.46
Site	8.82	18.87	9.16

Étape 2 — la moyenne des moyennes. Comparez la moyenne simple des trois taux « lignes » au taux global.

```
rc = x25.groupby("canal")["retourne"].agg(["mean", "size"])
print("moyenne simple :", round(rc["mean"].mean() * 100, 2), "| pondérée par les lignes :",
↳ round((rc["mean"] * rc["size"]).sum() / rc["size"].sum() * 100, 2))
```

moyenne simple : 6.36 | pondérée par les lignes : 6.29

À vous. Le taux de retour est-il plus élevé le Site que la Boutique dans chaque catégorie ? Calculez-le par canal et catégorie : un effet de mix est-il possible ?

Application 6.3 — Un arbre pour la marge (section 6.2)

Objectif. Décomposer l'écart de marge entre deux années et voir l'effet d'une convention.

Étape 1 — les facteurs.

```
def facteurs(g):
    n = g["id_commande"].nunique(); ca = g["montant"].sum()
    return n, ca / 1.2 / n, g["marge_ht"].sum() / (ca / 1.2)
```

```
(n0, h0, t0), (n1, h1, t1) = facteurs(x24), facteurs(x25)
print("2024 :", n0, round(h0, 2), round(t0 * 100, 2), "| 2025 :", n1, round(h1, 2), round(t1 * 100, 2))
```

```
2024 : 12031 82.39 36.17 | 2025 : 12946 85.27 37.96
```

Étape 2 — deux conventions. Valoriser les effets dans l'ordre (commandes, panier, taux) ou dans l'ordre inverse.

```
ordre1 = ((n1 - n0) * h0 * t0, n1 * (h1 - h0) * t0, n1 * h1 * (t1 - t0))
ordre2 = ((n1 - n0) * h1 * t1, n0 * (h1 - h0) * t1, n0 * h0 * (t1 - t0))
print("ordre 1 :", [round(v) for v in ordre1], "| somme", round(sum(ordre1)))
print("ordre 2 :", [round(v) for v in ordre2], "| somme", round(sum(ordre2)))
```

```
ordre 1 : [27270, 13517, 19662] | somme 60450
ordre 2 : [29615, 13180, 17654] | somme 60450
```

À vous. Les deux conventions donnent la **même somme** mais des **parts différentes**. Pourquoi ? Quelle convention vous semble la plus naturelle pour un lecteur non spécialiste, et comment l'écririez-vous dans une note ?

Application 6.4 — Entonnoir et cadre AARRR (section 6.2)

Objectif. Localiser l'étape du parcours où l'on perd le plus selon l'appareil et le type de visiteur.

```
s = d["sess"]
for col in ["appareil", "nouveau_visiteur"]:
    f = s.groupby(col)[["ajout_panier", "debut_paiement", "commande"]].mean().mul(100).round(1)
    f["paiement → commande (%)"] = (f["commande"] / f["debut_paiement"] * 100).round(0)
    print(f.to_string(), "\n")
```

	ajout_panier	debut_paiement	commande	paiement → commande (%)
appareil				
mobile	14.3	8.2	4.8	59.0
ordinateur	14.1	8.1	4.7	58.0
tablette	14.4	8.0	4.8	60.0
nouveau_visiteur				
0	14.6	8.5	5.0	59.0
1	14.0	7.9	4.5	57.0

À vous. Y a-t-il un appareil ou un type de visiteur à traiter en priorité ? Que répondez-vous à la gérante si les écarts entre groupes sont de l'ordre du bruit (comparez-les à ce que le hasard produit sur de tels effectifs) ?

Application 6.5 — Références et budget (section 6.3)

Objectif. Mesurer la précision du budget par catégorie et en déduire une tolérance.

```
b = d["budget"]
cat = b.groupby("categorie")[["ca_budget", "ca_reel"]].sum()
cat["écart (%)"] = (cat["ca_reel"] / cat["ca_budget"] - 1) * 100
mois_cat = b.groupby(["mois", "categorie"])[["ca_budget", "ca_reel"]].sum()
ecart = (mois_cat["ca_reel"] / mois_cat["ca_budget"] - 1) * 100
cat["écart-type mensuel (pts)"] = ecart.groupby("categorie").std()
print(cat[["écart (%)", "écart-type mensuel (pts)"]].round(1).to_string())
print("tolérance proposée (2 écarts-types, toutes catégories) :", round(2 * ecart.std(), 1), "%")
```

	écart (%)	écart-type mensuel (pts)
categorie		
Bien-être	6.5	11.2
Cuisine	3.7	7.1
Décoration	-3.9	10.5
Jardin	8.4	9.8
Maison	3.7	12.7
Papeterie	4.9	10.8
tolérance proposée (2 écarts-types, toutes catégories) : 21.3 %		

À vous. Une alerte « écart au budget supérieur à 5 % » sur une catégorie et un mois s'allumerait combien de fois sur 72 cases (12 mois × 6 catégories) ? Quelle tolérance retiendriez-vous pour une alerte qui s'allume environ une fois sur vingt ?

Application 6.6 — Cartes de contrôle et tableau de bord (section 6.3)

Objectif. Appliquer la carte de contrôle à un autre indicateur et résumer l'état des indicateurs hebdomadaires.

```
sh = 0.seuils_hebdo(d)
print(pd.DataFrame(sh).T[["p", "orange", "rouge", "n"]].round(1).to_string())
```

	p	orange	rouge	n
Taux de retour (lignes)	6.3	8.3	9.3	568.2
Livraisons à l'heure	78.2	71.1	67.5	134.0
Conversion du site	4.8	3.9	3.5	2396.6
Rupture de stock	7.2	11.6	13.8	139.2

Statut de la **dernière semaine complète** pour chaque indicateur (z calculé avec l'effectif de la semaine) :

```
def dernier_statut(w, p, sens, k=(2, 3)):
    ligne = w.iloc[-1]; z = (ligne["mean"] - p) / np.sqrt(p * (1 - p) / ligne["size"]) * sens
    return round(float(ligne["mean"]) * 100, 1), round(float(z), 1), "vert" if z > -k[0] else
        ("orange" if z > -k[1] else "rouge")
liv = d["liv"][d["liv"]["date_commande"] >= "2025-01-01"].assign(ok=lambda t: 1 - t["retard"])
wl = 0.semaines(liv, "date_commande", "ok"); wl = wl[wl["size"] >= 100]
print("livraisons à l'heure, dernière semaine retenue :", wl.index[-1].date(), dernier_statut(wl,
    sh["Livraisons à l'heure"]["p"] / 100, 1))
```

```
livraisons à l'heure, dernière semaine retenue : 2025-12-22 (44.7, -13.2, 'rouge')
```

À vous. Faites de même pour la **conversion du site** et la **rupture de stock** (elles figurent dans sh). Quelle semaine est la plus mal classée sur chacune ?

Exercices

Exercice 6.1 ★ — Quel chiffre garder ? (section 6.1.1)

Pour chacun des chiffres suivants, dites s'il s'agit d'un indicateur de **pilotage**, de **résultat**, de **contexte** ou à **supprimer**, et justifiez en une phrase : (a) le nombre de pages vues du site ; (b) le chiffre d'affaires du mois ; (c) le taux de livraisons à l'heure de la semaine ; (d) le nombre de clients inscrits à la lettre d'information ; (e) la marge brute du trimestre ; (f) le délai moyen d'expédition de la semaine.

Exercice 6.2 ★ — Une fiche pour le taux de rupture (section 6.1.2)

Rédigez la fiche du **taux de rupture de stock** (nom, décision liée, formule, périmètre, période, source, propriétaire, fréquence, sens favorable, limite). La table `stock_quotidien.csv` suit 20 produits : quel est le **dénominateur** naturel ? Calculez le taux pour 2025 avec votre définition.

Exercice 6.3 ★ — La moyenne des moyennes (section 6.1.5)

Deux canaux : le premier a 200 commandes et 5 % de retours, le second 800 commandes et 1 % de retours. (a) Quelle est la moyenne simple des deux taux ? (b) Quel est le taux global ? (c) Retrouvez-le avec `numpy.average` et des poids.

Exercice 6.4 ★★ — Cibler un indicateur sans le casser (section 6.1.6)

La gérante veut « 90 % de livraisons à l'heure ». Un transporteur propose de passer le **délai promis** de 6 à 8 jours. Avec les dates de commande et de livraison de `livraisons.csv` (2025), calculez le taux de livraisons à l'heure pour des délais promis de 6, 7, 8, 9 et 10 jours. Que vaut l'objectif de 90 % dans ces conditions ? Quel contre-indicateur proposez-vous ?

Exercice 6.5 ★★ — Le « client actif » (section 6.1.5)

Calculez, sur les 6 000 clients, la part de « clients actifs » selon trois définitions : au moins une commande dans les 12 derniers mois de 2025, au moins une commande dans les 6 derniers mois, au moins deux commandes dans l'année. Concluez sur le besoin d'une définition unique.

Exercice 6.6 ★★ — Décomposer le panier moyen (section 6.2.2)

Le panier moyen est le produit du nombre moyen de **lignes par commande** par le **montant moyen d'une ligne**. Vérifiez l'égalité en 2024 et en 2025, puis répartissez la hausse du panier moyen entre ces deux facteurs.

Exercice 6.7 ★★ — Prix, volume et mix par catégorie (section 6.2.3)

Pour chaque catégorie, calculez la variation de chiffre d'affaires entre 2024 et 2025 et séparez un **effet quantité** et un **effet prix moyen** ($\text{prix moyen} = \text{chiffre d'affaires} \div \text{quantités}$). Quelle catégorie a le plus augmenté ses prix moyens ? Quelle part de cette hausse vient d'un changement de **mix** à l'intérieur de la catégorie, plutôt que d'une hausse de prix catalogue (qui est de 3 % au 1er janvier 2025 pour tous les produits) ?

Exercice 6.8 ★★★ — Un OKR sur les ruptures (sections 6.2.4 et 6.3.1)

Rédigez un objectif et deux résultats clés pour réduire les ruptures de stock. Avec `stock_quotidien.csv`, calculez le taux de rupture par produit : combien de produits expliquent la moitié des jours de rupture ? Si les cinq pires produits avaient le taux de la médiane des autres, quel serait le taux global ? Cette cible est-elle atteignable avec ce seul levier ?

Exercice 6.9 ★ — Lire un benchmark (section 6.3.2)

À partir de `0.position_secteur(d)`, listez les indicateurs « meilleurs », « dans la norme » et « moins bons ». Pour deux d'entre eux, citez une raison pour laquelle la comparaison pourrait être **trompeuse** (définition, périmètre, saison).

Exercice 6.10 ★★ — La précision du budget par canal (section 6.3.3)

Calculez l'écart mensuel au budget par canal (Boutique, Site, Réseaux) et son écart-type. Quel canal est le plus difficile à prévoir ? Quelle tolérance mensuelle proposez-vous pour chaque canal ?

Exercice 6.11 ★★ — Bruit d'un taux de rupture (section 6.3.4)

Pour un taux de rupture hebdomadaire de 7,2 % sur 140 produit-jours, quel est l'écart-type du bruit ? Simulez 100 000 semaines à taux constant : quelle part s'écarte de plus de 2 points ? Comparez à l'écart-type observé des taux hebdomadaires de 2025.

Exercice 6.12 ★★★ — Une carte de contrôle de la conversion (section 6.3.5)

Tracez (ou calculez) la carte de contrôle de la **conversion hebdomadaire** du site en 2025, avec des limites à ± 3 écarts-types fondées sur les 40 premières semaines. Combien de semaines sortent des limites ? Si certaines sortent, proposez une explication à vérifier (saison, source de trafic) et dites si le signal serait **actionnable**.

Corrigés**Corrigé 6.1**

- (a) **À supprimer** ou à reléguer en contexte : aucune décision ne dépend d'un nombre de pages vues seul. (b) **Résultat** : à suivre chaque mois, il arrive tard. (c) **Pilotage** : une baisse déclenche un appel au transporteur. (d) **À supprimer** ou **à remplacer** par les clients actifs : un nombre d'inscrits cumulé ne baisse jamais. (e) **Résultat**. (f) **Pilotage** : un délai d'expédition qui s'allonge annonce des retards de livraison.

Corrigé 6.2

```
st = d["stock"]
print("produit-jours :", len(st), "| jours de rupture :", int(st["rupture"].sum()), "| taux :",
      ↪ round(st["rupture"].mean() * 100, 2), "%")
```

```
produit-jours : 7300 | jours de rupture : 537 | taux : 7.36 %
```

Fiche : **nom** : taux de rupture ; **décision** : passer commande plus tôt, changer de fournisseur ; **formule** : produit-jours où la demande dépasse le stock ÷ produit-jours ; **périmètre** : 20 produits principaux ; **période** : semaine ; **source** : stock_quotidien ; **propriétaire** : responsable des achats ; **fréquence** : hebdomadaire ; **sens favorable** : à la baisse ; **limite** : ne mesure pas la **demande perdue** (la quantité manquante) ni les produits hors des 20 principaux. Le dénominateur naturel est le nombre de **produit-jours** ($20 \times 365 = 7\,300$).

Corrigé 6.3

```
print("moyenne simple :", (5 + 1) / 2, "% | taux global :", round((200 * 0.05 + 800 * 0.01) / 1000 *
      ↪ 100, 2), "% | numpy :", round(float(np.average([5, 1], weights=[200, 800])), 2))
```

moyenne simple : 3.0 % | taux global : 1.8 % | numpy : 1.8

La moyenne simple (3 %) **surestime** le taux global (1,8 %) : le canal qui compte le plus a le taux le plus faible. On pondère par les dénominateurs.

Corrigé 6.4

```
l = d["liv"][d["liv"]["date_commande"] >= "2025-01-01"].copy()
duree = (pd.to_datetime(l["date_livraison"]) - pd.to_datetime(l["date_commande"])).dt.days
print({j: round(float((duree <= j).mean()) * 100, 1) for j in (6, 7, 8, 9, 10)})
```

{6: 73.5, 7: 89.1, 8: 96.3, 9: 98.9, 10: 99.7}

Avec 6 jours promis, 73,5 % des colis arrivent à l'heure ; avec 7 jours, 89,1 % ; avec 8 jours, 96,3 %. L'objectif de 90 % est donc « atteint » dès que l'on promet **un peu plus de sept jours**, sans qu'aucun colis n'arrive plus tôt. Le taux grimpe mécaniquement avec le délai promis : allonger la promesse fait « gagner » l'indicateur **sans que rien ne change pour le client**, qui attend davantage. C'est la loi de Goodhart. Un bon contre-indicateur : le **délai moyen réel** de livraison ou le taux de retours « livraison tardive ».

Corrigé 6.5

```
c = d["cmd"]; c25 = c[c["date_commande"] >= "2025-01-01"]
actifs12 = c25["id_client"].nunique()
actifs6 = c25[c25["date_commande"] >= "2025-07-01"]["id_client"].nunique()
deux = int((c25.groupby("id_client").size() >= 2).sum())
print("12 mois :", actifs12, round(actifs12 / 6000 * 100, 1), "% | 6 mois :", actifs6, round(actifs6 /
↳ 6000 * 100, 1), "% | au moins deux commandes :", deux, round(deux / 6000 * 100, 1), "%")
```

12 mois : 3875 64.6 % | 6 mois : 3156 52.6 % | au moins deux commandes : 2654 44.2 %

Trois définitions, trois chiffres très différents (64,6 %, 52,6 % et 44,2 %) : la définition doit être **écrite une fois** dans le glossaire et utilisée partout.

Corrigé 6.6

```
def lpc(g):
    n = g["id_commande"].nunique(); return len(g) / n, g["montant"].sum() / len(g), g["montant"].sum()
    ↳ / n
(l0, m0, p0), (l1, m1, p1) = lpc(x24), lpc(x25)
print("2024 :", round(l0, 3), "x", round(m0, 2), "=", round(l0 * m0, 2), "| 2025 :", round(l1, 3),
↳ "x", round(m1, 2), "=", round(l1 * m1, 2))
print("hausse du panier :", round(p1 - p0, 2), "€ = lignes", round((l1 - l0) * m0, 2), "+ montant par
↳ ligne", round(l1 * (m1 - m0), 2))
```

2024 : 2.3 x 42.98 = 98.87 | 2025 : 2.304 x 44.41 = 102.33
hausse du panier : 3.46 € = lignes 0.16 + montant par ligne 3.3

Les deux facteurs valent 2,300 lignes par commande et 42,98 € par ligne en 2024, 2,304 et 44,41 € en 2025 : leur produit redonne 98,87 € et 102,33 €. Sur les 3,46 € de hausse du panier, **0,16 € seulement** viennent d'un plus grand nombre de lignes par commande ; **3,30 € viennent du montant par ligne**. Les clients n'achètent pas plus d'articles, ils achètent des articles plus chers (prix catalogue en hausse de 3 % au 1er janvier 2025, et mix).

Corrigé 6.7

```
a = x24.groupby("categorie").agg(ca0=("montant", "sum"), q0=("quantite", "sum")); b =
↳ x25.groupby("categorie").agg(ca1=("montant", "sum"), q1=("quantite", "sum"))
t = a.join(b); t["p0"], t["p1"] = t["ca0"] / t["q0"], t["ca1"] / t["q1"]
t["effet quantité"] = (t["q1"] - t["q0"]) * t["p0"]; t["effet prix moyen"] = t["q1"] * (t["p1"] -
↳ t["p0"]); t["hausse prix moyen (%)"] = (t["p1"] / t["p0"] - 1) * 100
print(t[["effet quantité", "effet prix moyen", "hausse prix moyen (%)"]].round(1).to_string())
```

	effet quantité	effet prix moyen	hausse prix moyen (%)
categorie			
Bien-être	6668.7	3524.8	3.1
Cuisine	9130.7	6996.7	3.1
Décoration	17268.1	6400.9	2.5
Jardin	30096.1	14148.6	4.2
Maison	27192.3	8822.2	3.0
Papeterie	3417.8	1635.8	3.0

Le **Jardin** a la plus forte hausse de prix moyen (+4,2 %) : comme le prix catalogue a augmenté de 3 %, environ **1,2 point** vient du **mix** (références plus chères au sein de la catégorie) ou de remises moindres. À l'inverse, la **Décoration** (+2,5 %) est en dessous des 3 % de prix catalogue : son mix s'est déplacé vers des références moins chères. Les autres catégories (entre +3,0 % et +3,1 %) suivent le prix catalogue : leur hausse est surtout un effet **prix**.

Corrigé 6.8

```
st = d["stock"]; r = st.groupby("id_produit")["rupture"].sum().sort_values(ascending=False)
moitie = int((r.cumsum() / r.sum() < 0.5).sum() + 1)
pires = r.index[:5]; med =
↳ st[~st["id_produit"].isin(pires)].groupby("id_produit")["rupture"].mean().median()
nouveau = (st[~st["id_produit"].isin(pires)]["rupture"].sum() + med *
↳ st[st["id_produit"].isin(pires)].shape[0]) / len(st)
print("produits expliquant la moitié des ruptures :", moitie, "| taux actuel :",
↳ round(st["rupture"].mean() * 100, 2), "% | si les 5 pires étaient à la médiane :", round(nouveau *
↳ 100, 2), "%")
```

```
produits expliquant la moitié des ruptures : 9 | taux actuel : 7.36 % | si les 5 pires étaient à la
↳ médiane : 6.86 %
```

Exemple d'OKR : **objectif** « Ne plus manquer nos produits phares » ; **RC1** taux de rupture de 7,4 % à 4 % (la médiane du secteur) ; **RC2** aucun produit à plus de 10 % de rupture. Neuf produits sur vingt expliquent la moitié des jours de rupture, mais ramener les cinq pires à la médiane des autres ne fait passer le taux global que de **7,4 % à 6,9 %** : ce levier **ne suffit pas** à atteindre 4 %. Il faut un second levier : délais des fournisseurs, point de commande (chapitre 11).

Corrigé 6.9

```
pos = 0.position_secteur(d); print(pos.groupby("position")["indicateur"].apply(list).to_string())
```

position	
dans la norme	[Taux de marge brute (HT, %), Taux de retour (...]
meilleur	[Taux de conversion du site (%), Clients actif...]
moins bon	[Taux de rupture de stock (%), Livraisons à l'...]
à interpréter	[Frais de personnel / CA HT (%)]

Quelques raisons de prudence : la **conversion** dépend de la définition d'une session et de la qualité du trafic ; le **coût d'acquisition** dépend de ce que l'on compte dans les dépenses et de

ce qu'est un client « nouveau » ; la part du **site** dans le chiffre d'affaires dépend de l'activité des autres canaux. Les valeurs du secteur sont, de plus, **inventées**.

Corrigé 6.10

```
bc = d["budget"].groupby(["mois", "canal"])[["ca_budget", "ca_reel"]].sum()
ec = ((bc["ca_reel"] / bc["ca_budget"] - 1) * 100).groupby("canal")
print(pd.DataFrame({"moyenne (%)": ec.mean(), "écart-type (pts)": ec.std(), "tolérance 2 écarts-types
↳ (%)": 2 * ec.std()}).round(1).to_string())
```

	moyenne (%)	écart-type (pts)	tolérance 2 écarts-types (%)
canal			
Boutique	-6.2	8.9	17.9
Réseaux	4.3	19.3	38.6
Site	16.0	12.8	25.6

Le canal **Réseaux** est le plus difficile à prévoir (écart-type de 19,3 points, donc une tolérance de $\pm 39\%$), devant le Site (12,8 points, $\pm 26\%$) et la Boutique (8,9 points, $\pm 18\%$). Notez aussi les **moyennes** : le budget a **sous-estimé le Site de 16 %** et **surestimé la Boutique de 6 %** en moyenne, c'est-à-dire qu'il n'a pas anticipé le déplacement des ventes vers le site. Une tolérance symétrique autour d'un budget biaisé produit de fausses alertes d'un côté et en masque de l'autre : il faut d'abord **corriger le biais**.

Corrigé 6.11

```
rng = np.random.default_rng(11); n, p = 140, 0.072
sim = (rng.binomial(n, p, 100000) / n - p) * 100
print("écart-type du bruit :", round(float(np.sqrt(p * (1 - p) / n) * 100), 2), "pts | part à plus de
↳ 2 points :", round((abs(sim) > 2).mean() * 100, 1), "%")
sw = 0.semaines(d["stock"].assign(date=d["stock"]["date"]), "date", "rupture"); sw = sw[sw["size"] >=
↳ 100]
print("écart-type observé :", round(sw["mean"].std() * 100, 2), "pts sur", len(sw), "semaines")
```

```
écart-type du bruit : 2.18 pts | part à plus de 2 points : 41.2 %
écart-type observé : 7.18 pts sur 52 semaines
```

Le bruit d'une semaine de 140 produit-jours vaut 2,2 points : **un voyant à ± 2 points s'allumerait environ deux semaines sur cinq (41 %)** sans aucune cause. L'écart-type **observé** des taux hebdomadaires de 2025 est de **7,2 points**, plus de trois fois celui du bruit : la rupture n'est donc pas stable, elle monte nettement en fin d'année ; ici, la variabilité observée dépasse largement le hasard, il y a un **signal** à instruire.

Corrigé 6.12

```
cw = 0.semaines(d["sess"], "date", "commande"); cw = cw[cw["size"] >= 500]
base = cw.iloc[:40]; pb = base["sum"].sum() / base["size"].sum()
lo, hi = 0.limite_p(pb, cw["size"])
hors = cw[(cw["mean"] < lo) | (cw["mean"] > hi)]
print("centre :", round(pb * 100, 2), "% | semaines :", len(cw), "| hors limites :", len(hors),
↳ [str(i.date()) for i in hors.index][:6])
```

```
centre : 4.49 % | semaines : 53 | hors limites : 6 ['2025-09-29', '2025-11-24', '2025-12-01',
↳ '2025-12-08', '2025-12-15', '2025-12-22']
```

Le centre est à 4,5 % ; **6 semaines sur 53** sortent des limites : la semaine du 29 septembre et les **cinq dernières semaines de l'année** (24 novembre au 22 décembre). La conversion monte donc

en fin d'année : c'est un **effet saisonnier** (les commandes sont portées par la saison et le vendredi noir) et non une dérive à corriger. Une carte de contrôle sur un indicateur saisonnier doit être **calculée par saison** ou sur la comparaison à l'an dernier ; un signal n'est actionnable que s'il est **recoupé** (par source de trafic, par période de promotion) avant toute décision.

Pistes des applications

Quelques pistes pour les « À vous » des applications.

Application 6.1 (conversion). Le dénominateur est le nombre de **sessions** ; une session qui ajoute un article sans commander compte au dénominateur, pas au numérateur.

```
sess = d["sess"]
print("pandas :", round(sess["commande"].mean() * 100, 3), "% | SQL :", round(duckdb.sql(f"select
↳ 100.0 * sum(commande) / count(*) from read_csv_auto('{D}/sessions_web.csv')").fetchone()[0], 3),
↳ "%")
```

pandas : 4.785 % | SQL : 4.785 %

Application 6.2 (retours par canal et catégorie). On cherche un effet de mix : si le Site vend plus de catégories à fort retour, son taux global est tiré vers le haut.

```
t = x25.groupby(["categorie", "canal"])["retourne"].mean().unstack().mul(100).round(1)
t["Site - Boutique (pts)"] = (t["Site"] - t["Boutique"]).round(1)
print(t.to_string())
```

canal	Boutique	Réseaux	Site	Site - Boutique (pts)
categorie				
Bien-être	3.4	9.1	8.2	4.8
Cuisine	3.7	5.4	8.6	4.9
Décoration	3.4	6.7	8.8	5.4
Jardin	3.2	5.9	10.0	6.8
Maison	3.2	7.4	8.9	5.7
Papeterie	3.0	7.8	8.3	5.3

Le Site est au-dessus de la Boutique **dans chaque catégorie** : l'écart global ne vient donc pas d'un effet de mix mais d'un comportement propre au canal (on ne peut pas essayer un produit commandé en ligne).

Application 6.3 (conventions). Les deux ordres donnent la même somme parce que la décomposition est **exhaustive** (chaque convention répartit exactement l'écart total), mais le « terme croisé » (l'effet simultané du volume et du prix) est attribué différemment : au volume ou au prix selon l'ordre. Convention la plus lisible : valoriser le **volume au prix de départ**, puis le prix au volume d'arrivée, et l'écrire en une phrase dans la note.

Application 6.5 (alertes à 5 %).

```
mc = d["budget"].groupby(["mois", "categorie"])["ca_budget", "ca_reel"].sum(); e = (mc["ca_reel"] /
↳ mc["ca_budget"] - 1) * 100
print("cases hors ±5 % :", int((e.abs() > 5).sum()), "sur", len(e), "| écart absolu au 95e centile :",
↳ round(float(e.abs().quantile(0.95)), 1), "%")
```

cases hors ±5 % : 46 sur 72 | écart absolu au 95e centile : 20.1 %

Une alerte à $\pm 5\%$ s'allume sur **46 cases sur 72 (64 %)** : autant dire en permanence. Pour qu'elle ne s'allume qu'**une fois sur vingt**, il faut une tolérance de l'ordre du 95e centile des écarts absolus,

soit 20,1 % : à cette échelle de détail (mois × catégorie), le budget n'est pas un instrument de pilotage fin.

Application 6.6 (conversion et rupture).

```
st = d["stock"]; wr = 0.semaines(st, "date", "rupture"); wr = wr[wr["size"] >= 100]
wc = 0.semaines(d["sess"], "date", "commande"); wc = wc[wc["size"] >= 500]
for nom, w, sens in [("rupture", wr, -1), ("conversion", wc, 1)]:
    p = w["sum"].sum() / w["size"].sum(); z = (w["mean"] - p) / np.sqrt(p * (1 - p) / w["size"]) * sens
    print(nom, "| pire semaine :", w.index[z.argmax()].date(), "| z =", round(float(z.min()), 1), "|
    ↳ semaines rouges (z < -3) :", int((z < -3).sum()), "sur", len(w))
```

```
rupture | pire semaine : 2025-12-22 | z = -12.7 | semaines rouges (z < -3) : 6 sur 52
conversion | pire semaine : 2025-08-25 | z = -3.4 | semaines rouges (z < -3) : 2 sur 53
```

Pour ces deux indicateurs, la valeur centrale est calculée **sur toute l'année**, ce qui est une **faute de méthode** (6.3.5) quand une rupture de tendance existe en fin d'année : les limites sont déformées. Refaites le calcul en ne gardant que les semaines d'avant novembre pour la valeur centrale.

Chapitre 7 : ⊕ Analyse des écarts et des causes racines — exercices et applications

Ce chapitre du cahier prolonge le chapitre 7 du livre (chapitre complémentaire). Les **applications** sont de petites études guidées sur le budget et le réalisé de 2025 ; les **exercices** sont numérotés, avec leur niveau (★ de base, ★★ intermédiaire, ★★★ plus délicat) ; les **corrigés** sont à la fin. Tout le code est exécuté : essayez avant de regarder.

Une première cellule charge les bibliothèques et les fichiers ; les autres reprennent les noms ainsi définis.

```
import os, sys, warnings
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import outils_ch07 as O

D = os.environ["DONNEES"]
lire = lambda nom, **kw: pd.read_csv(os.path.join(D, nom), **kw)
bud = lire("budget_reel_2025.csv")
cmd, lig, prod = lire("commandes.csv", parse_dates=["date_commande"]), lire("lignes_commande.csv"),
↳ lire("produits.csv")
jours = lire("jours_exploitation.csv", parse_dates=["date"])
cmd["annee"] = cmd["date_commande"].dt.year
TVA = 0.20
print(len(bud), "lignes de budget |", len(cmd), "commandes |", len(lig), "lignes de commande")
```

```
216 lignes de budget | 36395 commandes | 83905 lignes de commande
```

Applications

Application 7.1 — Lire un écart et fixer un seuil (section 7.1)

Objectif. Construire le tableau d'écarts par catégorie et décider lesquels expliquer.

Étape 1 — le tableau par catégorie.

```
cat = bud.groupby("categorie")[["ca_budget", "ca_reel", "marge_budget", "marge_reelle"]].sum()
cat["écart CA"] = cat["ca_reel"] - cat["ca_budget"]
cat["écart CA %"] = (cat["écart CA"] / cat["ca_budget"] * 100).round(1)
cat["écart marge %"] = ((cat["marge_reelle"] / cat["marge_budget"] - 1) * 100).round(1)
print(cat[["écart CA", "écart CA %", "écart marge %"]].round(1).to_string())
```

	écart CA	écart CA %	écart marge %
categorie			
Bien-être	7157.5	6.5	12.1
Cuisine	8267.8	3.7	9.1
Décoration	-10410.1	-3.9	-0.3
Jardin	27469.2	8.4	15.0
Maison	10912.2	3.7	8.0
Papeterie	2667.0	4.9	10.8

Étape 2 — un seuil. Quelles catégories dépassent 5 % d'écart de chiffre d'affaires ?

```
print("catégories à plus de 5 % d'écart :", list(cat[cat["écart CA %"].abs() > 5].index))
print("catégorie en retard :", list(cat[cat["écart CA"] < 0].index))
```

```
catégories à plus de 5 % d'écart : ['Bien-être', 'Jardin']
catégorie en retard : ['Décoration']
```

À vous. Refaites le tableau par **canal** et par **mois** (le mois le plus en avance, le plus en retard) et commentez.

Application 7.2 — La décomposition à la main (section 7.2)

Objectif. Refaire le calcul prix-volume-mix sur deux produits, puis le confronter à la fonction.

Étape 1. Budget : produit X, 80 unités à 10 € ; produit Y, 20 unités à 40 €. Réalisé : X, 70 unités à 10 € ; Y, 40 unités à 42 €.

```
x = pd.DataFrame({"p": ["X", "Y"], "quantite_budget": [80, 20], "quantite_reel": [70, 40],
↳ "ca_budget": [800, 800], "ca_reel": [700, 1680]})
r = 0.pvm_ca(x, ["p"])
print({k: round(v, 2) for k, v in r.items()})
```

```
{'volume': 160.0, 'mix': 540.0, 'prix': 80.0, 'total': 780.0, 'ecart': 780.0}
```

Étape 2 — le calcul à la main. Prix moyen du budget : $1\,600/100 = 16$ € ; volume : $(110-100) \times 16 = 160$; mix : $(70 - 110 \times 0,8) \times 10 + (40 - 110 \times 0,2) \times 40 = -180 + 720 = 540$; prix : $40 \times 2 = 80$; total $160 + 540 + 80 = 780 = 2\,380 - 1\,600$.

```
print("volume", (110 - 100) * 16, "| mix", (70 - 110 * 0.8) * 10 + (40 - 110 * 0.2) * 40, "| prix", 40
↳ * (42 - 40), "| total", (110 - 100) * 16 + (70 - 110 * 0.8) * 10 + (40 - 110 * 0.2) * 40 + 40 * 2)
```

```
volume 160 | mix 540.0 | prix 80 | total 780.0
```

À vous. Que se passe-t-il si l'on inverse l'ordre de calcul (valoriser le volume au prix **réalisé**) ? Le total change-t-il ?

Application 7.3 — Décomposer la marge par canal (section 7.2)

Objectif. Appliquer la décomposition de la marge à chaque canal et comparer.

```
rows = {canal: 0.pvm_marge(d, ["categorie"]) for canal, d in bud.groupby("canal")}
print(pd.DataFrame(rows).T.round(0).to_string())
```

	volume	mix	marge_unitaire	total	ecart
Boutique	-12514.0	494.0	7936.0	-4084.0	-4084.0
Réseaux	1547.0	232.0	3127.0	4906.0	4906.0
Site	23025.0	-780.0	9995.0	32239.0	32239.0

Étape 2. La somme des trois effets de chaque canal est-elle égale à son écart de marge ?

```
t = pd.DataFrame(rows).T
print("somme des effets = écart, pour chaque canal :", bool(np.allclose(t["volume"] + t["mix"] +
↪ t["marge_unitaire"], t["ecart"])))
```

```
somme des effets = écart, pour chaque canal : True
```

À vous. Quel canal doit le plus de sa marge au volume ? À la marge unitaire ?

Application 7.4 — Le prix et le coût derrière la marge unitaire (section 7.2)

Objectif. Séparer, pour chaque canal, l'effet prix et l'effet coût dans l'effet « marge unitaire ».

```
res = {}
for canal, d in bud.groupby("canal"):
    ca, mg = 0.pvm_ca(d, ["categorie"]), 0.pvm_marge(d, ["categorie"])
    prix = ca["prix"] / (1 + TVA)
    res[canal] = {"marge unitaire": mg["marge_unitaire"], "dont prix (HT)": prix, "dont coût":
↪ mg["marge_unitaire"] - prix}
print(pd.DataFrame(res).T.round(0).to_string())
```

	marge unitaire	dont prix (HT)	dont coût
Boutique	7936.0	541.0	7396.0
Réseaux	3127.0	1238.0	1888.0
Site	9995.0	2004.0	7991.0

À vous. Dans quel canal l'effet « coût » est-il le plus élevé en proportion des unités vendues ?

Application 7.5 — Tester une hypothèse (section 7.3)

Objectif. Tester « le Site a pris des clients à la Boutique » par une autre méthode que la part des commandes : comparer le **nombre moyen de commandes en Boutique par client** en 2024 et en 2025, pour les clients actifs les deux années.

```
x = cmd[cmd["annee"].isin([2024, 2025])]
deux = x.groupby("id_client")["annee"].nunique()
ids = deux[deux == 2].index
nb = x[x["id_client"].isin(ids) & (x["canal"] == "Boutique")].groupby(["id_client",
↪ "annee"]).size().unstack(fill_value=0).reindex(ids, fill_value=0)
print("commandes en Boutique par client : 2024", round(nb[2024].mean(), 2), "| 2025",
↪ round(nb[2025].mean(), 2), "| clients :", len(ids))
```

```
commandes en Boutique par client : 2024 1.81 | 2025 1.62 | clients : 2828
```

Étape 2 — un test. Les moyennes diffèrent-elles de façon convaincante ? On calcule un intervalle par rééchantillonnage.

```
diff = nb[2025] - nb[2024]
graines = np.random.default_rng(1).integers(0, 10**6, 400)
boot = np.array([diff.sample(len(diff), replace=True, random_state=int(s)).mean() for s in graines])
print("variation moyenne :", round(diff.mean(), 3), "| IC à 95 % :", np.round(np.percentile(boot,
↪ [2.5, 97.5]), 3))
```

```
variation moyenne : -0.191 | IC à 95 % : [-0.256 -0.127]
```

À vous. Quelle conclusion honnête en tirez-vous (rejetée, probable, établie, non démontrée) ? Que faudrait-il de plus pour établir une cause ?

Application 7.6 — Rédiger le tableau d'hypothèses (section 7.3)

Objectif. Rédiger la conclusion d'une analyse d'écart : un tableau d'hypothèses avec leur statut, à partir de résultats calculés.

```
cout_budget = bud["ca_budget"].sum() / bud["quantite_budget"].sum() / (1 + TVA) -
↳ bud["marge_budget"].sum() / bud["quantite_budget"].sum()
lg = lig.merge(cmd[["id_commande", "annee"]], on="id_commande").merge(prod[["id_produit",
↳ "cout_achat"]], on="id_produit")
cu = lg.assign(c=lg["quantite"] * lg["cout_achat"]).groupby("annee").agg(c=("c", "sum"),
↳ q=("quantite", "sum"))
cu["u"] = cu["c"] / cu["q"]
jours["annee"] = jours["date"].dt.year
pluie = jours[jours["pluie_mm"] > 1].groupby("annee").size()
tab = pd.DataFrame([
    ["La pluie a freiné la Boutique", f"jours de pluie {pluie[2024]} (2024) puis {pluie[2025]}
↳ (2025)", "rejetée" if pluie[2025] < pluie[2024] else "à examiner"],
    ["Le budget anticipait un coût d'achat trop élevé", f"supposé {cout_budget:.2f} €, réalisé
↳ {cu.loc[2025, 'u']:.2f} €", "établie" if cout_budget > cu.loc[2025, "u"] * 1.02 else "à
↳ examiner"],
], columns=["hypothèse", "résultat", "statut"])
print(tab.to_string(index=False))
```

	hypothèse	résultat	statut
	La pluie a freiné la Boutique	jours de pluie 111 (2024) puis 92 (2025)	rejetée
	Le budget anticipait un coût d'achat trop élevé	supposé 19.58 €, réalisé 19.13 €	établie

À vous. Ajoutez une ligne pour « la hausse de tarif de janvier » à partir du rapport des prix catalogue d'une année à l'autre.

Exercices

Exercice 7.1 ★ (section 7.1). Le budget d'un canal est de 80 000 € et le réalisé de 72 000 €. Calculez l'écart absolu, l'écart relatif, et dites s'il est favorable. Même question pour des **coûts** budgétés à 30 000 € et réalisés à 33 000 €.

Exercice 7.2 ★★ (section 7.1). Trois lignes de budget ont pour écarts +9 000 €, −8 500 € et −400 €. Le total est de +100 €. Qu'en concluez-vous sur la lecture d'un écart total ? Quel indicateur complémentaire proposez-vous (par exemple la somme des écarts absolus) ?

Exercice 7.3 ★★ (section 7.1). Sur nos données, comptez les lignes mensuelles (216) dont l'écart relatif dépasse 15 % et l'écart absolu 1 500 €. Comparez à un seuil de 10 % et 800 € : quelle conclusion tirez-vous du choix du seuil ?

Exercice 7.4 ★ (section 7.2). Budget : A, 50 unités à 20 € ; B, 50 unités à 40 €. Réalisé : A, 60 unités à 20 € ; B, 40 unités à 40 €. Calculez les trois effets à la main, puis vérifiez avec 0. pvm_ca. Que révèle le fait que le prix ne bouge pas ?

Exercice 7.5 ★★ (section 7.2). Même budget qu'à l'exercice 7.4, réalisé : A, 50 unités à 22 € ; B, 50 unités à 40 €. Quel est l'effet prix ? Y a-t-il un effet volume ou mix ?

Exercice 7.6 ★★ (section 7.2). Démontrez que la somme volume + mix + prix égale l'écart total pour deux produits (utilisez la formule du livre et la relation $\sum_i s_{b,i} P_{b,i} = P_b^{\text{moy}}$).

Exercice 7.7 ★★★ (section 7.2). Décomposez l'écart de chiffre d'affaires de la **catégorie Jardin** seule, avec les canaux comme lignes. Quel effet domine ? Qu'apprend-on en comparant à la décomposition de la catégorie Décoration ?

Exercice 7.8 ★★ (section 7.3). Écrivez une hypothèse réfutable pour chacune des causes suivantes de l'écart de la Boutique : (a) la météo ; (b) les ruptures de stock ; (c) un transfert vers le Site. Pour chacune, indiquez le test et le résultat qui la réfuterait.

Exercice 7.9 ★★ (section 7.3). Menez les « cinq pourquoi » sur l'écart de marge **favorable** de l'entreprise (+8,6 %), en vous appuyant sur les chiffres de la section 7.2.

Exercice 7.10 ★★★ (section 7.3). Un collègue affirme : « le Site a fait perdre des ventes à la Boutique parce que la publicité en ligne a augmenté ». Quels sont les risques de raisonnement dans cette phrase ? Quelles données demanderiez-vous pour la tester ?

Corrigés

Corrigé 7.1. Écart absolu $72\,000 - 80\,000 = -8\,000$ €, relatif -10% ; **défavorable** pour un chiffre d'affaires. Coûts : $+3\,000$ € soit $+10\%$; **défavorable** aussi (des coûts supérieurs au budget) : même signe, mais le sens n'est pas le même que pour un chiffre d'affaires.

```
print("CA :", 72000 - 80000, round((72000 / 80000 - 1) * 100, 1), "% | coûts :", 33000 - 30000,
      ↪ round((33000 / 30000 - 1) * 100, 1), "%")
```

```
CA : -8000 -10.0 % | coûts : 3000 10.0 %
```

Corrigé 7.2. Le total de $+100$ € cache deux écarts importants qui se compensent presque. L'indicateur complémentaire : la somme des **écarts absolus** ($9\,000 + 8\,500 + 400 = 17\,900$ €), qui dit l'ampleur des écarts indépendamment du signe. Un écart total proche de zéro n'est pas un bon budget : c'est peut-être deux gros écarts de signes opposés.

```
e = np.array([9000, -8500, -400])
print("total :", e.sum(), "| somme des écarts absolus :", np.abs(e).sum())
```

```
total : 100 | somme des écarts absolus : 17900
```

Corrigé 7.3.

```
bud["e"] = bud["ca_reel"] - bud["ca_budget"]
bud["p"] = bud["e"] / bud["ca_budget"] * 100
for pc, eu in ((15, 1500), (10, 800)):
    print(f"plus de {pc} % et {eu} € :", int(((bud['p'].abs() > pc) & (bud['e'].abs() > eu)).sum()),
          ↪ "lignes sur", len(bud))
```

```
plus de 15 % et 1500 € : 34 lignes sur 216
plus de 10 % et 800 € : 79 lignes sur 216
```

Le nombre de lignes à expliquer **plus que double** quand on baisse les seuils : le seuil est un choix qui détermine la charge de travail, et il doit être écrit et justifié (et plutôt appliqué à un grain agrégé, où le bruit est moindre).

Corrigé 7.4. Prix moyen du budget : 30 €. Volume : $(100 - 100) \times 30 = 0$. Mix : $(60 - 50) \times 20 + (40 - 50) \times 40 = 200 - 400 = -200$. Prix : 0 . Total -200 ; vérification : réalisé $1\,200 + 1\,600 = 2\,800$ contre budget $1\,000 + 2\,000 = 3\,000$, soit -200 . Le prix ne bouge pas, donc **tout** l'écart est un effet de mix : on a vendu plus du produit bon marché.

```
ex = pd.DataFrame({"p": ["A", "B"], "quantite_budget": [50, 50], "quantite_reel": [60, 40],
                  ↪ "ca_budget": [1000, 2000], "ca_reel": [1200, 1600]})
print({k: round(v, 1) for k, v in 0.pvm_ca(ex, ["p"]).items()})
```

```
{'volume': 0.0, 'mix': -200.0, 'prix': 0.0, 'total': -200.0, 'ecart': -200.0}
```

Corrigé 7.5. Seul le prix de A change (+2 € sur 50 unités) : effet prix = $50 \times 2 = +100$ €. Les quantités sont inchangées : pas d'effet volume ni mix.

```
ex2 = pd.DataFrame({"p": ["A", "B"], "quantite_budget": [50, 50], "quantite_reel": [50, 50],
    ↪ "ca_budget": [1000, 2000], "ca_reel": [1100, 2000]})
print({k: round(v, 1) for k, v in 0.pvm_ca(ex2, ["p"]).items()})
```

```
{'volume': 0.0, 'mix': 0.0, 'prix': 100.0, 'total': 100.0, 'ecart': 100.0}
```

Corrigé 7.6. Volume + mix = $(Q_r - Q_b)P_b^{\text{moy}} + \sum_i (Q_{r,i} - Q_{rsb,i})P_{b,i}$. Or $\sum_i Q_{rsb,i}P_{b,i} = Q_r P_b^{\text{moy}}$, donc le mix vaut $\sum_i Q_{r,i}P_{b,i} - Q_r P_b^{\text{moy}}$ et la somme vaut $\sum_i Q_{r,i}P_{b,i} - Q_b P_b^{\text{moy}} = \sum_i Q_{r,i}P_{b,i} - \sum_i Q_{b,i}P_{b,i}$ (car $Q_b P_b^{\text{moy}} = \sum_i Q_{b,i}P_{b,i}$). En ajoutant le prix $\sum_i Q_{r,i}(P_{r,i} - P_{b,i})$, on obtient $\sum_i Q_{r,i}P_{r,i} - \sum_i Q_{b,i}P_{b,i}$: l'écart total. □ Vérification numérique sur 1 000 tirages aléatoires :

```
rng = np.random.default_rng(3)
ok = True
for _ in range(1000):
    d = pd.DataFrame({"p": [0, 1, 2], "quantite_budget": rng.integers(10, 100, 3), "quantite_reel":
    ↪ rng.integers(10, 100, 3)})
    d["ca_budget"] = d["quantite_budget"] * rng.uniform(5, 50, 3); d["ca_reel"] = d["quantite_reel"] *
    ↪ rng.uniform(5, 50, 3)
    r = 0.pvm_ca(d, ["p"])
    ok &= abs(r["total"] - r["ecart"]) < 1e-6
print("identité vérifiée sur 1 000 tirages :", bool(ok))
```

```
identité vérifiée sur 1 000 tirages : True
```

Corrigé 7.7.

```
for c in ("Jardin", "Décoration"):
    r = 0.pvm_ca(bud[bud["categorie"] == c], ["canal"])
    print(f"{c:11s}", {k: round(v) for k, v in r.items()})
```

```
Jardin      {'volume': 23042, 'mix': 217, 'prix': 4211, 'total': 27469, 'ecart': 27469}
Décoration  {'volume': -9576, 'mix': -87, 'prix': -747, 'total': -10410, 'ecart': -10410}
```

Pour le **Jardin**, l'écart est positif et presque entièrement un effet volume ; pour la **Décoration**, il est négatif et l'effet volume est également le plus important, avec un effet de prix très faible. On en tire que l'écart de la Décoration est une affaire de **quantités** (les clients en achètent moins que prévu, surtout en Boutique), pas de prix.

Corrigé 7.8. (a) *Météo* : « les jours de pluie ont été plus nombreux en 2025 qu'en 2024 » ; test : comparer le nombre de jours de pluie ; réfutée si 2025 en compte moins (c'est le cas). (b) *Ruptures* : « les jours-produits en rupture sont plus fréquents en 2025 qu'en 2024 » ; test : comparer les taux ; impossible ici faute de données 2024. (c) *Transfert* : « chez les clients actifs les deux années, la part de commandes en Boutique a baissé » ; test : comparer les parts avec un intervalle ; réfutée si l'intervalle contient zéro (ce n'est pas le cas).

Corrigé 7.9. (1) Pourquoi la marge dépasse-t-elle le budget de 8,6 % ? Parce que le volume est plus élevé et que la marge par unité est plus forte. (2) Pourquoi la marge par unité est-elle plus forte ? Pour le prix (3 783 € hors taxe) et surtout pour le coût (17 275 €). (3) Pourquoi le coût est-il plus bas ? Parce que le budget supposait +3,1 % et que le coût n'a augmenté que de 0,8 %. (4) Pourquoi cette hypothèse ? On ne sait pas : la règle de construction du budget n'est pas documentée (non démontré). (5) À documenter. Les trois premiers niveaux sont chiffrés, les deux derniers ne le sont pas.

Corrigé 7.10. Risques : (1) **post hoc** : la publicité a augmenté *et* la Boutique a reculé, sans lien prouvé ; (2) **cause commune** possible (saison, évolution des habitudes) ; (3) l'affirmation est **causale** alors que les données sont observationnelles ; (4) un seul mécanisme est avancé. Données demandées : les **dépenses publicitaires** par mois comparées à la **part** du Site, le **comportement des mêmes clients** avant et après une campagne, et idéalement une **expérience** (campagne testée sur une région ou une période et pas sur l'autre). Le test serait : la part des commandes passées en Boutique par les clients exposés baisse-t-elle davantage que celle des non exposés ?

Chapitre 8 : ⊕ Pareto, analyse ABC et benchmarking — exercices et applications

Ce chapitre du cahier prolonge le chapitre 8 du livre (chapitre complémentaire). Les **applications** sont de petites études guidées sur les produits, les clients et les indicateurs de la boutique ; les **exercices** sont numérotés, avec leur niveau (★ de base, ★★ intermédiaire, ★★★ plus délicat) ; les **corrigés** sont à la fin. Tout le code est exécuté : essayez avant de regarder.

Une première cellule charge les bibliothèques et les fichiers ; les autres reprennent les noms ainsi définis.

```
import os, sys, warnings
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import outils_ch08 as O

D = os.environ["DONNEES"]
cmd, lg, prod, ret, cli = O.charger(D)
sect = pd.read_csv(os.path.join(D, "benchmark_secteur.csv"))
l25 = lg[lg["annee"] == 2025]
ca_produit = l25.groupby("id_produit")["montant"].sum()
print(len(lg), "lignes de commande |", len(prod), "produits |", len(cli), "clients |", len(sect),
      ↪ "indicateurs de secteur (fictifs)")
```

83905 lignes de commande | 120 produits | 6000 clients | 12 indicateurs de secteur (fictifs)

Applications

Application 8.1 — Pareto d'une année à l'autre (section 8.1)

Objectif. Comparer la concentration du chiffre d'affaires par produit en 2023, 2024 et 2025.

```
for an in (2023, 2024, 2025):
    d = O.pareto(lg[lg["annee"] == an].groupby("id_produit")["montant"].sum())
    print(an, "| les 20 % premiers font", round(d.loc[d["part_elements"] <= 20, "part_cumulee"].max(),
          ↪ 1), "% | Gini", round(O.gini(d["valeur"]), 3))
```

2023 | les 20 % premiers font 50.5 % | Gini 0.479
2024 | les 20 % premiers font 50.3 % | Gini 0.476
2025 | les 20 % premiers font 51.0 % | Gini 0.479

Lecture. La part des 20 % premiers produits est stable (50,5 %, 50,3 %, 51,0 %) et le Gini aussi (0,479, 0,476, 0,479) : sur trois ans, la concentration ne bouge pas. Avec 120 produits, une différence d'un demi-point n'est de toute façon pas un signal.

À vous. La concentration augmente-t-elle ou baisse-t-elle ? Est-ce significatif, vu le nombre de produits ?

Application 8.2 — ABC par catégorie (section 8.1)

Objectif. Faire une analyse ABC à l'intérieur de chaque catégorie plutôt que sur tout le catalogue, et comparer.

```
global_abc = 0.classes_abc(ca_produit)
cat_de = prod.set_index("id_produit")["categorie"]
local = pd.concat([0.classes_abc(s) for _, s in
    ↪ ca_produit.groupby(cat_de.reindex(ca_produit.index))]).reindex(ca_produit.index)
print(pd.crosstab(global_abc.rename("classe globale"), local.rename("classe dans la
    ↪ catégorie")).to_string())
```

classe dans la catégorie	A	B	C
classe globale			
A	48	8	0
B	13	15	4
C	5	8	19

Étape 2. Quelle catégorie n'a **aucun** produit en classe A globale ?

```
print(pd.crosstab(cat_de.reindex(global_abc.index), global_abc).to_string())
```

col_0	A	B	C
categorie			
Bien-être	4	9	7
Cuisine	12	4	4
Décoration	14	4	2
Jardin	12	4	4
Maison	14	5	1
Papeterie	0	6	14

Lecture. La Papeterie n'a **aucun** produit en classe A (14 produits C sur 20) : ses produits sont peu chers (entre 3 et 25 €). Classés à l'intérieur de la catégorie, 48 des 56 produits A globaux restent A, mais 13 produits B globaux deviennent A dans leur catégorie : c'est utile pour juger la place de chaque produit **parmi ses semblables**, par exemple pour la Papeterie.

À vous. Quand est-il préférable de classer à l'intérieur d'une catégorie ?

Application 8.3 — Les remboursements par classe (section 8.1)

Objectif. Croiser les classes ABC (chiffre d'affaires) avec la part des remboursements.

```
rb = l25[l25["retournee"]].merge(ret[["id_ligne", "montant_rembourse"]],
    ↪ on="id_ligne").groupby("id_produit")["montant_rembourse"].sum()
t = pd.DataFrame({"remboursements": rb.groupby(global_abc.reindex(rb.index)).sum(), "ca":
    ↪ ca_produit.groupby(global_abc).sum()})
t["remboursé / CA %"] = (t["remboursements"] / t["ca"] * 100).round(2)
t["part des remboursements %"] = (t["remboursements"] / t["remboursements"].sum() * 100).round(1)
print(t.round(1).to_string())
```

	remboursements	ca	remboursé / CA %	part des remboursements %
A	67931.8	1068313.7	6.4	80.4
B	12250.1	192883.9	6.4	14.5

C	4287.0	63566.2	6.7	5.1
---	--------	---------	-----	-----

Lecture. Le taux de remboursement rapporté au chiffre d'affaires est presque le même dans les trois classes (6,4 %, 6,4 % et 6,7 %) : les produits A concentrent les remboursements (80,4 %) **en proportion** de leur chiffre d'affaires, pas davantage. Le Pareto des retours reproduit donc celui des ventes : un retour est un risque de **volume**, pas d'un produit en particulier.

À vous. Les produits A concentrent-ils les remboursements en proportion de leur chiffre d'affaires ?

Application 8.4 — Produits irréguliers (section 8.1)

Objectif. Repérer les produits de classe A dont la demande est irrégulière et estimer leur besoin de stock de sécurité.

```
mens = l25.groupby(["id_produit", "mois"])["quantite"].sum().unstack(fill_value=0)
cv = mens.std(axis=1) / mens.mean(axis=1)
irreguliers = cv[(global_abc == "A") & (cv >= 0.60)].sort_values(ascending=False)
print(len(irreguliers), "produits A à demande irrégulière (coefficient de variation d'au moins 0,60)")
print(pd.DataFrame({"produit": prod.set_index("id_produit").loc[irreguliers.index[:5],
↪ "nom_produit"].values, "cv": irreguliers.iloc[:5].round(2).values}).to_string(index=False))
```

```
12 produits A à demande irrégulière (coefficient de variation d'au moins 0,60)
      produit  cv
Guirlande compact 0.88
      Vase mat 0.76
      Gants design 0.74
Statuette rustique 0.70
      Transat nordique 0.69
```

Étape 2. Un stock de sécurité simple vaut $z \sigma \sqrt{L}$ avec $z = 1,65$ (95 %), σ l'écart-type de la demande **mensuelle** et L le délai de réapprovisionnement exprimé en **mois** (ici un demi-mois).

```
sig = mens.std(axis=1)
ss = (1.65 * sig * np.sqrt(0.5)).round(0)
print("stock de sécurité (unités) pour les cinq premiers :",
↪ ss.loc[irreguliers.index[:5]].astype(int).to_dict())
```

```
stock de sécurité (unités) pour les cinq premiers : {54: 17, 52: 19, 98: 9, 55: 15, 86: 29}
```

Lecture. Douze produits A ont une demande irrégulière ; le plus irrégulier, la « Guirlande compact », a un coefficient de variation de 0,88. Pour les cinq premiers, le stock de sécurité va de 9 à 29 unités, alors que leur demande mensuelle moyenne va de 10 à 36 unités.

À vous. Pourquoi le stock de sécurité est-il plus élevé pour un produit irrégulier que pour un produit régulier de même volume ?

Application 8.5 — Positionner la boutique (section 8.2)

Objectif. Positionner la boutique par rapport au secteur (données fictives), puis tester la sensibilité du verdict.

```
pos = 0.positionner(0.indicateurs_boutique(D, 2025), sect)
print(pos[["indicateur", "valeur", "mediane_secteur", "ecart_std",
↪ "verdict"]].round(2).to_string(index=False))
```

	indicateur	valeur	mediane_secteur	ecart_std	verdict
	Taux de marge brute (HT)	37.96	38.0	-0.01	proche de la médiane
	Taux de retour (lignes)	6.29	5.5	0.21	proche de la médiane

Panier moyen	102.33	92.0	0.29	favorable
Taux de conversion du site	4.78	2.6	1.47	favorable
Part du site dans le CA	46.63	35.0	0.52	neutre
Rotation du stock (par an)	4.23	4.2	0.01	proche de la médiane
Taux de rupture de stock	7.36	4.0	0.91	défavorable
Livraisons à l'heure	73.47	92.0	-2.50	défavorable
Coût d'acquisition d'un client	115.37	18.0	8.21	défavorable
Clients actifs à 12 mois	64.58	42.0	1.22	favorable
Part des frais de personnel dans le CA	12.78	24.0	-1.51	favorable

Étape 2. Que deviennent les verdicts si l'on considère qu'un écart de moins de 0,5 écart-type est « proche de la médiane » (au lieu de 0,25) ?

```
pos["verdict_05"] = np.where(pos["sens"] == 0, "neutre", np.where(pos["ecart_std"].abs() < 0.5,
↪ "proche de la médiane", pos["verdict"]))
print(pos.loc[pos["verdict"] != pos["verdict_05"], ["indicateur", "ecart_std", "verdict",
↪ "verdict_05"]].round(2).to_string(index=False))
```

indicateur	ecart_std	verdict	verdict_05
Panier moyen	0.29	favorable	proche de la médiane

Lecture. Un seul verdict change avec le seuil de 0,5 écart-type : le **panier moyen** (écart de 0,29), qui passe de « favorable » à « proche de la médiane ». Les autres verdicts — ruptures, livraisons, coût d'acquisition, conversion, clients actifs, frais de personnel — sont **robustes** : leurs écarts dépassent 0,9 écart-type.

À vous. Quel est l'indicateur dont le verdict est le plus **robuste** au choix du seuil ?

Exercices

Exercice 8.1 ★ (section 8.1). Cinq produits ont pour ventes 50, 30, 10, 6 et 4 €. Calculez les parts, les parts cumulées et les classes ABC (80 % et 95 %).

Exercice 8.2 ★★ (section 8.1). Calculez à la main le coefficient de Gini de quatre produits de ventes 1, 1, 2 et 6, avec la formule $G = \frac{2 \sum_i i x_{(i)}}{n \sum_i x_i} - \frac{n+1}{n}$ où $x_{(i)}$ sont les valeurs triées par ordre croissant. Vérifiez avec 0.gini.

Exercice 8.3 ★★ (section 8.1). Combien de produits faut-il pour atteindre 50 %, 70 % et 90 % du chiffre d'affaires de 2025 ? Qu'en concluez-vous sur la concentration ?

Exercice 8.4 ★★ (section 8.1). Calculez la part des produits A de 2023 qui sont encore A en 2025. Comparez avec le chiffre 2024–2025 du livre.

Exercice 8.5 ★★ (section 8.1). Refaites l'analyse ABC sur la **marge** de 2025 et comparez les produits A en chiffre d'affaires et A en marge : combien sont communs ? La Papeterie a-t-elle des produits A en marge ?

Exercice 8.6 ★★★ (section 8.1). Refaites la classification XYZ avec la variabilité **hebdomadaire** au lieu de la variabilité mensuelle (coefficient de variation des ventes par semaine). Les classes changent-elles ? Pourquoi un pas de temps plus fin donne-t-il des coefficients plus élevés ?

Exercice 8.7 ★ (section 8.2). Le taux de retour de la boutique est de 6,3 %, la médiane du secteur de 5,5 %, les quartiles de 3,5 % et 8,5 %. Calculez à la main l'écart standardisé robuste et dites si l'écart est favorable.

Exercice 8.8 ★★ (section 8.2). Recalculez le coût d'acquisition avec une définition plus étroite : seules les dépenses **payantes et réseaux sociaux** (fichier `campagnes.csv`) divisées par les nouveaux clients dont le canal d'acquisition est le **Site** ou les **Réseaux**. Cet indicateur se rapproche-t-il de la médiane du secteur ?

Exercice 8.9 ★★ (section 8.2). Les livraisons à l'heure dépendent du délai **promis**. Calculez la part des livraisons de 2025 effectuées en 6, 8 et 10 jours au plus (depuis la commande). Que concluez-vous sur la comparabilité avec un secteur dont on ignore le délai promis ?

Corrigés

Corrigé 8.1. Parts : 50 %, 30 %, 10 %, 6 %, 4 % ; cumul : 50, 80, 90, 96, 100 %. Avec la règle « A tant que le cumul **avant** l'élément est inférieur à 80 % » : le produit 1 (cumul avant 0) est A, le produit 2 (50) est A, le produit 3 (80) est B, le produit 4 (90) est B, le produit 5 (96) est C.

```
v = pd.Series([50, 30, 10, 6, 4], index=list("12345"), dtype=float)
print(0.pareto(v)[["valeur", "part_cumulee"]].round(1).assign(classe=0.classes_abc(v)).to_string())
```

	valeur	part_cumulee	classe
1	50.0	50.0	A
2	30.0	80.0	A
3	10.0	90.0	B
4	6.0	96.0	B
5	4.0	100.0	C

Corrigé 8.2. Valeurs triées 1, 1, 2, 6 ($n = 4$, somme 10). $\sum_i i x_{(i)} = 1 \cdot 1 + 2 \cdot 1 + 3 \cdot 2 + 4 \cdot 6 = 33$.

$$G = \frac{2 \times 33}{4 \times 10} - \frac{5}{4} = 1,65 - 1,25 = 0,40.$$

```
print("Gini :", round(0.gini([1, 1, 2, 6]), 2))
```

Gini : 0.4

Corrigé 8.3.

```
d = 0.pareto(ca_produit)
for part in (50, 70, 90):
    n = int((d["part_cumulee"] < part).sum() + 1)
    print(f"{part} % du chiffre d'affaires : {n} produits sur {len(d)} ({n / len(d) * 100:.0f} %)")
```

50 % du chiffre d'affaires : 24 produits sur 120 (20 %)
 70 % du chiffre d'affaires : 43 produits sur 120 (36 %)
 90 % du chiffre d'affaires : 74 produits sur 120 (62 %)

Il faut 24 produits (20 % du catalogue) pour la moitié du chiffre d'affaires, 43 (36 %) pour 70 % et 74 (62 %) pour 90 % : la concentration est **modérée** et la courbe monte régulièrement ; il n'y a pas de poignée de produits qui porte l'entreprise.

Corrigé 8.4.

```
abc23 = 0.classes_abc(lg[lg["annee"] == 2023].groupby("id_produit")["montant"].sum())
a23 = abc23[abc23 == "A"].index
print("produits A en 2023 :", len(a23), "| encore A en 2025 :",
      ↪ int(global_abc.reindex(a23).eq("A").sum()), "(", round(global_abc.reindex(a23).eq("A").mean() *
      ↪ 100, 1), "%)")
```

produits A en 2023 : 55 | encore A en 2025 : 54 (98.2 %)

Corrigé 8.5.

```
abc_marge = 0.classes_abc(l25.groupby("id_produit")["marge"].sum())
a_ca, a_m = set(global_abc[global_abc == "A"].index), set(abc_marge[abc_marge == "A"].index)
print("A en CA :", len(a_ca), "| A en marge :", len(a_m), "| communs :", len(a_ca & a_m))
print("produits A en marge dans la Papeterie :", int(sum(cat_de[p] == "Papeterie" for p in a_m)))
```

```
A en CA : 56 | A en marge : 54 | communs : 50
produits A en marge dans la Papeterie : 0
```

Corrigé 8.6.

```
l25b = l25.merge(cmd[["id_commande", "date_commande"]], on="id_commande")
sem = l25b.assign(semaine=l25b["date_commande"].dt.isocalendar().week).groupby(["id_produit",
↳ "semaine"])[["quantite"].sum().unstack(fill_value=0)
cv_h = sem.std(axis=1) / sem.mean(axis=1)
print("coefficient de variation médian : mensuel", round(cv.median(), 2), "| hebdomadaire",
↳ round(cv_h.median(), 2))
print(pd.crosstab(global_abc.rename("ABC"), pd.cut(cv_h, [0, 0.45, 0.6, 10], labels=["X", "Y", "Z"],
↳ right=False).rename("XYZ hebdo")).to_string())
```

```
coefficient de variation médian : mensuel 0.48 | hebdomadaire 0.66
XYZ hebdo  X   Y   Z
ABC
A           1  19  36
B           0  10  22
C           0   3  29
```

À pas de temps plus fin, une même demande moyenne se répartit en peu d'unités par période : le **hasard de Poisson** (une semaine ordinaire compte tantôt 3, tantôt 7 ventes) fait alors varier fortement le coefficient. La classe XYZ dépend donc du pas de temps ; on la choisit en cohérence avec la fréquence de réapprovisionnement.

Corrigé 8.7. Écart-type robuste = $(8,5 - 3,5) / 1,349 \approx 3,71$; écart standardisé = $(6,3 - 5,5) / 3,71 \approx 0,22$. Un taux de retour **plus bas** est favorable : ici il est **plus haut** que la médiane (légèrement défavorable), mais l'écart est inférieur à un quart d'écart-type : on le dit « proche de la médiane ».

```
print("écart-type robuste :", round((8.5 - 3.5) / 1.349, 2), "| écart standardisé :", round((6.3 -
↳ 5.5) / ((8.5 - 3.5) / 1.349), 2))
```

```
écart-type robuste : 3.71 | écart standardisé : 0.22
```

Corrigé 8.8.

```
camp = pd.read_csv(os.path.join(D, "campagnes.csv"))
dep = camp.loc[camp["source"].isin(["payant", "reseaux"]), "depense"].sum()
nouveaux = cli[(pd.to_datetime(cli["date_inscription"]).dt.year == 2025) &
↳ cli["canal_acquisition"].isin(["Site", "Réseaux"])]
print("dépenses payant + réseaux :", round(dep), "€ | nouveaux clients Site ou Réseaux :",
↳ len(nouveaux), "| coût :", round(dep / len(nouveaux), 1), "€ | médiane du secteur : 18 €")
```

```
dépenses payant + réseaux : 64903 € | nouveaux clients Site ou Réseaux : 337 | coût : 192.6 € |
↳ médiane du secteur : 18 €
```

Avec cette définition, le coût est de **193 €**, donc **plus élevé** que les 115 € de l'indicateur large : on retire les courriels (peu coûteux), mais aussi la plupart des nouveaux clients (ceux qui s'inscrivent en Boutique). Aucune de ces deux définitions ne rapproche l'indicateur de 18 € : **l'écart avec le secteur n'est pas qu'une affaire de définition**, et le fichier ne dit pas quels nouveaux clients viennent réellement de la publicité. La bonne conduite : ajuster la définition, mesurer ce qui reste, et en conclure que la donnée manque pour trancher.

Corrigé 8.9.

```
liv = pd.read_csv(os.path.join(D, "livraisons.csv"))
liv = liv[liv["date_commande"].str[:4] == "2025"]
duree = (pd.to_datetime(liv["date_livraison"]) - pd.to_datetime(liv["date_commande"])).dt.days
for j in (6, 8, 10):
    print(f"livrées en {j} jours au plus : {(duree <= j).mean() * 100:.1f} %")
```

```
livrées en 6 jours au plus : 73.5 %
livrées en 8 jours au plus : 96.3 %
livrées en 10 jours au plus : 99.7 %
```

La part de livraisons « à l'heure » passe de **73,5 %** (six jours) à **96,3 %** (huit jours) et **99,7 %** (dix jours), **sans livrer plus vite** : sans connaître le délai promis par le secteur, la comparaison avec une médiane de 92 % n'a pas de sens.

Chapitre 9 : ⊕ Analyse financière — exercices et applications

Ce chapitre du cahier prolonge le chapitre 9 du livre (complémentaire). Les **applications** sont de petites études guidées sur les comptes simulés de la boutique (compte de résultat mensuel, bilan annuel) ; vous les refaites pas à pas, puis vous prolongez dans les rubriques « À vous ». Les **exercices** (★ facile, ★★ moyen, ★★★ plus long) renvoient chacun à une section du livre ; leurs **corrigés** sont à la fin. Les comptes sont **simulés et simplifiés** (TVA fictive de 20 %, résultat net approché à 70 % du résultat d'exploitation, bilan équilibré par construction) : rien ici n'est un avis comptable ou fiscal.

Une première cellule charge les bibliothèques et les fichiers ; les autres reprennent les noms ainsi définis.

```
import os, sys, warnings
import numpy as np, pandas as pd, statsmodels.api as sm
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import outils_ch09 as O      # chargement, comptes annuels, ratios, régression des
    ↪ coûts, point mort, rentabilité par canal

pd.options.display.width = 150
pd.options.display.max_columns = 20
cr, bil, cmd, lig, prod, camp, bm = O.charger()
ann = O.annuel(cr)
bi = bil.set_index("annee")
fr = lambda x, nd=0: f"{x:,.{nd}f}".replace(",", " ").replace(".", ",").replace("-", "-")
print(len(cr), "mois de comptes |", len(bil), "bilans |", len(cmd), "commandes |", len(lig), "lignes
    ↪ de commande")
```

```
36 mois de comptes | 3 bilans | 36395 commandes | 83905 lignes de commande
```

Applications

Application 9.1 — Recouper les comptes avec les ventes (section 9.1 du livre)

Objectif. S'assurer que le chiffre d'affaires du compte de résultat se retrouve dans la base, puis que le bilan s'équilibre.

Étape 1 — Reconstituer le chiffre d'affaires hors taxes à partir des lignes de commande (montants toutes taxes comprises, divisés par 1,2).

```
ventes = 0.ventes_mensuelles(cmd, lig)
ecart = cr.set_index("mois")["ca_ht"] - ventes
print("écart absolu maximal :", round(ecart.abs().max(), 2), "€ | écart moyen :", round(ecart.mean(),
↳ 3), "€")
```

```
écart absolu maximal : 0.49 € | écart moyen : 0.007 €
```

Étape 2 — Mesurer l'effet d'une mauvaise TVA. Si l'on divisait par 1,196 au lieu de 1,2, quel écart relatif obtiendrait-on ?

```
ventes_mauvaise = ventes * 1.2 / 1.196
rel = (ventes_mauvaise / cr.set_index("mois")["ca_ht"] - 1) * 100
print("écart relatif moyen avec une TVA de 19,6 % :", round(rel.mean(), 2), "%")
```

```
écart relatif moyen avec une TVA de 19,6 % : 0.33 %
```

Étape 3 — Vérifier l'équilibre du bilan.

```
actif = bi[["immobilisations_nettes", "stock", "creances_clients", "tresorerie"]].sum(axis=1)
passif = bi[["capitaux_propres", "emprunt", "dettes_fournisseurs", "autres_dettes"]].sum(axis=1)
print((actif - passif).to_dict())
```

```
{2023: 0, 2024: 1, 2025: 0}
```

À vous. (1) Quel est le mois où l'écart absolu est maximal, et pourquoi n'est-il pas nul ? (2) Faites-vous confiance à un compte de résultat dont le chiffre d'affaires s'écarte de 0,35 % de la base ? Que feriez-vous ? (3) Un écart d'actif moins passif de 1 € est-il une erreur ?

Application 9.2 — Calculer les ratios à partir des comptes (section 9.2 du livre)

Objectif. Refaire les ratios de rentabilité, de cycle d'exploitation et de solidité **sans** la fonction du livre, pour en comprendre chaque dénominateur.

Étape 1 — Marges.

```
marge = pd.DataFrame({"taux_marge_brute": ann["marge_brute"] / ann["ca_ht"],
↳ "taux_marge_exploitation": ann["resultat_exploitation"] / ann["ca_ht"]})
print((marge * 100).round(1).T)
```

annee	2023	2024	2025
taux_marge_brute	36.4	36.4	37.8
taux_marge_exploitation	1.8	1.8	3.6

Étape 2 — Stock, délais et BFR.

```
rotation = ann["achats"] / bi["stock"]
delai_clients = bi["creances_clients"] / ann["ca_ht"] * 365
delai_fournisseurs = bi["dettes_fournisseurs"] / ann["achats"] * 365
bfr = bi["stock"] + bi["creances_clients"] - bi["dettes_fournisseurs"]
print(pd.DataFrame({"rotation": rotation, "jours de stock": 365 / rotation, "délai clients":
↳ delai_clients, "délai fournisseurs": delai_fournisseurs, "BFR": bfr}).round(1).T)
```

annee	2023	2024	2025
rotation	4.4	4.3	4.2
jours de stock	83.0	84.6	86.3
délai clients	10.6	10.6	10.6
délai fournisseurs	42.6	42.6	42.6
BFR	94583.0	101762.0	114186.0

Étape 3 — Rentabilité des capitaux propres et décomposition en trois facteurs (marge nette × rotation de l'actif × levier financier, dite décomposition de DuPont).

```
net = 0.7 * ann["resultat_exploitation"] # résultat net approché
total_actif = bi[["immobilisations_nettes", "stock", "creances_clients", "tresorerie"]].sum(axis=1)
roe = net / bi["capitaux_propres"]
facteurs = pd.DataFrame({"marge nette": net / ann["ca_ht"], "rotation de l'actif": ann["ca_ht"] /
    ↳ total_actif, "levier": total_actif / bi["capitaux_propres"]})
facteurs["produit"] = facteurs.prod(axis=1)
print(pd.concat([facteurs, roe.rename("ROE")], axis=1).round(3).T)
```

annee	2023	2024	2025
marge nette	0.013	0.012	0.025
rotation de l'actif	2.502	2.592	2.704
levier	2.344	2.196	2.020
produit	0.073	0.071	0.138
ROE	0.073	0.071	0.138

À vous. (1) Entre 2024 et 2025, lequel des trois facteurs explique la hausse de la rentabilité des capitaux propres ? (2) Calculez la liquidité générale en supposant que 30 000 € de l'emprunt sont exigibles à moins d'un an au lieu de 15 000 € : conclut-on autrement ? (3) Pourquoi le délai clients de 10,6 jours est-il identique les trois années, et que dirait-on s'il augmentait ?

Application 9.3 — Séparer coûts fixes et coûts variables (section 9.3 du livre)

Objectif. Estimer, ligne de coût par ligne de coût, la part qui suit le chiffre d'affaires, puis calculer le point mort.

Étape 1 — Une régression par ligne de coût (36 mois).

```
X = sm.add_constant(cr["ca_ht"])
lignes = ["achats", "frais_personnel", "loyers_charges", "marketing", "livraison", "frais_bancaires",
    ↳ "autres_charges"]
res = {c: (sm.OLS(cr[c], X).fit().params["ca_ht"], sm.OLS(cr[c], X).fit().rsquared) for c in lignes}
print(pd.DataFrame(res, index=["pente", "R2"]).T.round(3))
```

	pente	R2
achats	0.610	0.989
frais_personnel	0.046	0.923
loyers_charges	0.002	0.057
marketing	0.043	0.595
livraison	0.033	0.929
frais_bancaires	0.018	1.000
autres_charges	0.003	0.118

Étape 2 — Le point mort de 2025, en traitant le marketing comme un coût fixe.

```
x = ann.loc[2025]
b_pers = sm.OLS(cr["frais_personnel"], X).fit().params["ca_ht"]
cv = (x["achats"] - x["variation_stock"]) + x["livraison"] + x["frais_bancaires"] + b_pers *
    ↳ x["ca_ht"]
cf = x["charges_totales"] - x["livraison"] - x["frais_bancaires"] - b_pers * x["ca_ht"]
pm = 0.point_mort(x["ca_ht"], cv, cf)
print("taux de MCV :", round(pm["taux_mcv"] * 100, 1), "% | seuil :", round(pm["seuil"]), "€ | marge
    ↳ de sécurité :", round(pm["marge_securite"] * 100, 1), "% | levier :", round(pm["levier"], 1))
```

taux de MCV : 28.5 % | seuil : 963968 € | marge de sécurité : 12.7 % | levier : 7.9

Étape 3 — Traiter le marketing comme variable (pente de 0,043 € par euro de chiffre d'affaires) et recalculer le seuil.

```
b_mkt = sm.OLS(cr["marketing"], X).fit().params["ca_ht"]
cv2, cf2 = cv + b_mkt * x["ca_ht"], cf - b_mkt * x["ca_ht"]
pm2 = 0.point_mort(x["ca_ht"], cv2, cf2)
print("seuil si le marketing est variable :", round(pm2["seuil"]), "€ | marge de sécurité :",
      ↪ round(pm2["marge_securite"] * 100, 1), "%")
```

seuil si le marketing est variable : 939403 € | marge de sécurité : 14.9 %

À vous. (1) De combien le seuil change-t-il entre les deux traitements du marketing, et lequel est le plus prudent pour un budget ? (2) Calculez le seuil en nombre de commandes avec le panier moyen hors taxes de 2025. (3) Pourquoi la pente des loyers n'est-elle pas exactement nulle ?

Application 9.4 — Rentabilité par canal et clés de répartition (section 9.3 du livre)

Objectif. Voir comment le choix d'une clé de répartition change le résultat d'un canal, et pourquoi la contribution est plus fiable.

Étape 1 — Contribution et résultat par canal avec les deux clés du livre.

```
g, tot = 0.rentabilite_canal(cmd, lig, prod, cr, camp)
print(g[["ca_ht", "marge_brute", "contribution", "resultat_cle_ca",
      ↪ "resultat_cle_commandes"]].round(0).astype(int))
```

	ca_ht	marge_brute	contribution	resultat_cle_ca	resultat_cle_commandes
canal					
Boutique	467478	177622	169207	62194	62975
Réseaux	121729	46392	15683	-12183	-12154
Site	514763	195003	109595	-8242	-9052

Étape 2 — Une troisième clé : la marge brute. Répartissons les coûts communs au prorata de la marge brute.

```
communes = tot["frais_personnel"] + tot["loyers_charges"] + tot["amortissements"] +
      ↪ tot["autres_charges"]
g["resultat_cle_marge"] = g["contribution"] - g["marge_brute"] / g["marge_brute"].sum() * communes
print(g[["contribution", "resultat_cle_ca", "resultat_cle_commandes",
      ↪ "resultat_cle_marge"]].round(0).astype(int))
```

	contribution	resultat_cle_ca	resultat_cle_commandes	resultat_cle_marge
canal				
Boutique	169207	62194	62975	62080
Réseaux	15683	-12183	-12154	-12297
Site	109595	-8242	-9052	-8014

Étape 3 — Réconciliation avec le résultat de l'entreprise.

```
somme = g["resultat_cle_ca"].sum()
print("somme des résultats par canal :", round(somme), "€ | résultat d'exploitation :",
      ↪ round(ann.loc[2025, 'resultat_exploitation']), "€ | variation de stock :", round(ann.loc[2025,
      ↪ 'variation_stock']), "€ | reste :", round(somme + ann.loc[2025, 'variation_stock'] -
      ↪ ann.loc[2025, 'resultat_exploitation']), "€")
```

somme des résultats par canal : 41769 € | résultat d'exploitation : 39879 € | variation de stock :
 ↪ -1888 € | reste : 2 €

À vous. (1) La conclusion « le Site perd de l'argent » dépend-elle de la clé ? (2) Si l'on pouvait supprimer 30 % des coûts communs imputés à Réseaux en arrêtant ce canal, faut-il l'arrêter ? (3) Réaffectez le marketing de façon différente (tout au Site) et observez la contribution de Réseaux.

Application 9.5 — Hausse de prix ou hausse de volume ? (section 9.3 du livre)

Objectif. Chiffrer l'effet d'une hausse de prix et d'une hausse de volume, puis chercher la baisse de volume qui annule le gain d'une hausse de prix.

Étape 1 — Les deux leviers.

```
ca = x["ca_ht"]
liees_ca = x["frais_bancaires"] + b_pers * ca           # coûts proportionnels au chiffre d'affaires
prix = 0.01 * (ca - liees_ca)
volume = 0.01 * (ca - cv)
print("+1 % de prix :", round(prix), "€ | +1 % de volume :", round(volume), "€ | rapport :",
      ↪ round(prix / volume, 2))
```

+1 % de prix : 10328 € | +1 % de volume : 3145 € | rapport : 3.28

Étape 2 — Hausse de prix de 3 % : quelle perte de volume la compense ? Après la hausse de prix de p , le chiffre d'affaires est multiplié par $(1 + p)$ et le coût des marchandises ne change pas ; une baisse de volume de q retire q de toutes les ventes et de tous les coûts variables.

```
p = 0.03
gain = p * (ca - liees_ca)
def resultat(p, q):
    ca_ = ca * (1 + p) * (1 - q)
    cv_ = ((x["achats"] - x["variation_stock"]) + x["livraison"]) * (1 - q) + (x["frais_bancaires"] +
    ↪ b_pers * ca) * (1 + p) * (1 - q)
    return ca_ - cv_ - cf
q_zero = next(q for q in np.arange(0, 0.2, 0.0005) if resultat(p, q) <= resultat(0, 0))
print("gain d'une hausse de 3 % de prix à volume constant :", round(gain), "€ | baisse de volume qui
    ↪ l'annule :", round(q_zero * 100, 1), "%")
```

gain d'une hausse de 3 % de prix à volume constant : 30985 € | baisse de volume qui l'annule : 9.0 %

À vous. (1) Que devient ce seuil si la hausse de prix est de 1 % ? (2) Commentez : « une baisse de 5 % du volume après une hausse de prix de 3 % est-elle acceptable ? » (3) Pourquoi cette analyse est-elle en réalité un premier pas vers l'analyse de sensibilité du chapitre 13 ?

Exercices

Exercice 9.1 ★ — Une cascade à la main (section 9.1 du livre)

Un commerce a un chiffre d'affaires hors taxes de 240 000 €, des achats de marchandises de 150 000 € et une variation de stock de −5 000 € (le stock a baissé). Ses charges d'exploitation totalisent 70 000 €. Calculez la marge brute, son taux et le résultat d'exploitation.

Exercice 9.2 ★ — Marge ou marque ? (section 9.1 du livre)

Un article acheté 60 € hors taxes est vendu 120 € toutes taxes comprises (TVA de 20 %). Calculez son prix de vente hors taxes, sa marge, son taux de marge (sur le coût d'achat) et son taux de marque (sur le prix de vente).

Exercice 9.3 ★★ — Un bilan à équilibrer (section 9.1 du livre)

Un bilan compte : immobilisations nettes 80 000 €, stock 120 000 €, créances clients 30 000 €, capitaux propres 150 000 €, emprunt 50 000 €, dettes fournisseurs 60 000 €, autres dettes 40 000 €. Quelle est la trésorerie qui équilibre le bilan ? Quel est le total du bilan ?

Exercice 9.4 ★ — Rotation et délais à la main (section 9.2 du livre)

Un commerce a un stock de fin d'année de 80 000 €, des achats annuels de 400 000 €, des créances clients de 20 000 € pour un chiffre d'affaires de 480 000 €, et des dettes fournisseurs de 50 000 €. Calculez la rotation du stock, les jours de stock, le délai clients et le délai fournisseurs.

Exercice 9.5 ★★ — La croissance consomme de la trésorerie (section 9.2 du livre)

Avec les comptes de 2025, supposez que le chiffre d'affaires augmente de 10 % en 2026 et que le stock, les créances et les dettes fournisseurs augmentent dans la même proportion. De combien le BFR augmente-t-il ? Quelle part du résultat net approché de 2025 cela représente-t-il ?

Exercice 9.6 ★★ — Décomposer la rentabilité (section 9.2 du livre)

Un commerce a un chiffre d'affaires de 800 000 €, un résultat net de 24 000 €, un total d'actif de 400 000 € et des capitaux propres de 160 000 €. Calculez sa marge nette, la rotation de son actif, son levier financier et sa rentabilité des capitaux propres ; vérifiez que le produit des trois premiers donne la dernière.

Exercice 9.7 ★ — Un point mort à la main (section 9.3 du livre)

Un commerce a des coûts fixes de 60 000 € par an et un taux de marge sur coûts variables de 30 %. Calculez son seuil de rentabilité. S'il réalise 250 000 € de chiffre d'affaires, quels sont son résultat, sa marge de sécurité et son levier opérationnel ?

Exercice 9.8 ★★★ — Le coût d'une livraison (section 9.3 du livre)

Les frais de livraison ne concernent que les commandes du Site et des Réseaux. Estimez par régression le coût variable **par commande livrée** à partir des 36 mois de comptes, et comparez-le à la régression sur le chiffre d'affaires. Quelle explication est la plus fidèle ?

Exercice 9.9 ★★ — Arrêter ou garder un canal ? (section 9.3 du livre)

Pour le canal Réseaux en 2025, dites ce qui change dans le résultat de l'entreprise si on l'arrête, dans trois hypothèses : (a) aucun coût commun n'est économisé ; (b) 30 % des coûts communs qui lui sont imputés (clé « chiffre d'affaires ») sont réellement économisés ; (c) 60 % le sont. À partir de quelle part d'économie l'arrêt devient-il favorable ?

Corrigé 9.1

Marge brute = $240\,000 - 150\,000 + (-5\,000) = 85\,000$ € (un stock qui baisse veut dire que l'on a vendu des articles achetés avant : leur coût s'ajoute au coût des achats de l'année). Taux de marge brute = $85\,000/240\,000 = 35,4$ %. Résultat d'exploitation = $85\,000 - 70\,000 = 15\,000$ €, soit un taux de marge d'exploitation de 6,25 %.

Corrigé 9.2

Prix de vente hors taxes = $120/1,2 = 100$ €. Marge = $100 - 60 = 40$ €. Taux de marge (sur le coût) = $40/60 = 66,7$ % ; taux de marque (sur le prix de vente) = $40/100 = 40$ %. Les deux disent la même chose avec des dénominateurs différents : on précise toujours lequel.

Corrigé 9.3

L'actif hors trésorerie vaut $80\,000 + 120\,000 + 30\,000 = 230\,000$ €, le passif $150\,000 + 50\,000 + 60\,000 + 40\,000 = 300\,000$ €. La trésorerie qui équilibre est donc $300\,000 - 230\,000 = 70\,000$ €, et le total du bilan est de 300 000 €.

Corrigé 9.4

Rotation = $400\,000/80\,000 = 5$; jours de stock = $365/5 = 73$; délai clients = $20\,000/480\,000 \times 365 = 15,2$ jours ; délai fournisseurs = $50\,000/400\,000 \times 365 = 45,6$ jours. Le commerce paie ses fournisseurs 45,6 jours après la livraison et vend son stock en 73 jours : il finance donc environ 27 jours de stock avec son propre argent (sans compter le délai clients).

Corrigé 9.5

Le BFR de 2025 vaut 114 186 € ; si ses trois composantes augmentent de 10 %, il augmente de 10 %, soit 11 419 €, ce qui représente 40,9 % du résultat net approché de 27 915 €. **La croissance coûte du financement** : même une entreprise rentable doit trouver de l'argent pour financer le stock supplémentaire.

```
bfr25 = bi.loc[2025, "stock"] + bi.loc[2025, "creances_clients"] - bi.loc[2025, "dettes_fournisseurs"]
print(round(bfr25), round(0.10 * bfr25), round(0.10 * bfr25 / (0.7 * ann.loc[2025,
↪ "resultat_exploitation"]) * 100, 1))
```

```
114186 11419 40.9
```

Corrigé 9.6

Marge nette = $24\,000/800\,000 = 3,0$ % ; rotation de l'actif = $800\,000/400\,000 = 2,0$; levier financier = $400\,000/160\,000 = 2,5$; rentabilité des capitaux propres = $24\,000/160\,000 = 15$ %. Produit : $0,03 \times 2,0 \times 2,5 = 0,15$. On lit que la rentabilité vient de trois sources distinctes : ce que l'on gagne par euro vendu, la vitesse à laquelle l'actif produit des ventes, et la part de dette dans le financement.

Corrigé 9.7

Seuil = $60\,000/0,30 = 200\,000$ €. Pour 250 000 € de chiffre d'affaires, la marge sur coûts variables est de $0,30 \times 250\,000 = 75\,000$ €, donc un résultat de $75\,000 - 60\,000 = 15\,000$ €. Marge de sécurité = $(250\,000 - 200\,000)/250\,000 = 20$ %. Levier opérationnel = $75\,000/15\,000 = 5$: +1 % de chiffre d'affaires donne +5 % de résultat.

Corrigé 9.8

On régresse la livraison mensuelle sur le nombre de commandes livrées du mois (Site et Réseaux).

```
liv = cmd[cmd["canal"] != "Boutique"].assign(mois=lambda d:
↳ d["date_commande"].str[:7]).groupby("mois").size().rename("cmd_livrees")
d = cr.set_index("mois").join(liv)
m_cmd = sm.OLS(d["livraison"], sm.add_constant(d["cmd_livrees"])).fit()
m_ca = sm.OLS(d["livraison"], sm.add_constant(d["ca_ht"])).fit()
print("coût par commande livrée :", round(m_cmd.params["cmd_livrees"], 2), "€ | R2 :",
↳ round(m_cmd.rsquared, 3), "| R2 sur le CA :", round(m_ca.rsquared, 3))
```

```
coût par commande livrée : 4.2 € | R2 : 1.0 | R2 sur le CA : 0.929
```

Le coût par commande livrée est de 4,20 € avec un R^2 de 1,00 (la vérité programmée est de 4,20 € par commande livrée), contre un R^2 de 0,93 pour la régression sur le chiffre d'affaires. La livraison suit le **nombre de commandes livrées**, pas le chiffre d'affaires : c'est un coût variable **par commande**, qui pèse plus lourd sur les petits paniers. C'est un bon exemple de ce qu'un *inducteur de coût* mal choisi (le CA) donne une explication moins fidèle que le bon (les commandes livrées).

Corrigé 9.9

La contribution de Réseaux est de 15 683 € ; la part de coûts communs qui lui est imputée (clé « chiffre d'affaires ») est de 27 866 €. Si l'on arrête le canal : (a) aucun coût commun économisé : le résultat de l'entreprise baisse de 15 683 € ; (b) 30 % économisés (8 360 €) : il baisse de 15 683 – 8 360 = 7 323 € ; (c) 60 % économisés (16 719 €) : il augmente de 16 719 – 15 683 = 1 036 €. L'arrêt devient favorable à partir d'une économie de 15 683/27 866 = 56,3 % des coûts communs imputés : c'est la question à poser aux équipes (« quels coûts disparaissent réellement ? »), pas à la clé de répartition.

```
g, tot = 0.rentabilite_canal(cmd, lig, prod, cr, camp)
contrib, commun = g.loc["Réseaux", "contribution"], g.loc["Réseaux", "communes_prorata_ca"]
print(round(contrib), round(commun), {p: round(p * commun - contrib) for p in (0, 0.3, 0.6)},
↳ round(contrib / commun * 100, 1))
```

```
15683 27866 {0: -15683, 0.3: -7323, 0.6: 1036} 56.3
```

Pistes des applications

Application 9.1. (1) Le mois d'écart maximal est octobre 2024, avec 0,49 € : les comptes sont arrondis à l'euro, et les ventes de la base sont divisées par 1,2 sans arrondi. (2) Avec une TVA de 19,6 % au lieu de 20 %, l'écart relatif moyen est de 0,33 % : c'est un écart **systématique**, de même signe tous les mois, qui révèle un mauvais taux. Un écart de cet ordre se cherche (taux de TVA, ventes à taux réduit, canal oublié) avant d'analyser. (3) Un euro d'écart d'équilibre vient d'arrondis ; il n'y a pas d'erreur.

Application 9.2. (1) Entre 2024 et 2025, la marge nette double (de 1,2 % à 2,5 %), la rotation de l'actif progresse (de 2,59 à 2,70) et le levier baisse (de 2,20 à 2,02) : la hausse de la rentabilité vient de la **marge**, malgré moins d'endettement. (2) Avec 30 000 € d'emprunt exigible à moins d'un an, la liquidité générale tombe de 2,04 à 1,87 : elle reste supérieure à 1, la conclusion est la même. (3) Le délai clients est constant parce que les créances ont été fabriquées proportionnelles aux ventes ; s'il augmentait, les clients paieraient plus tard et le besoin en fonds de roulement croîtrait.

Application 9.3. (1) Le seuil passe de 963 968 € à 939 403 € (−2,5 %) quand le marketing est traité comme variable ; traiter le marketing comme fixe est plus **prudent** pour un budget (seuil plus haut, marge de sécurité de 12,7 % au lieu de 14,9 %). (2) $963\,968 / 85,27 \approx 11\,304$ commandes. (3) Les loyers ont augmenté par paliers en 2025, en même temps que les ventes : la pente (0,002 €) est un artefact, avec un R^2 de 0,06.

Application 9.4. (1) Non : le Site est en perte avec les trois clés (−8 242 €, −9 052 € et −8 014 €). (2) Il faut que l'économie dépasse 56,3 % des coûts communs imputés (exercice 9.9) : 30 % ne suffisent pas. (3) Si tout le marketing est affecté au Site, la contribution du Site tombe à 87 066 € et celle de Réseaux monte à 38 212 € : la contribution **dépend de l'affectation directe**, qu'il faut donc justifier.

Application 9.5. (1) Pour une hausse de prix de 1 %, la baisse de volume qui annule le gain est de 3,2 % ; pour 3 %, de 9,0 % (une perte de volume égale à trois fois la hausse de prix, environ). (2) Après +3 % de prix et −5 % de volume, le résultat est **encore supérieur de 13 712 €** à celui de départ : la hausse est favorable tant que le volume ne baisse pas de plus de 9 %. (3) Parce qu'on y fait varier **deux paramètres incertains** (prix, volume) et qu'on cherche le seuil de bascule : c'est exactement un calcul de sensibilité.

Chapitre 10 : ⊕ Analytique marketing et web — exercices et applications

Ce chapitre du cahier prolonge le chapitre 10 du livre. Les **applications** se font devant l'ordinateur, par petites étapes ; les **exercices** (★ facile, ★★ moyen, ★★★ difficile) ont des corrigés à la fin. Les données sont **simulées** ; les parcours multi-contacts, le groupe témoin, la vue d'outil et les doublons d'événements sont **fabriqués** par `build/outils_ch10.py` et signalés comme tels. Google Analytics n'est pas exécuté.

```
import os, sys, warnings
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import outils_ch10 as o

s, camp, cmd, lig, prod = o.charger()
m = o.marge_commandes(cmd, lig, prod)
w = s[s["id_commande"].notna()].merge(m, on="id_commande")
print(len(s), "sessions |", int(s["commande"].sum()), "commandes |", len(camp), "lignes de campagne")
```

```
127022 sessions | 6078 commandes | 36 lignes de campagne
```

Applications

Application 10.1 — La conversion par source, avec son incertitude (section 10.1.2)

Objectif. Calculer la conversion de chaque source et dire quelles différences sont fiables.

Étape 1 — la table.

```
g = s.groupby("source").agg(sessions=("commande", "size"), commandes=("commande", "sum"))
g["conversion_%"] = (g["commandes"] / g["sessions"] * 100).round(2)
ic = [0.wilson(k, n) for k, n in zip(g["commandes"], g["sessions"])]
g["ic_bas_%"], g["ic_haut_%"] = [round(a * 100, 2) for a, b in ic], [round(b * 100, 2) for a, b in ic]
print(g.sort_values("conversion_%").to_string())
```

	sessions	commandes	conversion_%	ic_bas_%	ic_haut_%
source					
reseaux	15243	329	2.16	1.94	2.40
payant	17783	535	3.01	2.77	3.27
referent	6351	239	3.76	3.32	4.26
organique	43187	1736	4.02	3.84	4.21
direct	35566	2467	6.94	6.68	7.21
email	8892	772	8.68	8.11	9.29

Étape 2 — deux sources se distinguent-elles ? Deux intervalles qui se recouvrent ne prouvent pas une égalité, mais l'absence de recouvrement est un signe net. Comparez le payant et l'organique, puis le référent et l'organique, avec un test de deux proportions.

```
from statsmodels.stats.proportion import proportions_ztest
for a, b in [("payant", "organique"), ("referent", "organique")]:
    ka, na, kb, nb = g.loc[a, "commandes"], g.loc[a, "sessions"], g.loc[b, "commandes"], g.loc[b,
    ↪ "sessions"]
    z, p = proportions_ztest([ka, kb], [na, nb])
    print(f"{a} contre {b} : écart {(ka / na - kb / nb) * 100:+.2f} point | z = {z:.2f} | p = {p:.4f}")
```

```
payant contre organique : écart -1.01 point | z = -5.99 | p = 0.0000
referent contre organique : écart -0.26 point | z = -0.98 | p = 0.3292
```

Lecture. Le payant convertit 1,01 point de moins que l'organique : l'écart est **très significatif**. Le référent (3,76 %) et l'organique (4,02 %) ont des intervalles qui se recouvrent, et le test donne un écart de -0,26 point avec une probabilité critique d'environ 0,33 : **on ne peut pas** dire qu'ils diffèrent.

À vous. Ajoutez la comparaison e-mail contre direct. Qu'en concluez-vous, et quelle taille d'écart un test aurait-il détectée avec 6 351 sessions ?

Application 10.2 — L'entonnoir par appareil (section 10.1.3)

Objectif. Trouver l'étape où l'on perd le plus selon l'appareil.

```
f = s.groupby("appareil")[["ajout_panier", "debut_paiement", "commande"]].sum()
f["sessions"] = s.groupby("appareil").size()
tab = pd.DataFrame({"panier_%": f["ajout_panier"] / f["sessions"] * 100, "paiement_%":
    ↪ f["debut_paiement"] / f["ajout_panier"] * 100,
                    "commande_%": f["commande"] / f["debut_paiement"] * 100, "conversion_%":
    ↪ f["commande"] / f["sessions"] * 100}).round(2)
print(tab.to_string())
```

	panier_%	paiement_%	commande_%	conversion_%
appareil				
mobile	14.35	57.17	58.69	4.81
ordinateur	14.11	57.37	58.41	4.73
tablette	14.35	56.01	60.23	4.84

Lecture. Les trois appareils ont des taux de passage voisins, et une conversion globale de 4,73 % à 4,84 % : dans ces données, **aucune étape n'est propre à un appareil**. Sur un site réel, un écart important au paiement sur mobile serait la première piste de travail.

À vous. Faites la même table par nouveau_visiteur, puis par source et appareil (six sources, trois appareils). Combien de cases ont moins de 500 sessions, et que cela change-t-il à la lecture ?

Application 10.3 — La rentabilité par source payante (section 10.2.2 et 10.2.3)

Objectif. Calculer le ROAS, le ROI, le coût par commande et le coût d'acquisition d'un client.

Étape 1 — dépense et marge.

```
dep = camp.groupby("source")["depense"].sum()
g2 = w.groupby("source").agg(commandes=("ca_ht", "size"), ca_ht=("ca_ht", "sum"), marge=("marge",
    ↪ "sum"), neufts=("premiere_commande", "sum"))
r = pd.DataFrame({"depense": dep}).join(g2)
r["cout_commande"] = r["depense"] / r["commandes"]
r["roas_ht"] = r["ca_ht"] / r["depense"]
r["roi_%"] = (r["marge"] - r["depense"]) / r["depense"] * 100
```

```
r["cac"] = r["depense"] / r["neufs"]
print(r[["depense", "commandes", "cout_commande", "roas_ht", "roi_%", "neufs",
↪ "cac"]].round(2).to_string())
```

	depense	commandes	cout_commande	roas_ht	roi_%	neufs	cac
source							
email	8239.76	772	10.67	7.51	180.23	42	196.18
payant	42374.19	535	79.20	1.04	-60.69	32	1324.19
reseaux	22528.77	329	68.48	1.28	-50.69	23	979.51

Étape 2 — la valeur d'un client. On compare le CAC à la marge qu'un client génère sur deux ans : pour les clients inscrits entre janvier et septembre 2023, on additionne la marge de leurs commandes dans les 730 jours suivant l'inscription.

```
cli = pd.read_csv(os.path.join(0.D, "clients.csv"), parse_dates=["date_inscription"])
nv = cli[(cli["date_inscription"] >= "2023-01-01") & (cli["date_inscription"] < "2023-10-01")]
mm = m.merge(nv[["id_client", "date_inscription"]], on="id_client")
mm = mm[mm["date_commande"] < mm["date_inscription"] + pd.Timedelta(days=730)]
print(len(nv), "clients | marge sur 24 mois par client :", round(mm["marge"].sum() / len(nv), 1), "€ |
↪ commandes par client :", round(len(mm) / len(nv), 2))
```

515 clients | marge sur 24 mois par client : 149.1 € | commandes par client : 4.9

Lecture. Un client inscrit en 2023 a rapporté en moyenne **149,1 € de marge** en 730 jours, avec 4,9 commandes. Le CAC de l'e-mail (196 €) est du même ordre ; celui de la publicité payante (1 324 €) et des réseaux (980 €) est **six à neuf fois** la valeur du client. Mais seules 345 commandes sont des premières commandes : le CAC attribue toute la dépense aux nouveaux clients, ce qui le surestime. Le vrai coût se situe entre le coût par commande (79 €) et le CAC ; **seul un test** peut le dire.

À vous. Recalculez le ROI de la publicité payante **mois par mois**. Dans quels mois est-il le moins mauvais ? Ce résultat vient-il de la dépense, de la saison ou du hasard ?

Application 10.4 — Attribution et groupe témoin (sections 10.2.5 et 10.2.6)

Objectif. Voir comment le modèle d'attribution déplace le mérite, puis mesurer l'incrémental avec son incertitude.

Étape 1 — comparer quatre modèles sur 4 000 parcours fabriqués.

```
parcours = 0.parcours_fabriques(4000)
cred = pd.DataFrame({mo: 0.attribution(parcours, mo) for mo in ["dernier clic", "premier clic",
↪ "linéaire", "en U"]}).mul(100).round(1)
print(cred.loc[["direct", "email", "organique", "payant", "reseaux", "referent"]].to_string())
```

	dernier clic	premier clic	linéaire	en U
direct	40.8	16.6	26.5	27.7
email	24.6	11.6	19.0	18.5
organique	17.8	25.5	22.6	22.0
payant	9.6	21.0	14.8	15.1
reseaux	3.8	20.8	12.9	12.6
referent	3.4	4.6	4.3	4.1

Étape 2 — l'incrémental avec son intervalle. Dans le test fabriqué, la différence entre les taux d'achat exposé et témoin est un écart de proportions ; son incertitude se calcule comme au chapitre 2.

```
t = 0.test_temoin()
exp, tem = t[t["expose"] == 1][["achat"], t[t["expose"] == 0][["achat"]]
```

```

p1, p0 = exp.mean(), tem.mean()
se = np.sqrt(p1 * (1 - p1) / len(exp) + p0 * (1 - p0) / len(tem))
print(f"écart : {(p1 - p0) * 100:.2f} point | intervalle à 95 % : de {(p1 - p0 - 1.96 * se) * 100:.2f}
↪ à {(p1 - p0 + 1.96 * se) * 100:.2f} point")
print("achats incrémentaux estimés :", round((p1 - p0) * len(exp)), "| intervalle :", round((p1 - p0 -
↪ 1.96 * se) * len(exp)), "à", round((p1 - p0 + 1.96 * se) * len(exp)))

```

```

écart : 0.36 point | intervalle à 95 % : de 0.02 à 0.70 point
achats incrémentaux estimés : 174 | intervalle : 10 à 338

```

Lecture. L'écart de 0,36 point a un intervalle de **0,02 à 0,70 point** (de 10 à 338 achats incrémentaux) : il exclut à peine zéro, et il est **très large**. Le dernier clic en attribuait 683. Même la borne haute de l'intervalle (338 achats) reste très en dessous.

À vous. Quelle taille de groupe témoin faudrait-il pour réduire de moitié l'intervalle ? (L'erreur type diminue en $1/\sqrt{n}$: pour la diviser par deux, il faut quatre fois plus de personnes.)

Application 10.5 — Réconcilier un outil d'analyse web (section 10.3)

Objectif. Mesurer la couverture d'un outil, puis corriger un ROAS avec le bon redressement.

```

vue = 0.vue_outil(s)
base_site = cmd[(cmd["canal"] == "Site") & (cmd["date_commande"] >= "2025-01-01")]
couverture = vue["commande"].sum() / len(base_site)
print("commandes du site en base :", len(base_site), "| vues par l'outil :",
↪ int(vue["commande"].sum()), f"| couverture : {couverture * 100:.1f} %")
par_source = (vue.groupby("source")["commande"].sum() / s.groupby("source")["commande"].sum() *
↪ 100).round(1)
print("couverture par source (%) :", par_source.to_dict())

```

```

commandes du site en base : 6078 | vues par l'outil : 4821 | couverture : 79.3 %
couverture par source (%) : {'direct': 84.5, 'email': 95.3, 'organique': 76.1, 'payant': 61.7,
↪ 'referent': 66.1, 'reseaux': 58.4}

```

Redressement global ou par source ? Si l'on divise les commandes vues par la couverture **globale**, on corrige en moyenne mais pas par source.

```

dep = camp.groupby("source")["depense"].sum()
wv = vue[vue["id_commande"].notna()].merge(m, on="id_commande")
ca_vu = wv.groupby("source")["ca_ht"].sum()
vrai = w.groupby("source")["ca_ht"].sum()
tab = pd.DataFrame({"roas_vrai": vrai / dep, "roas_outil": ca_vu / dep, "roas_corrige_global": ca_vu
↪ / couverture / dep}).dropna().round(2)
print(tab.loc[["email", "reseaux", "payant"]].to_string())

```

	roas_vrai	roas_outil	roas_corrige_global
source			
email	7.51	7.14	9.01
reseaux	1.28	0.74	0.93
payant	1.04	0.65	0.82

Lecture. Le redressement global donne un ROAS de la publicité payante de **0,82** pour une vérité de **1,04**, et de **0,93** pour les réseaux contre **1,28** : on **réduit** l'erreur mais l'écart reste de 20 à 27 %, parce que la couverture n'est pas la même partout (environ 62 % pour le payant, 58 % pour les réseaux et 95 % pour l'e-mail dans cette vue fabriquée) ; l'e-mail, lui, est **surestimé** (9,01 contre 7,51). Pour corriger source par source, il faudrait connaître la couverture de chacune : on ne l'a que si l'on peut comparer à une référence indépendante.

À vous. Quel serait le ROAS « corrigé » si l'on connaissait la couverture par source ? Que valent alors les trois ROAS ?

Exercices

Exercice 10.1 ★ — Un taux de conversion et son intervalle, à la main (section 10.1.2)

Une source a 2 000 sessions et 90 commandes. Calculez le taux de conversion, l'erreur type et l'intervalle de confiance à 95 % (approximation normale).

Exercice 10.2 ★ — Lire un entonnoir (section 10.1.3)

Sur 10 000 sessions, 1 200 ajoutent au panier, 600 commencent le paiement et 400 commandent. Calculez les taux de passage, la conversion globale, et dites où se situe la perte la plus massive et la plus actionnable.

Exercice 10.3 ★★ — Deux sources, une vraie différence ? (section 10.1.2)

Source A : 40 commandes sur 1 000 sessions. Source B : 50 commandes sur 1 000 sessions. L'écart de 1 point est-il significatif au seuil de 5 % ? Même question avec dix fois plus de sessions (400 contre 500 commandes). Que conclure ?

Exercice 10.4 ★★ — L'effet de mélange (section 10.1.5)

Le site a 100 000 sessions et 5 000 commandes. On ajoute 10 000 sessions d'une source qui convertit à 2 %. Calculez la nouvelle conversion globale et le nombre de commandes ajoutées. La campagne est-elle « mauvaise » ?

Exercice 10.5 ★ — ROAS et ROI (section 10.2.2)

On dépense 2 000 € et l'on attribue 5 000 € de chiffre d'affaires hors taxe, avec une marge brute de 38 % du chiffre d'affaires hors taxe. Calculez le ROAS, la marge dégagée, le ROI et dites si le canal est rentable.

Exercice 10.6 ★★ — Le seuil de rentabilité du ROAS (section 10.2.2)

Calculez le ROAS minimal (hors taxe) pour couvrir une dépense publicitaire quand la marge brute vaut 30 %, 38 % et 45 % du chiffre d'affaires hors taxe. Pourquoi un ROAS de 2,5 peut-il être rentable pour un produit et ruineux pour un autre ?

Exercice 10.7 ★★ — CAC et durée de retour (section 10.2.3)

On dépense 4 000 € pour acquérir 25 nouveaux clients. Un client génère 20 € de marge brute à sa première commande et 50 € de plus en moyenne sur les deux années suivantes. Calculez le CAC, le ratio valeur du client sur CAC, et dites si l'acquisition est rentable avec ou sans les commandes suivantes.

Exercice 10.8 ★★★ — Attribution à la main (section 10.2.5)

Trois parcours d'achat : (réseaux, organique, direct), (payant, email), (direct). Calculez le mérite de chaque source selon le dernier clic, le premier clic et le modèle linéaire. Quelle source gagne ou perd le plus selon le modèle ?

Exercice 10.9 ★★ — La couverture d'un outil (section 10.3.4)

Un outil voit 4 821 commandes quand la base en compte 6 078 pour le site. Calculez la couverture. Si l'outil annonce 90 000 € de chiffre d'affaires, quelle estimation donneriez-vous du chiffre d'affaires réel, et quelle hypothèse faites-vous ?

Exercice 10.10 ★★★ — Des événements en double (section 10.3.5)

Un outil reçoit 6 262 événements « achat » pour 6 078 commandes distinctes, d'un montant moyen de 100 € (non transmis dans l'événement). De combien le chiffre d'affaires de l'outil est-il gonflé ? Comment le corriger, et que se passe-t-il si les doublons ne portent pas sur les mêmes montants que les autres commandes ?

Corrigés

Corrigé 10.1

Taux $p = 90/2\,000 = 4,5\%$; erreur type $\sqrt{p(1-p)/n} = \sqrt{0,045 \times 0,955/2\,000} \approx 0,0046$; intervalle $4,5\% \pm 1,96 \times 0,46\%$, soit de **3,6 % à 5,4 %**.

```
p, n = 90 / 2000, 2000
se = np.sqrt(p * (1 - p) / n)
print(round(p * 100, 2), round(se * 100, 3), round((p - 1.96 * se) * 100, 2), round((p + 1.96 * se) * 100, 2))
```

```
4.5 0.464 3.59 5.41
```

Corrigé 10.2

Taux de passage : $1\,200/10\,000 = 12\%$; $600/1\,200 = 50\%$; $400/600 \approx 66,7\%$; conversion globale $400/10\,000 = 4\%$. La perte la plus **massive** est la première (88 % des sessions n'ajoutent rien) ; la plus **actionnable** est souvent la dernière : un tiers de ceux qui commencent à payer s'arrêtent.

Corrigé 10.3

Avec 1 000 sessions : $p_A = 4\%$, $p_B = 5\%$, proportion commune $4,5\%$, erreur type $\sqrt{0,045 \times 0,955 \times (2/1\,000)} \approx 0,0093$, $z \approx 1,08$, $p \approx 0,28$: **non significatif**. Avec dix fois plus de sessions, l'erreur type est divisée par $\sqrt{10}$, $z \approx 3,4$ et $p < 0,001$: **très significatif**.

```
from statsmodels.stats.proportion import proportions_ztest
for k, n in [([40, 50], [1000, 1000]), ([400, 500], [10000, 10000])]:
    z, p = proportions_ztest(k, n)
    print(f"z = {z:.2f}, p = {p:.4f}")
```

```
z = -1.08, p = 0.2807
z = -3.41, p = 0.0006
```

Le même écart d'un point est du bruit sur 1 000 sessions et une vraie différence sur 10 000 : **la taille de l'échantillon décide**.

Corrigé 10.4

Commandes ajoutées : $10\,000 \times 2\% = 200$. Nouvelle conversion : $(5\,000 + 200)/110\,000 \approx 4,73\%$, contre $5,00\%$ avant. La conversion globale **baisse** de $0,27$ point alors que les commandes **augmentent** de 200 : la campagne n'est pas « mauvaise », elle est moins bonne que la moyenne ; sa rentabilité se juge sur son coût et sa marge (section 10.2).

```
print(round(5200 / 110000 * 100, 2), 10000 * 0.02)
```

```
4.73 200.0
```

Corrigé 10.5

ROAS = $5\,000/2\,000 = 2,5$. Marge = $0,38 \times 5\,000 = 1\,900$ €. ROI = $(1\,900 - 2\,000)/2\,000 = -5\%$: le canal **perd** 100 €. Un ROAS de $2,5$ paraît bon mais ne couvre pas la dépense : le seuil de rentabilité est $1/0,38 \approx 2,63$.

Corrigé 10.6

Seuil = $1/\text{taux de marge}$: $1/0,30 \approx 3,33$; $1/0,38 \approx 2,63$; $1/0,45 \approx 2,22$. Un ROAS de $2,5$ est rentable pour un produit à 45% de marge (au-dessus de $2,22$), insuffisant pour un produit à 38% (seuil $2,63$) et ruineux pour un produit à 30% (seuil $3,33$).

```
print([round(1 / x, 2) for x in (0.30, 0.38, 0.45)])
```

```
[3.33, 2.63, 2.22]
```

Corrigé 10.7

CAC = $4\,000/25 = 160$ €. Valeur du client : 20 € à la première commande, 70 € au total sur deux ans. Ratio valeur/CAC : $70/160 \approx 0,44$, donc l'acquisition **ne se rentabilise pas**, même avec les commandes suivantes ; sur la seule première commande, le rapport est $20/160 = 0,125$. Il faudrait un client qui rapporte plus de 160 € de marge sur sa vie, ou un coût d'acquisition plus bas.

Corrigé 10.8

Dernier clic : direct 2 (parcours 1 et 3), email 1 ; réseaux, organique, payant 0. Premier clic : réseaux 1, payant 1, direct 1 ; organique, email 0. Linéaire : parcours 1 donne un tiers à chacun de réseaux, organique, direct ; parcours 2 donne $\frac{1}{2}$ à payant et $\frac{1}{2}$ à email ; parcours 3 donne 1 au direct ; total direct $1,33$, réseaux $0,33$, organique $0,33$, payant $0,5$, email $0,5$. Le **direct** gagne le plus au dernier clic (2 sur 3) et le moins au premier clic (1 sur 3) ; les **réseaux** et le **payant** passent de 0 à 1 selon qu'on regarde la fin ou le début.

```
parcours = [{"reseaux", "organique", "direct"}, {"payant", "email"}, {"direct"}]
for mo in ["dernier clic", "premier clic", "linéaire"]:
    print(mo, {k: round(v * 3, 2) for k, v in 0.attribution(parcours, mo).items() if v > 0})
```

```
dernier clic {'direct': 2.0, 'email': 1.0}
premier clic {'direct': 1.0, 'payant': 1.0, 'reseaux': 1.0}
linéaire {'direct': 1.33, 'organique': 0.33, 'payant': 0.5, 'email': 0.5, 'reseaux': 0.33}
```

Corrigé 10.9

Couverture = $4\,821 / 6\,078 \approx 79,3\%$. Estimation du chiffre d'affaires réel : $90\,000 / 0,793 \approx 113\,500$ €, **en supposant** que les commandes non vues ont le même montant moyen que les commandes vues et que la couverture est la même pour toutes les sources (hypothèse fautive dans notre vue fabriquée, où elle varie de 55 % à 95 % selon la source). Le chiffre est donc une **estimation grossière**, pas un redressement exact.

```
print(round(4821 / 6078 * 100, 1), round(90000 / (4821 / 6078)))
```

```
79.3 113466
```

Corrigé 10.10

Le surplus d'événements est $6\,262 - 6\,078 = 184$ achats, soit 3,0 %. Si chaque événement vaut 100 €, le chiffre d'affaires de l'outil est gonflé de **18 400 €** (3,0 %). On corrige en **dédoublonnant** sur l'identifiant de commande, présent dans l'événement. Si les doublons portent sur des commandes d'un montant différent de la moyenne (par exemple des commandes plus grosses, parce que les gros paniers rechargent plus souvent la page de confirmation), le gonflement du chiffre d'affaires n'est **pas** de 3 % : il peut être plus ou moins élevé que le surplus en nombre. Raison de plus de dédoublonner par identifiant plutôt que de corriger par un coefficient.

```
ev = 0.evenements_doublons(s)
print(len(ev) - ev["id_commande"].nunique(), (len(ev) - ev["id_commande"].nunique()) * 100,
      ↪ round((len(ev) / ev["id_commande"].nunique() - 1) * 100, 1))
```

```
184 18400 3.0
```

Chapitre 11 : ⊕ Analytique des opérations et de la chaîne logistique — exercices et applications

Ce chapitre du cahier accompagne le chapitre complémentaire 11 du livre. Les **applications** reprennent, par petites étapes, les études des sections 11.1 à 11.3 sur les données de la boutique ; les **exercices** (★ facile, ★★ moyen, ★★★ difficile) demandent des calculs à la main puis du code ; les **corrigés** sont à la fin. Les données sont **simulées** (graines fixes), et tout le chapitre est **autonome** : la cellule suivante charge ce qu'il faut.

```
import sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
from scipy import stats
import outils_ch11 as O

liv, rea, stk, prod = O.charger()
print(len(liv), "livraisons (hors retrait en magasin) |", len(rea), "commandes d'achat |", len(stk),
      ↵ "lignes de stock")
```

```
18047 livraisons (hors retrait en magasin) | 1500 commandes d'achat | 7300 lignes de stock
```

Applications

Application 11.1 — Délais, centiles et taux à l'heure (sections 11.1.1 et 11.1.2)

Objectif. Mesurer le service rendu par année et voir comment le choix de la promesse change le taux à l'heure.

Étape 1 — les délais par étape et par année.

```
par_an = liv.groupby("annee")[["preparation", "transport", "total"]].mean().round(2)
par_an["a_l_heure_%"] = (1 - liv.groupby("annee")["retard"].mean()).mul(100).round(1)
print(par_an)
```

	preparation	transport	total	a_l_heure_%
annee				
2023	1.61	4.12	5.74	72.6
2024	1.62	4.12	5.74	72.9
2025	1.60	4.14	5.74	73.0

Le taux à l'heure est de 72,6 % en 2023 et de 73,0 % en 2025 ; le délai total moyen est de 5,74 jours en 2023 et de 5,74 jours en 2025 : la qualité de service est **stable** sur trois ans, pas en dégradation.

Étape 2 — le taux à l'heure selon la promesse. Pour une promesse de p jours, le taux à l'heure est la part des commandes dont le délai total est au plus p .

```
for promesse in (5, 6, 7, 8, 10):
    print("promesse de", promesse, "jours :", round((liv["total"] <= promesse).mean() * 100, 1), "% à l'heure")
```

```
promesse de 5 jours : 47.0 % à l'heure
promesse de 6 jours : 72.8 % à l'heure
promesse de 7 jours : 88.9 % à l'heure
promesse de 8 jours : 96.1 % à l'heure
promesse de 10 jours : 99.7 % à l'heure
```

À vous. Quel délai promettre pour que 95 % des commandes arrivent à l'heure ? (Piste : le 95e centile du délai total.) Quelle est la conséquence commerciale d'une promesse plus longue ?

Piste. Le 95e centile du délai total est 8 jours : on promet 8 jours pour tenir 95 % de livraisons à l'heure. Plus long, la promesse est plus fiable mais moins attractive ; plus court, elle séduit mais se rompt. C'est un choix commercial éclairé par la distribution, pas un calcul.

Application 11.2 — Transporteurs, mode et décembre (sections 11.1.3 et 11.1.4)

Objectif. Comparer les transporteurs avec des intervalles, puis **standardiser** pour neutraliser décembre.

Étape 1 — taux de retard et intervalles de Wilson.

```
for nom, g in liv.groupby("transporteur"):
    p, lo, hi = 0.wilson(g["retard"].sum(), len(g))
    print(nom, len(g), round(p * 100, 1), round(lo * 100, 1), round(hi * 100, 1))
```

```
Transporteur A 8107 16.4 15.6 17.2
Transporteur B 6444 27.3 26.2 28.4
Transporteur C 3496 51.9 50.3 53.6
```

Étape 2 — standardisation directe. On calcule le taux de chaque transporteur dans chaque mois, puis on le pondère par la **répartition mensuelle de l'ensemble des envois** : c'est le taux qu'aurait chaque transporteur si tous avaient le même calendrier.

```
poids = liv["mois"].value_counts(normalize=True).sort_index()
taux = liv.pivot_table(index="mois", columns="transporteur", values="retard", aggfunc="mean")
standardise = (taux.mul(poids, axis=0)).sum() * 100
brut = liv.groupby("transporteur")["retard"].mean() * 100
print(pd.DataFrame({"brut %": brut.round(1), "standardisé %": standardise.round(1)}))
```

	brut %	standardisé %
transporteur		
Transporteur A	16.4	16.3
Transporteur B	27.3	27.5
Transporteur C	51.9	51.8

Les taux bruts (16,4 %, 27,3 %, 51,9 %) et standardisés (16,3 %, 27,5 %, 51,8 %) diffèrent d'au plus 0,2 point : le calendrier de chaque transporteur est le même, donc la standardisation ne change rien. Elle sera décisive quand les calendriers diffèrent (exercice 11.4).

À vous. Refaites la comparaison selon le **mode de livraison** (domicile ou point relais) au lieu du mois. Le classement des transporteurs tient-il ?

Piste. Pour le vérifier : `liv.pivot_table(index="mode_livraison", columns="transporteur", values="retard", aggfunc="mean")`. Les taux sont, à domicile, de 11,4 % (A), 20,2 % (B) et 42,0 % (C) ; en point relais, de 23,5 %, 38,0 % et 66,9 % : le classement tient dans les deux modes, car la répartition entre les modes est la même pour tous les transporteurs.

Application 11.3 — Colis abîmés, retours et coût (section 11.1.5)

Objectif. Chiffrer la casse et tester si le retard fait renvoyer.

Étape 1 — la casse par transporteur.

```
ab = liv.groupby("transporteur")["colis_abime"].agg(["sum", "count"])
ab["taux_%"] = (ab["sum"] / ab["count"] * 100).round(2)
print(ab)
```

	sum	count	taux_%
transporteur			
Transporteur A	72	8107	0.89
Transporteur B	93	6444	1.44
Transporteur C	147	3496	4.20

Étape 2 — le retard fait-il renvoyer ? On relie chaque livraison à un éventuel retour et on teste l'égalité des taux avec un test du khi-deux.

```
ret = pd.read_csv("donnees/retours.csv"); lig = pd.read_csv("donnees/lignes_commande.csv")
retourne = ret.merge(lig[["id_ligne", "id_commande"]], on="id_ligne")["id_commande"].unique()
liv["retour"] = liv["id_commande"].isin(retourne).astype(int)
tab = pd.crosstab(liv["retard"], liv["retour"])
chi2, p, _, _ = stats.chi2_contingency(tab, correction=False)
print(tab.assign(taux=(tab[1] / tab.sum(axis=1) * 100).round(1)))
print("p-valeur du khi-deux :", round(p, 3))
```

retour	0	1	taux
retard			
0	10757	2387	18.2
1	4022	881	18.0

p-valeur du khi-deux : 0.766

Le taux de retour est de 18,2 % après une livraison à l'heure et de 18,0 % après une livraison tardive ; la p-valeur de 0,77 est grande : **aucune différence détectable**.

À vous. Estimez le coût annuel de la casse du transporteur C si l'on suppose un coût de 100 € par colis abîmé. Que valent 100 € si le colis contenait un produit de marge moyenne 10 € ?

Piste. Environ 49 colis abîmés par an pour C (147 en trois ans), soit 4 900 € de remplacement à 100 € le colis ; en marge, la perte est de 10 € par colis si le client est remboursé et le produit revendu, beaucoup plus s'il faut le remplacer : le coût dépend de la politique, ce qui doit être précisé avec la gérante.

Application 11.4 — Ruptures et point de commande (sections 11.2.2 et 11.2.3)

Objectif. Calculer la couverture, les ruptures, et le point de commande pour trois niveaux de service.

Étape 1 — couverture et ruptures par produit.

```
pp = stk.groupby("id_produit").agg(d=("demande", "mean"), s=("demande", "std"),
    ↪ stock=("stock_fin_jour", "mean"), rupt=("rupture", "mean"))
pp["couverture"] = pp["stock"] / pp["d"]
print(pp[["d", "couverture", "rupt"]].describe().loc[["mean", "min", "max"]].round(2))
```

	d	couverture	rupt
mean	1.54	16.11	0.07
min	1.28	15.14	0.05
max	1.91	17.29	0.10

Étape 2 — le délai reconstitué et le point de commande.

```
L = 0.delais_reconstitues(stk)
Lm, Ls = L.mean(), L.std(ddof=1)
resultats = {}
for niveau, z in (("90 %", 1.2816), ("95 %", 1.6449), ("99 %", 2.3263)):
    R = np.ceil(pp["d"] * Lm + z * np.sqrt(Lm * pp["s"] ** 2 + pp["d"] ** 2 * Ls ** 2))
    resultats[niveau] = float(R.mean().round(1))
print(round(Lm, 1), round(Ls, 1), resultats, "| actuel :",
    ↪ round(float(stk.groupby("id_produit")["point_de_commande"].first().mean()), 1))
```

```
11.1 3.7 {'90 %': 27.8, '95 %': 30.6, '99 %': 36.1} | actuel : 13.3
```

Le délai moyen est de 11,1 jours (écart-type de 3,7). Le point de commande moyen serait de **27,8, 30,6 et 36,1** unités pour 90, 95 et 99 % de service, contre **13,3** actuellement : la règle actuelle est **en dessous même du niveau à 90 %**.

À vous. Le passage de 95 % à 99 % ajoute combien d'unités de stock de sécurité par produit ? Est-ce proportionnel au gain de service ?

Piste. Environ 5,5 unités de plus par produit (36,1 contre 30,6) pour passer de 95 à 99 %, alors que l'on gagne quatre points de service : le **dernier point coûte de plus en plus cher**, ce que la loi normale traduit par l'allongement des queues.

Application 11.5 — Rejeu de la politique et quantité économique (sections 11.2.4 et 11.2.5)

Objectif. Rejouer deux niveaux de service et mesurer la sensibilité de la quantité économique à l'hypothèse de coût.

Étape 1 — le rejeu. On compare la règle actuelle et le point de commande à 95 % (0. rejouer rejoue la demande de 2025 avec des délais tirés dans les délais observés).

```
actuel_rop = stk.groupby("id_produit")["point_de_commande"].first().to_dict()
R95 = np.ceil(pp["d"] * Lm + 1.6449 * np.sqrt(Lm * pp["s"] ** 2 + pp["d"] ** 2 * Ls **
    ↪ 2)).astype(int).to_dict()
Q = {p: int(pp.loc[p, "d"] * 30) + 5 for p in pp.index}
for nom, regle in (("actuelle", actuel_rop), ("95 %", R95)):
    r = 0.rejouer(stk, regle, Q, L, n_rep=10)
    print(nom, "| ruptures :", round(r[0] * 100, 1), "% | stock :", round(r[1], 1), "jours |
    ↪ commandes/an :", round(r[2], 1))
```

```
actuelle | ruptures : 7.1 % | stock : 16.1 jours | commandes/an : 9.9
95 % | ruptures : 1.4 % | stock : 26.1 jours | commandes/an : 11.0
```

La règle actuelle donne 7,1 % de ruptures pour 16,1 jours de stock ; la règle à 95 % 1,4 % pour 26,1 jours.

Étape 2 — sensibilité de la quantité économique au coût de commande.

```
cout_u = prod.set_index("id_produit")["cout_achat"].loc[pp.index]
D = pp["d"] * 365
for S in (10, 25, 50):
    qeco = np.sqrt(2 * D * S / (0.20 * cout_u))
    cout = (D / qeco * S + qeco / 2 * 0.20 * cout_u).sum()
    print("S =", S, "€ | quantité économique moyenne :", round(qeco.mean()), "unités | coût annuel :",
    ↪ round(cout), "€")
```

S = 10 € | quantité économique moyenne : 75 unités | coût annuel : 3574 €
 S = 25 € | quantité économique moyenne : 119 unités | coût annuel : 5651 €
 S = 50 € | quantité économique moyenne : 168 unités | coût annuel : 7992 €

À vous. Quand le coût d'une commande est multiplié par 5 (de 10 à 50 €), de combien la quantité économique est-elle multipliée ? (Piste : racine carrée.)

Piste. Par $\sqrt{5} \approx 2,2$: la quantité économique varie comme la **racine** du coût de commande, ce qui la rend peu sensible à une erreur sur S.

Application 11.6 — La carte de performance des fournisseurs (section 11.3)

Objectif. Construire la carte avec des intervalles par **bootstrap** (sans formule) et tester la stabilité du classement.

Étape 1 — intervalle bootstrap de l'écart moyen. On rééchantillonne les commandes de chaque fournisseur avec remise.

```
rng = np.random.default_rng(3)
carte = {}
for f, g in rea.groupby("fournisseur"):
    ec = g["ecart_j"].values
    moyennes = [rng.choice(ec, len(ec)).mean() for _ in range(2000)]
    carte[f[-1]] = (ec.mean(), np.percentile(moyennes, 2.5), np.percentile(moyennes, 97.5))
print(pd.DataFrame(carte, index=["moyenne", "bas", "haut"]).T.round(2))
```

	moyenne	bas	haut
A	0.67	0.25	1.16
B	1.06	0.52	1.64
C	1.08	0.64	1.60
D	1.06	0.55	1.60
E	3.89	3.20	4.59
F	1.29	0.73	1.85
G	0.48	0.09	0.95
H	0.92	0.49	1.36

Étape 2 — probabilité d'être le meilleur. À chaque rééchantillonnage, on classe les fournisseurs ; on compte la fréquence où chacun est premier (écart moyen le plus faible).

```
fr = {f[-1]: g["ecart_j"].values for f, g in rea.groupby("fournisseur")}
premiers = pd.Series([min(fr, key=lambda k: rng.choice(fr[k], len(fr[k])).mean()) for _ in
    range(2000)]).value_counts(normalize=True)
print(premiers.round(2).to_dict())
```

```
{'G': 0.65, 'A': 0.25, 'H': 0.04, 'B': 0.02, 'D': 0.02, 'C': 0.01, 'F': 0.0}
```

Le fournisseur G est premier dans 65 % des rééchantillonnages, A dans 25 %, et E dans 0 % : **personne n'est « le meilleur » avec certitude**, sauf que E ne l'est jamais.

À vous. Même question avec le taux de service au lieu de l'écart de délai.

Piste. Avec le taux de service, la première place est répartie entre 7 fournisseurs, le plus fréquent n'étant premier que dans 50 % des rééchantillonnages (leurs taux de service ne diffèrent que de quelques dixièmes de point), et E est premier dans 0 % des cas : même conclusion, aucun « meilleur » certain parmi les sept ordinaires.

Exercices

Exercice 11.1 ★ — Le taux à l'heure selon la promesse (section 11.1.2)

Sur le fichier des livraisons, calculez le taux de livraison à l'heure si l'on promettait 7 jours, puis 8 jours. Comparez au taux observé pour 6 jours.

Exercice 11.2 ★★ — Moyenne, médiane et 90e centile (section 11.1.2)

Neuf livraisons ont pris 4, 5, 5, 5, 6, 6, 6, 7 et 14 jours. Calculez à la main la moyenne, la médiane et le 90e centile (par interpolation linéaire), puis vérifiez avec NumPy. Lequel inscririez-vous dans un contrat de service ?

Exercice 11.3 ★ — Un intervalle de Wilson à la main (section 11.1.3)

Un transporteur est en retard sur 50 envois sur 200. Calculez à la main l'intervalle de Wilson à 95 % de son taux de retard, puis vérifiez avec `0.wilson`.

Exercice 11.4 ★★ — Un paradoxe de Simpson de transporteurs (section 11.1.4)

Le transporteur X assure 100 envois en décembre (80 en retard) et 400 les autres mois (100 en retard). Le transporteur Y assure 400 envois en décembre (300 en retard) et 100 les autres mois (20 en retard). Calculez les taux par strate et les taux globaux. Quel transporteur est meilleur ? Standardisez avec la répartition commune (500 envois en décembre, 500 les autres mois) pour conclure.

Exercice 11.5 ★★ — Basculer les envois du transporteur C (section 11.1.5)

Si l'on confiait tous les envois du transporteur C au transporteur A, combien de colis abîmés et combien de livraisons tardives éviterait-on sur trois ans (en supposant que les taux de A ne changent pas) ? Quelle limite voyez-vous à ce calcul ?

Exercice 11.6 ★ — Couverture et rotation à la main (section 11.2.1)

Un produit se vend 2 unités par jour et le stock moyen est de 36 unités. Calculez la couverture et la rotation. Que deviennent-elles si la demande double sans que le stock change ?

Exercice 11.7 ★★ — Un point de commande à la main (section 11.2.3)

Un produit a une demande journalière de moyenne 2 et d'écart-type 1, un délai de moyenne 9 jours et d'écart-type 2 jours. Calculez le point de commande pour 90 % de service ($z = 1,28$). Que devient-il si l'écart-type du délai est divisé par deux ?

Exercice 11.8 ★★ — La quantité économique et l'erreur sur le coût (section 11.2.5)

Un produit se vend 1 200 unités par an ; une commande coûte 30 €, la détention 3 € par unité et par an. Calculez la quantité économique. Si le coût de commande est en réalité de 60 €, de combien la quantité optimale change-t-elle, et de combien augmente le coût total quand on garde la quantité calculée avec 30 € ?

Exercice 11.9 ★★ — Deux fournisseurs, est-ce différent ? (section 11.3)

Le fournisseur X est en retard sur 30 de ses 100 commandes, le fournisseur Y sur 20 de ses 100. Les deux taux sont-ils significativement différents ? Combien de commandes par fournisseur faudrait-il pour détecter cet écart de 10 points avec une puissance de 80 % ?

Exercice 11.10 ★★★ — Une note multicritère et sa sensibilité (section 11.3)

Construisez pour chaque fournisseur une note de 0 à 100 à partir du taux de retard de 3 jours ou plus, du taux de service et de l'écart-type du délai, avec les pondérations de votre choix (justifiez-les). Faites varier les pondérations de ± 20 points : quels fournisseurs changent de rang ? Que recommandez-vous à la gérante ?

Corrigés

Corrigé 11.1

```
for promesse in (6, 7, 8):
    print("promesse de", promesse, "jours :", round((liv["total"] <= promesse).mean() * 100, 1), "% à l'heure")
```

```
promesse de 6 jours : 72.8 % à l'heure
promesse de 7 jours : 88.9 % à l'heure
promesse de 8 jours : 96.1 % à l'heure
```

Pour 6 jours, **72,8 %** ; pour 7 jours, **88,9 %** ; pour 8 jours, **96,1 %**. Allonger la promesse d'un jour ajoute 16,0 points de service, un deuxième jour n'en ajoute plus que 7,2 : la distribution est asymétrique et le gain **décroît**.

Corrigé 11.2

À la main : somme = $4 + 5 + 5 + 5 + 6 + 6 + 6 + 7 + 14 = 58$, moyenne $58/9 \approx 6,44$; médiane (5e valeur ordonnée) = 6 ; position du 90e centile = $0,9 \times (9 - 1) = 7,2$ entre la 8e valeur (7) et la 9e (14) : $7 + 0,2 \times 7 = 8,4$.

```
d = np.array([4, 5, 5, 5, 6, 6, 6, 7, 14])
print(round(d.mean(), 2), np.median(d), round(np.percentile(d, 90), 1))
```

```
6.44 6.0 8.4
```

On inscrit dans un contrat un **centile** (par exemple le 90e : 8,4 jours, soit « neuf commandes sur dix en 9 jours ou moins »), pas la moyenne, que la livraison de 14 jours tire vers le haut.

Corrigé 11.3

À la main, avec $p = 0,25$, $n = 200$, $z = 1,96$: centre = $\frac{0,25 + 1,96^2/400}{1 + 1,96^2/200} = \frac{0,25 + 0,0096}{1,0192} \approx 0,2547$; demi-largeur = $\frac{1,96\sqrt{0,25 \times 0,75/200 + 1,96^2/(4 \times 200^2)}}{1,0192} \approx 0,0596$. L'intervalle est donc d'environ 19,5 % à 31,4 %.

```
p, lo, hi = 0.wilson(50, 200)
print(round(p * 100, 1), round(lo * 100, 1), round(hi * 100, 1))
```

```
25.0 19.5 31.4
```

Corrigé 11.4

```
x = {"déc": (100, 80), "autres": (400, 100)}
y = {"déc": (400, 300), "autres": (100, 20)}
for nom, t in (("X", x), ("Y", y)):
    n = sum(v[0] for v in t.values()); k = sum(v[1] for v in t.values())
    print(nom, {s: round(v[1] / v[0] * 100) for s, v in t.items()}, "global :", round(k / n * 100), "%
    ↳ | standardisé :", round((0.5 * t["déc"][1] / t["déc"][0] + 0.5 * t["autres"][1] /
    ↳ t["autres"][0]) * 100, 1), "%")
```

```
X {'déc': 80, 'autres': 25} global : 36 % | standardisé : 52.5 %
Y {'déc': 75, 'autres': 20} global : 64 % | standardisé : 47.5 %
```

Par strate, X est **moins bon** que Y (80 % contre 75 % en décembre, 25 % contre 20 % ailleurs). Pourtant, globalement, X paraît **bien meilleur** (36 % contre 64 %) : c'est un paradoxe de Simpson, parce que Y a assuré surtout les envois de décembre, mois difficile. Standardisés sur la même répartition, X vaut 52,5 % et Y 47,5 % : **Y est le meilleur transporteur**, et le classement global était trompeur.

Corrigé 11.5

```
n_c = int((liv["transporteur"] == "Transporteur C").sum())
a = liv[liv["transporteur"] == "Transporteur A"]; c = liv[liv["transporteur"] == "Transporteur C"]
abimes = c["colis_abime"].sum() - n_c * a["colis_abime"].mean()
tardifs = c["retard"].sum() - n_c * a["retard"].mean()
print(n_c, round(abimes), round(tardifs))
```

```
3496 116 1243
```

Sur 3 496 envois du transporteur C, on éviterait environ **116 colis abîmés** et **1 243 livraisons tardives**. Limites : le transporteur A **n'a peut-être pas la capacité** d'absorber ce volume (sa performance pourrait se dégrader) ; ses taux sont mesurés sur ses envois actuels ; et le tarif de A peut être plus élevé : il faudrait comparer le **coût complet** (transport, casse, service).

Corrigé 11.6

Couverture = $36/2 = 18$ jours ; rotation = $365/18 \approx 20,3$ fois par an. Si la demande double sans que le stock change, la couverture tombe à $36/4 = 9$ jours et la rotation monte à environ 40,6 : le stock est « plus efficace », mais **deux fois plus exposé à la rupture**.

Corrigé 11.7

À la main : $\sigma_{DL} = \sqrt{9 \times 1^2 + 2^2 \times 2^2} = \sqrt{9 + 16} = 5$; stock de sécurité = $1,28 \times 5 = 6,4$; demande pendant le délai = $2 \times 9 = 18$; $R = 18 + 6,4 = 24,4$, soit 25 unités. Avec $\sigma_L = 1$: $\sigma_{DL} = \sqrt{9 + 4} \approx 3,61$, stock de sécurité $\approx 4,6$, $R \approx 22,6$.

```
for sL in (2, 1):
    s = np.sqrt(9 * 1 + 2 ** 2 * sL ** 2)
    print(sL, round(s, 2), round(1.28 * s, 1), round(18 + 1.28 * s, 1))
```

```
2 5.0 6.4 24.4
1 3.61 4.6 22.6
```

Diviser par deux l'incertitude sur le **délai** réduit le stock de sécurité de 6,4 à 4,6, soit environ 28 % : la fiabilité du fournisseur se paie en stock.

Corrigé 11.8

À la main : $Q^* = \sqrt{2 \times 1200 \times 30/3} = \sqrt{24\,000} \approx 155$. Avec $S = 60$: $Q^* = \sqrt{2 \times 1200 \times 60/3} = \sqrt{48\,000} \approx 219$, soit $\sqrt{2}$ fois plus. Garder $Q = 155$ alors que l'optimum est 219 : coût = $\frac{1200}{155} \times 60 + \frac{155}{2} \times 3 \approx 464,5 + 232,5 = 697$ € contre $\sqrt{2 \times 1200 \times 60 \times 3} \approx 657$ € à l'optimum : **6 % de plus** pour une erreur de 100 % sur le coût de commande.

```
f = lambda q, S: 1200 / q * S + q / 2 * 3
print(round(np.sqrt(2 * 1200 * 30 / 3), 1), round(np.sqrt(2 * 1200 * 60 / 3), 1), round(f(155, 60)),
      ↪ round(f(np.sqrt(2 * 1200 * 60 / 3), 60)))
```

```
154.9 219.1 697 657
```

Corrigé 11.9

```
from statsmodels.stats.proportion import proportions_ztest, proportion_confint
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize
z, pv = proportions_ztest([30, 20], [100, 100])
taille = NormalIndPower().solve_power(proportion_effectsize(0.30, 0.20), power=0.8, alpha=0.05)
print(round(z, 2), round(pv, 3), [round(v, 3) for v in proportion_confint(30, 100, method="wilson")],
      ↪ [round(v, 3) for v in proportion_confint(20, 100, method="wilson")], round(taille))
```

```
1.63 0.102 [0.219, 0.396] [0.133, 0.289] 292
```

La statistique vaut 1,63 et la p-valeur 0,102 : **non significatif** à 5 %, et les deux intervalles de Wilson se chevauchent largement. Il faudrait environ **292 commandes par fournisseur** pour détecter un écart de 10 points avec 80 % de puissance : le triple de ce que l'on a. Moralité : avec cent commandes chacun, on ne départage pas deux fournisseurs qui diffèrent de dix points.

Corrigé 11.10

```
reg = rea.groupby("fournisseur")["delai_reel_j"].std()
rea["retard_3j"] = (rea["ecart_j"] >= 3).astype(int)
crit = pd.DataFrame({"retard": rea.groupby("fournisseur")["retard_3j"].mean(), "service":
      ↪ rea.groupby("fournisseur")["taux_service"].mean(), "reg": reg})
norm = pd.DataFrame({"retard": 1 - (crit["retard"] - crit["retard"].min()) / (crit["retard"].max() -
      ↪ crit["retard"].min()),
      "service": (crit["service"] - crit["service"].min()) / (crit["service"].max() -
      ↪ crit["service"].min()),
      "reg": 1 - (crit["reg"] - crit["reg"].min()) / (crit["reg"].max() -
      ↪ crit["reg"].min())})
rangs = {}
for w in ((0.4, 0.4, 0.2), (0.6, 0.2, 0.2), (0.2, 0.6, 0.2), (0.2, 0.2, 0.6)):
    rangs[str(w)] = (norm @ pd.Series(w, index=norm.columns) *
      ↪ 100).round(0).rank(ascending=False).astype(int)
print(pd.DataFrame(rangs).T.rename(columns=lambda f: f[-1]))
```

```
fournisseur      A  B  C  D  E  F  G  H
(0.4, 0.4, 0.2)  3  5  4  7  8  5  1  2
```

(0.6, 0.2, 0.2)	4	4	4	7	8	6	1	2
(0.2, 0.6, 0.2)	3	6	5	7	8	4	1	2
(0.2, 0.2, 0.6)	3	6	4	7	8	4	1	2

Le fournisseur E est au rang 8 quelles que soient les pondérations : il est **toujours dernier**. Pour les autres, un même fournisseur peut changer de **2 rangs** selon les pondérations. Recommandation : **traiter E à part** (plan de redressement ou remplacement ; en attendant, un stock de sécurité adapté, section 11.2.3), et ne pas inventer de classement entre les sept autres, dont les écarts sont dans le bruit ; si la gérante veut une note, la lui donner avec les pondérations **qu'elle** choisit, et publier les intervalles.

Chapitre 12 : ⊕ Analytique RH et des ressources humaines — exercices et applications

Ce chapitre du cahier accompagne le chapitre complémentaire 12 du livre. Les **applications** reprennent, par petites étapes, les études des sections 12.1 à 12.3 sur les données de la boutique ; les **exercices** (★ facile, ★★ moyen, ★★★ difficile) demandent des calculs à la main puis du code ; les **corrigés** sont à la fin. Les données sont **simulées** (graines fixes) et **petites** (245 lignes, 20 départs) : gardez cette taille en tête. Le chapitre est **autonome**.

```
import sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
import outils_ch12 as O
```

```
ea, dep, emp = O.charger()
print(len(ea), "collaborateur-années |", ea["id_employe"].nunique(), "collaborateurs |",
      ↪ int(ea["depart_dans_l_annee"].sum()), "départs")
```

245 collaborateur-années | 64 collaborateurs | 20 départs

Applications

Application 12.1 — Rotation, intervalle et motifs (sections 12.1.1 à 12.1.3)

Objectif. Calculer les taux de rotation par année avec leur intervalle exact, puis isoler la rotation volontaire.

Étape 1 — par année.

```
rows = []
for annee, g in ea.groupby("annee"):
    k, E = int(g["depart_dans_l_annee"].sum()), len(g)
    t, bas, haut = O.poisson_ic(k, E)
    rows.append((annee, E, k, round(t * 100, 1), round(bas * 100, 1), round(haut * 100, 1)))
print(pd.DataFrame(rows, columns=["année", "effectif", "départs", "taux %", "bas",
      ↪ "haut"]).to_string(index=False))
```

année	effectif	départs	taux %	bas	haut
2021	47	5	10.6	3.5	24.8
2022	48	5	10.4	3.4	24.3
2023	50	2	4.0	0.5	14.4
2024	52	4	7.7	2.1	19.7
2025	48	4	8.3	2.3	21.3

Étape 2 — la rotation volontaire. On ne garde que les démissions.

```
dem = dep[dep["motif"] == "démission"]
k = len(dem); E = len(ea)
t, bas, haut = 0.poisson_ic(k, E)
print(k, "démissions | taux :", round(t * 100, 1), "% [", round(bas * 100, 1), ";", round(haut * 100, 1), "%]")
```

```
14 démissions | taux : 5.7 % [ 3.1 ; 9.6 ]
```

On compte 14 démissions, soit un taux de rotation volontaire de 5,7 % (intervalle de 3,1 à 9,6 %).

À vous. Les intervalles de 2021 et de 2023 se recouvrent-ils ? Que concluez-vous de la baisse apparente de 2023 ?

Piste. Oui : la borne haute de 2023 dépasse la borne basse de 2021 (indicateur ci-dessus : 1). La baisse de 2023 est compatible avec le hasard ; il faudrait davantage de départs pour la confirmer.

Application 12.2 — Comparer des équipes, et l'absentéisme (sections 12.1.4 et 12.1.5)

Objectif. Calculer les taux par site, puis relier absences et heures supplémentaires.

Étape 1 — par site, avec intervalles.

```
for site, g in ea.groupby("site"):
    t, bas, haut = 0.poisson_ic(int(g["depart_dans_l_annee"].sum()), len(g))
    print(site, len(g), "pers.-années |", round(t * 100, 1), "% [", round(bas * 100, 1), ";",
        round(haut * 100, 1), "%]")
```

```
Boutique 155 pers.-années | 7.7 % [ 4.0 ; 13.5 ]
Entrepôt 60 pers.-années | 6.7 % [ 1.8 ; 17.1 ]
Siège 30 pers.-années | 13.3 % [ 3.6 ; 34.1 ]
```

Étape 2 — l'absentéisme et les heures supplémentaires.

```
pente, ordonnee = np.polyfit(ea["heures_sup_mensuelles"], ea["jours_absence"], 1)
r, p = stats.pearsonr(ea["heures_sup_mensuelles"], ea["jours_absence"])
print("pente :", round(pente, 2), "jour par heure supplémentaire | corrélation :", round(r, 2), "| p
    :", round(p, 4))
```

```
pente : 0.47 jour par heure supplémentaire | corrélation : 0.43 | p : 0.0
```

Chaque heure supplémentaire mensuelle va avec **0,47 jour d'absence en plus** par an (corrélation de 0,43) : dix heures de plus vont avec 4,7 jours d'absence de plus.

À vous. Peut-on dire que les heures supplémentaires *causent* les absences ? Quelle information manque ?

Piste. Non : on n'a qu'une corrélation observée. Il faudrait savoir **qui** fait des heures supplémentaires et pourquoi (sous-effectif, volontariat), et suivre les mêmes personnes **avant et après** un changement d'organisation.

Application 12.3 — Courbe de survie (section 12.1.6)

Objectif. Estimer la durée de présence et comparer deux groupes.

Étape 1 — Kaplan-Meier.

```
from lifelines import KaplanMeierFitter
du = 0.durees(ea, dep)
km = KaplanMeierFitter().fit(du["fin"], du["depart"], entry=du["entree"])
print({t: round(float(km.survival_function_at_times([t]).iloc[0]), 2) for t in (1, 3, 5, 8)})
```

```
{1: 0.9, 3: 0.73, 5: 0.53, 8: 0.44}
```

Étape 2 — deux groupes : boutique et autres sites.

```
du["site"] = emp.set_index("id_employe")["site"].reindex(du.index).values
for nom, masque in (("Boutique", du["site"] == "Boutique"), ("Autres sites", du["site"] !=
↳ "Boutique")):
    d_ = du[masque]
    k = KaplanMeierFitter().fit(d_["fin"], d_["depart"], entry=d_["entree"])
    print(nom, len(d_), "| reste 5 ans :", round(float(k.survival_function_at_times([5]).iloc[0]) *
↳ 100), "%")
```

```
Boutique 38 | reste 5 ans : 56 %
Autres sites 26 | reste 5 ans : 45 %
```

À cinq ans, 56 % des collaborateurs de la boutique et 45 % des autres sites restent ; le test du log-rank donne une p-valeur de 0,69.

À vous. Ces deux courbes sont-elles différentes ? Que changerait un effectif dix fois plus grand ?

Piste. Avec une p-valeur de 0,69, on ne peut pas affirmer de différence ; un effectif dix fois plus grand réduirait l'intervalle d'environ un facteur trois (racine de dix), ce qui rendrait visible un écart de quelques points.

Application 12.4 — Écart brut, écart ajusté, compa-ratio (section 12.2)

Objectif. Mesurer l'écart de salaire entre femmes et hommes avant et après ajustement, et lire les compa-ratios.

Étape 1 — brut.

```
brut = ea.groupby("genre")["salaire_brut_mensuel"].mean()
print(brut.round(0).to_dict(), "| écart brut :", round((brut["F"] / brut["H"] - 1) * 100, 1), "%")
```

```
{'F': 2147.0, 'H': 2235.0} | écart brut : -3.9 %
```

Étape 2 — ajusté, avec erreurs types groupées par collaborateur.

```
m = smf.ols("np.log(salaire_brut_mensuel) ~ C(genre, Treatment('H')) + C(poste) + anciennete +
↳ C(annee)", data=ea).fit(cov_type="cluster", cov_kws={"groups": ea["id_employe"]})
cle = "C(genre, Treatment('H'))[T.F]"
lo, hi = m.conf_int().loc[cle]
print("écart ajusté :", round((np.exp(m.params[cle]) - 1) * 100, 1), "% [", round((np.exp(lo) - 1) *
↳ 100, 1), " ; ", round((np.exp(hi) - 1) * 100, 1), "%]")
```

```
écart ajusté : -3.8 % [ -4.5 ; -3.0 ]
```

Étape 3 — le compa-ratio selon le poste.

```
print(ea.groupby("poste")["compa_ratio"].agg(["count", "mean", "min", "max"]).round(2))
```

	count	mean	min	max
poste				
Administratif	2	1.00	1.00	1.00
Caissier	45	1.00	0.93	1.16
Logistique	60	1.00	0.90	1.12
Responsable	14	1.01	0.92	1.11
Service client	28	1.01	0.95	1.13
Vendeur	96	1.00	0.89	1.13

L'écart brut est de 3,9 % et l'écart ajusté de 3,8 %, tous deux en défaveur des femmes (intervalle de l'écart ajusté : de 3,0 à 4,5 %). Sans regrouper par collaborateur, l'intervalle aurait une largeur égale à 1,08 fois celle de l'intervalle regroupé (1 voudrait dire identique ; moins de 1, un intervalle trop étroit) : les lignes d'un même collaborateur ne sont pas indépendantes.

À vous. Dans les postes de moins de cinq personnes, que faites-vous du compa-ratio ? Que diriez-vous à la gérante sur l'écart ajusté ?

Piste. On ne publie pas les compa-ratios de postes de moins de cinq personnes (ils identifient les individus et ne mesurent rien). À la gérante : « à poste et ancienneté égaux, un écart de 3,8 % en défaveur des femmes subsiste ; c'est un signal à examiner (politiques d'embauche, de promotion, de négociation), pas une preuve de discrimination ; nos données ne contiennent ni le temps partiel ni l'évaluation de poste ».

Application 12.5 — Un modèle de départ et sa puissance (section 12.3)

Objectif. Ajuster le modèle, évaluer sa performance, puis mesurer la puissance par simulation.

Étape 1 — ajustement et AUC en validation croisée.

```
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
cols = ["heures_sup_mensuelles", "compa_ratio", "promo_3ans", "evaluation", "anciennete"]
X, y = ea[cols], ea["depart_dans_l_annee"]
auc = cross_val_score(make_pipeline(StandardScaler(), LogisticRegression(C=0.5, max_iter=1000)), X,
    ↪ y,
    ↪ cv=RepeatedStratifiedKFold(n_splits=5, n_repeats=20, random_state=0),
    ↪ scoring="roc_auc")
print(round(auc.mean(), 2), np.percentile(auc, [2.5, 97.5]).round(2))
```

```
0.53 [0.3 0.73]
```

Étape 2 — la puissance, par simulation. On simule 40 entreprises de même taille et l'on compte combien de fois l'effet des heures supplémentaires est détecté.

```
import donnees_a3 as G
def detecte_hs(graine):
    _, p, _ = G.rh(seed=graine)
    p = p.sort_values(["id_employe", "annee"])
    p["compa_ratio"] = p["salaire_brut_mensuel"] / p.groupby(["poste",
    ↪ "annee"])[["salaire_brut_mensuel"]].transform("median")
    p["promo_3ans"] = p.groupby("id_employe")["promotion"].transform(lambda s: s.rolling(3,
    ↪ min_periods=1).max())
    r = sm.Logit(p["depart_dans_l_annee"], sm.add_constant(p[cols])).fit(dis=0)
    return r.params["heures_sup_mensuelles"] > 0 and r.pvalues["heures_sup_mensuelles"] < 0.05
print(sum(detecte_hs(9000 + i) for i in range(40)), "détectations sur 40")
```

5 détections sur 40

L'AUC moyenne est de 0,53 (de 0,30 à 0,73) et l'effet des heures supplémentaires est détecté dans 5 tirages sur 40.

À vous. Que répondez-vous à une direction qui souhaite « un score de risque de départ pour chaque collaborateur » ?

Piste. Avec ces données, un tel score **ne prédit pas mieux que le hasard** (AUC proche de 0,5) ; il faudrait des dizaines de fois plus d'événements. En outre, un score individuel pose des questions d'information, d'équité et d'usage : on propose d'abord une **enquête d'engagement** et des **entretiens**.

Exercices

Exercice 12.1 ★ — Une rotation à la main (section 12.1.2)

Une équipe compte 52 personnes dans l'année et enregistre 4 départs, dont 3 démissions. Calculez le taux de rotation et le taux de rotation volontaire.

Exercice 12.2 ★★ — Un intervalle de Poisson exact (section 12.1.2)

Pour 4 départs en 52 personnes-années, calculez l'intervalle de Garwood à 95 % du taux, avec les quantiles du khi-deux ($\chi^2_{0,025;8}/2$ et $\chi^2_{0,975;10}/2$), puis vérifiez avec `0.poisson_ic`.

Exercice 12.3 ★★ — Kaplan-Meier à la main (section 12.1.6)

Six collaborateurs sont observés : durées de présence 0,5 (départ), 1 (départ), 1 (toujours là), 2 (départ), 3 (toujours là) et 4 (départ). Calculez à la main la courbe de survie à chaque départ, puis vérifiez avec `lifelines`.

Exercice 12.4 ★ — Absentéisme et coût (section 12.1.5)

Un collaborateur est absent 8,5 jours par an en moyenne pour 218 jours théoriques. Calculez le taux d'absentéisme. Si le salaire brut mensuel moyen est de 2 100 €, estimez le coût annuel des absences pour 50 personnes (en ne comptant que le salaire, hors remplacement).

Exercice 12.5 ★★ — Brut contre ajusté sur un exemple (section 12.2.4)

Deux postes : « A » (salaire 2 000 €) compte 30 hommes et 10 femmes ; « B » (salaire 3 000 €) compte 10 hommes et 30 femmes. Dans chaque poste, les femmes gagnent 3 % de moins que les hommes. Calculez l'écart brut global, puis l'écart « à poste égal ». Que montre l'exemple ?

Exercice 12.6 ★★ — Compa-ratio et petits groupes (section 12.2.5)

Un poste compte quatre personnes de salaires 2 000, 2 100, 2 200 et 3 500 €. Calculez la médiane et les compa-ratios. Que pensez-vous du compa-ratio de la personne à 3 500 €, et de sa publication ?

Exercice 12.7 ★★ — Combien d'années de données ? (section 12.3.1)

On observe environ 8 % de départs par an dans une équipe de 50 personnes. Combien d'années de données faudrait-il pour disposer de 10 événements par variable avec 5 variables explicatives ? Qu'en concluez-vous ?

Exercice 12.8 ★★★ — Puissance selon la taille (section 12.3.4)

Simulez la puissance de détection de l'effet des heures supplémentaires pour une, trois et dix entreprises empilées (mêmes règles, G. rh). Tracez ou tabulez la puissance en fonction du nombre d'événements.

Exercice 12.9 ★★★ — Auditer un modèle (section 12.3.5)

Ajustez le modèle de la section 12.3 et calculez le risque moyen prédit par genre et par site. Le genre est-il dans le modèle ? Un écart de risque entre les groupes signifie-t-il quelque chose ? Rédigez en cinq lignes une recommandation à la gérante.

Corrigés**Corrigé 12.1**

Taux de rotation = $4/52 \approx 7,7\%$; rotation volontaire = $3/52 \approx 5,8\%$. Avec si peu de départs, un seul départ de plus ou de moins fait varier le taux de près de deux points.

```
print(round(4 / 52 * 100, 1), round(3 / 52 * 100, 1), round(1 / 52 * 100, 1))
```

```
7.7 5.8 1.9
```

Corrigé 12.2

À la main : limite basse = $\chi^2_{0,025;8}/2/52 = 2,18/2/52 \approx 0,0210$; limite haute = $\chi^2_{0,975;10}/2/52 = 20,48/2/52 \approx 0,1969$. L'intervalle est donc d'environ 2,1 % à 19,7 %.

```
t, bas, haut = 0.poisson_ic(4, 52)
print(round(stats.chi2.ppf(0.025, 8) / 2, 2), round(stats.chi2.ppf(0.975, 10) / 2, 2), "|", round(t *
↪ 100, 1), round(bas * 100, 1), round(haut * 100, 1))
```

```
1.09 10.24 | 7.7 2.1 19.7
```

Quatre départs en 52 personnes donnent un taux de 7,7 % **avec un intervalle de 2,1 % à 19,7 %** : la borne haute est 9 fois la borne basse.

Corrigé 12.3

À la main. À 0,5 an : 6 personnes à risque, 1 départ, $S = 5/6 \approx 0,833$. À 1 an : 5 à risque, 1 départ (la personne « toujours là » à 1 an sort du risque après), $S = 0,833 \times 4/5 \approx 0,667$. À 2 ans : 3 à risque (les durées 2, 3 et 4), 1 départ, $S = 0,667 \times 2/3 \approx 0,444$. À 4 ans : 1 à risque, 1 départ, $S = 0,444 \times 0 = 0$.

Remarque : à 1 an, la convention est de compter les départs **avant** les sorties de l'observation survenues à la même durée ; on laisse donc 5 personnes à risque.

```
from lifelines import KaplanMeierFitter
kmf = KaplanMeierFitter().fit([0.5, 1, 1, 2, 3, 4], [1, 1, 0, 1, 0, 1])
print(kmf.survival_function_.round(3).T.to_string())
```

```
timeline      0.0      0.5      1.0      2.0      3.0      4.0
KM_estimate   1.0    0.833    0.667    0.444    0.444    0.0
```

Corrigé 12.4

Taux d'absentéisme = $8,5/218 \approx 3,9\%$. Un jour de salaire vaut environ $2\,100 \times 12/218 \approx 115,6$ € ; l'absence moyenne coûte $8,5 \times 115,6 \approx 983$ € par personne et par an, soit environ 49 000 € pour 50 personnes. C'est un coût **minimal** (il ignore le remplacement, la perte d'activité, la charge reportée).

```
jour = 2100 * 12 / 218
print(round(8.5 / 218 * 100, 1), round(jour, 1), round(8.5 * jour), round(8.5 * jour * 50))
```

```
3.9 115.6 983 49128
```

Corrigé 12.5

Salaire moyen des hommes : poste A, 2 000 € (30 hommes) ; poste B, 3 000 € (10 hommes) : moyenne = $(30 \times 2000 + 10 \times 3000)/40 = 2250$ €. Salaire des femmes : poste A, $0,97 \times 2000 = 1940$ € (10 femmes) ; poste B, $0,97 \times 3000 = 2910$ € (30 femmes) : moyenne = $(10 \times 1940 + 30 \times 2910)/40 = 2667,5$ €. L'écart brut est de $2667,5/2250 - 1 \approx +18,6\%$: les femmes semblent **mieux payées**, alors qu'à **poste égal** elles gagnent 3 % de **moins**.

```
h = (30 * 2000 + 10 * 3000) / 40
f = (10 * 0.97 * 2000 + 30 * 0.97 * 3000) / 40
print(round(h), round(f), round((f / h - 1) * 100, 1), "% brut | -3 % à poste égal")
```

```
2250 2668 18.6 % brut | -3 % à poste égal
```

L'exemple est un paradoxe de Simpson : la **répartition entre postes** (les femmes sont plus nombreuses dans le poste le mieux payé) masque l'écart à poste égal. D'où l'intérêt de **regarder les deux**, brut et ajusté, et de ne jamais communiquer l'un sans l'autre.

Corrigé 12.6

```
s = np.array([2000, 2100, 2200, 3500])
print(np.median(s), (s / np.median(s)).round(2))
```

```
2150.0 [0.93 0.98 1.02 1.63]
```

La médiane est $(2100 + 2200)/2 = 2150$; les compa-ratios sont 0,93, 0,98, 1,02 et **1,63**. Le compa-ratio de la personne à 3 500 € est élevé mais ne mesure presque rien : la médiane de quatre personnes est instable. Surtout, **publier** ce compa-ratio dans un tableau par poste **identifie** cette personne (sa rémunération se déduit de la médiane et du ratio) : on masque les groupes de moins de cinq personnes.

Corrigé 12.7

Il faut $10 \times 5 = 50$ événements. Avec environ $50 \times 0,08 = 4$ départs par an, il faudrait **12 à 13 ans** de données (sans compter que les effectifs et les conditions évoluent en douze ans).

```
print(10 * 5, 50 * 0.08, round(10 * 5 / (50 * 0.08), 1))
```

```
50 4.0 12.5
```

Conclusion : avec une petite équipe, on ne peut pas ajuster un modèle à cinq variables ; il faut soit **regrouper des entreprises ou des sites**, soit **réduire le modèle** à une ou deux variables (par exemple les heures supplémentaires seules), soit se contenter de **statistiques descriptives avec intervalles**.

Corrigé 12.8

```
import donnees_a3 as G
cols = ["heures_sup_mensuelles", "compa_ratio", "promo_3ans", "evaluation", "anciennete"]
def panel(graine, i):
    _, p, _ = G.rh(seed=graine)
    p = p.sort_values(["id_employe", "annee"]).assign(id_employe=lambda d: d["id_employe"] + 1000 * i)
    p["compa_ratio"] = p["salaire_brut_mensuel"] / p.groupby(["poste",
    ↪ "annee"])["salaire_brut_mensuel"].transform("median")
    p["promo_3ans"] = p.groupby("id_employe")["promotion"].transform(lambda s: s.rolling(3,
    ↪ min_periods=1).max())
    return p
resultats = {}
for taille in (1, 3, 10):
    ok, ev = 0, []
    for rep in range(25):
        g = pd.concat([panel(9500 + rep * 20 + j, j) for j in range(taille)])
        r = sm.Logit(g["depart_dans_l_annee"], sm.add_constant(g[cols])).fit(dis=0)
        ok += int(r.params["heures_sup_mensuelles"] > 0 and r.pvalues["heures_sup_mensuelles"] <
    ↪ 0.05); ev.append(g["depart_dans_l_annee"].sum())
    resultats[taille] = (round(float(np.mean(ev))), ok / 25 * 100)
print(resultats)
```

```
{1: (19, 12.0), 3: (58, 36.0), 10: (190, 80.0)}
```

Avec une entreprise (19 départs en moyenne), la puissance est de 12 % ; avec trois (58 départs), 36 % ; avec dix (190 départs), **80 %**. La puissance croît avec le **nombre d'événements**, pas avec le nombre de lignes : c'est le nombre de départs qui limite l'analyse. (Les valeurs dépendent des graines choisies : on garde l'ordre de grandeur.)

Corrigé 12.9

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
cols = ["heures_sup_mensuelles", "compa_ratio", "promo_3ans", "evaluation", "anciennete"]
mod = make_pipeline(StandardScaler(), LogisticRegression(C=0.5, max_iter=1000)).fit(ea[cols],
    ↪ ea["depart_dans_l_annee"])
ea["risque"] = mod.predict_proba(ea[cols])[:, 1]
print((ea.groupby("genre")["risque"].mean() * 100).round(1).to_dict(),
    ↪ (ea.groupby("site")["risque"].mean() * 100).round(1).to_dict())
```

```
{'F': 9.2, 'H': 6.9} {'Boutique': 8.1, 'Entrepôt': 8.4, 'Siège': 7.9}
```

Le **genre n'est pas une variable du modèle**. Le risque moyen prédit est de 9,2 % pour les femmes

et de 6,9 % pour les hommes, et de 7,9 % à 8,4 % selon le site : ces écarts sont **faibles et statistiquement fragiles** (quelques dizaines de départs au total). Un écart n'aurait de toute façon pas de sens sans savoir s'il correspond à un écart de risque **réel**.

Recommandation type à la gérante, en cinq lignes : (1) *les données (245 lignes, 20 départs) ne permettent pas de prédire les départs individuels : l'AUC est proche du hasard ; (2) nous ne recommandons donc pas de score individuel ; (3) nous proposons une enquête d'engagement anonyme et des entretiens de départ systématiques ; (4) pour l'équité salariale, un écart ajusté d'environ 3 à 4 % mérite un examen des politiques, avec les personnes compétentes ; (5) tout tableau par équipe de moins de cinq personnes sera masqué.*

Chapitre 13 : ⊕ Analyse de sensibilité, simulations « et si » et scénarios — exercices et applications

Ce chapitre du cahier prolonge le chapitre 13 du livre (complémentaire). Il comprend **six applications guidées** (le modèle de résultat de la boutique, de sa construction aux scénarios) puis **dix exercices** ★ à ★★★, tous corrigés. Les données sont **simulées** (la TVA est fictive, à 20 %). Les applications utilisent les fonctions de `build/outils_ch13.py` (`calibrer`, `resultat`, `tirages...`) : lisez-les, elles tiennent en une page.

```
import sys, os, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
from scipy.optimize import brentq
import outils_ch13 as O

T = O.charger()                # comptes mensuels, commandes, sessions, retours, livraisons...
b = O.calibrer(T)              # paramètres de base de 2025
R = lambda **k: O.resultat(b, **k) # le modèle : résultat après retours, en €, selon des variations
↳ de paramètres
base = R()
print("résultat de référence (2025, après retours) :", round(base), "€")
```

résultat de référence (2025, après retours) : 8501 €

Applications

Application 13.1 — Construire et vérifier le modèle (sections 13.1.1 et 13.1.2 du livre)

Objectif. Refaire à la main une version simplifiée du modèle, la comparer aux comptes, puis ajouter les retours.

Étape 1 — Les paramètres de base. On part des chiffres de 2025 : sessions, conversions, paniers, coût d'achat, personnel.

```
for k in ["sessions_pay", "sessions_aut", "conv_pay", "conv_aut", "panier_site", "n_bou",
↳ "panier_bou", "n_res", "panier_res", "cout_unitaire", "pers_fixe", "pers_var", "cout_par_colis"]:
    print(f"{k:16s}", round(b[k], 4) if b[k] < 10 else round(b[k], 1))
```

<code>sessions_pay</code>	33026
<code>sessions_aut</code>	93996

```

conv_pay      0.0262
conv_aut      0.0555
panier_site   101.6
n_bou         5442
panier_bou    103.1
n_res         1426
panier_res    102.4
cout_unitaire 19.1
pers_fixe     92603.8
pers_var      0.0439
cout_par_colis 4.2001

```

Étape 2 — Le chiffre d'affaires par la chaîne. Commandes du site = sessions × conversion ; CA TTC = Σ commandes × panier. On le compare au CA des commandes de 2025.

```

n_site = b["sessions_pay"] * b["conv_pay"] + b["sessions_aut"] * b["conv_aut"]
ca_ttc = n_site * b["panier_site"] + b["n_bou"] * b["panier_bou"] + b["n_res"] * b["panier_res"]
print("commandes du site :", round(n_site), "(observé :", b["n_site"], ")")
print("CA TTC modèle :", round(ca_ttc), "| CA TTC mesuré :", round(b["ca_ttc"]))

```

```

commandes du site : 6078 (observé : 6078 )
CA TTC modèle : 1324764 | CA TTC mesuré : 1324764

```

Étape 3 — Du chiffre d'affaires au résultat, sans retours. Complétez la formule avec les achats (articles × coût unitaire), le personnel, la livraison, les frais bancaires, le marketing et les charges fixes, puis comparez au résultat des comptes.

```

d = 0.resultat(b, detail=True)
print("résultat avant retours (modèle) :", round(d["resultat_avant_retours"]), "| comptes :",
      ↪ round(b["resultat_comptes"]))
stock = T["cr"].loc[T["cr"]["mois"].str.startswith("2025"), "variation_stock"].sum()
print("écart :", round(d["resultat_avant_retours"] - b["resultat_comptes"]), "| variation de stock
      ↪ :", round(stock))

```

```

résultat avant retours (modèle) : 41769 | comptes : 39879
écart : 1890 | variation de stock : -1888

```

Étape 4 — Ajouter les retours. Le coût net est le remboursement hors taxe moins le coût d'achat récupéré sur les articles remis en vente (85 %).

```

print("remboursements TTC (part du CA) :", round(b["taux_retour_ca"] * 100, 2), "%")
print("coût net des retours :", round(d["cout_retours"]), "| résultat après retours :",
      ↪ round(d["resultat"]))

```

```

remboursements TTC (part du CA) : 6.38 %
coût net des retours : 33268 | résultat après retours : 8501

```

À vous. Changez la part d'articles remis en vente (0.PART_REMISE_EN_STOCK, 85 % par défaut) à 60 %, puis à 100 %. Quel est le résultat de référence dans chaque cas ? Que vous apprend cet écart sur la fiabilité d'un modèle qui dépend d'une hypothèse non mesurée ?

Application 13.2 — Tornade et seuil d'élasticité (sections 13.1.3 et 13.1.4 du livre)

Objectif. Classer les paramètres par leur effet, d'abord avec des plages uniformes, puis avec des plages réalistes ; trouver l'élasticité-seuil.

Étape 1 — La tornade.

```

tor, _ = 0.tornado(b, 0.PLAGES_UNIFORMES)
print(tor[["libelle", "effet_bas", "effet_haut"]].round(0).to_string(index=False))

```

	libelle	effet_bas	effet_haut
	Prix de vente	-33176.0	29260.0
	Coût d'achat des produits	32391.0	-32391.0
	Articles par commande	-15868.0	15868.0
	Fréquentation de la boutique	-13639.0	13639.0
	Trafic du site (sessions)	-12038.0	12038.0
	Taux de conversion du site	-12038.0	12038.0
	Taux de retour (points)	10435.0	-10435.0
Charges fixes (loyer, personnel fixe...)		10210.0	-10210.0
	Coût de livraison par colis	6304.0	-6304.0
	Coût d'une session payante	4278.0	-2852.0

Étape 2 — Les mêmes paramètres, avec des plages tirées des données.

```
tor2, _ = 0.tornado(b, 0.PLAGES_DONNEES)
print(tor2[["libelle", "plage", "amplitude"]].round(3).to_string(index=False))
print("classement uniforme :", list(tor["parametre"][:4]), "| classement réaliste :",
      ↪ list(tor2["parametre"][:4]))
```

```
libelle  plage  amplitude
Prix de vente  0.050  33176.337
Coût d'achat des produits  0.030  19434.885
Trafic du site (sessions)  0.060  7222.519
Articles par commande  0.021  6664.568
Fréquentation de la boutique  0.040  5455.562
Taux de conversion du site  0.044  5296.514
Taux de retour (points)  0.400  2087.042
Charges fixes (loyer, personnel fixe...)  0.010  2041.988
Coût d'une session payante  0.100  1901.288
Coût de livraison par colis  0.050  1575.876
classement uniforme : ['prix', 'cout_achat', 'panier', 'frequentation'] | classement réaliste :
↪ ['prix', 'cout_achat', 'trafic', 'panier']
```

Étape 3 — Combien vaut un point ? L'effet de +1 % de chaque paramètre, en euros.

```
un = {k: round(R(**{k: 0.01}) - base) for k in ["prix", "cout_achat", "panier", "trafic", "conv",
      ↪ "frequentation", "fixes", "livraison", "cout_pub"]}
un["retours_pts (1 point)"] = round(R(retours_pts=1.0) - base)
print(un)
```

```
{'prix': 6145, 'cout_achat': -6478, 'panier': 3174, 'trafic': 1204, 'conv': 1204, 'frequentation':
↪ 1364, 'fixes': -2042, 'livraison': -315, 'cout_pub': -169, 'retours_pts (1 point)': -5218}
```

Étape 4 — L'élasticité-seuil. À partir de quelle élasticité une baisse de prix de 5 % cesse-t-elle de dégrader le résultat ?

```
seuil = brentq(lambda e: R(prix=-0.05, elasticite=e) - base, 0.1, 15)
print("élasticité-seuil :", round(seuil, 2))
print({e: round(R(prix=-0.05, elasticite=e) - base) for e in (0.6, 1.2, 1.8, 3.0, 4.0)})
```

```
élasticité-seuil : 3.61
{0.6: -40834, 1.2: -33176, 1.8: -25279, 3.0: -8737, 4.0: 5847}
```

À vous. Ajoutez à PLAGES_DONNEES un paramètre de votre choix (par exemple prix à $\pm 2\%$) et relisez le classement. Que dit-il de la différence entre une **décision** (le prix) et une **incertitude** (le trafic) ?

Application 13.3 — Les lois de la simulation et un premier Monte-Carlo (sections 13.2.1 à 13.2.3 du livre)

Objectif. Justifier les lois par les données, puis simuler l'année prochaine.

Étape 1 — La variabilité mensuelle de la conversion, du panier et des retours.

```
x = T["lig"].merge(T["cmd"][["id_commande", "date_commande", "canal"]], on="id_commande")
x = x[x["date_commande"] >= "2025-01-01"]
w = T["ses"].assign(m=lambda d: d["date"].str[:7])
cs = x[x["canal"] == "Site"].assign(m=lambda d:
    ↪ d["date_commande"].str[:7]).groupby("m").agg(ca=("montant", "sum"), n=("id_commande", "nunique"))
cs["conv"] = cs["n"] / w.groupby("m").size(); cs["panier"] = cs["ca"] / cs["n"]
for nom in ["conv", "panier"]:
    mensuel = cs[nom].std() / cs[nom].mean()
    print(nom, "variation mensuelle :", round(mensuel * 100, 1), "% | de la moyenne annuelle :",
    ↪ round(mensuel / np.sqrt(12) * 100, 1), "%")
```

```
conv variation mensuelle : 15.4 % | de la moyenne annuelle : 4.4 %
panier variation mensuelle : 7.2 % | de la moyenne annuelle : 2.1 %
```

Étape 2 — 2 000 tirages de l'année prochaine, prix indexés de 3 %.

```
P = 0.PARAMS_ANNEE_PROCHAINE
tir = 0.tirages(2000, seed=7, params=P)
res = 0.resultat_rapide(b, tir, prix=0.03)
print("médiane :", round(np.median(res)), "| 10 % :", round(np.quantile(res, 0.1)), "| 90 % :",
    ↪ round(np.quantile(res, 0.9)), "| perte :", round((res < 0).mean() * 100, 1), "%")
```

```
médiane : 17480 | 10 % : -12044 | 90 % : 47433 | perte : 22.2 %
```

Étape 3 — Quels paramètres expliquent la variance ? Régression du résultat centré-réduit sur les paramètres centrés-réduits.

```
noms = list(P)
X = np.column_stack([tir[k] for k in noms]); Xs = (X - X.mean(0)) / X.std(0); rs = (res - res.mean())
    ↪ / res.std()
beta = np.linalg.lstsq(Xs, rs, rcond=None)[0]
print((pd.Series(beta ** 2, index=noms).sort_values(ascending=False) * 100).round(1).to_string())
```

```
cout_achat      66.9
trafic           10.3
panier           8.9
frequentation    5.4
conv             5.3
retours_pts      0.9
fixes            0.8
cout_pub         0.5
livraison        0.4
```

À vous. Remplacez l'écart-type du coût d'achat (3 %) par 5 %, puis par 1 %. Que devient la probabilité de perte ? Quelle conclusion en tirez-vous pour la gérante : faut-il discuter de la loi du coût d'achat ou de celle du trafic ?

Application 13.4 — Stabilité et dépendance (sections 13.2.4 et 13.2.5 du livre)

Objectif. Mesurer combien de tirages suffisent, puis l'effet d'une corrélation entre trafic et conversion.

Étape 1 — Étendue de la médiane selon le nombre de tirages (10 graines).

```
for n in (100, 1000, 10000):
    med = [np.median(0.resultat_rapide(b, 0.tirages(n, seed=s, params=P), prix=0.03)) for s in
    ↪ range(10)]
    print(n, "tirages : étendue de la médiane =", round(max(med) - min(med)), "€")
```

```
100 tirages : étendue de la médiane = 7912 €
1000 tirages : étendue de la médiane = 3090 €
10000 tirages : étendue de la médiane = 1205 €
```

Étape 2 — Corrélation entre trafic et conversion.

```
for rho in (-0.5, 0.0, 0.5):
    r = 0.resultat_rapide(b, 0.tirages(10000, seed=1, rho=rho, params=P), prix=0.03)
    print(f"rho = {rho:+.1f} : écart-type {r.std():8.0f} € | perte {100 * (r < 0).mean():.1f} %")
```

```
rho = -0.5 : écart-type    23018 € | perte 22.1 %
rho = +0.0 : écart-type    23926 € | perte 22.6 %
rho = +0.5 : écart-type    24816 € | perte 23.1 %
```

Étape 3 — Une dépendance forte, pour voir. On force une corrélation de $-0,9$: que devient l'écart-type ? Et avec $+0,9$?

```
for rho in (-0.9, 0.9):
    r = 0.resultat_rapide(b, 0.tirages(10000, seed=1, rho=rho, params=P), prix=0.03)
    print(f"rho = {rho:+.1f} : écart-type {r.std():8.0f} €")
```

```
rho = -0.9 : écart-type    22277 €
rho = +0.9 : écart-type    25527 €
```

À vous. Pourquoi l'effet de la corrélation est-il si modeste dans ce modèle ? (Indice : regardez la part de variance du coût d'achat dans l'application 13.3.)

Application 13.5 — Scénarios et « et si » (sections 13.3.1 et 13.3.2 du livre)

Objectif. Calculer trois scénarios, les situer dans la distribution simulée, puis chiffrer deux « et si » à partir des données.

Étape 1 — Trois scénarios cohérents.

```
res_sc = {k: 0.resultat(b, detail=True, **p) for k, p in 0.SCENARIOS.items()}
for k, d in res_sc.items():
    print(f"{k:11s} commandes {d['commandes']:8.0f} | CA HT {d['ca_ht']:10.0f} | résultat
    ↪ {d['resultat']:9.0f} | centile dans la simulation {100 * (res < d['resultat']).mean():.1f}")
```

```
central      commandes    12793 | CA HT    1134860 | résultat    17979 | centile dans la simulation
↪ 51.4
pessimiste   commandes    11161 | CA HT    961002 | résultat   -53287 | centile dans la simulation
↪ 0.2
optimiste    commandes    13724 | CA HT    1241183 | résultat    64901 | centile dans la simulation
↪ 97.9
```

Étape 2 — L'effet d'une promotion plus longue, tiré des données. Une régression donne l'effet sur les commandes ; les marges par commande viennent des lignes de commande.

```
promo = 0.promo_longue(T)
print({k: round(v, 3) if isinstance(v, float) else v for k, v in promo.items()})
```

```
{'uplift': 0.194, 'uplift_bas': 0.148, 'uplift_haut': 0.242, 'commandes_base': 193, 'part_soldes':
↳ 0.555, 'marge_normale': 32.542, 'marge_soldes': 15.737, 'marge_promo': 23.218, 'delta': -929.054,
↳ 'seuil_uplift': 0.402}
```

Étape 3 — La perte d'un transporteur.

```
t = 0.transporteurs(T)
c = t.loc["Transporteur C"]
print(t.round(3).to_string())
print("surcoût de remplacement (+8 %) :", round(c["colis"] * b["cout_par_colis"] * 0.08), "€")
```

	colis	abimes	retards
transporteur			
Transporteur A	3363	0.010	0.153
Transporteur B	2663	0.015	0.264
Transporteur C	1478	0.042	0.522
surcoût de remplacement (+8 %) :	497 €		

À vous. Dans l'étape 1, quel est le **scénario du milieu de la simulation** ? Dans quel centile se trouve le scénario pessimiste, et que concluez-vous de la différence entre un scénario de crise et un mauvais dixième d'années ?

Application 13.6 — Options, regret et seuils (sections 13.3.3 et 13.3.4 du livre)

Objectif. Comparer des options sur les mêmes tirages, calculer le regret, trouver les seuils de bascule.

Étape 1 — Cinq options sur les mêmes tirages.

```
sim = {nom: 0.resultat_rapide(b, tir, **opt) for nom, opt in 0.OPTIONS.items()}
ref = sim["Indexation de 3 %"]
for nom, v in sim.items():
    print(f"{nom:40s} moyenne {v.mean():9.0f} | perte {100 * (v < 0).mean():5.1f} % | meilleure que
↳ l'indexation dans {100 * ((v - ref) > 0).mean():5.1f} % des tirages")
```

Prix inchangé	moyenne	-1308		perte	52.8 %		meilleure que l'indexation
↳ dans 0.0 % des tirages							
Indexation de 3 %	moyenne	17654		perte	22.2 %		meilleure que l'indexation
↳ dans 0.0 % des tirages							
Hausse de 5 %	moyenne	29552		perte	10.5 %		meilleure que l'indexation
↳ dans 100.0 % des tirages							
Baisse de 5 %	moyenne	-36254		perte	93.8 %		meilleure que l'indexation
↳ dans 0.0 % des tirages							
Indexation de 3 % et publicité +20 %	moyenne	8213		perte	36.1 %		meilleure que
↳ l'indexation dans 0.0 % des tirages							

Étape 2 — La table de regret sur les trois scénarios.

```
M = pd.DataFrame({nom: {s: 0.resultat(b, **{**0.SCENARIOS[s], **{k: v for k, v in opt.items() if k ==
↳ "prix"}, **({"budget_pub": 0.2} if "publicité" in nom else {}))} for s in 0.SCENARIOS} for nom,
↳ opt in 0.OPTIONS.items()})
regret = M.rsub(M.max(axis=1), axis=0)
print(regret.max().round(0).to_string())
```

Prix inchangé	33237.0
Indexation de 3 %	12807.0
Hausse de 5 %	0.0
Baisse de 5 %	70924.0

Indexation de 3 % et publicité +20 % 21415.0

Étape 3 — Les seuils de bascule.

```
cen = lambda **k: {**0.SCENARIOS["central"], **k}
print("hausse du coût d'achat qui annule le résultat 2025 :", round(brentq(lambda x: R(cout_achat=x),
↪ -0.05, 0.2) * 100, 2), "%")
print("élasticité où +5 % cesse de battre +3 % :", round(brentq(lambda e: R(**cen(prix=0.05,
↪ elasticite=e)) - R(**cen(prix=0.03, elasticite=e)), 0.5, 10), 2))
```

hausse du coût d'achat qui annule le résultat 2025 : 1.31 %
élasticité où +5 % cesse de battre +3 % : 3.22

À vous. Rédigez en trois phrases ce que vous diriez à la gérante sur la politique de prix, en incluant un seuil. Relisez-vous : une personne qui ne connaît pas le modèle comprend-elle chaque chiffre ?

Exercices

Exercice 13.1 ★ — Un modèle de poche, à la main (section 13.1.1 du livre)

Un site reçoit 20 000 sessions par mois, converties à 4 %. Le panier moyen est de 100 € TTC (TVA 20 %), le coût d'achat représente 50 % du chiffre d'affaires hors taxe, la livraison coûte 4 € par commande et les charges fixes sont de 15 000 € par mois. (a) Calculez le résultat mensuel. (b) Que devient-il si la conversion passe à 4,4 % (+10 %) ?

Exercice 13.2 ★ — Un écart à expliquer (section 13.1.2 du livre)

Un modèle donne un résultat de 41 769 € quand les comptes disent 39 879 €. Quel est l'écart, en euros et en pourcentage ? Quelle composante des comptes pourrait l'expliquer ? Que feriez-vous si l'écart n'avait **aucune** explication ?

Exercice 13.3 ★★ — La marge de contribution d'une commande (section 13.1.4 du livre)

Mesurez l'effet sur le résultat d'une hausse de 1 % du trafic, comptez le nombre de commandes du site supplémentaires, et déduisez ce que **rapporte une commande du site de plus**. Vérifiez l'ordre de grandeur par un raisonnement à la main (prix hors taxe, coût d'achat, livraison, frais bancaires, personnel variable, coût net des retours).

Exercice 13.4 ★★ — Quelle plage pour que le trafic passe devant ? (section 13.1.6 du livre)

Avec les plages réalistes, le coût d'achat (± 3 %) a une amplitude de 19 435 € et le trafic (± 6 %) de 7 223 €. De combien devrait varier le trafic pour avoir la même amplitude que le coût d'achat ? Cette plage est-elle plausible compte tenu de ce que vous savez des sessions mensuelles ?

Exercice 13.5 ★ — Une loi normale pour un coût (section 13.2.2 du livre)

Le coût d'achat varie de +2 % en moyenne avec un écart-type de 3 %, suivant une loi normale. Quelle est la probabilité que le coût **baisse** ? Cela vous semble-t-il plausible ? Proposez une loi qui évite le problème.

Exercice 13.6 ★★ — Combien de tirages pour 500 € près ? (section 13.2.4 du livre)

L'écart-type du résultat simulé est d'environ 23 900 €. Combien de tirages faut-il pour connaître le **résultat moyen** à ± 500 € près, avec 95 % de confiance ? Vérifiez par simulation avec 40 graines.

Exercice 13.7 ★★★ — Répercuter les hausses de coût (sections 13.2.5 et 13.3.3 du livre)

La gérante propose une règle : si le coût d'achat dépasse de 1 point l'hypothèse (+2 %), elle augmente ses prix de 0,5 point en plus des 3 %. Simulez cette règle et comparez le résultat moyen, l'écart-type et la probabilité de perte à ceux d'un prix fixé à +3 %. Que montre cet exemple sur la différence entre **une décision fixe** et **une règle de décision** ?

Exercice 13.8 ★★ — Un autre scénario de transporteur (section 13.3.2 du livre)

On remplace le transporteur C par un transporteur dont le coût par colis est supérieur de 20 % et qui abîme 1,5 % des colis. Calculez le surcoût, l'économie sur les colis abîmés (même hypothèse de remboursement qu'au livre) et le gain net. À partir de quel surcoût de remplacement l'opération devient-elle perdante ?

Exercice 13.9 ★★ — Le regret selon l'élasticité (section 13.3.3 du livre)

Calculez le résultat de quatre options de prix (inchangé, +3 %, +5 %, -5 %) dans le scénario central pour des élasticités de 0,6, 1,2, 3,0 et 4,0. Quelle option est la meilleure pour chaque élasticité ? Laquelle minimise le plus grand regret ?

Exercice 13.10 ★★★ — La note à la gérante (sections 13.3.4 et 13.3.5 du livre)

Rédigez la note d'une page : réponse, preuve (un tableau), réserves. Imposez-vous trois chiffres : la **probabilité de perte** avec indexation, le **seuil de hausse du coût d'achat** qui annule le résultat de 2025, et l'**élasticité-seuil** de la baisse de prix. Dites clairement quelles hypothèses ne sont pas mesurées.

Corrigés

Corrigé 13.1

```
def mini(s=20000, c=0.04, p=100, tva=0.2, cout=0.5, fixes=15000, liv=4):
    n = s * c
    ca_ht = n * p / (1 + tva)
    return ca_ht - cout * ca_ht - liv * n - fixes

print("résultat de base :", round(mini()), "€ | avec conversion +10 % :", round(mini(c=0.044)), "€ |
↳ écart :", round(mini(c=0.044) - mini()), "€")
```

résultat de base : 15133 € | avec conversion +10 % : 18147 € | écart : 3013 €

- (a) 800 commandes ; chiffre d'affaires TTC 80 000 €, soit 66 667 € hors taxe ; achats 33 333 € ; livraison 3 200 € ; charges fixes 15 000 € : **résultat de 15 133 €**. (b) Avec 4,4 % de conversion, 880 commandes : le résultat monte à **18 147 €**, soit **+3 013 €**. Un point de vue utile : dix pour cent de conversion en plus fournissent 20 % de résultat en plus, parce que les charges fixes ne bougent pas (**levier opérationnel**).

Corrigé 13.2

```
d = 0.resultat(b, detail=True)
ecart = d["resultat_avant_retours"] - b["resultat_comptes"]
print(round(ecart), "€", round(ecart / b["resultat_comptes"] * 100, 1), "% | variation de stock des
↳ comptes :", round(T["cr"].loc[T["cr"]["mois"].str.startswith("2025"), "variation_stock"].sum()),
↳ "€")
```

1890 €, 4.7 % | variation de stock des comptes : -1888 €

L'écart est de **1 890 €**, soit **4,7 %**. Il correspond, à 2 € près, à la **variation de stock** de l'année (−1 888 €) que le modèle ne représente pas. Si l'écart n'avait eu **aucune** explication, il aurait fallu **chercher avant d'utiliser le modèle** : un paramètre mal estimé, un poste de coût oublié, une période mal alignée. Un modèle dont on ne sait pas expliquer l'écart ne doit pas servir à décider.

Corrigé 13.3

```
d_orders = (b["sessions_pay"] * b["conv_pay"] + b["sessions_aut"] * b["conv_aut"]) * 0.01
effet = R(trafic=0.01) - base
print("effet de +1 % de trafic :", round(effet), "€ | commandes en plus :", round(d_orders, 2), "|
↳ résultat par commande de plus :", round(effet / d_orders, 2), "€")
```

effet de +1 % de trafic : 1204 € | commandes en plus : 60.78 | résultat par commande de plus : 19.81 €

+1 % de trafic donne **+1 204 €** pour **60,8 commandes** de plus : une commande du site de plus rapporte environ **19,81 €**. À la main : un panier de 101,63 € TTC fait 84,69 € hors taxe ; le coût d'achat de ses 2,77 articles vaut environ 52,9 € ; la livraison coûte 4,20 € ; les frais bancaires et le personnel variable environ 1,5 € + 3,7 € ; le coût net moyen des retours environ 2,5 €. Il reste de l'ordre de 20 €, ce que donne le modèle : la **marge de contribution** d'une commande est de près d'un **quart de son chiffre d'affaires hors taxe**.

Corrigé 13.4

Il faut une amplitude de 19 435 € pour 7 223 € à $\pm 6\%$, soit **un facteur 2,7** : le trafic devrait varier d'environ **$\pm 16\%$** (19 435 / 1 204 € par point). Sur les sessions mensuelles, l'écart d'un mois à l'autre atteint 18 % (mais en grande partie par saison) : un écart annuel de **$\pm 16\%$** est **peu plausible** pour une année sans événement exceptionnel. Le classement « coût d'achat avant trafic » est donc robuste pour la boutique, ce qui n'est pas vrai de toutes les plages.

Corrigé 13.5

```
from scipy.stats import norm
print("P(baisse) =", round(norm.cdf(-0.02 / 0.03), 4))
```

$P(\text{baisse}) = 0.2525$

$P(X < 0) = \Phi\left(\frac{0-0,02}{0,03}\right) = \Phi(-0,667) \approx 25,25\%$: une chance sur quatre que le coût d'achat **baisse**, ce qui n'est guère plausible quand les fournisseurs annoncent des hausses. On corrige en choisissant une loi qui ne prend que des valeurs positives pour le **facteur** de variation (par exemple une loi **log-normale** sur $1 + \Delta$), ou en tronquant la normale à zéro ; l'essentiel est d'**écrire** pourquoi.

Corrigé 13.6

```
sigma = 23926
print("n =", round((1.96 * sigma / 500) ** 2))
P = 0.PARAMS_ANNEE_PROCHAINE
moy = [0.resultat_rapide(b, 0.tirages(8800, seed=s, params=P), prix=0.03).mean() for s in range(40)]
print("écart-type de la moyenne sur 40 graines :", round(np.std(moy)), "€ | demi-largeur à 95 % :",
      ↪ round(1.96 * np.std(moy)), "€")
```

$n = 8797$

écart-type de la moyenne sur 40 graines : 274 € | demi-largeur à 95 % : 537 €

L'erreur de la moyenne de n tirages est $\sigma/\sqrt{n}$; pour qu'elle soit de 500/1,96, il faut $n = (1,96\sigma/500)^2 \approx 8\,797$ **tirages**. La simulation avec 8 800 tirages et 40 graines donne un écart-type de la moyenne de 274 €, donc une demi-largeur à 95 % d'environ 537 € : de l'ordre de la précision visée (les 40 graines ne donnent qu'une estimation de cet écart-type).

Corrigé 13.7

```
P = 0.PARAMS_ANNEE_PROCHAINE
tir = 0.tirages(10000, seed=1, params=P)
fixe = 0.resultat_rapide(b, tir, prix=0.03)
regle = 0.resultat_rapide(b, tir, prix=0.03 + 0.5 * (tir["cout_achat"] - 0.02))
for nom, v in (("prix fixé à +3 %", fixe), ("règle de répercussion", regle)):
    print(f"{nom:24s} moyenne {v.mean():7.0f} | écart-type {v.std():7.0f} | perte {100 * (v <
      ↪ 0).mean():.1f} %")
```

prix fixé à +3 %	moyenne	18213	écart-type	23926	perte	22.6 %
règle de répercussion	moyenne	18302	écart-type	17204	perte	14.2 %

Le résultat moyen est quasiment le même (18 213 € contre 18 302 €), mais l'**écart-type tombe de 23 926 € à 17 204 €** et la probabilité de perte de **22,6 % à 14,2 %**. Une **règle de décision** (« je répercute la moitié des hausses ») **réduit le risque** sans coûter d'espérance, parce qu'elle fait varier le prix **en sens contraire** du coût : c'est une corrélation voulue entre un paramètre incertain et une décision. Un prix fixé ne profite pas de cette information. (Le modèle suppose que l'élasticité reste la même : une règle de prix qui bouge peut aussi perturber la clientèle, ce que le modèle ignore.)

Corrigé 13.8

```
t = 0.transporteurs(T); c = t.loc["Transporteur C"]
surcout = c["colis"] * b["cout_par_colis"] * 0.20
evites = c["colis"] * (c["abimes"] - 0.015)
eco = evites * b["panier_site"] / (1 + 0.TVA)
print("surcoût :", round(surcout), "| colis abîmés évités :", round(evites, 1), "| économie :",
      ↪ round(eco), "| gain net :", round(eco - surcout))
print("surcoût de remplacement maximal :", round(eco / (c["colis"] * b["cout_par_colis"]) * 100, 1),
      ↪ "%")
```

```
surcoût : 1242 | colis abîmés évités : 39.8 | économie : 3373 | gain net : 2132
surcoût de remplacement maximal : 54.3 %
```

Surcoût de 20 % : **1 242 €** ; colis abîmés évités : $1\,478 \times (4,2\% - 1,5\%) \approx \mathbf{39,8}$; économie (remboursement HT d'un panier moyen) : **3 373 €** ; **gain net : 2 132 €**. L'opération reste gagnante tant que le surcoût de remplacement est inférieur à environ **54 %** ($3\,373 / 6\,208$ €, le coût actuel de livraison des colis du transporteur C). C'est un **seuil de bascule** que l'on peut présenter à la gérante, avec le rappel que l'hypothèse « chaque colis abîmé est remboursé en totalité » est discutable.

Corrigé 13.9

```
opts = {"Prix inchangé": 0.0, "Indexation de 3 %": 0.03, "Hausse de 5 %": 0.05, "Baisse de 5 %": -0.05}
tab = pd.DataFrame({e: pd.Series({k: R(**{**0.SCENARIOS["central"], "prix": v, "elasticite": e}) for
      ↪ k, v in opts.items()}) for e in (0.6, 1.2, 3.0, 4.0)})
print(tab.round(0).astype(int).to_string())
regret = tab.rsub(tab.max(axis=0), axis=1)
print("meilleure option par élasticité :", tab.idxmax().to_dict())
print("plus grand regret par option :", regret.max(axis=1).round(0).astype(int).to_dict())
```

	0.6	1.2	3.0	4.0
Prix inchangé	-1079	-1079	-1079	-1079
Indexation de 3 %	23373	17979	2359	-5966
Hausse de 5 %	39238	29928	3579	-10090
Baisse de 5 %	-43717	-36224	-12309	1962

```
meilleure option par élasticité : {0.6: 'Hausse de 5 %', 1.2: 'Hausse de 5 %', 3.0: 'Hausse de 5 %',
      ↪ 4.0: 'Baisse de 5 %'}
plus grand regret par option : {'Prix inchangé': 40317, 'Indexation de 3 %': 15865, 'Hausse de 5 %':
      ↪ 12052, 'Baisse de 5 %': 82955}
```

La **hausse de 5 %** est la meilleure pour les élasticités de 0,6, 1,2 et 3,0 ; pour 4,0 (au-delà de tous les seuils), c'est la **baisse de 5 %**. Le plus grand regret est de **12 052 €** pour la hausse de 5 %, **15 865 €** pour l'indexation, **40 317 €** pour le prix inchangé et **82 955 €** pour la baisse : la hausse de 5 % **minimise le regret maximal** même en couvrant des élasticités élevées. On voit aussi que la **recommandation ne change qu'au-delà d'une élasticité de 3,2** : tant que l'on n'y croit pas, la hausse domine.

Corrigé 13.10

Éléments de réponse (une note acceptable contient ces trois chiffres, chacun avec sa phrase).

- **Probabilité de perte avec indexation de 3 % : 22,6 %** (10 000 tirages, médiane 17 834 €, intervalle à 80 % de -11 755 € à 49 067 €). « Une année sur quatre environ, on perd de l'argent. »
- **Seuil de coût d'achat : +1,31 %**. « Une hausse de 1,3 % non répercutée annule le résultat de 2025. »

- **Élasticité-seuil de la baisse de 5 % : 3,6.** « Baisser les prix ne rapporte que si chaque 1 % de baisse fait gagner plus de 3,6 % de commandes. »
- **Réserves** : le modèle suppose une élasticité constante (non mesurée), six lois sur neuf sont des hypothèses, les retours ne figurent pas dans le compte de résultat simulé et sont ajoutés par le modèle, les chiffres sont des ordres de grandeur.
- **Recommandation** : indexer les prix, tester une hausse plus forte sur quelques produits (test A/B), sécuriser le coût d'achat.

Une bonne note met la **réponse en tête**, le **tableau** en milieu de page et les **réserves** en bas, et n'utilise aucun mot que la gérante ne comprendrait pas (pas de « tirage », « centile » sans explication).

Projet du volume et auto-évaluation

— exercices et applications

Ce chapitre du cahier clôt le volume III. Il contient le **projet du volume** : répondre **de bout en bout** à une vraie question métier, de la question de la gérante à la recommandation chiffrée, en passant par l'exploration, une régression, un contrefactuel, de l'incertitude et une analyse de sensibilité ; puis l'**auto-évaluation** (quarante questions). Il ne demande aucune notion nouvelle : chaque étape s'appuie sur un chapitre du livre.

Projet du volume

P.1 La question et le cahier des charges

La gérante vous écrit : « *Nous faisons des promotions trois fois par an (soldes d'hiver, soldes d'été, semaine du « Vendredi noir »). Les ventes montent, et l'on me dit que c'est un succès. Je voudrais savoir si ces promotions nous font vraiment **gagner de l'argent**, et si je dois les reconduire telles quelles l'an prochain.* »

Avant tout calcul, vous **reformulez la question** (livre, introduction et 6.1) : « gagner de l'argent » n'est pas « vendre davantage ». La question testable est :

Sur les jours de promotion, la marge brute dégagée est-elle supérieure à celle que l'on aurait dégagée sans promotion ?

Elle suppose un **contrefactuel** (ce qui se serait passé sans promotion) que l'on ne verra jamais : il faudra l'**estimer**, et dire avec quelle incertitude. La méthode suit huit étapes, chacune appuyée sur un chapitre du livre :

Étape	Question	Chapitre du livre
P.2 Explorer	À quoi ressemblent les jours de promotion ? La comparaison brute est-elle équitable ?	1
P.3 Effet sur les commandes	Combien de commandes la promotion ajoute-t-elle, à saison égale ?	3, 5
P.4 Effet sur la marge	Le contrefactuel de marge : gagne-t-on ou perd-on ?	3, 9
P.5 Incertitude	Le résultat tient-il si l'effet est un peu plus fort ou plus faible ?	2, 13

Étape	Question	Chapitre du livre
P.6 Seuil de bascule	À partir de quel effet la promotion devient-elle rentable ?	13
P.7 Mesurer mieux la prochaine fois	Comment concevoir un test pour trancher ?	2.5
P.8 La recommandation	Qu'écrit-on à la gérante ?	6

Les données. `donnees/jours_exploitation.csv` (1 096 jours : commandes, chiffre d'affaires, météo, promotion, dépense publicitaire) et `donnees/commandes.csv`, `lignes_commande.csv`, `produits.csv`. Elles sont **simulées** et leur vérité est programmée (volume I) : la promotion augmente les commandes de **18 %** ; nous comparerons à la fin. Les marges sont **hors taxe** (TVA fictive de 20 %).

P.2 Étape 1 : explorer

On commence par regarder, sans modèle : combien de jours de promotion, quand, et ce que dit la comparaison brute (livre, 1.1 et 1.2).

```
import numpy as np, pandas as pd
import statsmodels.formula.api as smf
import warnings; warnings.filterwarnings("ignore")

j = pd.read_csv("donnees/jours_exploitation.csv", parse_dates=["date"])
j["mois"], j["annee"], j["t"] = j["date"].dt.month, j["date"].dt.year, np.arange(len(j)) / 365.25
print("jours de promotion :", int(j["promo_active"].sum()), "sur", len(j), "| mois concernés :",
      ↪ [int(m) for m in sorted(j.loc[j["promo_active"] == 1, "mois"].unique())])
brut = j.groupby("promo_active")[["nb_commandes", "chiffre_affaires"]].mean().round(1)
print(brut)
print("écart brut des commandes :", round((brut.loc[1, "nb_commandes"] / brut.loc[0, "nb_commandes"]
      ↪ - 1) * 100, 1), "% | écart brut du CA :", round((brut.loc[1, "chiffre_affaires"] / brut.loc[0,
      ↪ "chiffre_affaires"] - 1) * 100, 1), "%")
```

```
jours de promotion : 153 sur 1096 | mois concernés : [1, 6, 7, 11]
      nb_commandes  chiffre_affaires
promo_active
0              32.8             3338.5
1              35.4             3300.1
écart brut des commandes : 7.9 % | écart brut du CA : -1.2 %
```

Lecture. 153 jours de promotion sur 1 096, en janvier, juin-juillet et novembre. La comparaison brute donne 7,9 % de commandes en plus et même **1,2 % de chiffre d'affaires en moins** : un jour de promotion rapporte en moyenne un peu moins qu'un jour ordinaire, ce qui annonce déjà la question de la marge.

La comparaison brute est **trompeuse** : les promotions tombent en janvier et en été, deux **saisons creuses** ; comparer des jours de soldes à des jours « normaux » compare aussi des saisons différentes. Il faut contrôler le mois, le jour de la semaine, la tendance, la pluie et la publicité.

P.3 Étape 2 : l'effet sur les commandes

Une régression sur le logarithme du nombre de commandes donne l'effet en **pourcentage**, « toutes choses égales par ailleurs » (livre, 3.1 et 3.2). Les jours se suivent : on utilise des **erreurs robustes** à l'autocorrélation (livre, 3.1).

```
j["pub7"] = j["depense_pub"].rolling(7, min_periods=1).sum() / 1000
j["pluie"] = (j["pluie_mm"] > 1).astype(int)
formule = "np.log(nb_commandes) ~ promo_active + C(mois) + C(jour_semaine) + t + pluie + pub7"
mod = smf.ols(formule, data=j).fit(cov_type="HAC", cov_kwds={"maxlags": 7})
pct = lambda b: (np.exp(b) - 1) * 100
b, ic = mod.params["promo_active"], mod.conf_int().loc["promo_active"]
print("effet de la promotion sur les commandes : %+.1f %% (IC à 95 %% : %+.1f à %+.1f)" % (pct(b),
    pct(ic[0]), pct(ic[1])))
icp = mod.conf_int().loc["pub7"]
print("effet de 1 000 € de publicité hebdomadaire : %+.2f %% (IC à 95 %% : %+.2f à %+.2f) | R² : %.2f"
    pct(mod.params["pub7"]), pct(icp[0]), pct(icp[1]), mod.rsquared))
naif = smf.ols("np.log(nb_commandes) ~ promo_active", data=j).fit()
print("régression sans contrôle : %+.1f %%" % pct(naif.params["promo_active"]))
```

```
effet de la promotion sur les commandes : +19.2 % (IC à 95 % : +13.5 à +25.1)
effet de 1 000 € de publicité hebdomadaire : +0.73 % (IC à 95 % : -4.55 à +6.31) | R² : 0.78
régression sans contrôle : +8.7 %
```

Lecture. Avec les contrôles, l'effet de la promotion sur les commandes est de **+19,2 %**, avec un intervalle de 13,5 à 25,1 % : plus du double de l'écart brut (7,9 %), parce que les promotions tombent en saison creuse. Sans contrôle, la régression ne trouve que 8,7 %. Pour la publicité, l'estimation (+0,73 % par 1 000 € hebdomadaires) a un intervalle de -4,6 à +6,3 % qui contient à la fois zéro et l'effet programmé (+1,5 %) : les données ne permettent pas de trancher.

P.4 Étape 3 : le contrefactuel de marge

La **marge brute** d'un jour est la somme, sur ses lignes de commande, du montant hors taxe moins le coût d'achat (livre, 9.1). Pendant la promotion, deux choses se passent : on vend **plus** de commandes (effet positif), mais chaque commande **rapporte moins** (remises).

```
cmd = pd.read_csv("donnees/commandes.csv"); lig = pd.read_csv("donnees/lignes_commande.csv"); prod =
    pd.read_csv("donnees/produits.csv")
x = lig.merge(cmd[["id_commande", "date_commande"]], on="id_commande").merge(prod[["id_produit",
    "cout_achat"]], on="id_produit")
x["marge"] = x["montant"] / 1.2 - x["quantite"] * x["cout_achat"]
marge_j = x.groupby("date_commande")["marge"].sum().rename("marge").reset_index().rename(columns={
    "date_commande": "date"})
marge_j["date"] = pd.to_datetime(marge_j["date"])
j = j.merge(marge_j, on="date")
pr, npr = j[j["promo_active"] == 1], j[j["promo_active"] == 0]
mo_np, mo_p = npr["marge"].sum() / npr["nb_commandes"].sum(), pr["marge"].sum() /
    pr["nb_commandes"].sum()
print("marge par commande : hors promotion", round(mo_np, 2), "€ | en promotion", round(mo_p, 2), "€")
```

```
marge par commande : hors promotion 32.08 € | en promotion 23.62 €
```

Lecture. Une commande rapporte en moyenne 32,08 € de marge hors promotion et 23,62 € en promotion : les remises (55,5 % des commandes en promotion portent le code « SOLDES », à -20 %) font perdre un quart de la marge par commande.

Le **contrefactuel** : sans promotion, il y aurait eu $N/(1+e)$ commandes (où N est le nombre réel et e l'effet estimé), chacune rapportant la marge d'une commande ordinaire. L'**incrément de marge** est la marge réelle moins cette marge contrefactuelle.

```
def increment(e, marge_ordre=mo_np):
    commandes_sans = pr["nb_commandes"].sum() / (1 + e)
    return pr["marge"].sum() - commandes_sans * marge_ordre

e_hat, e_bas, e_haut = np.exp(b) - 1, np.exp(ic[0]) - 1, np.exp(ic[1]) - 1
print("marge réelle des jours de promotion :", round(pr["marge"].sum()), "€")
for nom, e in [("effet estimé", e_hat), ("effet bas de l'IC", e_bas), ("effet haut de l'IC", e_haut)]:
    print(f"{nom:20s} e = {e * 100:5.1f} % | marge sans promotion {pr['nb_commandes'].sum() / (1 + e)} € | "
          f"incrément de marge {increment(e):8.0f} €")
```

```
marge réelle des jours de promotion : 127977 €
effet estimé      e = 19.2 % | marge sans promotion 145861 € | incrément de marge -17884 €
effet bas de l'IC e = 13.5 % | marge sans promotion 153101 € | incrément de marge -25125 €
effet haut de l'IC e = 25.1 % | marge sans promotion 138963 € | incrément de marge -10986 €
```

La promotion **vend plus mais gagne moins** : même avec l'extrémité haute de l'intervalle de confiance de l'effet, l'incrément de marge est négatif. On n'a pas encore compté le surcoût publicitaire des jours de promotion.

```
pub_supp = pr["depense_pub"].sum() - len(pr) * npr["depense_pub"].mean()
print("surcoût publicitaire des jours de promotion :", round(pub_supp), "€ | incrément de marge après "
      f"publicité :", round(increment(e_hat) - pub_supp), "€")
```

```
surcoût publicitaire des jours de promotion : 7132 € | incrément de marge après publicité : -25016 €
```

Lecture. Le gain de commandes ne compense pas la perte de marge par commande : l'incrément de marge est de **-17 884 €** à l'effet estimé, de **-10 986 €** à la borne haute de l'intervalle, et de **-25 016 €** une fois la publicité supplémentaire (7 132 €) comptée.

P.5 Étape 4 : l'incertitude et la sensibilité

L'effet estimé est incertain, et le contrefactuel suppose que la marge par commande « normale » aurait été la même. On simule : l'effet est tiré selon l'incertitude de la régression (loi normale sur le coefficient), et la marge par commande varie de plus ou moins 10 % (livre, 13.1 et 13.2).

```
rng = np.random.default_rng(42)
se = (ic[1] - ic[0]) / (2 * 1.96)
sims = np.array([increment(np.exp(rng.normal(b, se)) - 1, mo_np * rng.uniform(0.9, 1.1)) for _ in
                  range(5000)])
print("incrément de marge simulé : médiane", round(np.median(sims)), "€ | intervalle à 90 % :",
      f"round(np.percentile(sims, 5)), \"à\", round(np.percentile(sims, 95)), \"€\")
print("part des simulations où la promotion gagne de l'argent :", round((sims > 0).mean() * 100, 1),
      f"%")
```

```
incrément de marge simulé : médiane -17864 € | intervalle à 90 % : -32456 à -3416 €
part des simulations où la promotion gagne de l'argent : 1.0 %
```

Lecture. L'incrément de marge simulé a pour médiane **-17 864 €** (90 % des simulations entre **-32 456 €** et **-3 416 €**) ; la promotion ne gagne de l'argent que dans 1 % des simulations.

La conclusion est **robuste** aux deux sources d'incertitude que l'on a modélisées : dans presque toutes les simulations, la promotion fait perdre de la marge. Il reste des incertitudes **non modélisées** (voir P.8).

P.6 Étape 5 : le seuil de bascule

À partir de quel effet sur les commandes la promotion serait-elle neutre en marge ? C'est un **seuil de bascule** (livre, 13.3) : on cherche e^* tel que l'incrément soit nul, c'est-à-dire $1 + e^* = N m_0 / M$, avec m_0 la marge d'une commande ordinaire et M la marge réelle des jours de promotion.

```
e_seuil = pr["nb_commandes"].sum() * mo_np / pr["marge"].sum() - 1
print("effet nécessaire pour ne pas perdre de marge : %+.1f %% de commandes (effet estimé : %+.1f %%)"
↳ % (e_seuil * 100, e_hat * 100))
```

```
effet nécessaire pour ne pas perdre de marge : +35.8 % de commandes (effet estimé : +19.2 %)
```

Lecture. Il faudrait +35,8 % de commandes pour ne pas perdre de marge, contre +19,2 % observés : même la borne haute de l'intervalle (+25,1 %) est loin du seuil.

```
cp = cmd.merge(j[["date", "promo_active"]].assign(date=lambda d: d["date"].dt.strftime("%Y-%m-%d")),
↳ left_on="date_commande", right_on="date")
codes = cp.loc[cp["promo_active"] == 1, "code_promo"].fillna("aucun").replace("",
↳ "aucun").value_counts(normalize=True)
print("codes promotionnels des commandes en promotion (%) :", (codes * 100).round(1).to_dict())
```

```
codes promotionnels des commandes en promotion (%) : {'SOLDES': 55.5, 'aucun': 44.5}
```

P.7 Étape 6 : mesurer mieux la prochaine fois

On n'a **pas** randomisé la promotion : on a estimé son effet par régression, ce qui suppose que les variables de contrôle suffisent. Pour trancher une prochaine fois, on peut tester une promotion sur **une partie** des jours ou des clients, et la question devient : **combien de jours** faut-il (livre, 2.5) ? On utilise l'écart-type des résidus de la régression comme mesure du bruit.

```
from statsmodels.stats.power import TTestIndPower
sigma = np.sqrt(mod.scale)
for effet in (0.05, 0.10, 0.20):
    n_jours = TTestIndPower().solve_power(effect_size=np.log(1 + effet) / sigma, alpha=0.05,
↳ power=0.8)
    print(f"détecter +{effet * 100:.0f} % de commandes : environ {np.ceil(n_jours):.0f} jours par
↳ groupe (écart-type résiduel du log : {sigma:.3f})")
```

```
détecter +5 % de commandes : environ 212 jours par groupe (écart-type résiduel du log : 0.179)
détecter +10 % de commandes : environ 57 jours par groupe (écart-type résiduel du log : 0.179)
détecter +20 % de commandes : environ 17 jours par groupe (écart-type résiduel du log : 0.179)
```

Lecture. Pour détecter +10 % de commandes avec un test sur des jours, il faut environ 57 jours par groupe ; +5 % en exigerait 212. Une expérience sur quelques semaines ne tranche que les effets d'au moins 10 à 20 %.

À retenir. Quand on ne peut pas expérimenter, on **estime** un contrefactuel ; quand on peut, on **expérimente**. Dans les deux cas, on annonce l'incertitude et le seuil de bascule, pas seulement une moyenne.

P.8 Étape 7 : la recommandation à la gérante

Le message tient en cinq lignes, chacune avec un chiffre ; on le **produit par le calcul**, pas à la main.

```
fr = lambda v, nd=0: f"{v:,.{nd}f}".replace(",", " ").replace(".", ",")
print(f"1) Les {len(pr)} jours de promotion ont vu {fr(pct(b), 1)} % de commandes en plus que des
↳ jours comparables (intervalle : {fr(pct(ic[0]), 1)} à {fr(pct(ic[1]), 1)} %).")
```

```
print(f"2) La marge par commande tombe de {fr(mo_np, 1)} € à {fr(mo_p, 1)} € à cause des remises.")
print(f"3) La marge brute des jours de promotion est inférieure de {fr(-increment(e_hat))} € à ce
↳ qu'elle aurait été sans promotion (de {fr(-increment(e_haut))} à {fr(-increment(e_bas))} €).")
print(f"4) Il aurait fallu {fr(e_seuil * 100, 0)} % de commandes en plus pour ne pas perdre de marge.")
print(f"5) Sur {fr(len(sims))} simulations, la promotion gagne de l'argent dans {fr((sims > 0).mean()
↳ * 100, 1)} % des cas.")
```

- 1) Les 153 jours de promotion ont vu 19,2 % de commandes en plus que des jours comparables (intervalle
↳ : 13,5 à 25,1 %).
- 2) La marge par commande tombe de 32,1 € à 23,6 € à cause des remises.
- 3) La marge brute des jours de promotion est inférieure de 17 884 € à ce qu'elle aurait été sans
↳ promotion (de 10 986 à 25 125 €).
- 4) Il aurait fallu 36 % de commandes en plus pour ne pas perdre de marge.
- 5) Sur 5 000 simulations, la promotion gagne de l'argent dans 1,0 % des cas.

Recommandation proposée. Ne pas reconduire la promotion « telle quelle » : réduire la **profondeur** des remises (la remise de 20 % est le principal levier), la **concentrer** sur les produits à forte marge ou sur les clients à acquérir, et **tester** la prochaine édition sur une partie des jours ou des clients.

Ce que cette analyse ne dit pas : la **valeur à long terme** des clients acquis pendant les soldes (ils reviennent peut-être) ; l'effet sur l'**image** ou sur le **déstockage** ; l'effet de la promotion sur les **autres jours** (reports d'achats) ; la marge sur les ventes **perdues** si l'on avait été en rupture.

P.9 Les limites de l'étude

- **Contrefactuel estimé, non observé.** Le modèle suppose que mois, jour, tendance, pluie et publicité contrôlent bien la saison ; une cause oubliée biaiserait l'effet (livre, 3.1).
- **Marge par commande « ordinaire » supposée constante :** en réalité, le mélange de produits varie selon la saison. La simulation de P.5 en tient compte par ± 10 %.
- **Reports d'achats et effets de long terme non mesurés :** une promotion peut décaler des achats (donc surestimer l'effet) ou acquérir des clients durables (donc le sous-estimer).
- **Données simulées.** La vérité (+18 % de commandes) est connue ; en réalité, aucune vérité n'est disponible.

P.10 La vérité programmée

Après coup seulement, on compare à ce qui a été programmé. L'effet de la promotion sur les commandes est **+18 %** dans le simulateur ; la régression avec contrôles en trouve environ **+19 %**, avec un intervalle qui contient 18 %, alors que la comparaison brute en trouve moins de la moitié. C'est le résultat que l'on attend d'une bonne analyse : on retrouve l'ordre de grandeur, avec son incertitude, **et** on sait pourquoi la comparaison naïve se trompait.

P.11 Variante : l'objet d'e-mail

La même démarche s'applique au test A/B d'un objet d'e-mail (`donnees/ab_email.csv`) : **mesurer l'écart, tester, calculer la puissance** (livre, 2.2 et 2.5), puis **décider**.

```
from statsmodels.stats.proportion import proportions_ztest, proportion_confint
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

ab = pd.read_csv("donnees/ab_email.csv")
t = ab.groupby("groupe")["achat_7j"].agg(["sum", "size"])
p = t["sum"] / t["size"]
```

```

stat, pval = proportions_ztest(t["sum"].values, t["size"].values)
print("taux d'achat : A", round(p["A"] * 100, 2), "% | B", round(p["B"] * 100, 2), "% | écart",
      ↪ round((p["B"] - p["A"]) * 100, 2), "points | p-valeur :", round(pval, 3))
pui = NormalIndPower().solve_power(effect_size=proportion_effectsize(0.034, 0.030), nobs1=6000,
      ↪ alpha=0.05)
print("puissance pour détecter un vrai écart de 3,0 % à 3,4 % avec 6 000 par groupe :", round(pui *
      ↪ 100, 1), "%")
n_req = NormalIndPower().solve_power(effect_size=proportion_effectsize(0.034, 0.030), alpha=0.05,
      ↪ power=0.8)
print("destinataires par groupe pour 80 % de puissance :", int(np.ceil(n_req)))

```

```

taux d'achat : A 2.92 % | B 3.38 % | écart 0.47 points | p-valeur : 0.143
puissance pour détecter un vrai écart de 3,0 % à 3,4 % avec 6 000 par groupe : 23.8 %
destinataires par groupe pour 80 % de puissance : 30362

```

Lecture. L'écart observé (0,47 point, $p = 0,14$) n'est **pas significatif** ; ce n'est pas la preuve qu'il n'y a pas d'effet : avec 6 000 destinataires par groupe, la puissance n'est que de 24 % pour un vrai écart de 0,4 point, et il en faudrait environ 30 000 par groupe pour atteindre 80 %. Décision raisonnable : ne pas conclure, ou refaire le test avec plus de monde.

Auto-évaluation

Mode d'emploi. Répondez à voix haute ou par écrit **avant** de lire le corrigé, en une ou deux phrases. Les sections à relire sont indiquées dans le corrigé. Trente-cinq bonnes réponses sur quarante signalent un volume bien assimilé ; les questions des sections et chapitres facultatifs (⊕ 1.4, 2.4, 2.5, 3.3, 4.3, 4.4, 5.3 et chapitres 7 à 13) comptent si vous les avez lus.

Exploration (chapitre 1)

1. Le panier moyen vaut 100,4 € et la médiane 79,8 € ; 61 % des commandes sont inférieures à la moyenne. Que dit ce couple de chiffres, et lequel annoncez-vous ?
2. En moyenne, un jour de promotion rapporte **moins** de chiffre d'affaires qu'un jour ordinaire, et pourtant, mois par mois, il rapporte **plus**. Comment est-ce possible ?
3. Pourquoi la règle des 1,5 écart interquartile appliquée au chiffre d'affaires quotidien signale-t-elle surtout des jours de novembre et de décembre ? Que faire à la place ?
4. Distinguez une anomalie, une erreur et un événement, avec un exemple pour chacun.

Tests, A/B et corrélation (chapitre 2)

5. Que mesure une p-valeur, et que ne mesure-t-elle pas ? Donnez deux interprétations fausses courantes.
6. Un test d'e-mail compare 2,92 % d'achats (A) à 3,38 % (B), avec 6 000 destinataires par groupe et une p-valeur de 0,14. Que concluez-vous ? Que faire ensuite ?
7. Une refonte de page est répartie 50/50 mais l'on observe 20 048 sessions en A et 18 574 en B. Est-ce un hasard ? Que faire avant de lire les conversions ?
8. Pourquoi regarder les résultats d'un test A/B tous les jours et s'arrêter dès que $p < 0,05$ est-il dangereux ?
9. Combien de personnes par groupe faut-il pour détecter, avec 80 % de puissance et un seuil de 5 %, le passage d'un taux de 4,0 % à 4,6 % ?
10. La corrélation entre dépense publicitaire et commandes est de 0,53 sur l'année, et de 0,11 à mois égal. Que s'est-il passé ?

Régression (chapitre 3)

11. Dans une régression de $\ln(\text{commandes})$ sur une indicatrice de promotion et des contrôles, le coefficient de la promotion vaut 0,176. Que signifie-t-il en pourcentage ?
12. Que veut dire « toutes choses égales par ailleurs » dans l'interprétation d'un coefficient ? Pourquoi la régression simple et la régression multiple donnent-elles des effets différents ?
13. Pourquoi emploie-t-on des erreurs standard robustes sur des données journalières ?
14. Le rapport de cotes d'un retour pour le Site contre la Boutique est de 3,11 ; la probabilité de retour en Boutique est de 3 %. Quelle est la probabilité pour le Site ?
15. Un modèle expliquant les commandes a un R^2 de 0,78. Peut-on dire qu'il « explique la cause » de 78 % des ventes ? Quelle est la différence entre expliquer et prédire ?

Segmentation et cohortes (chapitre 4)

16. Pourquoi standardise-t-on les variables avant une segmentation par k-moyennes ?
17. Une segmentation en quatre groupes a une silhouette de 0,28. Que faut-il en penser ?
18. Pourquoi les cohortes les plus récentes semblent-elles « moins fidèles » dans une matrice de rétention ?
19. Un client a commandé il y a 20 jours, 12 fois dans l'année, pour 1 200 € : que dit son score RFM ?
20. Marge annuelle par client actif de 69 €, rétention annuelle de 80 %, taux d'actualisation de 10 % : quelle est la valeur vie client par la formule simple $m r / (1 + d - r)$?

Séries temporelles (chapitre 5)

21. Un indice saisonnier de janvier vaut 0,82 : que signifie-t-il ?
22. Pourquoi compare-t-on souvent un mois au même mois de l'année précédente plutôt qu'au mois précédent ?
23. Quelle est la prévision de référence à battre pour une série saisonnière, et pourquoi le MAPE est-il un indicateur imparfait ?
24. Les ventes de quatre mois valent 100, 110, 90 et 120. Calculez la moyenne mobile sur trois mois pour le troisième et le quatrième mois.

KPI (chapitre 6)

25. Quels éléments doit contenir la fiche d'un KPI ?
26. Donnez un exemple de la loi de Goodhart avec un indicateur de la boutique.
27. Le chiffre d'affaires du site se décompose en sessions $\times$ conversion $\times$ panier : vérifiez-le avec 127 022 sessions, 4,785 % de conversion et 101,63 € de panier.
28. Un taux de livraisons à l'heure oscille autour de 92 % avec un écart-type hebdomadaire de 2 points. Quelles limites de contrôle à ± 3 écarts-types ? Une semaine à 87 % est-elle un signal ?

Chapitres complémentaires (7 à 13)

29. Budget : 100 unités à 10 € ; réalisé : 110 unités à 9,50 €. Décomposez l'écart de chiffre d'affaires en effet volume et effet prix.
30. Que sont les « 5 pourquoi », et pourquoi faut-il tester une cause supposée plutôt que de l'affirmer ?
31. Les 20 % de produits les plus vendus font 51 % du chiffre d'affaires, et non 80 % : que dit-on de la « règle des 80/20 » ? Que mesure une analyse ABC ?

32. Votre coût d'acquisition d'un client est de 98,6 € contre une médiane de secteur de 18 €. Peut-on conclure que vous dépensez cinq fois trop ?
33. Coûts fixes annuels de 400 000 € et taux de marge sur coûts variables de 38 % : quel est le seuil de rentabilité ?
34. Résultat d'exploitation de 39 879 € pour un chiffre d'affaires hors taxe de 1 103 969 € : quelle est la marge d'exploitation, et en quoi diffère-t-elle de la marge brute ?
35. Quel est le seuil de rentabilité d'une dépense publicitaire, exprimé en ROAS (chiffre d'affaires sur dépense), si la marge est de 38 % du chiffre d'affaires hors taxe ?
36. 127 022 sessions, 6 078 commandes : quel est le taux de conversion ? Pourquoi dépend-il de la source de trafic ?
37. Demande moyenne de 12 unités par jour, écart-type journalier de 4, délai de 10 jours, niveau de service visé de 95 % ($z = 1,65$) : stock de sécurité et point de commande ?
38. Sept départs sur un effectif moyen de 64 : quel est le taux de turnover ? Pourquoi faut-il être prudent avec ce taux ?
39. Qu'apporte un diagramme en tornade ? Quelle est sa limite ?
40. Une simulation de Monte-Carlo donne un résultat médian de 17 834 € et une probabilité de perte de 22,6 %. Comment le présentez-vous, et que ne dit-elle pas ?

Corrigés des questions

Pour les questions numériques, **calculons** plutôt que de nous fier à la mémoire.

```
import numpy as np
from scipy.stats import chi2, norm
from statsmodels.stats.power import NormalIndPower
from statsmodels.stats.proportion import proportion_effectsize

print("Q7 chi2 de la répartition :", round(((20048 - 19311) ** 2 / 19311) * 2, 1), "| p =",
      f"{{chi2.sf(((20048 - 19311) ** 2 / 19311) * 2, 1):.1e}}", "| part de B :", round(18574 / 38622 *
      100, 1), "%")
print("Q9 n par groupe :",
      int(np.ceil(NormalIndPower().solve_power(effect_size=proportion_effectsize(0.046, 0.040),
      alpha=0.05, power=0.8))))
print("Q11 effet en % :", round((np.exp(0.176) - 1) * 100, 1))
cote = 0.03 / 0.97 * 3.11
print("Q14 probabilité pour le Site :", round(cote / (1 + cote) * 100, 1), "%")
print("Q20 valeur vie client :", round(69 * 0.8 / (1 + 0.10 - 0.8), 1), "€")
v = [100, 110, 90, 120]
print("Q24 moyennes mobiles à 3 mois :", [round(float(np.mean(v[i - 2:i + 1])), 1) for i in (2, 3)])
print("Q27 CA du site :", round(127022 * 0.04785 * 101.63))
print("Q28 limites :", 92 - 3 * 2, "à", 92 + 3 * 2, "| z de 87 % :", (87 - 92) / 2)
print("Q29 volume :", (110 - 100) * 10, "| prix :", round((9.5 - 10) * 110, 2), "| total :", 110 * 9.5
      - 100 * 10)
print("Q33 seuil :", round(400000 / 0.38))
print("Q34 marge d'exploitation :", round(39879 / 1103969 * 100, 1), "%")
print("Q35 ROAS de bascule :", round(1 / 0.38, 2))
print("Q36 conversion :", round(6078 / 127022 * 100, 2), "%")
ss = 1.65 * 4 * np.sqrt(10)
print("Q37 stock de sécurité :", round(ss, 1), "| point de commande :", round(12 * 10 + ss, 1))
print("Q38 turnover :", round(7 / 64 * 100, 1), "%")
```

Q7 chi2 de la répartition : 56.3 | p = 6.4e-14 | part de B : 48.1 %
 Q9 n par groupe : 17923
 Q11 effet en % : 19.2
 Q14 probabilité pour le Site : 8.8 %
 Q20 valeur vie client : 184.0 €
 Q24 moyennes mobiles à 3 mois : [100.0, 106.7]

Q27 CA du site : 617707
 Q28 limites : 86 à 98 | z de 87 % : -2.5
 Q29 volume : 100 | prix : -55.0 | total : 45.0
 Q33 seuil : 1052632
 Q34 marge d'exploitation : 3.6 %
 Q35 ROAS de bascule : 2.63
 Q36 conversion : 4.78 %
 Q37 stock de sécurité : 20.9 | point de commande : 140.9
 Q38 turnover : 10.9 %

1. La moyenne dépasse la médiane de 26 % et 61 % des commandes sont sous la moyenne : la distribution est **asymétrique à droite** (quelques gros paniers). On annonce la **médiane** (79,8 €) avec les quartiles, et l'on peut citer la moyenne pour le chiffre d'affaires total (1.1.1, 1.1.6).
2. C'est le **paradoxe de Simpson** : les promotions tombent en saison creuse (janvier, été), donc la moyenne globale compare des mois différents ; **dans chaque mois comparable**, la promotion rapporte plus. Il faut comparer à **période égale**, jamais sur la moyenne globale (1.2.5).
3. Parce que le chiffre d'affaires est **saisonnier** : novembre et décembre sont naturellement hauts, la règle globale les prend pour des extrêmes. On compare chaque jour à une **référence locale** (même saison, même jour de semaine) et l'on utilise un écart robuste (1.3.3, 1.3.4).
4. Une **anomalie** est un écart par rapport à l'attendu (une journée à -55 % de ventes) ; une **erreur** a une cause technique (un chiffre d'affaires multiplié par 10 par une faute de saisie) ; un **événement** a une cause réelle (une panne du site, une fermeture, une grosse commande professionnelle). On ne les traite pas de la même façon : corriger, annoter ou garder (1.3.5).
5. La p-valeur est la probabilité, si l'hypothèse nulle est vraie, d'observer un écart au moins aussi grand que celui mesuré. Ce n'est **pas** la probabilité que l'hypothèse nulle soit vraie, ni la probabilité que le résultat soit dû au hasard, ni une mesure de la **taille** ou de l'**importance** de l'effet (2.1.2, 2.1.3).
6. L'écart (0,46 point) n'est **pas significatif** ($p = 0,14$) ; cela ne prouve pas l'absence d'effet : avec 6 000 par groupe, la puissance pour un vrai écart de 0,4 point n'est que d'environ 24 %. On ne conclut pas, et l'on refait le test avec plus de monde (environ 30 000 par groupe pour 80 % de puissance) (2.2.2, 2.5).
7. Non : $\chi^2 \approx 56$, $p \approx 6 \times 10^{-14}$ (code ci-dessus) ; 48,1 % des sessions en B au lieu de 50 %. C'est un **ratio d'échantillon défectueux** (ici le filtre de robots n'a été appliqué qu'au groupe B). Il faut d'abord comprendre et corriger la répartition : tant qu'elle est faussée, la comparaison ne l'est pas moins (2.2.4).
8. Chaque coup d'œil est un **test de plus** : le risque de faux positif s'accumule. Dans une simulation sans aucun effet, regarder tous les jours conduit à conclure à tort dans 27 % des cas, au lieu de 5 %. Il faut fixer la taille et la durée à l'avance, ou utiliser des méthodes séquentielles (2.2.6).
9. Environ **17 900** personnes par groupe (code ci-dessus) : passer de 4,0 % à 4,6 % est un petit effet (0,6 point) qui exige beaucoup de monde (2.5.2).
10. La publicité et les commandes montent **ensemble en novembre-décembre** : la saison est une **variable de confusion**. À mois égal, la corrélation tombe à 0,11, et un modèle avec contrôles ne permet pas de conclure sur l'effet de la publicité (2.3.4).
11. $e^{0,176} - 1 \approx 19,2$ % : à contrôles égaux, un jour de promotion s'accompagne d'environ **19 % de commandes en plus** (3.1.5, 3.2.2).
12. C'est l'effet d'une variable lorsque **les autres variables du modèle sont gardées constantes**. Une régression simple attribue à la promotion ce qui vient de la saison, du jour de la semaine, etc. : la régression multiple **sépare** ces effets, d'où des coefficients différents (3.1.3).

13. Les erreurs de jours consécutifs sont **corrélées** (autocorrélation) et leur variance n'est pas constante : les erreurs standard classiques sont trop petites et les intervalles trop étroits. Les erreurs robustes (HAC) corrigent cet effet (3.1.7).

14. Cote en Boutique = $0,03/0,97 \approx 0,031$; cote pour le Site $\approx 0,031 \times 3,11 \approx 0,096$, soit une probabilité d'environ **8,8 %** (code ci-dessus). Un rapport de cotes de 3 ne triple pas la probabilité, sauf quand elle est petite (3.3.1).

15. Non. Un R^2 élevé dit que le modèle **reproduit** bien les variations observées, pas qu'il en détermine la **cause**. **Expliquer** demande un modèle interprétable et des hypothèses causales ; **prédire** demande de bonnes prévisions sur des données nouvelles (jeu de test), quelle que soit l'interprétation des coefficients (3.1.8).

16. Parce que les k-moyennes reposent sur des **distances** : une variable en euros (milliers) écraserait une variable en nombre de commandes (dizaines). On ramène les variables à des échelles comparables, par exemple en les centrant et réduisant (4.1.4).

17. Une silhouette de 0,28 indique une **structure faible** : les groupes se recouvrent. La segmentation peut quand même être **utile** si elle est stable et si les groupes se comportent différemment (réachat), mais on ne doit pas la présenter comme des « familles naturelles » de clients (4.1.5, 4.1.7).

18. À cause de l'**observation tronquée à droite** : une cohorte récente n'a pas encore vécu autant de mois que les anciennes ; ses cellules lointaines sont vides, ou reposent sur peu de données. Comparer des cohortes à **âge égal**, jamais à date égale (4.2.6).

19. Un score **élevé** sur les trois axes : récence (très récent), fréquence (12 commandes) et montant (1 200 €) la placent dans les meilleurs quintiles. Le score classe des clients **relativement** aux autres, il ne dit pas pourquoi (4.3.1).

20. $69 \times 0,8 / (1 + 0,10 - 0,8) = 184$ € (code ci-dessus). La formule suppose une rétention constante et un horizon infini : elle donne un ordre de grandeur, pas une valeur à garantir (4.3.2).

21. En janvier, les ventes valent en moyenne **82 % d'un mois moyen** (18 % de moins) : l'indice est le rapport entre le niveau de janvier et le niveau de référence de la tendance (5.1.4).

22. Parce que la **saison** est la même : comparer janvier à décembre compare un creux à un pic. La comparaison au même mois de l'an dernier neutralise la saisonnalité, et mesure la croissance annuelle (5.1.3).

23. La référence à battre est la **prévision naïve saisonnière** (« comme à la même période l'an dernier »). Le MAPE est imparfait : il explose quand la valeur réelle est proche de zéro, et il pénalise différemment les surestimations et les sous-estimations (5.2.2, 5.2.3).

24. Troisième mois : $(100 + 110 + 90) / 3 = 100$; quatrième mois : $(110 + 90 + 120) / 3 \approx 106,7$ (code ci-dessus) (5.2.1).

25. Le **nom**, la **formule** exacte, le **périmètre** (quoi est inclus), la **période**, la **source**, le **propriétaire**, la **fréquence** de mise à jour, la **décision** qu'il éclaire, et éventuellement sa cible et ses seuils (6.1.2).

26. Si l'on fixe pour cible le **nombre de commandes** en promotion, on pousse les remises : les commandes montent, la marge baisse. Autre exemple : fixer un panier minimum pour la livraison gratuite augmente le panier moyen mais peut faire baisser le chiffre d'affaires. Dès qu'un indicateur devient une cible, il cesse d'être un bon indicateur (6.1.6).

27. $127\,022 \times 0,04785 \times 101,63 \approx 617\,707$ € avec les valeurs arrondies de l'énoncé (code ci-dessus) et exactement 617 715,45 € avec les valeurs non arrondies : l'égalité est exacte à l'euro près. L'arbre sert à **localiser** l'origine d'une variation (6.2.1).

28. Limites : 92 ± 6 , soit de 86 % à 98 %. Une semaine à 87 % reste **dans les limites** ($z = -2,5$) : ce n'est pas un signal à elle seule ; plusieurs semaines consécutives sous la moyenne, ou une semaine sous 86 %, en seraient un (6.3.4, 6.3.5).
29. Effet volume = $(110 - 100) \times 10 = +100$ € ; effet prix = $(9,5 - 10) \times 110 = -55$ € ; total +45 € = $1\,045 - 1\,000$ (code ci-dessus). La convention utilisée (volume au prix budgété, prix aux quantités réelles) fait que les deux effets **somment** à l'écart (7.2).
30. Les « 5 pourquoi » consistent à demander successivement « pourquoi ? » pour remonter d'un symptôme à une cause. Chaque cause supposée est une **hypothèse** : on la teste avec les données (corrélation, comparaison, expérience) ; sinon on ne peut écrire que « cause probable » (7.3).
31. La règle des 80/20 est un **ordre de grandeur** qui ne se vérifie pas toujours : ici il faut 56 produits sur 120 pour 80 % du chiffre d'affaires. L'**analyse ABC** classe les éléments en trois groupes selon leur contribution cumulée (par exemple 80 / 15 / 5 %), pour adapter la gestion à chaque classe (8.1).
32. Non, pas sans comprendre : les **définitions** peuvent différer (quels coûts compte-t-on, quels clients sont « nouveaux » ?), tout comme la période et la taille. Un écart avec une référence est une **question**, pas une conclusion (8.2).
33. $400\,000/0,38 \approx 1\,052\,632$ € de chiffre d'affaires (code ci-dessus) : en dessous, l'entreprise perd de l'argent (9.3).
34. $39\,879/1\,103\,969 \approx 3,6$ %. La **marge brute** ne retire que les achats ; la **marge d'exploitation** retire aussi le personnel, les loyers, le marketing, la livraison et les autres charges (9.2).
35. Il faut un ROAS d'au moins $1/0,38 \approx 2,6$: en dessous, chaque euro de publicité rapporte moins d'un euro de marge (10.2).
36. $6\,078/127\,022 \approx 4,78$ %. La conversion dépend de la **source** (un trafic issu d'un e-mail, déjà client, convertit bien mieux qu'un trafic de réseaux sociaux) : la conversion globale varie avec le **mix** du trafic, pas seulement avec la qualité du site (10.1).
37. Stock de sécurité = $1,65 \times 4 \times \sqrt{10} \approx 20,9$; point de commande = $12 \times 10 + 20,9 \approx 140,9$, soit environ **141 unités** (code ci-dessus) (11.2).
38. $7/64 \approx 10,9$ %. Avec **peu d'événements** (7 départs), l'intervalle de confiance est très large : un taux de 11 % n'est pas distinguable de 5 % ou de 20 % ; on compare avec prudence entre postes ou sites (12.1).
39. Le diagramme en tornade classe les paramètres selon l'**effet sur le résultat** d'une variation de chacun, **un à la fois** : il montre où l'incertitude compte le plus. Sa limite : il ignore les **dépendances** entre paramètres et suppose des plages choisies à la main (13.1).
40. On présente la **médiane**, un **intervalle** (par exemple 80 %) et la **probabilité de perte** : « dans un cas sur cinq, l'année serait déficitaire ». La simulation ne prédit rien : elle ne vaut que par les **hypothèses** (lois, plages, dépendances) qui l'alimentent (13.2, 13.3).

Compétence	Où la retravailler
------------	--------------------

Votre grille d'auto-évaluation

Pour chaque ligne : **je sais l'expliquer** / **je sais le faire** / **à revoir**.

Compétence	Où la retravailler
Explorer une variable et repérer des motifs	1.1, 1.3
Croiser des variables sans tomber dans un paradoxe de Simpson	1.2
Choisir et lire un test, interpréter une p-valeur	2.1, 2.4
Concevoir et lire un test A/B	2.2
Calculer une taille d'échantillon, une puissance	2.5
Lire une corrélation et la distinguer d'une causalité	2.3
Ajuster et diagnostiquer une régression	3.1
Interpréter les coefficients pour des non-spécialistes	3.2
Modéliser une probabilité (régression logistique)	3.3
Segmenter des clients et valider la segmentation	4.1
Lire une matrice de cohortes sans se tromper	4.2, 4.4
Calculer un RFM, une valeur vie client, un churn	4.3
Décomposer une série, comparer à l'an dernier	5.1
Lisser et prévoir honnêtement	5.2, 5.3
Définir un bon KPI et un arbre d'indicateurs	6.1, 6.2
Fixer cibles, références et seuils	6.3
Décomposer un écart et remonter aux causes	7
Faire un Pareto, une analyse ABC, un benchmarking	8
Lire des comptes, des ratios, un seuil de rentabilité	9
Analyser l'acquisition et la rentabilité marketing	10
Analyser livraisons, stocks et fournisseurs	11
Analyser effectifs et équité avec prudence	12
Mesurer sensibilité, simuler, comparer des scénarios	13
Répondre de bout en bout à une question métier	Projet du volume